REACT JS

INTERVIEW GUIDE

Read This Once. Crack Every React Interview.

From "What is JSX?" to Senior-Level Machine Coding — Concepts, Hooks, Redux, Performance, Custom Hooks, API Patterns, React 18 & 10 Real Projects.

16	381+	160+	10
Chapters	Pages	Interview Q&As	Live Projects

WHAT'S INSIDE

JSX & Components	React Hooks	Context API	React Router	Redux Toolkit	Performance
Custom Hooks	API & Async	Error Boundaries	React 18	Portals & Fiber	Machine Coding

By Syed Shaaz Akhtar

React Developer · Interview Coach · Full-Stack Engineer

Interview-Focused · Concept-Driven · Production-Ready Code · Real Projects

TABLE OF CONTENTS

React JS Interview Guide · By Syed Shaaz Akhtar · 16 Chapters · 381+ Pages

Ch 01 Introduction to React

- What is React? — Why React over vanilla JS
- JSX — JavaScript + HTML syntax
- Virtual DOM — how it makes React fast
- Components — functional vs class
- SPA vs MPA — when to use React
- React ecosystem overview (Vite, CRA, Next.js)
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 02 Props & Prop Drilling

- What are Props? — passing data parent → child
- Props are read-only — why you must not mutate them
- children prop — composing UI with nested content
- Default props and prop validation
- Prop Drilling — the problem with deep component trees
- When to solve drilling: Context vs Redux
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 03 Lifting State Up

- What is Lifting State Up?
- Sibling communication — via shared parent
- Controlled Components — React owns the value
- The single source of truth principle
- When NOT to lift — keeping local state local
- Data flow diagrams — props down, callbacks up
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 04 React Hooks

- useState — reactive state and functional updates
- useEffect — side effects and cleanup
- useRef — DOM access and mutable values
- useReducer — complex state with reducer pattern
- useContext — consuming shared context
- useMemo + useCallback — memoization hooks
- Rules of Hooks — why and what they are
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 05 Component Lifecycle

- Mount → Update → Unmount lifecycle phases
- useEffect covers all 3 phases
- Dependency array — [], [deps], no array
- Cleanup function — why and when to return one
- StrictMode double-fire — what it means
- Effect execution order across components

- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 06 Event Handling & Forms

- Synthetic Events — React's event wrapper
- e.preventDefault() — stopping default form behaviour
- Controlled vs Uncontrolled inputs
- Handling all input types (text, checkbox, select, file)
- Form validation — patterns and best practices
- useRef for uncontrolled components
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 07 Conditional Rendering & Lists

- &&, ternary, and early return patterns
- Rendering lists with .map()
- The key prop — why it is required and what breaks without it
- Never use array index as key — the exact reason
- Nested lists and fragments
- Performance considerations for large lists
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 08 Context API

- createContext, Provider, useContext — 3 step pattern
- Custom context hook pattern (useAuth, useTheme)
- Auth context — complete login/logout example
- Performance — when Context causes unnecessary re-renders
- Context vs Props vs Redux — when to choose what
- Multiple contexts — composing providers
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 09 React Router

- BrowserRouter, Routes, Route setup
- useNavigate — programmatic navigation
- useParams — reading URL parameters
- useSearchParams — query string handling
- Nested Routes and Outlet
- Protected Routes — auth-gated navigation
- Lazy loading routes with React.lazy + Suspense
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 10 Redux Toolkit

- Why Redux — when Context is not enough
- Store, Slice, Action, Reducer, Dispatch explained
- createSlice — auto-generated action creators
- configureStore — combining slices
- useSelector and useDispatch hooks
- createAsyncThunk — API calls in Redux
- Complete Cart example with Redux
- Redux DevTools — time-travel debugging
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 11 Performance Optimization

- What causes re-renders — 4 triggers
- React.memo — skip re-render when props unchanged
- useMemo — cache expensive computed values
- useCallback — stable function references
- The Optimization Trinity — memo + useMemo + useCallback
- Code splitting — React.lazy + Suspense
- Virtualization — react-window for huge lists
- Debounce and Throttle patterns
- React DevTools Profiler — measuring before optimizing
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 12 Custom Hooks

- What is a custom hook — rules and naming
- Logic shared vs state NOT shared
- useFetch — loading / error / data + AbortController
- useLocalStorage — state that survives refresh
- useDebounce — debounce any value
- useForm — complete form state management
- useIntersectionObserver — infinite scroll trigger
- useClickOutside, useWindowSize, useMediaQuery
- Composing hooks — hooks calling hooks
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 13 API Calls & Async Patterns

- async/await and Promise fundamentals
- fetch() — why you must check response.ok
- Axios — interceptors, auto JSON, timeout
- Complete data fetching pattern with useEffect
- Race conditions — what they are and how to prevent them
- AbortController — cancel requests on unmount

- Error handling — network, HTTP, parse, timeout
- Optimistic Updates — instant UI with rollback
- Pagination — page numbers and infinite scroll
- React Query — useQuery, useMutation, cache invalidation
- API Service Layer — organising all HTTP calls
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 14 Advanced Topics

- Error Boundaries — contain render errors, show fallback
- Portals — render outside parent DOM (modals)
- Suspense — declarative loading state
- Reconciliation — virtual DOM diffing algorithm
- Keys and component identity — when state resets
- forwardRef — pass ref to inner DOM element
- useImperativeHandle — control what ref exposes
- Higher-Order Components (HOC) — withAuth, withLoading
- Compound Components — Tabs.Tab, Tabs.Panel pattern
- Render Props pattern
- React Fiber — interruptible reconciliation engine
- 12 Interview Q&As · Common Mistakes · Quick Reference

Ch 15 React 18 Features & Final Review

- createRoot() — the one change that unlocks everything
- Automatic Batching — free performance everywhere
- Concurrent Mode — interruptible rendering (default)
- useTransition — mark state updates as non-urgent
- useDeferredValue — defer expensive value processing
- useId — stable unique IDs for SSR-safe accessibility
- useSyncExternalStore — subscribe to external stores
- Improved Suspense — sibling independence
- Streaming SSR — progressive HTML delivery
- Complete 15-chapter summary table
- Top 20 most-asked React interview questions

Ch 16 Machine Coding Round

- Project 1 — Counter App [Beginner] — useState, disabled state
- Project 2 — Todo List [Beginner-Medium] — CRUD, derived state, filters
- Project 3 — Search + Filter + Sort [Medium] — useMemo, chained ops
- Project 4 — Star Rating [Medium] — hover state, controlled component
- Project 5 — Accordion / FAQ [Medium] — animation, single/multi open
- Project 6 — Infinite Scroll Feed [Medium-Hard] — IntersectionObserver
- Project 7 — Multi-Step Form [Hard] — useReducer, validation, progress
- Project 8 — GitHub Live Search [Hard] — debounce + AbortController + real API
- Project 9 — Shopping Cart + Context [Hard] — Context + useReducer + localStorage
- Project 10 — Kanban Board [Senior] — nested state, move between columns
- 5-Minute Planning Framework · 8 Common Mistakes · 10 Follow-Up Q&As

★ *Every chapter includes Interview Q&As · Code Examples with Comments · Comparison Tables · Common Mistakes to Avoid · Quick Reference Summary*

React JS

Easy Explanation — Everything You Need for Interview

By Syed Shaaz Akhtar | Interview Focused Guide

This guide covers React JS from the very beginning. Every concept is explained in simple English with real examples. Each chapter ends with the most-asked interview questions and their complete answers. Read this like you are reading a story — not like a textbook.

Chapter 1: Introduction to React JS

What React is, how it works, and why every company uses it

Before learning React, you need to understand WHY it was created. What problem existed before React? What does React solve? These are not just interview questions — understanding this will help you understand every single concept in React much better.

Topic 1: What is React?

React is a JavaScript **library** for building user interfaces. It was created by **Jordan Walke** at Facebook in 2011 and was open-sourced in 2013. Today it is maintained by **Meta (Facebook)** and a large open-source community.

Think of it this way:

- **HTML** gives structure to a webpage — it is the skeleton
- **CSS** makes it look good — it is the clothes
- **JavaScript** makes it interactive — it gives actions
- **React** is a smarter, organised way to write that JavaScript — it makes building complex interfaces easy

For example, when you open Gmail and click different folders — the page does NOT reload. Only the content changes. Or when you type in the Facebook search box and results appear instantly. That is React in action.

> 💡 **Tip**
>
> When the interviewer asks "What is React?", always say:
>
> "React is a JavaScript LIBRARY (not a framework) created by Meta for building user interfaces using a component-based approach and a Virtual DOM for performance."
>
> Those three things — library, component-based, Virtual DOM — are what the interviewer wants to hear.

Topic 2: Library vs Framework — Very Important Difference!

This is one of the first questions interviewers ask: "Is React a library or a framework?" The answer is library — but you must know WHY.

Aspect	Library	Framework
Definition	A set of tools you pick and use when you need them	A complete structure that controls how you build things
Who is in control?	**You** control the flow of the app	The **framework** controls the flow — you fill in the blanks
Flexibility	Very flexible — choose your own tools	Less flexible — must follow framework rules
Example	**React** — only handles the UI layer	**Angular** — has routing, forms, HTTP, everything built-in
Analogy	Buying ingredients — you cook your own dish	Meal kit — everything is given, you just follow steps

> ✅ **Remember**
>
> React is a LIBRARY. Angular is a FRAMEWORK. Vue is in between — called a "progressive framework".
>
> React only handles the View (UI) layer. For routing you use React Router. For global state you use Redux or Context API. You pick and choose.
>
> This is asked in nearly every React interview. Do not say "React is a framework."

Topic 3: The Problem Before React — Why Was It Created?

To understand React, you need to understand the pain developers faced before it.

The Old Way — Vanilla JavaScript + jQuery

Before React, developers built websites using plain JavaScript or jQuery. Every time something changed on the page — for example, a counter increased or a user logged in — developers had to manually go into the HTML, find the right element, and update it.

Imagine a Facebook-like app with thousands of elements on screen. A new notification comes in. You have to:

1. Find the notification bell element in the DOM
2. Update the count number inside it
3. Maybe also update a dropdown list somewhere else
4. Maybe also update a counter in the sidebar

Doing all of this manually — in a big app — was a nightmare. It was:

- **Slow** — the browser had to re-render large parts of the page
- **Buggy** — easy to forget to update one element

- **Messy** — JavaScript code was all over the place
- **Hard to reuse** — you could not easily reuse a piece of UI

> 📝 **Note**
> jQuery was the most popular library before React. It made selecting DOM elements easier. But it still had all the same problems with large apps.
> jQuery is fine for small websites. For big, complex apps like Facebook or Gmail — it falls apart.

How React Solved All of This

Problem Before React	React Solution
Manual DOM updates — find element, change it by hand	React updates the DOM automatically when data changes
Hard to reuse UI pieces across the app	Components — write once, reuse anywhere
Logic and HTML were mixed together — messy	JSX keeps UI and logic together cleanly in one component
Page had to reload to show new content	SPA (Single Page Application) — no reload needed
Tracking what changed was very hard	Virtual DOM — React tracks exactly what changed efficiently

Topic 4: Virtual DOM — The Most Asked React Topic

If there is ONE topic you must understand deeply for a React interview, it is the Virtual DOM. This question is asked in almost every interview.

First — What is the Real DOM?

DOM stands for Document Object Model. It is a tree-like structure the browser creates from your HTML. Every element on your page — every div, button, heading — is a node in this tree.

When something changes on the page, the browser has to:

5. Rebuild part of this DOM tree
6. Recalculate the layout of all affected elements (Reflow)
7. Repaint those elements on screen (Repaint)

This is expensive. The browser has to do a lot of work. And if you change many things at once — like in a React app where state changes — doing this for every single change would make the app very slow.

> 📝 **Note**
> Real DOM operations are slow because they trigger layout recalculations and screen repaints in the browser.

> The more elements you update at once, the slower it gets. This is why large apps built with plain JS felt laggy.

What is the Virtual DOM?

The Virtual DOM is a lightweight JavaScript copy of the real DOM. It is just a plain JavaScript object that describes what the UI should look like — it has no connection to the actual browser.

Since it is just a JavaScript object — no browser APIs, no layout calculations, no painting — operations on the Virtual DOM are extremely fast.

> 💡 **Tip**
> A simple Virtual DOM node looks like this in memory:
> { type: "div", props: { className: "box" }, children: [...] }
> This is just a JavaScript object. React creates millions of these instantly — no browser work needed at this stage.

How React Uses the Virtual DOM — Step by Step

This is the process React follows every time state or props change. It is called Reconciliation.

```
// Step 1: Initial Render
// Your JSX is converted to a Virtual DOM tree (v-old)
// React passes this to ReactDOM which paints it on the real DOM

// Step 2: State Changes
// Example: user clicks a button, count goes from 0 to 1

// Step 3: React creates a NEW Virtual DOM tree (v-new)
// This new tree reflects the updated state

// Step 4: Diffing Algorithm
// React compares v-new with v-old, node by node
// It finds exactly WHAT changed (in this case: count = 1)

// Step 5: Patching
// React updates ONLY the changed node in the REAL DOM
// Everything else stays untouched

// Result: The browser repaints only the tiny changed part — very fast!
```

Aspect	Real DOM	Virtual DOM
What is it?	Actual HTML tree in the browser	JavaScript object copy of the DOM
Speed of updates	Slow — triggers layout + repaint	Fast — just JavaScript object operations

On every change	Can re-render large parts of tree	Only updates the nodes that actually changed
Who manages it?	The browser	React (completely in memory)
Memory usage	Higher	Lower — just plain JS objects

✅ Remember

Virtual DOM is NOT magic. It is just a plain JavaScript object describing what the UI looks like.

The key benefit is not that Virtual DOM is faster — it is that React uses it to minimise the number of real DOM updates.

The process: State changes → New Virtual DOM → Diff with old → Update only what changed in Real DOM.

This process is called RECONCILIATION. The algorithm React uses is called the DIFFING ALGORITHM.

🎯 Interview Tip

How to answer "What is Virtual DOM?" in an interview:

"Virtual DOM is a lightweight JavaScript object that React keeps in memory as a copy of the real DOM. When state changes, React creates a new Virtual DOM tree and uses a Diffing Algorithm to compare it with the previous tree. It finds the minimum number of changes needed and updates only those nodes in the real DOM. This process, called Reconciliation, makes React very efficient because it avoids unnecessary browser repaints."

Topic 5: SPA vs MPA — Single Page vs Multi Page App

React is used to build SPAs. You need to understand what this means and why it matters.

Multi Page Application (MPA) — The Old Way

In a traditional website, every time you click a link or button — the browser sends a request to the server, the server sends back a whole new HTML page, and the browser loads that new page from scratch.

Think of a news website from 2005. You click on "Sports" — the whole page reloads. You see a loading spinner. The header, footer, sidebar all disappear and come back again. Every navigation = full page reload.

Single Page Application (SPA) — The React Way

In an SPA, there is only ONE HTML file. When you navigate to a different section, React does NOT reload the page. Instead it swaps out the components on screen — using JavaScript. The URL changes, the content changes, but the page never reloads.

Open Gmail right now. Click on different labels — Inbox, Sent, Drafts. Notice the URL changes but the page does NOT reload. The sidebar stays. The header stays. Only the email list updates. That is SPA behaviour.

Feature	SPA (React)	MPA (Traditional)
Page reload on navigation?	**No** — only components swap	**Yes** — full HTML reload every time
Speed after first load	Very fast — feels like a native app	Slower — needs server round trip each time
Initial load time	Slightly slower (downloads JS bundle)	Fast first load (server sends ready HTML)
SEO (Search engines)	Needs extra work — SSR or SSG via Next.js	Naturally good — each page has its own URL
User experience	Smooth, app-like feel	Page flicker on every navigation
Real examples	Gmail, Facebook, Figma, Airbnb	Wikipedia, government sites, news articles

> 📝 **Note**
> SEO means Search Engine Optimisation — how well Google can find and index your pages.
> SPAs have SEO challenges because the content is loaded via JavaScript, not HTML. Google struggles to read JavaScript-rendered content.
> This is why Next.js was created — it does Server Side Rendering (SSR) for React apps, giving you the best of both worlds.

Topic 6: Components — The Building Blocks of React

The most important concept in React is the **Component**. Everything in React is a component.

What is a Component?

A component is a reusable piece of UI. Think of it like a LEGO block. A LEGO block is a small, independent piece. You can combine many blocks to build a big structure. Each block can be reused — you can place the same block in multiple places.

In React, a component is a JavaScript function that returns JSX (which looks like HTML). Each component represents a piece of your UI — a button, a card, a header, a whole page.

```
// This is a React Component — just a JavaScript function
function Greeting() {
  return <h1>Hello, Shaaz!</h1>;
}

// Use it anywhere in your app like an HTML tag:
<Greeting />

// You can reuse it as many times as you want:
<Greeting />
<Greeting />
```

```
<Greeting />
```

> 💡 **Tip**
>
> Component names MUST start with a capital letter. <greeting /> means an HTML element. <Greeting /> means a React component.
>
> This is how React tells the difference between built-in HTML elements (div, button, h1) and your custom components.

Component Tree — How React Apps Are Structured

Every React app is a tree of components. The App component is the root. It contains child components. Each child can have its own children. This is called the Component Tree.

```
// Component Tree — like a family tree
App                        // Root component
  Header                   // Child of App
    Logo                   // Child of Header
    NavBar                 // Child of Header
  Main                     // Child of App
    HeroSection            // Child of Main
    ProductList            // Child of Main
      ProductCard          // Child of ProductList (used many times)
        ProductImage       // Child of ProductCard
        ProductInfo        // Child of ProductCard
        BuyButton          // Child of ProductCard
  Footer                   // Child of App
```

> ✅ **Remember**
>
> When a component's STATE changes, React re-renders that component AND all its children.
>
> It does NOT re-render the parent or sibling components.
>
> This is why deciding WHERE to put state is very important in React — putting it too high causes unnecessary re-renders of many child components.

Topic 7: JSX — JavaScript + HTML Together

When you look at React code for the first time, you will notice something strange — HTML-like code sitting inside a JavaScript function. That is JSX.

What is JSX?

JSX stands for JavaScript XML. It is a syntax extension that lets you write HTML-like code inside JavaScript. It is NOT real HTML — it just looks like it.

Under the hood, a tool called Babel compiles every JSX element into a React.createElement() function call. This is why JSX works in the browser — it gets converted to pure JavaScript before running.

```
// What YOU write (JSX):
function UserCard({ name, role }) {
```

```
  return (
    <div className="card">
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}

// What BABEL converts it to (React.createElement calls):
function UserCard({ name, role }) {
  return React.createElement(
    "div",
    { className: "card" },
    React.createElement("h2", null, name),
    React.createElement("p",  null, role)
  );
}
```

> 📝 **Note**
>
> This is why React 16 and older required "import React from 'react'" at the top of every file — because JSX compiled to React.createElement() which needed React to be in scope.
>
> Since React 17, the new JSX transform handles this automatically. You no longer need to import React manually just to use JSX.

JSX Rules — Must Know for Interviews

JSX looks like HTML but it has strict rules. Breaking any of these causes an error.

Rule 1: Only One Root Element

Every JSX return must have exactly ONE parent element wrapping everything else.

```
// ❌ WRONG — two root elements, no parent wrapper
return (
  <h1>Title</h1>
  <p>Description</p>
);

// ✅ CORRECT — wrap in a div
return (
  <div>
    <h1>Title</h1>
    <p>Description</p>
  </div>
);

// ✅ BETTER — use a Fragment (does not add extra div to the DOM)
return (
  <>
    <h1>Title</h1>
```

```
    <p>Description</p>
  </>
);
```

Rule 2: Use className, Not class

Because JSX is JavaScript, and class is a reserved keyword in JavaScript. So React uses className instead.

```
// ❌ WRONG
<div class="box">Hello</div>

// ✅ CORRECT
<div className="box">Hello</div>
```

Rule 3: All Tags Must Be Closed

In HTML, tags like br and img can be left unclosed. In JSX, every tag must be closed — either with a closing tag or self-closing slash.

```
// ❌ WRONG (valid HTML but invalid JSX)
<br>
<img src="photo.jpg">
<input type="text">

// ✅ CORRECT
<br />
<img src="photo.jpg" />
<input type="text" />
```

Rule 4: JavaScript Goes Inside Curly Braces {}

Any JavaScript expression inside JSX must go inside curly braces. This can be a variable, a function call, or a ternary expression.

```
const name = "Shaaz";
const isLoggedIn = true;

return (
  <div>
    <h1>Hello {name}</h1>           // variable
    <p>{2 + 2}</p>                   // expression
    <p>{name.toUpperCase()}</p>      // function call
    {isLoggedIn ? <p>Welcome!</p> : <p>Please log in</p>}
  </div>
);
```

Rule 5: camelCase for HTML Attributes

HTML attributes become camelCase in JSX. onclick becomes onClick, onchange becomes onChange, tabindex becomes tabIndex.

```
// ❌ WRONG
<button onclick={handleClick}>Click</button>

// ✅ CORRECT
<button onClick={handleClick}>Click</button>

// More examples:
<input onChange={handleChange} />
<label htmlFor="email">Email</label>   // "for" becomes "htmlFor"
<div tabIndex={0}></div>               // "tabindex" becomes "tabIndex"
```

✅ Remember

JSX is NOT HTML. It looks like HTML but it is JavaScript under the hood.

Rules: One root element, className not class, all tags closed, JS in {}, camelCase attributes.

Fragment <> </> is used to wrap multiple elements without adding an extra div to the DOM.

Babel compiles JSX to React.createElement() calls. The browser never sees JSX directly.

Topic 8: Setting Up React — The Modern Way with Vite

There are two ways to create a React project. The old way is Create React App (CRA). The modern, recommended way is Vite. Vite is 10-20x faster.

Aspect	Create React App (CRA)	Vite
Status	Officially deprecated — avoid for new projects	**Recommended** — the modern standard
Dev server start time	20-40 seconds on large projects	Under 1 second — almost instant
Build tool	Webpack (older, slower)	esbuild + Rollup (extremely fast)
Hot Module Reload	Slower to reflect changes	Instant updates on file save
Bundle size	Larger default setup	Leaner and faster by default

Setting Up a React Project with Vite

```
# Step 1: Create the project
npm create vite@latest my-react-app -- --template react

# Step 2: Move into the folder
cd my-react-app

# Step 3: Install all dependencies
```

```
npm install

# Step 4: Start the development server
npm run dev

# Your app is now running at: http://localhost:5173
```

Project Folder Structure — What Each File Does

```
my-react-app/
  public/           --> Static files (images, favicon) — not processed by Vite
  src/
    assets/         --> Images, icons used in components
    App.jsx         --> Root component — the parent of everything
    App.css         --> Styles for App component
    main.jsx        --> Entry point — this is where React starts
    index.css       --> Global styles
  index.html        --> The ONE html file (this is the SPA shell)
  vite.config.js    --> Vite configuration
  package.json      --> Project dependencies and scripts
```

main.jsx — The Entry Point (Very Important)

main.jsx is the starting point of every React app. This file is where React takes control and "mounts" your app onto the HTML page.

```
// main.jsx — Entry Point
import React from "react"
import ReactDOM from "react-dom/client"
import App from "./App.jsx"
import "./index.css"

// Find the <div id="root"> in index.html
// Mount the entire React app inside it
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

```
<!-- index.html — This is the ONE HTML file in our SPA -->
<!DOCTYPE html>
<html>
  <body>
    <!-- React injects your entire app here -->
    <div id="root"></div>
    <!-- Vite injects the main.jsx script automatically -->
  </body>
```

```
</html>
```

> 📝 **Note**
>
> ReactDOM.createRoot() was introduced in React 18. It replaces the older ReactDOM.render() from React 17 and below.
>
> React.StrictMode is a developer tool only. It does NOT affect your production build. It intentionally runs your component functions twice during development to help you find bugs.
>
> If you see your components rendering twice — that is StrictMode. It is normal, not a bug.

Topic 9: React Architecture — The Full Picture

Now that you understand all the individual pieces, let us see how they all fit together when React renders your app.

```
// THE FULL REACT RENDERING PIPELINE

// 1. You write JSX in component files (App.jsx, Header.jsx, etc.)

// 2. Vite + Babel compiles JSX → React.createElement() calls

// 3. React runs those calls and creates a Virtual DOM tree in memory
//    (just JavaScript objects — no browser involved yet)

// 4. ReactDOM.createRoot().render() is called in main.jsx
//    React compares Virtual DOM with what is on screen (initially empty)
//    It builds the real DOM and injects it into <div id="root">

// 5. User interacts — clicks a button, types in a field
//    → State changes

// 6. React re-runs the component function
//    → Creates a NEW Virtual DOM tree

// 7. React DIFFS new Virtual DOM with old Virtual DOM
//    → Finds exactly what changed

// 8. React updates ONLY those changed nodes in the Real DOM
//    → Browser repaints only the changed area

// 9. Repeat from Step 5 on every state change
```

> 💡 **Tip**
>
> react and react-dom are two SEPARATE packages. This is intentional.
>
> "react" contains all the component logic, hooks, Virtual DOM — the core React brain.
>
> "react-dom" is the renderer — it takes the Virtual DOM and paints it on a browser.
>
> Why separate? Because React Native also uses "react" but uses its own renderer for mobile — not react-dom. The same brain, different output device.

Topic 10: Your First Real Component — Line by Line

Let us write a proper React component from scratch and understand every single line.

```
// File: UserCard.jsx
// Purpose: A reusable card that shows a user's name and role

// Props are the inputs to your component (like function arguments)
// We destructure them directly in the function parameters
function UserCard({ name, role, avatar }) {

  // JSX is returned — this describes what the UI looks like
  return (
    <div className="user-card">

      {/* This is a comment in JSX — use {/* */} syntax */}
      <img src={avatar} alt={name} />    // self-closing tag — required in JSX

      <h2>{name}</h2>                    // JavaScript variable inside {}

      <p className="role">{role}</p>     // className, not class

    </div>
  );
}

// This line is REQUIRED — without it you cannot use this component elsewhere
export default UserCard;
```

```
// File: App.jsx
// Purpose: Using the UserCard component multiple times

import UserCard from "./UserCard";   // Import the component first

function App() {
  return (
    <div className="app">
      <h1>Our Team</h1>

      {/* Each usage is an independent INSTANCE of UserCard */}
      {/* Props are passed like HTML attributes */}
      <UserCard
        name="Shaaz Akhtar"
        role="Frontend Developer"
        avatar="https://example.com/shaaz.jpg"
      />

      <UserCard
```

```
        name="Ali Hassan"
        role="Backend Developer"
        avatar="https://example.com/ali.jpg"
      />

      <UserCard
        name="Priya Sharma"
        role="UI Designer"
        avatar="https://example.com/priya.jpg"
      />
    </div>
  );
}

export default App;
```

✅ **Remember**

Every component is just a JavaScript function that returns JSX.

Component names MUST start with a capital letter — UserCard, not userCard.

Every component file should have "export default ComponentName" at the bottom.

Props are read-only — a child component CANNOT modify the props it receives from its parent.

You can use the same component as many times as you want — each usage is an independent instance.

Chapter 1: Interview Questions with Full Answers

These are the most commonly asked React Chapter 1 interview questions. Read each question and answer carefully. The answers are written exactly the way you should say them in an interview.

Q1: What is React? How is it different from Angular?

Answer:

React is a JavaScript LIBRARY created by Meta for building user interfaces. The keyword is library — not framework. React only handles the View layer. You choose your own tools for routing, state management, etc.

Angular is a full FRAMEWORK created by Google. It comes with everything built in — routing, HTTP, forms, validation. It tells you how to structure your entire app.

The key difference: With React, YOU are in control of the app structure. With Angular, the framework is in control.

React uses a Virtual DOM for fast updates. Angular uses change detection with Zone.js. React uses JSX (JS + HTML together). Angular uses TypeScript with separate HTML templates.

Both are popular, but React has more job demand because of its simplicity and flexibility.

Q2: What is the Virtual DOM? How does it work?

Answer:

Virtual DOM is a lightweight JavaScript object that React keeps in memory as a copy of the real DOM.

How it works: When state changes, React creates a new Virtual DOM tree. It then uses a Diffing Algorithm to compare this new tree with the previous one. It finds the exact nodes that changed. Then it updates ONLY those changed nodes in the real DOM.

This process is called Reconciliation. It is very efficient because real DOM operations are expensive — they trigger browser reflow and repaint. By batching and minimising DOM updates, React makes the app very fast.

The Virtual DOM is NOT what makes React fast — it is the smart strategy of minimising real DOM updates that makes React fast.

Q3: What is Reconciliation?

Answer:

Reconciliation is the process React uses to update the real DOM to match the Virtual DOM after state or props change.

When a component re-renders, React creates a new Virtual DOM tree and compares it with the previous one using its Diffing Algorithm. The algorithm checks elements by type and position. If the type is the same — it updates the existing node. If the type changed — it destroys the old node and creates a new one.

Keys play a big role in reconciliation for lists. When you provide a unique key to each list item, React can correctly identify which items were added, removed, or reordered.

Without keys, React would have to re-render the entire list on any change — which is very inefficient.

Q4: What is JSX? Is it required to use React?

Answer:

JSX is JavaScript XML — a syntax extension that lets you write HTML-like code inside JavaScript. Babel compiles JSX to React.createElement() function calls.

No, JSX is NOT required. You can write React using pure React.createElement() calls. But JSX makes code much more readable and maintainable.

Without JSX: React.createElement("div", {className: "box"}, React.createElement("h1", null, "Hello"))

With JSX: <h1>Hello</h1>

In any real project you will always use JSX. No one writes raw createElement calls.

Q5: What is the difference between a Component and an Element?

Answer:

This is a tricky but common question.

A **React Element** is a plain JavaScript object that describes what you want to render. It is what React.createElement() returns. It is immutable — once created, you cannot change it.

Example of a React Element: { type: "h1", props: { children: "Hello" } }

A **React Component** is a function (or class) that returns React Elements. It can receive props as input and can have internal state.

Simple rule: A component is a factory that PRODUCES elements. Elements are the product.

When you write <UserCard />, React calls the UserCard function which returns elements.

Q6: Why do component names need to start with a capital letter?

Answer:

This is how React tells the difference between a built-in HTML element and a custom React component.

 — lowercase means a built-in HTML element. React renders it as an actual DOM node.

<UserCard /> — uppercase means a React component. React calls the UserCard function to get JSX.

If you wrote <userCard />, React would treat it as an unknown HTML element and not call your function.

This is not a convention — it is a requirement. React will not work correctly if a component starts with a lowercase letter.

Q7: What is React.StrictMode? Why does my component render twice?

Answer:

React.StrictMode is a developer tool that helps you find bugs in your React app. It wraps your app in the main.jsx file.

It does NOT affect production builds at all. It is completely removed in production.

In development mode, StrictMode intentionally calls your component function twice. This is to help you find components that have side effects in the render phase — which is not allowed in React.

So if you see your component rendering twice in development — that is StrictMode working correctly. It is NOT a bug.

StrictMode also warns you if you use deprecated React APIs, so you can update your code before they are removed.

Q8: What is a Single Page Application? What are its advantages and disadvantages?

Answer:

A Single Page Application (SPA) is a web app that loads a single HTML file and dynamically updates the content using JavaScript without full page reloads.

Advantages: Very fast after initial load. Smooth user experience — feels like a native app. No full page reloads means no white flash between navigations. Better for user interactions.

Disadvantages: Slower initial load — must download the entire JavaScript bundle. SEO challenges — content is loaded via JS which search engines struggle to index. Needs JavaScript enabled in the browser. Back button and bookmarking need special handling via React Router.

Examples of SPAs: Gmail, Facebook, Figma, Airbnb, Netflix.

The SEO problem is solved by Next.js which does Server Side Rendering (SSR) for React apps.

Q9: What does "export default" mean? What is the difference between default and named exports?

Answer:

In JavaScript, export makes something available to be used in other files. React components must be exported to be used elsewhere.

Default export: export default MyComponent — each file can have only ONE default export. When you import it, you can name it anything: import MyComp from "./MyComponent"

Named export: export function Header() {} or export const PI = 3.14 — a file can have MANY named exports. When you import them, you must use the exact name in curly braces: import { Header } from "./Header"

In React, most component files use default export for the main component. Named exports are used for utility functions or multiple components from one file.

Hooks like useState and useEffect use named exports — that is why you write: import { useState } from "react"

Q10: What are props in React?

Answer:

Props (short for properties) are the way to pass data from a parent component to a child component. They work exactly like function arguments.

When you write <UserCard name="Shaaz" role="Developer" />, you are passing props to the UserCard component.

Inside UserCard, you receive them as: function UserCard({ name, role }) { ... }

The most important rule about props: they are **READ-ONLY**. A child component CANNOT modify the props it receives. This is called one-way data flow — data only flows from parent to child, never the other way.

If a child needs to change something in the parent, it cannot modify props directly. Instead, the parent passes a callback function as a prop, and the child calls that function. This is how React handles upward communication.

Q11: What is the difference between Vite and Create React App (CRA)?

Answer:

Both are tools to create a React project with all the setup done for you.

Create React App (CRA) uses Webpack under the hood. It is the old official way. It is now officially deprecated — the React team no longer recommends it for new projects.

Vite uses esbuild for development and Rollup for production builds. It is the new recommended tool.

The key difference is speed. CRA dev server can take 20-40 seconds to start. Vite starts in under 1 second because it uses native ES Modules in development — it does not bundle everything, it just serves files directly.

Hot Module Replacement (HMR) — when you change a file and save, the browser updates. With Vite this is nearly instant. With CRA it was noticeably slower.

For any new React project, always use Vite.

Q12: What happens step by step when you run npm run dev?

Answer:

Step 1: Vite starts a local development server, typically at http://localhost:5173

Step 2: You open that URL in the browser. Browser requests index.html.

Step 3: index.html loads. It has a script tag pointing to /src/main.jsx.

Step 4: Vite intercepts this request and compiles main.jsx using esbuild (JSX → JavaScript). It serves this compiled JS to the browser.

Step 5: main.jsx runs in the browser. It calls ReactDOM.createRoot(document.getElementById("root")).render(<App />).

Step 6: React calls your App function, which calls child component functions, which create a Virtual DOM tree.

Step 7: ReactDOM takes this Virtual DOM tree and creates actual DOM elements and inserts them into the #root div in index.html.

Step 8: Vite watches your files for changes. When you save a file, Vite updates only that module in the browser (HMR) — no full page reload needed.

Common Mistakes Beginners Make in Chapter 1

These mistakes are very common and interviewers can spot them immediately. Avoid all of these.

Mistake 1: Calling React a Framework

React is a LIBRARY, not a framework. This mistake shows you have not understood the fundamentals. Always say library.

Mistake 2: Using class instead of className

```
// ❌ WRONG — gives a warning and may break styles
<div class="container">

// ✅ CORRECT
<div className="container">
```

Mistake 3: Using lowercase for component names

```
// ❌ WRONG — React treats this as an unknown HTML element
function userCard() { return <div>...</div>; }
<userCard />

// ✅ CORRECT
function UserCard() { return <div>...</div>; }
<UserCard />
```

Mistake 4: Forgetting to export the component

```
// ❌ WRONG — UserCard.jsx with no export
function UserCard() { return <div>Hello</div>; }
// Missing export! Importing this file gives undefined.

// ✅ CORRECT
function UserCard() { return <div>Hello</div>; }
export default UserCard;
```

Mistake 5: Using if/else directly inside JSX return

```
// ❌ WRONG — if/else statements are not allowed in JSX
return (
  <div>
    if (isLoggedIn) { <Dashboard /> }
  </div>
);

// ✅ CORRECT — use ternary operator
return (
  <div>
    {isLoggedIn ? <Dashboard /> : <Login />}
  </div>
);

// ✅ ALSO CORRECT — for show-only (no else needed), use &&
return (
  <div>
    {isLoggedIn && <Dashboard />}
  </div>
);
```

Tips & Tricks — How to Stand Out in Your Interview

💡 Tip

When asked "What is React?" — always mention THREE things in one sentence:

"React is a JavaScript LIBRARY (not framework) by Meta for building user interfaces using a COMPONENT-BASED approach with a VIRTUAL DOM for performance."

Library + Component-based + Virtual DOM. These three keywords will impress any interviewer.

🎯 Interview Tip

When asked about Virtual DOM — do NOT just say "it is faster than real DOM".

Say: "Virtual DOM allows React to batch and minimise real DOM updates. On every state change, React creates a new Virtual DOM tree, runs a Diffing Algorithm to compare it with the previous tree, and only updates the changed DOM nodes. This process is called Reconciliation."

This answer shows you understand the HOW, not just the WHAT.

💡 **Tip**

Reconciliation analogy you can use in an interview:

"React is like a smart editor. Instead of rewriting the entire document every time a word changes, it finds the exact word that changed and updates only that. That is what React does with the DOM — it is surgical, not wholesale."

Analogies show the interviewer you deeply understand the concept, not just memorised it.

✅ **Remember**

SPA question: If asked about Single Page Application — mention the SEO challenge and say Next.js solves it with Server Side Rendering. This shows advanced knowledge.

Fragment question: <> </> is a shorthand for <React.Fragment>. Use it when you do not want to add an extra div to the DOM. The full <React.Fragment key={id}> form is used in lists where you need a key prop.

JSX compilation: Always mention that Babel compiles JSX to React.createElement(). This shows you understand what happens under the hood.

react vs react-dom: These are separate packages on purpose. "react" is the core logic. "react-dom" is the browser renderer. React Native uses the same "react" package with a different renderer — that is the power of this separation.

Quick Reference — Chapter 1 Summary

Concept	What It Is	Key Point
React	JavaScript library for UI	Library NOT framework. Only handles View layer.
Virtual DOM	JS copy of the real DOM in memory	Used to find minimal real DOM changes needed
Reconciliation	Process of updating real DOM	Diff old & new Virtual DOM → update only changed nodes
Component	Function that returns JSX	Reusable, starts with uppercase, must export
JSX	HTML-like syntax in JavaScript	Compiled to React.createElement() by Babel
Props	Data passed from parent to child	READ-ONLY. Cannot be modified by child.

SPA	Single Page Application	One HTML file. React Router handles navigation.
Vite	Modern React project setup tool	Replaces Create React App. Much faster.
Fragment <>	Wrapper with no DOM element	Use instead of div when you need a root element
export default	Makes component usable elsewhere	Required in every component file

✅ **Remember**

Most asked in interviews from Chapter 1: Virtual DOM + Reconciliation explanation, Library vs Framework, SPA vs MPA, JSX rules, Props being read-only.

Topics that trip up beginners: Component names must be uppercase, className not class, no if/else in JSX, all tags must close, only one root element.

Advanced points that impress interviewers: Mentioning Diffing Algorithm and O(n) complexity, knowing that react and react-dom are separate packages, understanding why Next.js solves the SPA SEO problem.

End of Chapter 1 — Good Luck! 🚀

Chapter 2: Props & Prop Drilling

How data flows between components — explained from scratch

By Syed Shaaz Akhtar | Interview Focused Guide

Chapter 1 taught you what a component is. Now the big question is: how do components talk to each other? How does a parent component send data to its child? That is exactly what Props are for. This chapter also covers Prop Drilling — one of the most discussed interview topics in React.

Read this chapter carefully. Props and Prop Drilling are asked in EVERY React interview — from fresher level to senior level.

Topic 1: What are Props?

Props is short for **Properties**. Props are the way a parent component passes data to a child component. They work exactly like arguments you pass to a function.

Think of it this way. You have a function called greet. You can call it with different names each time — greet("Shaaz"), greet("Ali"), greet("Priya"). Each call gets the same function but different data. In React, props work exactly like that.

```
// Without props — this component is rigid, always shows same name
function Greeting() {
  return <h1>Hello, Shaaz!</h1>;  // hardcoded — cannot reuse
}

// With props — this component is flexible, can show any name
function Greeting({ name }) {    // "name" is the prop
  return <h1>Hello, {name}!</h1>;
}

// Now you can reuse it for different data:
<Greeting name="Shaaz" />    // Output: Hello, Shaaz!
<Greeting name="Ali" />      // Output: Hello, Ali!
<Greeting name="Priya" />    // Output: Hello, Priya!
```

> 💡 **Tip**
>
> Props make components REUSABLE. Without props, every component would show the same hardcoded data.
>
> With props, one component can display hundreds of different data — like a ProductCard that shows different products each time it is used.

Topic 2: How to Pass and Receive Props

Step 1 — Passing Props (from Parent)

You pass props to a component exactly like you write HTML attributes. Each prop is a key="value" pair written inside the JSX tag.

```
// App.jsx — Parent Component
function App() {
  return (
    <div>
      {/* Passing props to UserCard — like HTML attributes */}
      <UserCard
        name="Shaaz Akhtar"
        role="Frontend Developer"
        age={23}
        isActive={true}
        skills={["React", "JavaScript", "CSS"]}
      />
    </div>
  );
}
```

> 📝 **Note**
>
> String props use quotes: name="Shaaz"
>
> All other types — numbers, booleans, arrays, objects, functions — use curly braces: age={23}, isActive={true}, skills={["React"]}
>
> If a boolean prop is true, you can write it as just the prop name: isActive={true} is the same as writing isActive

Step 2 — Receiving Props (in Child)

Props arrive at the child component as a single JavaScript object. You can receive them in two ways:

```
// Way 1 — Receive the full props object, then access each property
function UserCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>{props.role}</p>
      <p>Age: {props.age}</p>
    </div>
  );
}

// Way 2 — Destructure props directly in the function parameter (PREFERRED)
function UserCard({ name, role, age, isActive, skills }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{role}</p>
      <p>Age: {age}</p>
      <p>Status: {isActive ? "Active" : "Inactive"}</p>
```

```
    </div>
  );
}
```

> ✅ **Remember**
>
> Way 2 (destructuring) is what you will see in all real projects and is the preferred modern way.
>
> Both ways work identically — destructuring just makes the code cleaner and easier to read.
>
> When you destructure { name, role } from props, you are doing the exact same thing as props.name and props.role.

Topic 3: The #1 Rule — Props are Read-Only

This is the most important rule about props. You will be asked this in every React interview.

Props are **READ-ONLY**. This means a child component can READ the props it receives but it can NEVER MODIFY them. If a child tries to change a prop, React will either throw an error or silently ignore it.

```
// ❌ WRONG — never try to modify props inside a child component
function UserCard({ name }) {
  name = "Changed Name";  // This is WRONG. Do not do this.
  return <h2>{name}</h2>;
}

// ✅ CORRECT — only READ props, never write to them
function UserCard({ name }) {
  return <h2>{name}</h2>;  // Just read and display
}
```

This rule exists for a very important reason. In React, data always flows in ONE direction — from parent to child. This is called **one-way data flow** or **unidirectional data flow**. It makes apps predictable and easy to debug.

Think of it like a TV remote. The remote (parent) sends commands (props) to the TV (child). The TV displays the result — but the TV cannot pick up the remote and change its own buttons. Data goes one way only.

> 🎯 **Interview Tip**
>
> How to answer "Can a child component modify its props?" in an interview:
>
> "No. Props are read-only in React. A child component can only READ the props it receives from its parent — it cannot modify them. This is by design. React enforces one-way data flow, which means data always flows from parent to child, never the other way. This makes the app predictable and easy to debug."
>
> Then add: "If a child needs to update something in the parent, the parent passes a callback function as a prop, and the child calls that function. This pattern keeps data flow clean and traceable."

Topic 4: Types of Data You Can Pass as Props

You can pass any valid JavaScript value as a prop. Here are all the types with examples:

```
// 1. String
<Button label="Click Me" />

// 2. Number
<ProductCard price={499} rating={4} />

// 3. Boolean
<Modal isOpen={true} />
<Button disabled={false} />
<Toggle isActive />    // shorthand for isActive={true}

// 4. Array
<TagList tags={["React", "JavaScript", "CSS"]} />

// 5. Object
<UserProfile user={{ name: "Shaaz", age: 23, city: "Delhi" }} />

// 6. Function — this is how child communicates BACK to parent
<Button onClick={() => console.log("Clicked!")} />
<Form onSubmit={handleFormSubmit} />

// 7. JSX / React Element — this is called "children" prop (Topic 6)
<Card><p>Content inside card</p></Card>

// 8. Variable — passing a variable stored in parent
const userName = "Shaaz";
<Greeting name={userName} />
```

> 💡 **Tip**
>
> Passing a function as a prop is how a child talks back to its parent.
>
> Example: Parent passes onDelete={handleDelete} to a Card component. When user clicks the delete button inside Card, Card calls props.onDelete() — which runs handleDelete in the parent.
>
> This pattern is called "Lifting State Up" and is very commonly asked in interviews.

Topic 5: Default Props — Fallback Values

What happens if a parent forgets to pass a required prop? The component receives **undefined** for that prop — which can break your UI or cause errors.

Default props let you set a fallback value that is used when a prop is not passed. There are two ways to set default props.

Way 1 — Default Value in Destructuring (Modern, Preferred)

```
function UserCard({ name = "Guest", role = "No Role Assigned", age = 0 }) {
```

```
  return (
    <div>
      <h2>{name}</h2>
      <p>{role}</p>
      <p>Age: {age}</p>
    </div>
  );
}

// Using it without passing any props:
<UserCard />
// Output: Guest, No Role Assigned, Age: 0

// Using it with some props:
<UserCard name="Shaaz" />
// Output: Shaaz, No Role Assigned, Age: 0
```

Way 2 — defaultProps (Old Way, Still Valid)

```
function UserCard({ name, role, age }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}

// Set defaults on the function itself
UserCard.defaultProps = {
  name: "Guest",
  role: "No Role Assigned",
  age: 0
};
```

> 📝 **Note**
> defaultProps is being deprecated slowly. The modern approach is to use default values directly in destructuring.
> Use destructuring defaults for all new code. You may see defaultProps in older React codebases.
> Functional components: use destructuring defaults. Class components: use defaultProps.

Topic 6: The children Prop — Special & Very Important

React has one very special built-in prop called **children**. You do not pass it like a normal attribute. Instead, it represents whatever JSX you write BETWEEN the opening and closing tags of a component.

```
// Defining a Card component that uses the children prop
```

```
function Card({ children, title }) {
  return (
    <div className="card">
      <h2>{title}</h2>
      <div className="card-body">
        {children}    {/* Whatever is passed between <Card> tags */}
      </div>
    </div>
  );
}

// Using it — anything between <Card> and </Card> becomes "children"
<Card title="User Info">
  <p>Name: Shaaz Akhtar</p>
  <p>Role: Developer</p>
  <button>Edit Profile</button>
</Card>

// You can put ANYTHING inside — text, elements, even other components
<Card title="Dashboard">
  <UserStats />
  <RecentActivity />
</Card>
```

The children prop is used to build **wrapper** or **layout** components. Instead of creating a new component for every variation, you create a flexible wrapper and put any content inside it.

> 💡 **Tip**
>
> Real-world use cases for children prop:
>
> — Modal component: <Modal><YourContent /></Modal>
>
> — Layout wrapper: <PageLayout><YourPage /></PageLayout>
>
> — Button with icon: <Button><img src="icon.png" /> Save</Button>
>
> This pattern is used everywhere in React. If you see a component with a closing tag (not self-closing), it is probably using children.

Topic 7: Prop Drilling — The Problem

Now we come to one of the most discussed topics in React interviews. You must understand Prop Drilling deeply.

What is Prop Drilling?

Prop Drilling is when you pass props through multiple levels of components just to get data from a top-level component to a deeply nested one — even though the middle components do not need that data at all.

Think of an office building. The CEO (top) needs to send a document to a junior developer (bottom). But instead of emailing directly, the CEO gives it to the VP, the VP gives it to the Manager, the Manager gives it to the Team Lead, and the Team Lead gives it to the junior developer. None of the middle people needed the document — they just passed it along. That is Prop Drilling.

```
// PROP DRILLING EXAMPLE — data has to travel 4 levels down

// Level 1: App — has the user data
function App() {
  const user = { name: "Shaaz", role: "Admin" };
  return <Dashboard user={user} />;  // pass to Dashboard
}

// Level 2: Dashboard — does NOT use user, just passes it down
function Dashboard({ user }) {
  return <Sidebar user={user} />;    // pass to Sidebar
}

// Level 3: Sidebar — does NOT use user either, just passes it down
function Sidebar({ user }) {
  return <UserProfile user={user} />; // pass to UserProfile
}

// Level 4: UserProfile — FINALLY uses the user data
function UserProfile({ user }) {
  return <h2>Welcome, {user.name}! Role: {user.role}</h2>;
}
```

In this example, Dashboard and Sidebar do not use the user prop at all. They just receive it and immediately pass it to the next component. This is Prop Drilling. Now imagine this going 8 or 10 levels deep in a large app.

Why is Prop Drilling a Problem?

- **Hard to maintain** — if the prop name or shape changes, you must update every component in the chain
- **Code becomes messy** — middle components have props they do not use, making code confusing
- **Difficult to refactor** — moving a component to a different level breaks the prop chain
- **Harder to read** — developers have to trace props across many files to understand the data flow

> ✅ **Remember**
>
> Prop Drilling is NOT a bug. It is a pattern. It becomes a PROBLEM when it goes 3 or more levels deep.
>
> For 1-2 levels, passing props directly is perfectly fine and is actually the recommended approach.
>
> Prop Drilling becomes a problem in large apps where data needs to travel many levels deep.
>
> The interview question is: "What is Prop Drilling and how do you solve it?" — know both parts.

Topic 8: Solutions to Prop Drilling

There are multiple ways to solve prop drilling. An interviewer will likely ask you about all of them. Know the tradeoffs.

Solution 1 — Context API (Built into React)

The Context API lets you create a global store of data that any component in the tree can access directly — without passing it through props. Think of it like a radio station. Instead of passing a message through each person in a chain, you broadcast it once and anyone with a radio can tune in.

```
// Step 1: Create a Context
import { createContext, useContext } from "react";

const UserContext = createContext(null);  // null is the default value

// Step 2: Provide the Context at the top level
function App() {
  const user = { name: "Shaaz", role: "Admin" };

  return (
    // Wrap your component tree in the Provider
    // Any component inside this Provider can access "user"
    <UserContext.Provider value={user}>
      <Dashboard />    // No need to pass user prop here!
    </UserContext.Provider>
  );
}

// Step 3: Any nested component can consume the context directly
function UserProfile() {
  const user = useContext(UserContext);  // Access data directly!
  return <h2>Welcome, {user.name}! Role: {user.role}</h2>;
}

// Dashboard and Sidebar do NOT need to pass user prop anymore
function Dashboard() {
  return <Sidebar />;  // Clean — no prop passing needed
}

function Sidebar() {
  return <UserProfile />;  // Clean — no prop passing needed
}
```

> 📝 **Note**
>
> Context API is built into React — no extra library needed.
>
> Best for data that many components need: logged-in user, theme (dark/light), language/locale.
>
> NOT recommended for very frequently changing data (like a counter that updates every second) because every component consuming the context will re-render.
>
> Context API is covered in detail in Chapter 8.

Solution 2 — Redux / Redux Toolkit (External Library)

Redux is an external state management library. It creates a single global store for your entire app. Any component can read from or write to this store without prop drilling.

```
// Redux gives you ONE global store
// Any component can read from it using useSelector
// Any component can update it using useDispatch

import { useSelector } from "react-redux";

function UserProfile() {
  // Read directly from Redux store — no prop drilling needed
  const user = useSelector(state => state.user);
  return <h2>Welcome, {user.name}!</h2>;
}
```

Solution 3 — Component Composition (Avoid Drilling Entirely)

Sometimes the best solution is to restructure your components so that the parent directly renders the deeply nested component — eliminating the middle layers entirely.

```
// Instead of drilling user through Dashboard and Sidebar...
// Let App itself render UserProfile and pass it down as children

function App() {
  const user = { name: "Shaaz", role: "Admin" };
  return (
    <Dashboard>
      <Sidebar>
        <UserProfile user={user} />  {/* Directly placed here */}
      </Sidebar>
    </Dashboard>
  );
}

// Dashboard and Sidebar just render their children — no user prop needed
function Dashboard({ children }) {
  return <div className="dashboard">{children}</div>;
}

function Sidebar({ children }) {
  return <div className="sidebar">{children}</div>;
}
```

Solution	Best For	Tradeoff
Props (direct)	1-2 levels deep — simple parent-child	Gets messy beyond 2-3 levels
Context API	App-wide data: user, theme, language	Causes re-renders when context value changes

Redux / Zustand	Complex state, frequent updates, large apps	Extra library, more setup/boilerplate
Composition	Layout components, avoiding middle props	Requires rethinking component structure

🎯 **Interview Tip**

Perfect answer to "What is Prop Drilling and how do you solve it?":

"Prop Drilling is when you pass props through multiple component levels just to reach a deeply nested component — even though the middle components do not need those props."

"For example, if App has user data and needs to pass it to UserProfile which is 4 levels deep, every component in between must accept and forward the user prop even if they do not use it."

"Solutions: (1) Context API — built into React, creates a global context that any component can consume with useContext. Best for semi-static data like user or theme. (2) Redux or Zustand — external state management for complex apps. (3) Component Composition — restructure components so the parent renders the nested component directly as children."

Topic 9: How React Re-renders When Props Change

This is a very important concept that ties directly into React performance. You must understand when and why React re-renders a component.

A component re-renders in React when:

1. Its **state** changes (covered in Chapter 3)
2. Its **props** change — specifically, when the parent re-renders and passes new prop values
3. Its **parent** re-renders (even if the props passed down did not change — this is the default behaviour)

```
// Parent component — manages count state
function Parent() {
  const [count, setCount] = React.useState(0);

  return (
    <div>
      <button onClick={() => setCount(count + 1)}>Click</button>
      <Child name="Shaaz" count={count} />
    </div>
  );
}

// Child re-renders every time Parent re-renders
// because Parent is re-created and Child gets new props reference
function Child({ name, count }) {
  console.log("Child rendered");  // prints on every Parent render
  return <p>{name}: {count}</p>;
}
```

📝 **Note**

By default, when a parent re-renders, ALL its children re-render too — even if their props did not actually change.

This is React's default behaviour and is usually fine for small apps.

For large apps with heavy components, you can use React.memo() to prevent unnecessary re-renders.

React.memo() is covered in the Performance chapter.

React.memo — Preventing Unnecessary Re-renders

React.memo is a higher-order component (HOC) that wraps a component and prevents it from re-rendering if its props have not changed.

```
// Without React.memo — Child re-renders every time Parent does
function Child({ name }) {
  console.log("Child rendered");
  return <p>Hello, {name}!</p>;
}

// With React.memo — Child ONLY re-renders if "name" prop changes
const Child = React.memo(function Child({ name }) {
  console.log("Child rendered");
  return <p>Hello, {name}!</p>;
});

// Now if Parent re-renders but name prop stays the same,
// Child will NOT re-render — it is memoized.
```

💡 **Tip**

Do NOT wrap every component with React.memo. Use it only when:

— A component renders often but its props rarely change

— The component is expensive to render (heavy calculations, large lists)

Using React.memo everywhere adds overhead and can actually make your app slower.

Topic 10: Spreading Props — The ...spread Pattern

Sometimes you need to pass many props to a child component. You can spread an object as props using the spread operator. This is a very common pattern in real projects.

```
// Without spreading — verbose, repetitive
function App() {
  const userProps = {
    name: "Shaaz",
    role: "Developer",
    age: 23,
    city: "Delhi"
```

```
  };

  return (
    <UserCard
      name={userProps.name}
      role={userProps.role}
      age={userProps.age}
      city={userProps.city}
    />
  );
}

// With spreading — clean and short
function App() {
  const userProps = { name: "Shaaz", role: "Developer", age: 23, city:
"Delhi" };

  return <UserCard {...userProps} />;  // spreads all properties as props
}

// Can also mix spread with additional props:
<UserCard {...userProps} isAdmin={true} />
// Same as: name="Shaaz" role="Developer" age={23} city="Delhi" isAdmin={true}
```

✅ **Remember**

Spreading props is useful when you have many props or when you are building wrapper components.

Be careful with spreading — if the object has many properties, the child gets all of them even if it does not need them. This can cause issues with unknown HTML attributes on DOM elements.

Example of the danger: if you spread props on a native HTML element like <button {...props}>, unexpected props might show up as HTML attributes and cause console warnings.

Topic 11: Real World Example — Building a ProductCard

Let us put everything together and build a realistic ProductCard component that a company like Amazon or Flipkart might have. This example covers all the props concepts together.

```
// ProductCard.jsx — A realistic reusable component
function ProductCard({
  name,
  price,
  rating = 0,             // default prop — 0 stars if not passed
  inStock = true,         // default prop — assume in stock
  image,
  onAddToCart,            // function prop — callback to parent
  onWishlist,             // function prop — another callback
  badge = null,           // default prop — no badge by default
}) {
  return (
```

```
    <div className="product-card">

      {/* Conditional rendering using && */}
      {badge && <span className="badge">{badge}</span>}

      <img src={image} alt={name} />

      <h3>{name}</h3>

      <p className="price">₹{price.toLocaleString()}</p>

      <div className="rating">
        {"⭐".repeat(Math.round(rating))} ({rating})
      </div>

      {/* Conditional rendering using ternary */}
      {inStock
        ? <button onClick={onAddToCart}>Add to Cart</button>
        : <button disabled>Out of Stock</button>
      }

      <button onClick={onWishlist}>♡ Wishlist</button>
    </div>
  );
}

export default ProductCard;
```

```
// App.jsx — Using ProductCard with different data
import ProductCard from "./ProductCard";

function App() {
  const products = [
    {
      id: 1,
      name: "Wireless Headphones",
      price: 2499,
      rating: 4.5,
      inStock: true,
      image: "headphones.jpg",
      badge: "BESTSELLER"
    },
    {
      id: 2,
      name: "Bluetooth Speaker",
      price: 1299,
      rating: 3.8,
      inStock: false,
      image: "speaker.jpg",
      badge: null
```

```
    }
  ];

  function handleAddToCart(productId) {
    console.log("Added product", productId, "to cart");
  }

  function handleWishlist(productId) {
    console.log("Wishlisted product", productId);
  }

  return (
    <div className="product-list">
      {products.map(product => (
        <ProductCard
          key={product.id}
          {...product}    // spread all product properties as props
          onAddToCart={() => handleAddToCart(product.id)}
          onWishlist={() => handleWishlist(product.id)}
        />
      ))}
    </div>
  );
}
```

💡 **Tip**

Notice how onAddToCart and onWishlist are functions passed as props. The child component does not know what happens when the button is clicked — it just calls the function.

The parent controls the logic. The child controls only the display. This is the correct separation of concerns in React.

This is also called the "Lifting State Up" pattern — keeping logic in the parent and passing callbacks to children.

Topic 12: Props vs State — A Crucial Difference

Beginners often confuse Props and State. This question is asked in almost every React interview. You must know the difference clearly.

Aspect	Props	State
What is it?	Data passed FROM parent TO child	Data managed INSIDE the component itself
Who controls it?	The PARENT component	The component ITSELF
Can it change?	Child CANNOT change it (read-only)	Yes, using setState or useState

When to use?	When parent needs to configure the child	When data needs to change over time inside the component
Analogy	TV remote sends commands to TV	TV's internal volume setting
Example	name="Shaaz" passed to UserCard	count = 0, increases on button click

✅ **Remember**

Props = from parent, read-only, cannot change inside child.

State = inside component, can change, triggers re-render when changed.

When a component needs data that changes over time internally — use State.

When a component needs to display data configured by its parent — use Props.

State is covered in full detail in Chapter 3.

Chapter 2: Interview Questions with Full Answers

These are the most asked interview questions on Props and Prop Drilling. Every answer is written exactly the way a senior developer would answer in an interview.

Q1: What are props in React?

Answer:

Props (short for properties) are the mechanism by which data is passed from a parent component to a child component in React.

They work like function arguments — when you use a component, you pass props to it like HTML attributes: <UserCard name="Shaaz" age={23} />. Inside the component, you access them as function parameters.

The most important rule about props: **they are read-only**. A child component can only read the props it receives — it cannot modify them. This enforces React's one-way data flow.

Props can be any JavaScript value: strings, numbers, booleans, arrays, objects, functions, or even JSX elements.

Q2: Can a child component modify its props?

Answer:

No. Props are strictly read-only in React.

If a child component tries to modify a prop, it will not work and React may throw an error. This is by design.

React enforces one-way data flow — data always flows from parent to child, never the other way. This makes the application predictable. If any component could modify props freely, it would be very difficult to trace where data changes are happening.

If a child needs to update data in the parent, the parent passes a callback function as a prop. The child calls that function when needed — like onDelete, onSubmit, onChange.

This pattern keeps data flow clean and traceable.

Q3: What is the difference between props and state?

Answer:

Props are data passed from a parent component to a child. The child cannot modify them. They are controlled by the parent.

State is data that is managed inside a component itself. The component can change its own state using useState. When state changes, the component re-renders.

Analogy: Think of a form. The placeholder text "Enter your name" is a prop — the parent configured it. But the actual text the user types is state — it changes inside the component as the user types.

Rule of thumb: If data comes from outside the component — it is a prop. If data is created and managed inside the component — it is state.

Q4: What is Prop Drilling? When does it become a problem?

Answer:

Prop Drilling is when you pass props through multiple component levels just to get data from a top-level component to a deeply nested one — even though the middle components do not use that data at all.

Example: App has user data. UserProfile (4 levels deep) needs it. So user prop is passed through Dashboard, Sidebar, and Wrapper — none of which need it.

It becomes a problem when: (1) It goes 3 or more levels deep. (2) You need to change the prop structure — then you must update every file in the chain. (3) Middle components become polluted with props they do not use.

For 1-2 levels, prop drilling is fine and actually recommended. The problem starts deeper.

Q5: How do you solve Prop Drilling?

Answer:

Option 1 — Context API: Built into React. Create a context with createContext, provide it at the top level with Provider, and consume it anywhere with useContext. Best for semi-static app-wide data like logged-in user, theme, language.

Option 2 — Redux / Zustand: External state management libraries. Redux creates a single global store. Any component can read from it with useSelector and update it with useDispatch. Best for complex apps with frequently changing shared state.

Option 3 — Component Composition: Restructure your components so that the parent directly renders the deeply nested component as children — eliminating the middle layers. This avoids the problem entirely without any extra tools.

My preferred approach: Use props for 1-2 levels, Context API for app-wide data, and Redux/Zustand only when the app is complex enough to justify it.

Q6: What is the children prop in React?

Answer:

The children prop is a special built-in prop in React. You do not pass it as an attribute. Instead, it automatically represents whatever JSX you place between the opening and closing tags of a component.

Example: <Card> Hello </Card> — inside the Card component, props.children equals the Hello element.

This is used to build wrapper or container components — like modals, cards, layout wrappers, and panels — where the component provides structure but the content inside can vary.

Any valid JSX can be passed as children — text, elements, arrays of elements, or other components.

Q7: What are default props? How do you set them?

Answer:

Default props are fallback values used when a parent does not pass a required prop. This prevents the component from receiving undefined for missing props.

The modern way is to use default values directly in destructuring:

function Button({ label = "Click Me", disabled = false }) { ... }

The older way is using defaultProps: Button.defaultProps = { label: "Click Me" };

Always prefer the destructuring approach for new code. defaultProps is being deprecated and removed from future React versions for function components.

Q8: When does a component re-render in React?

Answer:

A component re-renders in three situations:

1. State changes — when useState's setter function is called with a new value inside the component.

2. Props change — when the parent re-renders and passes new prop values to the child.

3. Parent re-renders — by default, when a parent component re-renders, ALL its children re-render too — even if their props did not change.

The third point is important for performance. To prevent unnecessary re-renders of children when props did not change, wrap the child with React.memo().

Note: Context value changes also cause all consuming components to re-render.

Q9: What is React.memo? When should you use it?

Answer:

React.memo is a higher-order component that wraps a functional component and memoizes (caches) its rendered output.

When the parent re-renders, React.memo checks if the child's props have changed. If they have NOT changed, React skips re-rendering the child and reuses the previous output.

Use React.memo when: A component re-renders frequently but its props rarely change. The component is expensive to render (heavy computations, large DOM).

Do NOT use it everywhere. Wrapping every component adds memory overhead and React.memo's shallow comparison itself takes time. Use it intentionally for genuinely expensive components.

React.memo does a shallow comparison of props. For objects and arrays, if the reference changes on every parent render (which it usually does), React.memo will not help without also using useMemo or useCallback.

Q10: What is one-way data flow in React?

Answer:

One-way data flow (also called unidirectional data flow) means data in React only flows in ONE direction — from parent component to child component, through props.

A child cannot send data back up to its parent by modifying props. If a child needs to communicate upward, the parent passes a callback function as a prop and the child calls that function.

Why is this important? It makes the app predictable and easy to debug. When data only flows one way, you always know where a piece of data comes from. If something is wrong, you trace it up the component tree.

In contrast, two-way binding (like in older Angular) allows data to flow in both directions, which can make large apps harder to debug.

Q11: What is the difference between controlled and uncontrolled components? (Related to Props)

Answer:

A **controlled component** is a form element (input, select, textarea) whose value is controlled by React state. The value is set via a prop (value={stateVariable}) and updated via onChange. React is the single source of truth.

An **uncontrolled component** lets the DOM handle its own state. You access the value using a ref when needed, not on every change.

Example of controlled: <input value={name} onChange={e => setName(e.target.value)} />

Example of uncontrolled: <input ref={inputRef} /> — read with inputRef.current.value when needed.

For most cases, use controlled components. They give React full control, make form validation easy, and are easier to test.

Q12: How do you pass a function as a prop? Why would you do this?

Answer:

You pass a function as a prop the same way as any other value — using curly braces: <Button onClick={handleClick} />

Inside the Button component: function Button({ onClick }) { return <button onClick={onClick}>Click</button>; }

You pass functions as props to allow child components to trigger logic that lives in the parent. This is the correct way for a child to communicate upward.

> Real example: A ProductCard has a "Delete" button. The deletion logic lives in the parent (App) because App manages the list. So App passes onDelete={handleDelete} to ProductCard. When the button is clicked, ProductCard calls onDelete() — which runs handleDelete in App.
>
> This pattern keeps the parent in control of business logic while the child only handles UI rendering.

Common Mistakes Beginners Make with Props

Mistake 1: Trying to modify props

```
// ❌ WRONG — modifying props directly
function UserCard({ name }) {
  name = name.toUpperCase();  // DO NOT modify the prop variable
  return <h2>{name}</h2>;
}

// ✅ CORRECT — create a new variable, leave props untouched
function UserCard({ name }) {
  const displayName = name.toUpperCase();  // new variable, prop unchanged
  return <h2>{displayName}</h2>;
}
```

Mistake 2: Forgetting curly braces for non-string props

```
// ❌ WRONG — price becomes the string "499", not the number 499
<ProductCard price="499" inStock="true" />

// ✅ CORRECT — use curly braces for numbers and booleans
<ProductCard price={499} inStock={true} />

// This matters! "499" + 100 = "499100", but 499 + 100 = 599
```

Mistake 3: Deeply nesting props unnecessarily

```
// ❌ AVOID — prop drilling 4+ levels just to share simple data
<App>
  <Page user={user}>
    <Content user={user}>
      <Sidebar user={user}>
        <Profile user={user} />  // finally used here
      </Sidebar>
    </Content>
  </Page>
</App>

// ✅ BETTER — use Context API for data needed deeply across the tree
```

```
<UserContext.Provider value={user}>
  <App>
    <Page>
      <Content>
        <Sidebar>
          <Profile />  // accesses user via useContext — no prop drilling
        </Sidebar>
      </Content>
    </Page>
  </App>
</UserContext.Provider>
```

Mistake 4: Passing the wrong data type

```
// ❌ WRONG — passing array as string causes bugs
<TagList tags="React,JavaScript,CSS" />  // This is a string, not array

// ✅ CORRECT — pass an actual array
<TagList tags={["React", "JavaScript", "CSS"]} />
```

Tips & Tricks — Stand Out in Your Interview

💡 Tip

Mention the analogy for one-way data flow:

"React's data flow is like a waterfall — water only flows downward (parent to child via props). If a child needs to send something up, it uses a rope (callback function) that the parent threw down."

Analogies like this show the interviewer you truly understand the concept.

🎯 Interview Tip

When asked about Prop Drilling, always structure your answer in 3 parts:

1. Define it clearly with an example
2. Explain WHY it is a problem (maintenance, readability)
3. Give 3 solutions with tradeoffs (Props, Context, Redux)

This 3-part answer shows you understand the full picture — not just the definition.

✅ Remember

Props — pass down data from parent to child, read-only, any JS value.

children prop — special prop for JSX between component tags, used for wrappers.

Default props — set in destructuring with = operator, fallback if prop not passed.

Prop Drilling — passing props many levels deep through middle components.

Solutions — Context API (built-in), Redux/Zustand (external), Composition (restructure).
React.memo — skips re-render if props have not changed, use sparingly.
Callback props — how child talks back to parent, keeps data flowing one way.
Props vs State — props from outside (read-only), state from inside (changeable).

Quick Reference — Chapter 2 Summary

Concept	What It Is	Key Rule
Props	Data from parent to child	READ-ONLY — child cannot modify
children prop	JSX between component tags	<Card>content</Card> → props.children
Default props	Fallback if prop not passed	Set in destructuring: { name = "Guest" }
Prop Drilling	Props passed many levels deep	Problem when 3+ levels, use Context to solve
Context API	Global data without prop drilling	createContext → Provider → useContext
React.memo	Skip re-render if props unchanged	Use only for expensive components
Callback props	Function passed as prop	Child calls it to communicate UP to parent
Spread props	Pass object as props: {...obj}	Shorthand, watch for unintended props
One-way flow	Data flows parent → child only	Makes app predictable and debuggable

End of Chapter 2 • Next: Chapter 3 — State & useState Hook

Chapter 3: Lifting State Up

How sibling components share data — the most important React pattern

By Syed Shaaz Akhtar | Interview Focused Guide

In Chapter 2, you learned that props flow DOWN from parent to child. But what happens when two sibling components need to share the same data? Neither can pass props to the other directly. The solution is called "Lifting State Up" — and it is one of the most fundamental React patterns. This chapter explains it from scratch, with real examples.

This topic is asked in almost every React interview at every level. Read it carefully.

Topic 1: What is State?

Before we talk about Lifting State Up, you need a solid understanding of what State is. We will cover useState in full detail in Chapter 4 — but here is what you need to know now.

State is data that lives inside a component and can change over time. When state changes, React automatically re-renders that component and shows the updated UI.

Think of state like a whiteboard inside a room. Only the people in that room can see and change what is written on it. Other rooms cannot see it unless you bring the whiteboard to a common area.

```
// A simple counter component with local state
import { useState } from "react";

function Counter() {
  // useState returns [currentValue, setterFunction]
  // 0 is the initial value
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
      <button onClick={() => setCount(count - 1)}>Decrease</button>
    </div>
  );
}
```

> 📝 **Note**
> State is LOCAL by default. The count variable above only exists inside the Counter component. No other component can read or change this count directly.
> This is the key problem Lifting State Up solves — when TWO components need the same state.

Topic 2: The Problem — When Two Components Need the Same Data

Imagine you are building a temperature converter. You have two input boxes side by side — one for Celsius and one for Fahrenheit. When the user types in the Celsius box, the Fahrenheit box should update automatically, and vice versa.

These two boxes are SIBLING components — they are at the same level in the component tree. Neither is the parent of the other. So they cannot directly share data.

```
//  The PROBLEM: Two siblings need the same temperature value

//  Component Tree:
//  App
//    CelsiusInput    ← needs temperature value
//    FahrenheitInput ← also needs the SAME temperature value

// ❌ WRONG APPROACH — each has its own separate state
function CelsiusInput() {
  const [celsius, setCelsius] = useState(0);
  return <input value={celsius} onChange={e => setCelsius(e.target.value)} />;
}

function FahrenheitInput() {
  const [fahrenheit, setFahrenheit] = useState(32);
  return <input value={fahrenheit} />;
}

// Problem: CelsiusInput has NO idea what FahrenheitInput is doing.
// They are completely isolated. Changing one does NOT update the other.
```

This is the problem. Each component has its own isolated state. They cannot see each other's data. No matter how much you type in Celsius, Fahrenheit never knows about it.

> 💡 **Tip**
>
> Rule to remember: In React, data flows DOWN (parent to child via props). It never flows SIDEWAYS between siblings.
>
> The only way two siblings can share data is if that data lives in their common parent, and the parent passes it down to both.

Topic 3: The Solution — Lifting State Up

The solution is simple but powerful: **move the shared state UP to the closest common parent** of the two components that need it. Then the parent passes the state down to both children as props.

This is called "Lifting State Up" because you are taking state that was in a child component and moving it UP to the parent.

```
// ✅ CORRECT APPROACH — Lift state UP to the common parent (App)
import { useState } from "react";
```

```
// App is the common parent of both inputs
// It owns the temperature state and passes it down to both children
function App() {
  const [celsius, setCelsius] = useState(0);

  // Derived value — calculated from celsius, NOT a separate state
  const fahrenheit = (celsius * 9/5) + 32;

  // When Celsius changes, update the shared state
  function handleCelsiusChange(value) {
    setCelsius(Number(value));
  }

  // When Fahrenheit changes, convert back and update state
  function handleFahrenheitChange(value) {
    setCelsius((Number(value) - 32) * 5/9);
  }

  return (
    <div>
      <CelsiusInput
        value={celsius}
        onChange={handleCelsiusChange}
      />
      <FahrenheitInput
        value={fahrenheit}
        onChange={handleFahrenheitChange}
      />
    </div>
  );
}

// CelsiusInput is now a "controlled" component — no own state
// It receives value and onChange from parent via props
function CelsiusInput({ value, onChange }) {
  return (
    <div>
      <label>Celsius: </label>
      <input
        value={value}
        onChange={e => onChange(e.target.value)}
      />
    </div>
  );
}

// FahrenheitInput is also a "controlled" component — no own state
function FahrenheitInput({ value, onChange }) {
  return (
    <div>
      <label>Fahrenheit: </label>
```

```
      <input
        value={value.toFixed(2)}
        onChange={e => onChange(e.target.value)}
      />
    </div>
  );
}
```

✅ Remember

After lifting state up: the parent (App) owns ONE state variable (celsius).

Fahrenheit is NOT a separate state — it is a DERIVED VALUE calculated from celsius.

Both child components are now "controlled" — they have no state of their own. They receive value via props and report changes via onChange callback.

This is the Lifting State Up pattern. The parent is the single source of truth.

Topic 4: Single Source of Truth

"Single Source of Truth" is a principle that goes hand-in-hand with Lifting State Up. It means there should be exactly ONE place in your app where a particular piece of data lives. All other components that need that data get it from that one place.

In the temperature example: the celsius state in App is the single source of truth. Both CelsiusInput and FahrenheitInput get their values from this one place. There is no duplication of data — just one number, shared everywhere.

```
// ❌ WRONG — data duplicated in two places (not a single source of truth)
function App() {
  return (
    <div>
      <Cart />       // Cart has its own count state
      <NavBar />     // NavBar ALSO has its own count state
    </div>
  );
}
// Problem: If Cart updates its count, NavBar never knows.
// The two states get out of sync. Bugs everywhere.

// ✅ CORRECT — one source of truth in the parent
function App() {
  const [cartCount, setCartCount] = useState(0);

  return (
    <div>
      <NavBar cartCount={cartCount} />
      <Cart
        cartCount={cartCount}
        onAddItem={() => setCartCount(cartCount + 1)}
        onRemoveItem={() => setCartCount(cartCount - 1)}
      />
```

```
    </div>
  );
}
// Now cartCount lives in ONE place — App.
// Both NavBar and Cart always show the same number. Always in sync.
```

🎯 **Interview Tip**

How to answer "What is Single Source of Truth?" in an interview:

"Single Source of Truth means having exactly one place in the application where a particular piece of data lives. In React, this is achieved by Lifting State Up — moving shared state to the closest common parent. All components that need that data receive it from this one source via props. This ensures all components always display consistent, in-sync data. There is no duplication, no risk of different components showing different values for the same data."

Topic 5: The Lifting State Up Pattern — Step by Step

Here is a clear, repeatable process for applying the Lifting State Up pattern whenever you face the "two siblings need the same data" problem.

1. **Identify** which components need the shared data
2. **Find their closest common ancestor** in the component tree
3. **Move the state** from the children to that common ancestor
4. **Pass the state DOWN** to each child as a prop
5. **Pass a callback function DOWN** so children can update the state
6. **Remove the local state** from the children — they become controlled

Let us apply this to a real e-commerce example — a cart that shows a count in the navbar:

```
// STEP 1: Identify — NavBar needs cartCount, Cart needs cartCount
// STEP 2: Common ancestor — App is the parent of both
// STEP 3 & 4: Move state to App, pass it down

import { useState } from "react";

function App() {
  // STEP 3: State lives here now — single source of truth
  const [cartItems, setCartItems] = useState([]);

  // STEP 5: Callback functions to update state
  function addToCart(product) {
    setCartItems([...cartItems, product]);
  }

  function removeFromCart(productId) {
    setCartItems(cartItems.filter(item => item.id !== productId));
  }
```

```
  return (
    <div>
      {/* STEP 4: Pass state DOWN as props */}
      <NavBar cartCount={cartItems.length} />

      <ProductList onAddToCart={addToCart} />

      <Cart
        items={cartItems}
        onRemove={removeFromCart}
      />
    </div>
  );
}

// STEP 6: NavBar — controlled, no own state
function NavBar({ cartCount }) {
  return (
    <nav>
      <span>My Shop</span>
      <span>🛒 Cart ({cartCount})</span>   {/* always in sync! */}
    </nav>
  );
}

// STEP 6: ProductList — controlled, calls parent's callback
function ProductList({ onAddToCart }) {
  const products = [
    { id: 1, name: "Headphones", price: 2499 },
    { id: 2, name: "Keyboard",   price: 1299 },
  ];
  return (
    <div>
      {products.map(p => (
        <div key={p.id}>
          <span>{p.name} — ₹{p.price}</span>
          <button onClick={() => onAddToCart(p)}>Add to Cart</button>
        </div>
      ))}
    </div>
  );
}

// STEP 6: Cart — controlled, shows items from parent
function Cart({ items, onRemove }) {
  if (items.length === 0) return <p>Your cart is empty.</p>;
  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>
          {item.name}
```

```
          <button onClick={() => onRemove(item.id)}>Remove</button>
        </li>
      ))}
    </ul>
  );
}
```

> 💡 **Tip**
>
> Notice that when a user adds a product, ProductList calls onAddToCart (which is App's addToCart function).
>
> App updates cartItems state. React re-renders App. NavBar gets the new cartCount. Cart gets the new items.
>
> Everything stays in sync because there is ONE source of truth — cartItems in App.
>
> This is the power of Lifting State Up.

Topic 6: Controlled vs Uncontrolled Components

When you lift state up, the child components that previously had their own state become "controlled" components. This is a key concept that comes up in interviews.

Aspect	Uncontrolled Component	Controlled Component
State location	State lives INSIDE the component	State lives in the PARENT, passed via props
Who controls data?	The component manages itself	The parent manages the data
How to update?	Component calls its own setState	Component calls a callback prop from parent
Source of truth	Local — only the component knows	Parent — single source of truth
When to use?	Simple, isolated UI elements	Shared data, forms, synced components
Example	A counter only one component uses	Any input whose value other parts need

```
// UNCONTROLLED — manages its own state
function SearchBox() {
  const [query, setQuery] = useState("");
  return (
    <input
      value={query}
      onChange={e => setQuery(e.target.value)}
      placeholder="Search..."
    />
  );
```

```
  // Only this component knows what is in the search box.
  // Parent has no idea. Fine if search is purely local.
}

// CONTROLLED — parent manages state, passes it down
function SearchBox({ query, onQueryChange }) {
  return (
    <input
      value={query}
      onChange={e => onQueryChange(e.target.value)}
      placeholder="Search..."
    />
  );
  // Parent knows the query. Can use it to filter a list elsewhere.
  // This is the controlled pattern — used after lifting state up.
}
```

✅ **Remember**

After lifting state up, the child becomes a CONTROLLED component.

A controlled component has no state of its own. It only reads from props and calls callbacks.

Controlled = the PARENT is in control. Uncontrolled = the component controls itself.

In React forms, controlled inputs (where value={state} and onChange={setter}) are the recommended approach because React becomes the single source of truth for the input value.

Topic 7: Real World Example — Search Filter with Results

This is one of the most common real-world examples of Lifting State Up. You have a search input and a results list. They need to share the search query — lifting state up to the parent solves this cleanly.

```
// Real example: Search + Results need to share the query
import { useState } from "react";

const ALL_PRODUCTS = [
  { id: 1, name: "React Book",         category: "Education" },
  { id: 2, name: "JavaScript Course",  category: "Education" },
  { id: 3, name: "Wireless Keyboard",  category: "Tech" },
  { id: 4, name: "Mechanical Mouse",   category: "Tech" },
  { id: 5, name: "Desk Lamp",          category: "Home" },
];

// App is the common parent — holds the shared search query
function App() {
  const [query, setQuery]         = useState("");
  const [category, setCategory]   = useState("All");

  // Derived data — filtered based on shared state
  const filtered = ALL_PRODUCTS.filter(p => {
    const matchesQuery    = p.name.toLowerCase().includes(query.toLowerCase());
```

```
    const matchesCategory = category === "All" || p.category === category;
    return matchesQuery && matchesCategory;
  });

  return (
    <div>
      {/* SearchBar is controlled — query lives in App, not here */}
      <SearchBar
        query={query}
        category={category}
        onQueryChange={setQuery}
        onCategoryChange={setCategory}
      />

      {/* ResultsCount gets derived data from App */}
      <ResultsCount count={filtered.length} />

      {/* ProductList receives filtered results from App */}
      <ProductList products={filtered} />
    </div>
  );
}

// SearchBar — controlled, all state comes from parent
function SearchBar({ query, category, onQueryChange, onCategoryChange }) {
  return (
    <div>
      <input
        value={query}
        onChange={e => onQueryChange(e.target.value)}
        placeholder="Search products..."
      />
      <select value={category} onChange={e => onCategoryChange(e.target.value)}>
        <option>All</option>
        <option>Education</option>
        <option>Tech</option>
        <option>Home</option>
      </select>
    </div>
  );
}

// ResultsCount — simple display component
function ResultsCount({ count }) {
  return <p>Showing {count} result{count !== 1 ? "s" : ""}</p>;
}

// ProductList — renders filtered products from parent
function ProductList({ products }) {
  if (products.length === 0) return <p>No products found.</p>;
  return (
```

```
    <ul>
      {products.map(p => (
        <li key={p.id}>{p.name} — {p.category}</li>
      ))}
    </ul>
  );
}
```

> 📝 **Note**
>
> Notice that "filtered" is NOT a separate state. It is a derived value calculated directly from query and category.
>
> This is important — never create a second state variable that is derived from another state variable. Always calculate derived values on the fly.
>
> Bad: const [filtered, setFiltered] = useState([]) with a useEffect to keep it in sync.
>
> Good: const filtered = products.filter(...) calculated directly during render.

Topic 8: When to Lift State Up vs When NOT To

Lifting state up is powerful, but you should not lift every piece of state. Here is a clear guide on when to lift and when to keep state local.

Keep State Local When:

- The data is only used by ONE component and no sibling needs it
- It is UI state — like whether a dropdown is open, whether a tooltip is visible
- It is temporary form data that will be submitted and not shared
- No parent or sibling component will ever need to know about it

```
// ✅ Keep local — only this dropdown cares about isOpen
function Dropdown({ options }) {
  const [isOpen, setIsOpen] = useState(false);
  return (
    <div>
      <button onClick={() => setIsOpen(!isOpen)}>Menu</button>
      {isOpen && <ul>{options.map(o => <li key={o}>{o}</li>)}</ul>}
    </div>
  );
}
// No other component needs to know if this dropdown is open.
// Keep it local.
```

Lift State Up When:

- **Two or more siblings** need to read the same data
- **A parent needs to know** what is happening inside a child
- **Changes in one component** must be reflected in another component

- **The data drives multiple parts** of the UI at the same time

```
// ✅ Lift up — both FilterPanel and ProductGrid need the same filter
function App() {
  const [filter, setFilter] = useState("all");  // lifted here
  return (
    <div>
      <FilterPanel filter={filter} onFilterChange={setFilter} />
      <ProductGrid filter={filter} />
    </div>
  );
}
```

Situation	Decision	Reason
Modal open/close — only one component	Keep LOCAL	No sibling needs to know
Form input in one form — submitted together	Keep LOCAL	Form manages its own fields
Cart count shown in NavBar AND Cart	LIFT UP to App	Two siblings share it
Search query used by SearchBar AND ResultsList	LIFT UP to App	Two siblings share it
Active tab in a Tabs component	Keep in Tabs (LOCAL)	Only Tabs and its children need it
Logged-in user info — needed everywhere	Use Context API	Too many levels — lifting is not enough

Topic 9: The Downside — Re-renders When Lifting State

When you lift state up, there is an important performance consideration you should know about — and be able to explain in an interview.

When state in the parent changes, the parent re-renders. And when the parent re-renders, ALL its children re-render by default — even if their props did not change.

```
// Example: App lifts cartCount state
function App() {
  const [cartCount, setCartCount] = useState(0);

  return (
    <div>
      <NavBar cartCount={cartCount} />
      <HeroSection />    // This re-renders too! Even though it has no
      <ProductList onAdd={() => setCartCount(c => c + 1)} />  // props from App
      <Footer />         // This also re-renders unnecessarily
```

```
    </div>
  );
}

// Every time cartCount changes, ALL four children re-render.
// HeroSection and Footer do not even use cartCount — wasted renders.
```

For small apps this is not a problem. React's re-renders are fast. But for large apps with heavy components, this can hurt performance.

Solution: React.memo to prevent unnecessary re-renders

```
// Wrap components that do NOT depend on the changed state with React.memo

const HeroSection = React.memo(function HeroSection() {
  console.log("HeroSection rendered");
  return <div className="hero">Welcome to My Shop!</div>;
});

const Footer = React.memo(function Footer() {
  console.log("Footer rendered");
  return <footer>© 2025 My Shop</footer>;
});

// Now when cartCount changes:
// NavBar re-renders  ✓ (it receives cartCount prop)
// ProductList re-renders  ✓ (it receives onAdd — but this is a new function
ref!)
// HeroSection does NOT re-render  ✓ (memoized, no props changed)
// Footer does NOT re-render  ✓ (memoized, no props)
```

> 💡 **Tip**
>
> There is a subtle issue with React.memo and function props.
>
> In the example above, onAdd={() => setCartCount(c => c + 1)} creates a NEW function on every render of App.
>
> ProductList receives a new function reference each time, so React.memo thinks props changed and re-renders anyway.
>
> The fix is useCallback — it memoizes the function so the reference stays stable. Covered in Chapter 6 (Hooks).

Topic 10: Lifting State Up vs Context API — When to Use Which

As your app grows, you might find yourself lifting state very high — sometimes all the way to the root App component. At that point, prop drilling becomes a problem again. This is where Context API comes in.

Aspect	Lifting State Up	Context API
What it does	Moves state to the closest common	Creates a global store any

	parent	component can access
Data travel	Through props (explicit)	Directly to any consumer (no props needed)
Best for	Nearby siblings (1-3 levels)	Data needed deep in the tree or globally
Performance	Only affected children re-render	ALL context consumers re-render on change
Complexity	Simple — just move state and pass props	More setup — createContext, Provider, useContext
Examples	Cart + NavBar, Search + Results	Logged-in user, theme, language settings

Think of it as a progression:

7. **1-2 levels deep** — just pass props normally
8. **2-4 levels, 2-3 siblings** — Lift State Up to the common parent
9. **Many levels deep or globally needed** — use Context API
10. **Very complex, frequently changing** — use Redux or Zustand

> ✅ **Remember**
>
> Lifting State Up is the first tool you reach for. Context API is the next step when lifting is not enough.
>
> Do NOT jump straight to Context API for every shared state. If two siblings are close in the tree, lifting state up is simpler and more explicit.
>
> The progression: local state → lifted state → context → Redux/Zustand.

Topic 11: Derived State — Do Not Duplicate State

A common mistake when lifting state up is creating extra state variables for data that can be calculated from existing state. This is called "derived state" and you should calculate it directly, not store it separately.

```
// ❌ WRONG — totalPrice is derived from cartItems. Do not store separately.
function App() {
  const [cartItems, setCartItems] = useState([]);
  const [totalPrice, setTotalPrice] = useState(0);  // WRONG — derived state

  function addToCart(item) {
    setCartItems([...cartItems, item]);
    setTotalPrice(totalPrice + item.price);  // Must remember to update both
  }
  // Problem: cartItems and totalPrice can get OUT OF SYNC.
  // If you update one and forget the other, bugs happen.
}
```

```
// ✅ CORRECT — derive totalPrice from cartItems directly
function App() {
  const [cartItems, setCartItems] = useState([]);

  // Derived — calculated fresh every render from the single source of truth
  const totalPrice = cartItems.reduce((sum, item) => sum + item.price, 0);

  function addToCart(item) {
    setCartItems([...cartItems, item]);  // Only ONE state to update
    // totalPrice updates automatically on next render. No extra work.
  }
}
```

✅ Remember

If a value can be CALCULATED from existing state or props — calculate it, do not store it as separate state.

Storing derived state creates two sources of truth which can go out of sync.

Common examples of derived values: filtered list from a query, total price from cart items, full name from first + last name, error message from form fields.

Rule: Ask yourself "Can I calculate this from existing state?" If yes, calculate it. Do not useState it.

Topic 12: Complete Real-World Example — Tabs Component

Let us build a Tabs component that perfectly demonstrates Lifting State Up, controlled components, and the single source of truth principle all together.

```
// Complete Tabs Example — shows all Lifting State Up concepts together
import { useState } from "react";

const TABS = [
  { id: "profile",  label: "Profile",  content: "Your profile info here." },
  { id: "settings", label: "Settings", content: "App settings go here." },
  { id: "billing",  label: "Billing",  content: "Payment methods here." },
];

// Tabs — the common parent, holds activeTab state
function Tabs() {
  const [activeTab, setActiveTab] = useState("profile");

  // Find the currently active tab data
  const currentTab = TABS.find(t => t.id === activeTab);

  return (
    <div className="tabs-container">
      {/* TabList is controlled — activeTab comes from parent */}
      <TabList
        tabs={TABS}
```

```
        activeTab={activeTab}
        onTabChange={setActiveTab}
      />

      {/* TabPanel is controlled — content comes from parent */}
      <TabPanel content={currentTab.content} />
    </div>
  );
}

// TabList — controlled component, highlights active tab from prop
function TabList({ tabs, activeTab, onTabChange }) {
  return (
    <div className="tab-list">
      {tabs.map(tab => (
        <button
          key={tab.id}
          className={tab.id === activeTab ? "active" : ""}
          onClick={() => onTabChange(tab.id)}
        >
          {tab.label}
        </button>
      ))}
    </div>
  );
}

// TabPanel — simple display component
function TabPanel({ content }) {
  return <div className="tab-panel">{content}</div>;
}
```

In this example: Tabs holds the activeTab state (the single source of truth). TabList receives it as a prop and highlights the correct tab. TabPanel receives the content derived from it. When a tab is clicked, TabList calls onTabChange which updates the state in Tabs. Everything re-renders and stays in sync.

> 💡 **Tip**
>
> This Tabs example is perfect for an interview answer. It is small, real, and shows all the concepts cleanly.
>
> You can say: "I use the Lifting State Up pattern. The Tabs parent holds activeTab state. Both TabList and TabPanel receive this as props. TabList reports clicks via a callback. This gives a single source of truth and keeps the components in sync."

Chapter 3: Interview Questions with Full Answers

These are the most asked interview questions about Lifting State Up. Study each answer carefully.

Q1: What is "Lifting State Up" in React?

Answer:

"Lifting State Up" is a pattern where you move a shared state variable from a child component up to the closest common parent of the components that need it.

The problem it solves: when two sibling components need the same data, they cannot share it directly because data in React only flows from parent to child. The solution is to put the state in their common parent and pass it down to both as props.

After lifting: the parent becomes the single source of truth for that data. Both children receive it via props and report changes via callback functions. This keeps all components in sync.

Q2: Why can two sibling components not share state directly?

Answer:

In React, data flows in only ONE direction — from parent to child through props. There is no mechanism for sibling-to-sibling communication.

Component A cannot read or write to Component B's state even if they are next to each other in the component tree. They are completely isolated.

The only way siblings can share data is through a common parent. The parent holds the data and passes it down to both siblings via props. When a sibling needs to change the data, it calls a callback function that the parent provided — which updates the parent's state — which flows back down to both siblings.

Q3: What is Single Source of Truth?

Answer:

"Single Source of Truth" means there is exactly ONE place in the application where a particular piece of data lives. All components that need that data read it from this one place.

In React, this is achieved through Lifting State Up. When you move shared state to a common parent, that parent becomes the single source of truth.

Why it matters: If the same data existed in multiple places (duplicated state), the copies could go out of sync — one component shows value 5 while another shows value 3 for the same thing. With a single source of truth, all components always show consistent data.

Q4: What is a controlled component?

Answer:

A controlled component is a component — usually a form element — whose value is completely controlled by React state from a parent.

It has no state of its own. Its value is set via a prop (value={stateVariable}) and it reports changes via an onChange callback prop. The parent updates the state in response to the callback, which flows back down and the component re-renders with the new value.

Example: <input value={name} onChange={e => setName(e.target.value)} />

This is the opposite of uncontrolled components where the DOM holds the value and you read it with a ref.

After lifting state up, child components naturally become controlled components.

Q5: What is derived state? Why should you avoid storing it in useState?

Answer:

Derived state is any value that can be calculated from existing state or props. Examples: total price calculated from cart items, a filtered list calculated from all items and a search query, full name from first + last name.

You should NOT store derived values in useState because it creates two sources of truth. The two can go out of sync — if you update the original state but forget to update the derived state, your UI shows stale data.

Instead, calculate derived values directly during render: const total = items.reduce((sum, item) => sum + item.price, 0)

The derived value recalculates automatically every render without any extra work or risk of getting out of sync.

Q6: When should you use Lifting State Up vs Context API?

Answer:

Use Lifting State Up when: two or three components that are close together in the tree need to share data. The common parent is 1-3 levels above. It is the simpler, more explicit solution.

Use Context API when: the data needs to reach many components spread throughout the tree, or when lifting would require passing props through many levels of intermediate components (prop drilling).

Example: Cart count shared between NavBar and Cart — lift it to App. That is simple. But logged-in user info needed by Header, Sidebar, Profile, Settings, and Dashboard — lifting creates too much prop passing. Use Context for that.

Rule: Lift first. If you find yourself passing props through 3 or more levels just to reach the component that needs it, switch to Context.

Q7: What happens to performance when you lift state up?

Answer:

When state in a parent changes, React re-renders the parent and all its children by default — even if some children do not use the changed state.

For example, if App lifts cartCount and it changes, all of App's children re-render: NavBar, HeroSection, Footer — even though only NavBar uses cartCount.

For small apps, this is not a problem — React's re-renders are very fast.

For large apps with expensive components, you can use React.memo to wrap components that should not re-render when irrelevant state changes. React.memo skips re-rendering if the component's props have not changed.

For memoizing function props (callbacks), use useCallback to keep the same function reference across renders.

Q8: How does a child component communicate back to its parent after lifting state up?

Answer:

Through callback functions passed as props.

The parent defines a function that updates its state, then passes that function as a prop to the child. When the child needs to communicate upward — like when a button is clicked — it calls that callback function.

Example: Parent defines const handleAddToCart = (item) => setCartItems([...cartItems, item]). Parent passes it as <ProductCard onAddToCart={handleAddToCart} />. Child calls props.onAddToCart(item) when button is clicked.

This keeps one-way data flow intact. Data flows down via props. Events flow up via callbacks. The parent always remains in control.

Q9: Give a real-world example where you would use Lifting State Up.

Answer:

Classic example: E-commerce site with a NavBar and a Cart component side by side.

NavBar needs to show the cart item count (e.g., 🛒 3). Cart needs to show the actual list of items. When a user adds an item from ProductList, both NavBar count and Cart list must update.

Solution: Lift cartItems state up to App. App passes cartItems.length to NavBar. App passes cartItems to Cart. App passes addToCart function to ProductList. When ProductList calls addToCart, App's state updates and both NavBar and Cart re-render with the new data.

Other real examples: Search box + results list, filter panel + product grid, form steps where step 3 needs data from step 1.

Q10: What is the difference between Lifting State Up and Prop Drilling?

Answer:

Lifting State Up is a SOLUTION — you move state to a common parent so siblings can share it. It is intentional and correct.

Prop Drilling is a PROBLEM — you pass props through many intermediate components that do not need them, just to reach a deeply nested component.

They are related but different:

Lifting State Up sometimes CAUSES prop drilling if the state needs to reach very deep components. You lift it to the top, but then have to drill it down through many levels.

When Lifting State Up causes too much prop drilling, you switch to Context API which eliminates the drilling while keeping a single source of truth.

Q11: What is the difference between state and derived state?

Answer:

State is data that React manages and persists between renders. When you call useState, React stores that value and gives you a setter to update it. It is the source of truth.

Derived state is data that is COMPUTED from state or props. It is not stored — it is recalculated every render. Examples: filtered array, total sum, formatted date string.

You should only put the minimal amount of data in useState. Everything that can be computed should be computed — not stored.

The test: "Can I always calculate this value from my current state and props?" If yes — it is derived state. Do not store it in useState.

Q12: Walk me through Lifting State Up step by step.

Answer:

Step 1: Identify which components need the shared data.

Step 2: Find their closest common ancestor in the component tree.

Step 3: Move the state from the children up to that common ancestor.

Step 4: Pass the state DOWN to each child as a prop.

Step 5: Pass a callback function DOWN so each child can update the state when needed.

Step 6: Remove the local state from the children. They become controlled components.

Result: The parent is the single source of truth. All children are in sync. Data flows down via props. Changes flow up via callbacks.

Common Mistakes Beginners Make

Mistake 1: Storing derived state in useState

```
// ❌ WRONG — totalPrice is derived from cartItems
const [cartItems, setCartItems] = useState([]);
const [totalPrice, setTotalPrice] = useState(0);  // redundant!

// ✅ CORRECT — calculate it directly
const [cartItems, setCartItems] = useState([]);
const totalPrice = cartItems.reduce((sum, item) => sum + item.price, 0);
```

Mistake 2: Lifting state too high unnecessarily

```
// ❌ WRONG — lifting modal state to App when only one component uses it
function App() {
  const [isModalOpen, setIsModalOpen] = useState(false);  // too high!
  return <ProductCard isModalOpen={isModalOpen} onToggle={setIsModalOpen} />;
}

// ✅ CORRECT — modal state stays inside ProductCard (only it uses it)
function ProductCard() {
  const [isModalOpen, setIsModalOpen] = useState(false);
  return (
    <div>
      <button onClick={() => setIsModalOpen(true)}>View Details</button>
      {isModalOpen && <Modal onClose={() => setIsModalOpen(false)} />}
    </div>
  );
```

```
}
```

Mistake 3: Directly mutating state instead of using setter

```
// ❌ WRONG — directly pushing to array does NOT trigger re-render
function App() {
  const [items, setItems] = useState([]);
  function addItem(item) {
    items.push(item);         // WRONG — mutating state directly
    console.log(items);       // items changed but React does NOT know
  }
}

// ✅ CORRECT — always create a new array, use the setter
function App() {
  const [items, setItems] = useState([]);
  function addItem(item) {
    setItems([...items, item]);  // new array — React sees the change
  }
}
```

Mistake 4: Not passing callbacks — child has no way to update parent

```
// ❌ WRONG — parent lifted state but forgot to pass callback to child
function App() {
  const [count, setCount] = useState(0);
  return <Counter count={count} />;  // missing onCountChange!
}

function Counter({ count }) {
  // How do I update count? I have no callback!
  return <button onClick={() => {}}>Count: {count}</button>;
}

// ✅ CORRECT — always pass the callback
function App() {
  const [count, setCount] = useState(0);
  return <Counter count={count} onCountChange={setCount} />;
}

function Counter({ count, onCountChange }) {
  return <button onClick={() => onCountChange(count + 1)}>Count:
{count}</button>;
}
```

Tips & Tricks — Stand Out in Your Interview

💡 **Tip**

The classic interview answer analogy for Lifting State Up:

"Lifting State Up is like two siblings in a house who both need the TV remote. Instead of each sibling having their own separate remote that controls nothing, you give the remote (state) to the parent. The parent passes it to whichever sibling needs it, and when they press a button (callback), the parent changes the channel (updates state) and both siblings see the new channel (re-render with new props)."

🎯 **Interview Tip**

When asked about Lifting State Up, structure your answer in 4 parts:

1. Define the problem — siblings need shared data but cannot share directly
2. Explain the solution — move state to closest common parent
3. Mention the result — parent is single source of truth, children are controlled
4. Mention when to go further — if too many levels, use Context API

This shows you understand the full picture and not just the definition.

✅ **Remember**

Lifting State Up — move shared state to the closest common parent of the components that need it.

Single Source of Truth — ONE place holds the data, all others read from it.

Controlled Component — no own state, receives value via prop, reports changes via callback.

Derived State — calculate from existing state, never store separately in useState.

When to lift — two or more siblings need the same data.

When NOT to lift — UI-only state used by one component only (dropdowns, tooltips).

When to use Context — too many levels, lifting causes prop drilling.

Callback pattern — parent passes function as prop, child calls it to communicate upward.

Quick Reference — Chapter 3 Summary

Concept	What It Means	Key Point
Lifting State Up	Move state to common parent	Solves sharing between siblings
Single Source of Truth	One place holds the data	All components stay in sync
Controlled Component	No own state, driven by props	Value from parent, changes via

		callback
Derived State	Calculated from existing state	Never store in useState — calculate directly
Callback prop	Function passed down to child	Child calls it to update parent state
When to lift	Two+ siblings share data	Move to their closest common ancestor
When to use Context	Many levels or global data	When lifting causes prop drilling
React.memo	Skip re-render if props unchanged	Optimises lifted-state re-renders

End of Chapter 3 • Next: Chapter 4 — useState & useEffect Hooks

Chapter 4: React Hooks — Complete Guide

useState, useEffect, useRef, useContext, useReducer, useMemo, useCallback & Custom Hooks

By Syed Shaaz Akhtar | Interview Focused Guide

Hooks are the heart of modern React. Before hooks (React 16.8, 2019), you needed class components for state and lifecycle. Hooks let you do all of that inside simple function components. This chapter covers every hook you need for interviews — from the basic useState to advanced useMemo and Custom Hooks.

Every single hook in this chapter is asked in interviews. Read carefully and understand the WHY behind each hook — not just the syntax.

Section A: useState

Managing data that changes inside a component

Topic 1: What is useState?

useState is a hook that lets you add a state variable to a function component. State is data that the component remembers between renders. When state changes, React re-renders the component and the UI updates automatically.

Think of state like a sticky note on your desk. You write something on it. You can change it later. Every time you look at it (render), you see the latest value.

```
import { useState } from "react";

// Syntax:
const [stateVariable, setterFunction] = useState(initialValue);

// Real example — a simple counter
function Counter() {
  const [count, setCount] = useState(0);  // 0 is the initial value

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(count - 1)}>-</button>
      <button onClick={() => setCount(0)}>Reset</button>
```

```
    </div>
  );
}
```

> 💡 **Tip**
> useState returns an ARRAY of exactly two things: [currentValue, setterFunction].
> The names (count, setCount) are up to you — but the convention is "name" and "setName".
> Every time setCount is called with a new value, React re-renders the component. You see the new count on screen.

Topic 2: useState with Objects and Arrays

State can hold any JavaScript value — numbers, strings, booleans, objects, and arrays. But with objects and arrays, there is a critical rule: **never mutate state directly. Always create a new copy.**

State with an Object

```
function UserForm() {
  const [user, setUser] = useState({ name: "", age: 0, city: "" });

  function updateName(newName) {
    // ❌ WRONG — mutating existing object does NOT trigger re-render
    user.name = newName;   // React does not know this changed!
    setUser(user);         // same reference, React skips re-render

    // ✅ CORRECT — spread old state, override changed field
    setUser({ ...user, name: newName });  // new object = re-render
  }

  return (
    <div>
      <input value={user.name} onChange={e => setUser({...user, name:
e.target.value})} />
      <input value={user.age}  onChange={e => setUser({...user, age:
Number(e.target.value)})} />
    </div>
  );
}
```

State with an Array

```
function TodoList() {
  const [todos, setTodos] = useState(["Buy milk", "Read book"]);

  // ADD — spread existing + new item
  function addTodo(text) {
    setTodos([...todos, text]);
  }
```

```
  // REMOVE — filter out the item
  function removeTodo(index) {
    setTodos(todos.filter((_, i) => i !== index));
  }

  // UPDATE — map and replace specific item
  function updateTodo(index, newText) {
    setTodos(todos.map((todo, i) => i === index ? newText : todo));
  }

  return (
    <ul>
      {todos.map((todo, i) => (
        <li key={i}>
          {todo}
          <button onClick={() => removeTodo(i)}>x</button>
        </li>
      ))}
    </ul>
  );
}
```

> ✅ **Remember**
>
> React uses SHALLOW comparison to detect state changes. If you pass the same object reference, React thinks nothing changed and skips re-render.
>
> Always create a NEW array or object: [...arr] or {...obj}.
>
> Never use: arr.push(), arr.splice(), obj.key = value — these mutate in place and do not trigger re-renders.

Topic 3: Functional Updates — When New State Depends on Old State

There is a subtle bug that happens when you call the setter function multiple times in the same event handler. If the new state depends on the old state, you must use the **functional update form**.

```
// THE BUG — calling setCount multiple times based on count
function Counter() {
  const [count, setCount] = useState(0);

  function incrementThrice() {
    // ❌ WRONG — all three use the SAME stale count value (0)
    setCount(count + 1);  // count is 0, schedules: set to 1
    setCount(count + 1);  // count is still 0! schedules: set to 1
    setCount(count + 1);  // count is still 0! schedules: set to 1
    // Result: count becomes 1 — not 3!
  }

  function incrementThriceCorrect() {
    // ✅ CORRECT — functional update receives LATEST state
```

```
    setCount(prev => prev + 1);  // 0 → 1
    setCount(prev => prev + 1);  // 1 → 2
    setCount(prev => prev + 1);  // 2 → 3
    // Result: count becomes 3 — correct!
  }

  return <button onClick={incrementThriceCorrect}>Count: {count}</button>;
}
```

> ✅ **Remember**
>
> When new state depends on previous state — ALWAYS use functional update: setCount(prev => prev + 1).
>
> This is especially important in setTimeout, event handlers, and async code where the state value might be stale.
>
> React batches multiple state updates inside event handlers — functional updates ensure each gets the latest value.

Topic 4: useState — Lazy Initialization

If the initial value of your state requires a heavy calculation, you can pass a **function** to useState instead of a value. React will call this function only ONCE on the first render — not on every render.

```
// ❌ SLOW — expensiveCalc() runs on EVERY render
const [data, setData] = useState(expensiveCalc());

// ✅ FAST — expensiveCalc() runs only ONCE (first render)
const [data, setData] = useState(() => expensiveCalc());

// Real example: reading from localStorage
const [theme, setTheme] = useState(() => {
  return localStorage.getItem("theme") || "light";
  // Without lazy init: reads localStorage on EVERY render
  // With lazy init: reads localStorage ONCE at mount
});
```

Section B: useEffect

Running code after render — API calls, subscriptions, timers

Topic 5: What is useEffect?

useEffect is a hook for running **side effects** in a function component. A side effect is anything that happens outside of rendering — like fetching data from an API, setting up a timer, manually changing the DOM, or subscribing to an event.

The rule in React: the render function must be pure — no side effects during rendering. useEffect gives you a safe place to run side effects AFTER the component renders.

```
import { useState, useEffect } from "react";

// Syntax:
useEffect(() => {
  // Side effect code runs here — AFTER render
  // ...
  return () => {
    // Optional cleanup — runs before next effect OR component unmounts
  };
}, [dependencies]);  // dependency array controls WHEN effect runs
```

Topic 6: The Dependency Array — The Most Important Part

The second argument to useEffect is the **dependency array**. This array controls WHEN the effect runs. This is the most commonly misunderstood part of useEffect and is asked in almost every interview.

Dependency Array	When Effect Runs	Use Case
No array — useEffect(() => {})	After EVERY render	Almost never — very rare
Empty array — useEffect(() => {}, [])	Once only — after first render (mount)	Fetch initial data, set up listeners
With values — useEffect(() => {}, [a, b])	After first render AND whenever a or b changes	Re-fetch when filter changes, sync with prop

```
// 1. NO dependency array — runs after every single render
useEffect(() => {
  console.log("Runs after every render — avoid this usually");
});

// 2. EMPTY array — runs once after first render (componentDidMount)
useEffect(() => {
  console.log("Runs once — good for initial API calls");
  fetchUserData();
}, []);

// 3. WITH dependencies — runs when userId changes
useEffect(() => {
  console.log("userId changed, re-fetching...");
  fetchUserById(userId);
}, [userId]);  // re-runs every time userId changes
```

> ✅ **Remember**
> Empty [] = runs once. [val] = runs when val changes. No array = runs every render.
> If your effect uses a variable, that variable should be in the dependency array.

> Missing dependencies can cause stale closures — your effect reads an old value of the variable.
> Extra dependencies cause unnecessary re-runs — but missing dependencies cause bugs.

Topic 7: Fetching Data with useEffect — Complete Pattern

This is the most common use of useEffect. Every real React app fetches data. Here is the complete, correct pattern including loading state, error handling, and cleanup.

```
import { useState, useEffect } from "react";

function UserProfile({ userId }) {
  const [user, setUser]       = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  useEffect(() => {
    // Reset state when userId changes
    setLoading(true);
    setError(null);

    // Abort controller — cancels fetch if component unmounts
    const controller = new AbortController();

    fetch(`https://api.example.com/users/${userId}`, {
      signal: controller.signal
    })
      .then(res => {
        if (!res.ok) throw new Error("User not found");
        return res.json();
      })
      .then(data => {
        setUser(data);
        setLoading(false);
      })
      .catch(err => {
        if (err.name === "AbortError") return; // ignore cancelled fetch
        setError(err.message);
        setLoading(false);
      });

    // Cleanup — cancel the fetch if userId changes or component unmounts
    return () => controller.abort();

  }, [userId]);  // Re-runs when userId prop changes

  if (loading) return <p>Loading...</p>;
  if (error)   return <p>Error: {error}</p>;
  if (!user)   return null;
```

```
  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.email}</p>
    </div>
  );
}
```

💡 **Tip**

Three states to always handle with API calls: loading, error, data.

AbortController cancels in-flight requests when the component unmounts or userId changes — prevents "state update on unmounted component" warnings.

This pattern is what interviewers want to see. Most beginners miss the cleanup function and error handling.

Topic 8: Cleanup Function — Preventing Memory Leaks

The function you return from useEffect is the **cleanup function**. It runs before the NEXT effect runs and when the component unmounts. Without cleanup, you can get memory leaks, stale timers, and bugs.

```
// Example 1: Cleanup a timer
function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const interval = setInterval(() => {
      setSeconds(prev => prev + 1);
    }, 1000);

    // Cleanup — clear the interval when component unmounts
    return () => clearInterval(interval);
    // Without this: timer keeps running even after component is gone!
  }, []);

  return <p>Seconds: {seconds}</p>;
}

// Example 2: Cleanup an event listener
function MouseTracker() {
  const [pos, setPos] = useState({ x: 0, y: 0 });

  useEffect(() => {
    function handleMove(e) {
      setPos({ x: e.clientX, y: e.clientY });
    }

    window.addEventListener("mousemove", handleMove);
```

```
    // Cleanup — remove listener when component unmounts
    return () => window.removeEventListener("mousemove", handleMove);
    // Without this: listener piles up on every render!
  }, []);

  return <p>Mouse: {pos.x}, {pos.y}</p>;
}
```

✅ **Remember**

Always cleanup: timers (clearInterval/clearTimeout), event listeners (removeEventListener), API calls (AbortController), WebSocket connections (ws.close()).

Without cleanup — memory leaks. Timer runs after unmount. Listener fires on a dead component. Console shows "setState on unmounted component" warning.

The cleanup function runs: (1) before the NEXT effect runs, (2) when the component unmounts.

Section C: Other Essential Hooks

useRef, useContext, useReducer, useMemo, useCallback

Topic 9: useRef — Persistent Values Without Re-renders

useRef creates a mutable container that persists across renders but does NOT cause a re-render when it changes. It returns an object with a single property: **.current**.

useRef has two main use cases:

- **Accessing DOM elements directly** — like focusing an input, measuring size
- **Storing a value that should persist across renders but NOT trigger re-render** — like a previous value, timer ID, or flag

Use Case 1 — Accessing DOM Elements

```
import { useRef } from "react";

function SearchBox() {
  const inputRef = useRef(null);

  function focusInput() {
    // Directly access the DOM input element and call .focus()
    inputRef.current.focus();
  }

  return (
    <div>
      {/* Attach the ref to the DOM element using the "ref" attribute */}
      <input ref={inputRef} placeholder="Search..." />
```

```
      <button onClick={focusInput}>Focus Input</button>
    </div>
  );
}
```

Use Case 2 — Storing Previous Values

```
import { useState, useEffect, useRef } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  const prevCountRef = useRef(0);

  useEffect(() => {
    // After every render, update the ref to the current count
    // This does NOT cause a re-render
    prevCountRef.current = count;
  });

  return (
    <p>
      Current: {count} | Previous: {prevCountRef.current}
      <button onClick={() => setCount(c => c + 1)}>+</button>
    </p>
  );
}

// Another use — store interval ID to clear it later
function Stopwatch() {
  const [time, setTime] = useState(0);
  const intervalRef = useRef(null);

  function start() {
    intervalRef.current = setInterval(() => setTime(t => t + 1), 1000);
  }

  function stop() {
    clearInterval(intervalRef.current);  // access stored interval ID
  }

  return <div><button onClick={start}>Start</button><button
onClick={stop}>Stop</button> {time}s</div>;
}
```

Aspect	useState	useRef
Causes re-render?	YES — every update triggers re-render	NO — changes never trigger re-render
Access value	const [val, setVal] = useState()	ref.current

Mutable?	Use setter — never mutate directly	ref.current can be freely mutated
Use case	UI data that should be shown on screen	DOM refs, timers, flags, previous values
Persists across renders?	YES	YES

Topic 10: useContext — Consuming Context Without Prop Drilling

useContext is the hook for consuming data from a React Context. Without it, you would need to wrap every consumer in a Context.Consumer render prop — very messy. useContext makes it one clean line.

```
import { createContext, useContext, useState } from "react";

// Step 1: Create the Context
const ThemeContext = createContext("light");  // "light" is default

// Step 2: Provide the context at the top level
function App() {
  const [theme, setTheme] = useState("light");

  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Header />
      <MainContent />
      <Footer />
    </ThemeContext.Provider>
  );
}

// Step 3: Consume ANYWHERE in the tree with useContext
function Header() {
  const { theme, setTheme } = useContext(ThemeContext);

  return (
    <header className={theme}>
      <h1>My App</h1>
      <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
        Toggle Theme
      </button>
    </header>
  );
}

// Works any level deep — no prop passing needed
function DeepChild() {
  const { theme } = useContext(ThemeContext);
  return <div className={theme}>Deep content</div>;
```

```
}
```

> 📝 **Note**
>
> Every component that calls useContext(SomeContext) re-renders when the context VALUE changes.
>
> If your context value is an object like { theme, setTheme }, a new object is created on every parent render — causing all consumers to re-render unnecessarily.
>
> Fix: wrap the value in useMemo: value={useMemo(() => ({ theme, setTheme }), [theme])}
>
> For very frequently changing data (like a counter), Context causes too many re-renders. Use Redux/Zustand instead.

Topic 11: useReducer — Complex State Logic Made Clean

useReducer is an alternative to useState for managing complex state. Instead of calling setState directly, you dispatch **actions** that describe what happened. A **reducer function** handles those actions and returns the new state.

Think of it like a bank. Instead of directly changing your balance, you submit a transaction (action) to the bank teller (reducer) who processes it and updates your account (state).

```
import { useReducer } from "react";

// The reducer — a pure function: (currentState, action) => newState
function cartReducer(state, action) {
  switch (action.type) {
    case "ADD_ITEM":
      return { ...state, items: [...state.items, action.payload] };

    case "REMOVE_ITEM":
      return {
        ...state,
        items: state.items.filter(item => item.id !== action.payload)
      };

    case "CLEAR_CART":
      return { ...state, items: [] };

    default:
      return state;  // unknown action — return unchanged state
  }
}

const initialState = { items: [], discount: 0 };

function Cart() {
  const [state, dispatch] = useReducer(cartReducer, initialState);

  return (
    <div>
```

```
      <p>Items: {state.items.length}</p>

      {/* Dispatch an action — like sending a command */}
      <button onClick={() => dispatch({
        type: "ADD_ITEM",
        payload: { id: 1, name: "Headphones", price: 2499 }
      })}>
        Add Headphones
      </button>

      <button onClick={() => dispatch({ type: "CLEAR_CART" })}>
        Clear Cart
      </button>
    </div>
  );
}
```

Aspect	useState	useReducer
Best for	Simple, independent state values	Complex state with multiple sub-values
State updates	Call setter directly: setCount(5)	Dispatch actions: dispatch({type: "ADD"})
Logic location	Inline in event handlers	Centralised in reducer function
Testing	Harder — logic spread across handlers	Easy — reducer is a pure function, unit test it
When to switch	When you have 3+ related state updates	When state transitions become complex

💡 **Tip**

Rule of thumb: If you find yourself writing 3 or more useState calls that are all related, consider useReducer instead.

useReducer is the foundation of Redux. If you understand useReducer, you understand Redux's core concept.

The reducer must be a PURE function — same input always gives same output, no side effects inside the reducer.

Topic 12: useMemo — Caching Expensive Calculations

useMemo memoizes (caches) the result of a computation. It only re-calculates when its dependencies change. This prevents expensive calculations from running on every render.

```
import { useState, useMemo } from "react";

function ProductList({ products, searchQuery }) {
```

```
  // ❌ WITHOUT useMemo — this filters on EVERY render
  // Even if products and searchQuery have not changed
  const filtered = products.filter(p =>
    p.name.toLowerCase().includes(searchQuery.toLowerCase())
  );

  // ✅ WITH useMemo — only re-filters when products or searchQuery changes
  const filteredMemo = useMemo(() => {
    console.log("Filtering products..."); // Only logs when deps change
    return products.filter(p =>
      p.name.toLowerCase().includes(searchQuery.toLowerCase())
    );
  }, [products, searchQuery]);  // Only re-run when these change

  return (
    <ul>
      {filteredMemo.map(p => <li key={p.id}>{p.name}</li>)}
    </ul>
  );
}

// Another example — expensive calculation
function Stats({ numbers }) {
  const stats = useMemo(() => {
    const sum = numbers.reduce((a, b) => a + b, 0);
    const avg = sum / numbers.length;
    const max = Math.max(...numbers);
    return { sum, avg, max };
  }, [numbers]);  // Only recalculate when numbers array changes

  return <p>Sum: {stats.sum} | Avg: {stats.avg} | Max: {stats.max}</p>;
}
```

✅ **Remember**

useMemo caches a COMPUTED VALUE. useCallback caches a FUNCTION. Remember this difference.

Do NOT use useMemo everywhere. It has its own overhead — storing the cached value and comparing dependencies. Use it only for genuinely expensive calculations.

Rule: If removing useMemo and the app is still fast — it did not need useMemo.

Topic 13: useCallback — Caching Functions for Stability

useCallback returns a memoized (cached) version of a function. The function is only recreated when its dependencies change. This is useful when passing callbacks to child components wrapped with React.memo.

```
import { useState, useCallback } from "react";
```

```
// THE PROBLEM without useCallback:
function Parent() {
  const [count, setCount] = useState(0);
  const [name, setName]   = useState("Shaaz");

  // Every render of Parent creates a NEW function reference for handleClick
  const handleClick = () => console.log("clicked");

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
      <ExpensiveChild onClick={handleClick} />
      {/* When count changes, Parent re-renders, handleClick gets a new
          reference. React.memo on ExpensiveChild sees new props → re-renders
          even though the function does the same thing! */}
    </div>
  );
}

// THE FIX with useCallback:
function Parent() {
  const [count, setCount] = useState(0);

  // Same function reference across renders — only new if deps change
  const handleClick = useCallback(() => {
    console.log("clicked");
  }, []);  // empty deps = function never changes

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
      <ExpensiveChild onClick={handleClick} />
      {/* Now handleClick has same reference → React.memo works correctly
          → ExpensiveChild does NOT re-render when count changes */}
    </div>
  );
}

const ExpensiveChild = React.memo(function ExpensiveChild({ onClick }) {
  console.log("ExpensiveChild rendered");
  return <button onClick={onClick}>Expensive Button</button>;
});
```

Hook	What it Caches	Returns	Use When
useMemo	Result of a function call	The computed VALUE	Expensive calculations (filtered list, stats)
useCallback	The function itself	The FUNCTION	Passing stable callbacks

			to memoized children

🎯 **Interview Tip**

The classic interview question: "What is the difference between useMemo and useCallback?"

Answer: "useMemo caches a computed value — the result of calling a function. useCallback caches the function itself — the reference. useMemo(() => a + b, [a, b]) returns the sum. useCallback(() => doSomething(), []) returns the function."

One-liner to remember: useMemo = cache VALUE. useCallback = cache FUNCTION.

Topic 14: Custom Hooks — Reusing Logic Across Components

Custom Hooks are functions that start with "use" and can call other hooks inside them. They let you extract component logic into reusable functions. This is one of the most powerful features of React hooks.

Before custom hooks, reusing stateful logic required complex patterns (HOCs, render props). Now you just write a function.

Example 1 — useFetch: Reusable Data Fetching

```
// Custom Hook — useFetch
// Can be used by ANY component that needs to fetch data
import { useState, useEffect } from "react";

function useFetch(url) {
  const [data, setData]       = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  useEffect(() => {
    setLoading(true);
    const controller = new AbortController();

    fetch(url, { signal: controller.signal })
      .then(res => res.json())
      .then(data => { setData(data); setLoading(false); })
      .catch(err => {
        if (err.name !== "AbortError") {
          setError(err.message); setLoading(false);
        }
      });

    return () => controller.abort();
  }, [url]);

  return { data, loading, error };
}

// Now ANY component can use it in ONE line:
```

```
function UserProfile({ userId }) {
  const { data: user, loading, error } = useFetch(`/api/users/${userId}`);
  if (loading) return <p>Loading...</p>;
  if (error)   return <p>Error: {error}</p>;
  return <h2>{user.name}</h2>;
}

function ProductPage({ productId }) {
  const { data: product, loading } = useFetch(`/api/products/${productId}`);
  if (loading) return <p>Loading...</p>;
  return <h2>{product.name} — ₹{product.price}</h2>;
}
```

Example 2 — useLocalStorage: Persistent State

```
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try {
      const stored = localStorage.getItem(key);
      return stored ? JSON.parse(stored) : initialValue;
    } catch {
      return initialValue;
    }
  });

  function setStoredValue(newValue) {
    setValue(newValue);
    localStorage.setItem(key, JSON.stringify(newValue));
  }

  return [value, setStoredValue];
}

// Usage — exactly like useState but persists in localStorage
function ThemeToggle() {
  const [theme, setTheme] = useLocalStorage("theme", "light");
  return (
    <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
      Current: {theme}
    </button>
  );
}
```

Example 3 — useDebounce: Delay Input Processing

```
function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
```

```
    return () => clearTimeout(timer);  // cancel if value changes before delay
  }, [value, delay]);

  return debouncedValue;
}

// Usage — API only called 500ms after user stops typing
function SearchBar() {
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 500);

  // Only runs when debouncedQuery changes (500ms after typing stops)
  useEffect(() => {
    if (debouncedQuery) fetchSearchResults(debouncedQuery);
  }, [debouncedQuery]);

  return <input value={query} onChange={e => setQuery(e.target.value)} />;
}
```

✅ Remember

Custom hooks MUST start with "use" — this tells React it follows hook rules.

Custom hooks can use other hooks (useState, useEffect, etc.) inside them.

Custom hooks do NOT share state between components — each component gets its own independent state.

When two components use the same custom hook, they each get their own copy of all state and effects.

Custom hooks are just functions — they do not render any JSX.

Topic 15: Rules of Hooks — Must Know

React has two strict rules about where hooks can be called. Breaking these rules causes bugs and errors.

Rule 1: Only Call Hooks at the Top Level

Never call hooks inside loops, conditions, or nested functions. Always call them at the top level of your component function.

```
// ❌ WRONG — hook inside a condition
function UserProfile({ isLoggedIn }) {
  if (isLoggedIn) {
    const [user, setUser] = useState(null);  // WRONG! Hook inside if
  }
}

// ❌ WRONG — hook inside a loop
function List({ items }) {
  items.forEach(item => {
    const [selected, setSelected] = useState(false);  // WRONG! Hook inside loop
```

```
  });
}

// ✅ CORRECT — all hooks at the top level
function UserProfile({ isLoggedIn }) {
  const [user, setUser] = useState(null);  // Always called, every render
  // Then use isLoggedIn inside JSX or effects
}
```

Rule 2: Only Call Hooks from React Functions

Call hooks only from React function components or custom hooks. Never from regular JavaScript functions, class components, or event handlers.

```
// ❌ WRONG — hook in regular JS function
function regularFunction() {
  const [count, setCount] = useState(0);  // WRONG! Not a React function
}

// ✅ CORRECT — hook in React component
function Counter() {
  const [count, setCount] = useState(0);  // Correct
  return <button onClick={() => setCount(c => c + 1)}>{count}</button>;
}

// ✅ CORRECT — hook in custom hook
function useCounter(initial = 0) {
  const [count, setCount] = useState(initial);  // Correct in custom hook
  return { count, increment: () => setCount(c => c + 1) };
}
```

> 📝 **Note**
>
> WHY these rules exist: React relies on the ORDER hooks are called to correctly associate state with each hook.
>
> If you call a hook inside a condition, the order can change between renders — React gets confused about which state belongs to which hook.
>
> The eslint-plugin-react-hooks package enforces these rules automatically in your code editor.

Chapter 4: Interview Questions with Full Answers

Q1: What are React Hooks? Why were they introduced?

Answer:

React Hooks are special functions that let you "hook into" React features — like state and lifecycle — from inside function components.

Before hooks (React 16.8, February 2019), function components were stateless. You needed class components for state, lifecycle methods, and logic reuse.

Hooks were introduced to solve three problems: (1) Hard to reuse stateful logic between components — required HOCs or render props which were complex. (2) Complex components became hard to understand — lifecycle logic was split across componentDidMount, componentDidUpdate, componentWillUnmount. (3) Classes confused developers — "this" binding, class syntax was harder for both people and build tools.

With hooks: function components can do everything class components could, and logic can be reused via custom hooks.

Q2: What is useState? How does it work?

Answer:

useState is a hook that adds a state variable to a function component.

You call useState(initialValue) and it returns an array of two things: the current state value, and a setter function to update it.

When you call the setter with a new value, React schedules a re-render of the component. On the next render, useState returns the new value.

The initial value is only used on the first render. After that, React ignores it and returns the stored value.

Important: state updates from useState are ASYNCHRONOUS. The state does not change immediately — it changes on the next render.

Q3: What is useEffect? What are its different use cases based on the dependency array?

Answer:

useEffect is a hook for running side effects after a component renders. Side effects include API calls, timers, event listeners, DOM manipulation.

Dependency array controls when it runs:

— No array: runs after EVERY render. Rarely useful.

— Empty array []: runs ONCE after the first render only. Like componentDidMount.

— With values [a, b]: runs after first render AND whenever a or b changes. Like componentDidUpdate for specific values.

The function returned from useEffect is the cleanup function — it runs before the next effect and on unmount. Use it to cancel timers, remove listeners, abort API calls.

Q4: What is the cleanup function in useEffect? Why is it important?

Answer:

The cleanup function is the function you return from inside useEffect. It runs in two situations: before the NEXT effect runs, and when the component unmounts.

It is important because without cleanup you get memory leaks and bugs. Example: if you set up a setInterval in useEffect without clearing it, the timer keeps running even after the component is removed from the screen.

What needs cleanup: intervals (clearInterval), timeouts (clearTimeout), event listeners (removeEventListener), API requests (AbortController.abort()), WebSocket connections (ws.close()), any subscription.

The pattern: const id = setInterval(fn, 1000); return () => clearInterval(id);

Q5: What is the difference between useMemo and useCallback?

Answer:

Both are performance hooks that memoize (cache) something to prevent unnecessary recalculation.

useMemo caches a COMPUTED VALUE — the RESULT of calling a function. It re-runs the function and returns a new value only when dependencies change.

useCallback caches a FUNCTION itself — the REFERENCE to a function. It returns the same function object across renders until dependencies change.

useMemo(() => computeExpensiveValue(a, b), [a, b]) — returns the value.

useCallback(() => doSomething(a, b), [a, b]) — returns the function.

When to use: useMemo for expensive calculations. useCallback for callbacks passed to React.memo child components to prevent unnecessary re-renders.

Q6: What is useRef? How is it different from useState?

Answer:

useRef returns an object { current: initialValue } that persists across renders. You can store any mutable value in ref.current.

Key difference: changing ref.current does NOT cause a re-render. This is the opposite of useState where every update triggers a re-render.

Two main use cases: (1) Accessing DOM elements directly — attach ref to a JSX element with the ref attribute, then access the DOM node via ref.current. (2) Storing a value that should persist across renders without causing re-renders — like a timer ID, a flag, or a previous value.

Rule: If you need to display the value in the UI — use useState. If you need to store it silently without affecting the UI — use useRef.

Q7: What is useReducer? When would you use it instead of useState?

Answer:

useReducer is an alternative to useState for managing more complex state. It takes a reducer function and initial state, and returns the current state and a dispatch function.

Instead of calling a setter directly, you dispatch action objects that describe what happened. The reducer function takes the current state and action, and returns the new state.

Use useReducer when: state has multiple sub-values that update together, the next state depends on the previous in complex ways, you have many different operations on the same state, or you want to centralise state logic for testability.

Simple rule: if you have 2-3 related useState calls and the update logic is getting complex, consider switching to useReducer.

useReducer is essentially what Redux does — same concept, local scope.

Q8: What are Custom Hooks? Why are they useful?

Answer:

Custom hooks are JavaScript functions whose names start with "use" and that can call other hooks inside them.

They let you extract stateful logic from a component and share it across multiple components. Before custom hooks, you needed patterns like HOCs (Higher Order Components) or render props — both complex.

Example: useFetch(url) extracts the entire fetch + loading + error logic. Any component that needs to fetch data just calls useFetch and gets data, loading, error back.

Important: custom hooks do NOT share state. Each component using the same custom hook gets its own independent copy of all the state and effects inside it.

Custom hooks are just functions — no JSX, no rendering. They return whatever the component needs.

Q9: What are the Rules of Hooks?

Answer:

There are two official rules of hooks.

Rule 1: Only call hooks at the TOP LEVEL. Never call hooks inside conditions (if), loops (for), or nested functions. This ensures hooks are always called in the same order on every render — which React relies on to correctly associate state with each hook.

Rule 2: Only call hooks from React FUNCTION COMPONENTS or CUSTOM HOOKS. Never from regular JavaScript functions, class components, or event callbacks.

Why these rules: React tracks hooks by their call order. If you call a hook inside an if statement, the order can change between renders — React would then associate the wrong state with the wrong hook, causing subtle bugs.

The eslint-plugin-react-hooks package automatically enforces these rules and warns you when you break them.

Q10: What is the difference between useEffect with empty deps [] vs no deps array?

Answer:

useEffect with empty array [] — runs ONCE, after the first render only. This is equivalent to componentDidMount in class components. Use this for one-time setup: fetch initial data, add a global event listener, initialise a library.

useEffect with NO array — runs after EVERY single render, every time the component re-renders for any reason. This is almost never what you want. It would re-run on every state change, every props change.

Practical: always use [] if you only want the effect to run once. Only omit the array if you genuinely need to run on every render — which is extremely rare.

Common interview trap: "Why is my API called infinitely?" — usually because the developer wrote useEffect(() => { fetch(...); setData(...) }) without [] — setData triggers re-render, which triggers useEffect, which calls setData again, infinite loop.

Q11: What is stale closure in useEffect? How do you fix it?

Answer:

A stale closure happens when an effect captures a variable value at the time of creation, and that variable later changes — but the effect still uses the old (stale) value.

Example: useEffect(() => { const id = setInterval(() => console.log(count), 1000); return () => clearInterval(id); }, []) — if count is not in the dependency array, the interval always logs the initial value of count (0) — never the updated value.

Fixes: (1) Add the variable to the dependency array — the effect re-runs with the latest value. (2) Use functional updates — setCount(prev => prev + 1) does not need count in dependencies. (3) Use useRef to store the latest value.

This is a common interview trap question. The answer shows you deeply understand closures and React's render cycle.

Q12: How do you prevent infinite loops in useEffect?

Answer:

Infinite loops in useEffect happen when: the effect updates state → re-render → effect runs again → updates state → re-render... forever.

Common causes: (1) No dependency array — effect runs every render. (2) Setting state inside effect without proper deps — setData(result) inside useEffect with [data] in deps. (3) Object or array in deps — new reference created every render even if content is same.

Fixes: (1) Use empty [] if the effect should run once. (2) Check your dependencies — are you updating something that is also in deps? (3) For objects in deps, compare specific properties: [user.id] instead of [user]. (4) useMemo or useCallback to stabilise references.

Debugging tip: if you see rapid re-renders or infinite network calls, check useEffect dependencies first.

Common Mistakes with Hooks

Mistake 1: Forgetting cleanup in useEffect

```
// ❌ WRONG — timer never cleared, runs after unmount
useEffect(() => {
  setInterval(() => setCount(c => c + 1), 1000);
}, []);

// ✅ CORRECT — always return cleanup
useEffect(() => {
  const id = setInterval(() => setCount(c => c + 1), 1000);
  return () => clearInterval(id);
}, []);
```

Mistake 2: Mutating state directly

```
// ❌ WRONG — push mutates the array, React does not detect change
const [items, setItems] = useState([]);
items.push("new item");  // mutates — no re-render!
setItems(items);          // same reference — React skips re-render

// ✅ CORRECT — create new array
setItems([...items, "new item"]);
```

Mistake 3: Creating infinite loop with useEffect

```
// ❌ WRONG — sets state inside effect with that state in deps
const [data, setData] = useState([]);
useEffect(() => {
  setData([...data, "new"]);  // updates data → re-runs effect → infinite!
}, [data]);

// ✅ CORRECT — use functional update or fix deps
useEffect(() => {
  setData(prev => [...prev, "new"]);  // does not need data in deps
}, []);  // runs once
```

Mistake 4: Using useMemo/useCallback everywhere unnecessarily

```
// ❌ WRONG — over-optimising simple values adds overhead
const doubled = useMemo(() => count * 2, [count]);  // count * 2 is instant!
const name = useMemo(() => "Shaaz", []);             // constant — no need

// ✅ CORRECT — only for genuinely expensive work
const filtered = useMemo(() =>
  hugeList.filter(item => item.matches(complexQuery)),
[hugeList, complexQuery]);  // this is actually expensive — useMemo helps
```

All Hooks — Quick Reference

Hook	Purpose	Returns	Key Rule
useState	State variable in component	[value, setter]	Never mutate directly — use setter
useEffect	Side effects after render	Nothing (optionally cleanup fn)	Return cleanup function always
useRef	Mutable value / DOM access	{ current: value }	Change does NOT trigger re-render

useContext	Consume context value	Context value	All consumers re-render on change
useReducer	Complex state with actions	[state, dispatch]	Reducer must be a pure function
useMemo	Cache expensive calculation	Computed value	Cache VALUE, not function
useCallback	Cache function reference	Memoized function	Cache FUNCTION, not value
Custom Hook	Reuse stateful logic	Whatever you return	Name must start with "use"

✅ **Remember**

useState — UI data that changes and must show on screen.
useEffect — side effects: API calls, timers, listeners. Always cleanup.
useRef — DOM access or storing values without triggering re-render.
useContext — consume global context, avoid prop drilling.
useReducer — complex state, multiple related updates, centralised logic.
useMemo — cache EXPENSIVE computed values. Use sparingly.
useCallback — cache FUNCTION references for React.memo children.
Custom Hook — extract and reuse logic. Starts with "use". Each call is independent.
Rules — only at top level, only in React functions or custom hooks.

End of Chapter 4 • Next: Chapter 5 — Component Lifecycle

Chapter 5: Component Lifecycle

Mount → Update → Unmount — explained with stories you will never forget

By Syed Shaaz Akhtar | Interview Focused Guide

Every living thing is born, lives its life, and eventually dies. A React component does the exact same thing. It appears on screen (birth), it can change and update (life), and it disappears from screen (death). This journey is called the **Component Lifecycle**.

This chapter explains lifecycle in THREE ways: with a real-life analogy, with class components (old way), and with hooks (modern way). You will be asked this topic in every interview from fresher to senior level. Read it like a story — and it will stay with you forever.

> 💡 **Tip**
>
> 🏪 THE RESTAURANT ANALOGY — remember this for the entire chapter:
>
> Imagine a component is a RESTAURANT BRANCH.
>
> — MOUNT = Opening Day. Staff hired, kitchen set up, doors opened for the first time.
>
> — UPDATE = Daily Operations. Menu changes, staff added, prices updated, orders come in.
>
> — UNMOUNT = Closing Day. Staff let go, kitchen cleaned, doors locked — branch never opens again.
>
> Every single lifecycle concept maps to this analogy. Keep it in your head.

Topic 1: The Three Phases of Every Component

Every React component goes through exactly three phases. No exceptions. Understanding WHAT happens in each phase is the foundation of everything else.

Phase	Plain English	Restaurant	When it happens
MOUNT	Component appears on screen for the first time	Opening Day — first ever customer walks in	Once — the very first render
UPDATE	Component re-renders because state or props changed	Daily Operations — prices change, new dishes added	Every time state or props change
UNMOUNT	Component is removed from screen permanently	Closing Day — shutters down forever	When component is hidden or parent is gone

> 📝 **Note**
>
> A component can go through the UPDATE phase hundreds of times in its life.
>
> But MOUNT happens exactly ONCE (first appearance) and UNMOUNT happens exactly ONCE (final disappearance).

> Some components never unmount — like a NavBar that is always visible. They only mount and update.

Topic 2: Class Component Lifecycle — The Old Way

Before React 16.8 (2019), there were no hooks. Lifecycle was handled using special methods in class components. You need to know these for interviews — especially when the interviewer asks you to compare old and new React.

The Three Most Important Class Lifecycle Methods

```
class UserDashboard extends React.Component {

  // ── PHASE 1: MOUNT ──────────────────────────────────────────────
  // Called once, AFTER the component is added to the DOM
  // Perfect for: API calls, setting up subscriptions, timers
  componentDidMount() {
    console.log("Restaurant OPENED — fetch initial data");
    fetch(`/api/users/${this.props.userId}`)
      .then(r => r.json())
      .then(user => this.setState({ user, loading: false }));
  }

  // ── PHASE 2: UPDATE ─────────────────────────────────────────────
  // Called after EVERY re-render (state/props change)
  // prevProps and prevState let you compare old vs new
  componentDidUpdate(prevProps, prevState) {
    // IMPORTANT: always compare! Without this check = infinite loop
    if (prevProps.userId !== this.props.userId) {
      console.log("userId changed — fetch new user");
      this.fetchUser(this.props.userId);
    }
  }

  // ── PHASE 3: UNMOUNT ────────────────────────────────────────────
  // Called just BEFORE component is removed from DOM
  // Perfect for: clearing timers, removing listeners, cancelling requests
  componentWillUnmount() {
    console.log("Restaurant CLOSING — clean everything up");
    clearInterval(this.timerID);
    window.removeEventListener("resize", this.handleResize);
  }

  render() {
    const { user, loading } = this.state;
    if (loading) return <p>Loading...</p>;
    return <h2>Welcome, {user.name}</h2>;
  }
}
```

✅ **Remember**

componentDidMount → runs ONCE after first render → "Doors just opened"

componentDidUpdate → runs after EVERY re-render → "Something changed in the restaurant"

componentWillUnmount → runs ONCE before removal → "Closing time — clean up"

Always put the comparison check (prevProps.x !== this.props.x) inside componentDidUpdate. Without it you will create infinite re-renders.

Topic 3: The Complete Class Lifecycle — All Methods

There are more lifecycle methods beyond the big three. The diagram below shows the complete journey. You do not need to memorise all of them — but you should know they exist and what they do.

Method	Phase	When Called	Typical Use
constructor()	Mount	Before anything, when class is created	Initialise state, bind methods
render()	Mount + Update	Every render — returns JSX	Return JSX — must be pure, no side effects
componentDidMount()	Mount	After first render, DOM is ready	API calls, subscriptions, timers
shouldComponentUpdate()	Update	Before re-render — return true/false	Performance: skip re-render if not needed
componentDidUpdate()	Update	After every re-render	React to prop/state changes, re-fetch data
componentWillUnmount()	Unmount	Just before removed from DOM	Cleanup: timers, listeners, cancel requests
componentDidCatch()	Error	When child component throws an error	Error boundaries — show fallback UI

💡 **Tip**

The interview question "Explain React lifecycle" usually expects you to cover these five in order:

constructor → render → componentDidMount → componentDidUpdate → componentWillUnmount

Know these five and you will answer 95% of lifecycle questions perfectly.

Topic 4: Modern Way — Hooks Replace Lifecycle Methods

With React 16.8, hooks replaced class lifecycle methods entirely. Modern React uses **useEffect** to handle all three phases. Here is the exact mapping — this is the most important table in this chapter.

Class Method	Hooks Equivalent	How to write it
componentDidMount	useEffect with empty []	useEffect(() => { doSomething(); }, [])
componentDidUpdate	useEffect with [dep]	useEffect(() => { doOnChange(); }, [dep])
componentWillUnmount	Cleanup function in useEffect	useEffect(() => { return () => cleanup(); }, [])
All three combined	One useEffect with [] + return	useEffect(() => { setup(); return () => cleanup(); }, [])

```
// ── CLASS WAY (old) ──────────────────────────────────────
class Timer extends React.Component {
  componentDidMount()    { this.id = setInterval(tick, 1000); }
  componentWillUnmount() { clearInterval(this.id); }
  render() { return <p>{this.state.time}</p>; }
}

// ── HOOKS WAY (modern) — same result, cleaner code ──────
function Timer() {
  const [time, setTime] = useState(0);

  useEffect(() => {
    // componentDidMount — runs once, after first render
    const id = setInterval(() => setTime(t => t + 1), 1000);

    // componentWillUnmount — runs when component is removed
    return () => clearInterval(id);

  }, []); // [] means: only mount + unmount, no updates

  return <p>Seconds: {time}</p>;
}
```

🎯 Interview Tip

Perfect answer to "How do lifecycle methods map to hooks?":

"componentDidMount = useEffect with empty dependency array []. It runs once after the first render."

"componentDidUpdate = useEffect with dependencies [dep]. It runs after first render AND every time dep changes."

"componentWillUnmount = the return function inside useEffect — the cleanup. It runs when the component is removed."

"All three can live inside a single useEffect. That is the beauty of hooks — related logic is together instead of split across three separate methods."

Topic 5: MOUNT Phase — Deep Dive

Mount is the birth of a component. It happens ONCE. Understanding the exact ORDER of events during mount is crucial — especially for interviews that ask "when does useEffect run relative to render?".

Exact Order of Events During Mount:

1. React **calls your function** (or class render method)
2. All **useState hooks initialise** with their initial values
3. Your **JSX is returned** — React builds the Virtual DOM
4. React **updates the real DOM** — HTML appears on screen
5. **Browser paints** the pixels — user can now SEE the component
6. **useEffect runs** — AFTER painting, not before

```
// MOUNT ORDER demonstration
function UserCard({ userId }) {

  console.log("1. Component function called");

  const [user, setUser] = useState(() => {
    console.log("2. useState initialises (lazy init runs here)");
    return null;
  });

  useEffect(() => {
    // This runs LAST — after DOM is painted
    console.log("6. useEffect runs — DOM is already visible");
    fetch(`/api/users/${userId}`)
      .then(r => r.json())
      .then(data => setUser(data)); // triggers UPDATE phase
  }, []);

  console.log("3. JSX about to be returned");

  return (
    // 4. React puts this in real DOM
    // 5. Browser paints it on screen
    <div>
      {user ? <h2>{user.name}</h2> : <p>Loading...</p>}
    </div>
  );
}

// Console output order:
// 1. Component function called
// 2. useState initialises
// 3. JSX about to be returned
// [React updates DOM and browser paints]
// 6. useEffect runs
```

📝 **Note**

useEffect runs AFTER the browser paints. Not before. Not during. AFTER.
This means the user sees the initial UI (loading state) BEFORE the API call starts.
This is why you always show a loading spinner — the data is not there yet on first render.
If you need code to run BEFORE paint (rare), use useLayoutEffect instead of useEffect.

Topic 6: UPDATE Phase — Deep Dive

Update is the most frequent phase. Every time state changes or a parent passes new props, the component goes through the Update phase. React re-runs your function and re-renders the UI with fresh data.

What Triggers an Update?

- **setState / useState setter called** — most common trigger
- **New props from parent** — parent re-renders with different data
- **Context value changes** — any component consuming a changed context
- **forceUpdate()** — class only, forces a re-render (avoid this)

```
// UPDATE PHASE — tracking what changes
function SearchPage() {
  const [query, setQuery]     = useState("");
  const [results, setResults] = useState([]);

  // This effect watches "query" — runs on MOUNT and every UPDATE of query
  useEffect(() => {
    console.log(`Query changed to: "${query}" — fetching results`);

    if (!query.trim()) {
      setResults([]);
      return;
    }

    const controller = new AbortController();

    fetch(`/api/search?q=${query}`, { signal: controller.signal })
      .then(r => r.json())
      .then(data => setResults(data))
      .catch(() => {});

    return () => controller.abort(); // cancel if query changes again

  }, [query]); // <-- only re-run when query changes

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      <ul>{results.map(r => <li key={r.id}>{r.title}</li>)}</ul>
    </div>
```

```
  );
}

// What happens when user types "react":
// 1. setQuery("r")   → UPDATE → useEffect runs → fetches "r"
// 2. setQuery("re")  → UPDATE → cleanup aborts "r" fetch → fetches "re"
// 3. setQuery("rea") → UPDATE → cleanup aborts "re" fetch → fetches "rea"
// 4. setQuery("reac")→ UPDATE → cleanup aborts "rea" fetch → fetches "reac"
// 5. setQuery("react")→UPDATE → cleanup aborts "reac" fetch → fetches "react"
```

✅ **Remember**

The cleanup function runs BEFORE the next effect — not just on unmount.

So with [query], the sequence is: query changes → cleanup old effect → run new effect.

This is why AbortController in the cleanup is so powerful — it cancels the previous API call when a new one starts.

Without cleanup here: all 5 API calls would finish in unpredictable order and the last one to arrive might not be "react".

Topic 7: UNMOUNT Phase — Deep Dive

Unmount is the death of a component. It happens when the component is removed from the UI — either because a condition turned false, or the parent removed it, or the user navigated to a different page.

What Causes Unmount?

- **Conditional rendering** — {isOpen && <Modal />} — when isOpen becomes false, Modal unmounts
- **Routing** — when the user navigates away, the old page component unmounts
- **Parent unmounts** — if a parent unmounts, all its children unmount too
- **Key changes** — giving a component a new key forces React to unmount + remount it

```
// UNMOUNT example — Modal that cleans up on close
function App() {
  const [isOpen, setIsOpen] = useState(false);

  return (
    <div>
      <button onClick={() => setIsOpen(true)}>Open Modal</button>
      {/* When isOpen becomes false, Modal UNMOUNTS */}
      {isOpen && <Modal onClose={() => setIsOpen(false)} />}
    </div>
  );
}

function Modal({ onClose }) {
  useEffect(() => {
    // MOUNT: disable background scroll when modal opens
```

```
    document.body.style.overflow = "hidden";
    console.log("Modal MOUNTED — scroll disabled");

    // UNMOUNT: re-enable scroll when modal closes
    return () => {
      document.body.style.overflow = "auto";
      console.log("Modal UNMOUNTED — scroll re-enabled");
    };
  }, []);

  return (
    <div className="modal">
      <p>Modal Content</p>
      <button onClick={onClose}>Close</button>
    </div>
  );
}

// What happens:
// User clicks "Open Modal"  → isOpen=true → Modal MOUNTS → scroll disabled
// User clicks "Close"       → isOpen=false → Modal UNMOUNTS → scroll re-enabled
```

💡 **Tip**

Never forget to clean up in unmount. Real bugs from missing cleanup:
— Timer fires after component is gone → "setState on unmounted component" warning
— Event listener piles up → multiple handlers firing for the same event
— API response arrives after unmount → state update on dead component
— Background scroll stays disabled → modal closes but page cannot scroll
The cleanup function prevents ALL of these. Always return it.

Topic 8: The Key Prop Trick — Force Remount

There is a very useful React trick that most developers do not know: giving a component a **different key** forces React to completely UNMOUNT it and MOUNT a brand new one.

Normally React reuses existing component instances when re-rendering. But when the key changes, React throws away the old instance and creates a completely fresh one — with fresh state, fresh effects, everything reset.

```
// PROBLEM: ChatWindow keeps old messages when switching users
function App() {
  const [selectedUserId, setSelectedUserId] = useState(1);

  return (
    <div>
      <UserList onSelect={setSelectedUserId} />
      {/* Without key — React REUSES the same ChatWindow instance */}
      {/* The component updates but never fully resets */}
```

```
      <ChatWindow userId={selectedUserId} />
    </div>
  );
}

// SOLUTION: Add key={selectedUserId}
function App() {
  const [selectedUserId, setSelectedUserId] = useState(1);

  return (
    <div>
      <UserList onSelect={setSelectedUserId} />
      {/* key changes when userId changes → React UNMOUNTS old + MOUNTS new */}
      {/* Fresh state, fresh effects — like opening a brand new chat window */}
      <ChatWindow key={selectedUserId} userId={selectedUserId} />
    </div>
  );
}

// Now switching users gives a completely fresh ChatWindow every time.
// Old messages gone, old scroll position gone, fresh fetch starts.
```

🎯 **Interview Tip**

This key trick is a hidden gem that impresses interviewers. Most developers never think to use it.

"If you need to completely reset a component when a prop changes — instead of writing complex useEffect logic to reset all state — just give the component key={propValue}. React will unmount the old instance and mount a fresh one automatically."

Common use cases: chat windows when switching contacts, form resets when navigating between items, profile pages when switching users.

Topic 9: React.StrictMode and Double Invocation

If you have used Create React App or Vite, you might have noticed your component function runs TWICE in development — even though you only rendered it once. This is **React.StrictMode** at work.

In development mode, StrictMode intentionally calls your component function and effects TWICE to help you find bugs. In production, it runs normally once.

```
// main.jsx — StrictMode wraps your app
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

// What StrictMode does in DEVELOPMENT:
// 1. Calls your component function TWICE
//    → helps find components with side effects in render
//
```

```
// 2. Mounts → runs effects → UNMOUNTS → REMOUNTS → runs effects again
//    → helps find effects missing cleanup functions
//
// 3. Warns about deprecated API usage

// Example: if you see this in console:
// "Subscribed!" → "Unsubscribed!" → "Subscribed!"
// That is StrictMode doing mount → unmount → remount
// If your cleanup is correct, this is fine
// If cleanup is missing — StrictMode exposes the bug early
```

📝 **Note**

StrictMode does NOT affect production builds. Zero performance impact in production.

If your component mounts twice in development — that is StrictMode, not a bug.

If your app breaks when mounted twice — that IS a bug that StrictMode just helped you find.

If an API call fires twice in development — add an AbortController cleanup. StrictMode revealed a missing cleanup.

Topic 10: useLayoutEffect — The Synchronous sibling of useEffect

useEffect runs **after** the browser paints. But sometimes you need to run code **before** the browser paints — like measuring a DOM element and repositioning a tooltip. That is what **useLayoutEffect** is for.

Aspect	useEffect	useLayoutEffect
When it runs	AFTER browser paints — async	BEFORE browser paints — synchronous
Blocks paint?	No — paint happens first	Yes — paint waits for this to finish
Use for	API calls, subscriptions, timers	DOM measurements, tooltip positioning, animations
Performance	Better — does not delay paint	Can cause visual lag if heavy work done here
Common usage	95% of cases	5% of cases — DOM-read-then-write scenarios

```
// useLayoutEffect — tooltip that must know its position before paint
import { useRef, useState, useLayoutEffect } from "react";

function Tooltip({ text, children }) {
  const [position, setPosition] = useState({ top: 0, left: 0 });
  const triggerRef = useRef(null);

  useLayoutEffect(() => {
    // Measure the DOM element BEFORE browser paints
```

```
    // If we used useEffect, tooltip would flash in wrong position first
    const rect = triggerRef.current.getBoundingClientRect();
    setPosition({
      top: rect.bottom + window.scrollY,
      left: rect.left + window.scrollX,
    });
  });

  return (
    <span ref={triggerRef}>
      {children}
      <div style={{ position: "absolute", top: position.top, left: position.left
}}>
        {text}
      </div>
    </span>
  );
}

// Rule: default to useEffect. Only switch to useLayoutEffect
//       when you see a visual flicker that useEffect causes.
```

✅ Remember

useEffect = async, after paint. useLayoutEffect = sync, before paint.

Use useLayoutEffect ONLY when you need to read DOM measurements and apply changes before the user sees anything.

If unsure: start with useEffect. Only switch if you see a visible flicker.

useLayoutEffect with heavy work = blocks rendering = bad performance. Keep it fast.

Topic 11: shouldComponentUpdate and React.PureComponent (Performance)

During the Update phase, React re-renders a component even if the new props are identical to the old ones. For class components, you can control this with **shouldComponentUpdate** and **React.PureComponent**.

```
// shouldComponentUpdate — manually control re-render
class ProductCard extends React.Component {
  shouldComponentUpdate(nextProps, nextState) {
    // Return TRUE to allow re-render, FALSE to skip it
    // Only re-render if price or name actually changed
    return (
      nextProps.price !== this.props.price ||
      nextProps.name  !== this.props.name
    );
  }

  render() {
```

```
    return <div>{this.props.name}: ₹{this.props.price}</div>;
  }
}

// PureComponent — automatic shallow comparison (easier)
// Automatically implements shouldComponentUpdate with shallow compare
class ProductCard extends React.PureComponent {
  render() {
    return <div>{this.props.name}: ₹{this.props.price}</div>;
  }
}

// HOOKS EQUIVALENT — React.memo
const ProductCard = React.memo(function ProductCard({ name, price }) {
  return <div>{name}: ₹{price}</div>;
});
// React.memo = PureComponent for function components
// Skips re-render if all props are shallowly equal
```

Class (Old Way)	Hooks (Modern Way)	What it does
shouldComponentUpdate()	React.memo + custom compare	Skip re-render if props unchanged
React.PureComponent	React.memo	Auto shallow-compare props
componentDidMount()	useEffect(() => {}, [])	Run once after first render
componentDidUpdate()	useEffect(() => {}, [dep])	Run when dep changes
componentWillUnmount()	return () => {} in useEffect	Cleanup on removal

Topic 12: Complete Lifecycle Diagram — The Full Picture

Here is the complete lifecycle flow from birth to death, combining everything you have learned. This is what you draw on a whiteboard in interviews.

```
// ═══════════════════════════════════════════════════
//  COMPLETE REACT COMPONENT LIFECYCLE
// ═══════════════════════════════════════════════════

//   ┌──────────────────────────────────────────┐
//   │              MOUNT PHASE                 │
//   │  1. Component function called            │
//   │  2. useState initialises                 │
//   │  3. JSX returned → Virtual DOM built     │
//   │  4. Real DOM updated → HTML on screen    │
//   │  5. Browser paints pixels                │
//   │  6. useEffect(fn, []) runs               │
```

```
//   └──────────────────────────┬──────────────────────────┘
//                              │
//                              ▼ (state/props change)
//   ┌──────────────────────────────────────────────────────┐
//   │                UPDATE PHASE                          │
//   │  1. Trigger: setState / new props / context          │
//   │  2. Component function called again                  │
//   │  3. New Virtual DOM built                            │
//   │  4. Diffing: old vs new Virtual DOM                  │
//   │  5. Only changed nodes updated in real DOM           │
//   │  6. Browser repaints changed area                    │
//   │  7. Previous useEffect cleanup runs                  │
//   │  8. useEffect(fn, [dep]) runs with new dep           │
//   └──────────────────────────┬──────────────────────────┘
//                              │
//                              │
//     (can loop many times|through update phase)
//                              │
//                              ▼ (component removed)
//   ┌──────────────────────────────────────────────────────┐
//   │                UNMOUNT PHASE                         │
//   │  1. Trigger: conditional false / navigate            │
//   │  2. Cleanup function from useEffect runs             │
//   │  3. Component removed from real DOM                  │
//   │  4. All state and refs destroyed                     │
//   └──────────────────────────────────────────────────────┘
```

💡 Tip

THE RESTAURANT SUMMARY to remember everything:

MOUNT = Opening Day. Hire staff (useState), open doors (render), first customers served (DOM painted), manager does opening checklist (useEffect []).

UPDATE = Daily Ops. New menu item added (setState) → kitchen updates (re-render) → only changed dishes remade (Virtual DOM diff) → delivery team reacts (useEffect [dep]).

UNMOUNT = Closing Day. Manager does closing checklist (cleanup fn) → staff let go (state destroyed) → keys handed back (DOM removed).

Chapter 5: Interview Questions with Full Answers

Q1: What are the lifecycle phases of a React component?

Answer:

A React component has three lifecycle phases.

Mount — the component appears on screen for the first time. React calls your function, initialises state, builds the Virtual DOM, updates the real DOM, browser paints, then useEffect with [] runs. This happens exactly once.

Update — the component re-renders because state or props changed. React re-runs your function, builds a new Virtual DOM, diffs it with the old one, updates only changed DOM nodes,

runs cleanup from previous effect, then runs useEffect with matching deps. This can happen many times.

Unmount — the component is removed from the screen. The cleanup function returned from useEffect runs, then the component is destroyed from the DOM and all its state is gone. Happens exactly once.

Q2: What is componentDidMount? What is its hooks equivalent?

Answer:

componentDidMount is a class lifecycle method that runs once, after the component is first rendered to the DOM.

It is the perfect place for: API calls to fetch initial data, setting up event listeners, initialising third-party libraries, starting timers.

Its hooks equivalent is useEffect with an empty dependency array: useEffect(() => { doSomething(); }, [])

The empty array [] tells React "only run this after the first render, never again". This maps exactly to componentDidMount.

Important: unlike componentDidMount which runs synchronously, useEffect always runs asynchronously — after the browser has painted the screen.

Q3: What is componentWillUnmount? What is its hooks equivalent?

Answer:

componentWillUnmount is a class lifecycle method that runs just before the component is removed from the DOM. It is the cleanup method — used to prevent memory leaks.

What you do here: clear intervals and timeouts, remove event listeners, cancel API requests, close WebSocket connections, unsubscribe from any subscriptions.

Its hooks equivalent is the return function inside useEffect: useEffect(() => { setup(); return () => cleanup(); }, [])

The function you return from useEffect is the cleanup function. It runs both when the component unmounts AND before the next effect runs on update.

Missing this is one of the most common sources of memory leaks in React apps.

Q4: What is the order of execution during a component's first render?

Answer:

The order is: (1) Component function is called. (2) useState hooks initialise with their initial values. (3) JSX is returned and React builds the Virtual DOM. (4) React compares with the real DOM and injects the HTML. (5) Browser paints the pixels — user can now see the component. (6) useEffect with [] runs — after painting.

The key point interviewers love: useEffect runs AFTER the browser paints, not before.

This means on first render, the user sees the loading state (null data) first. Then useEffect fires, data is fetched, setState is called, UPDATE phase happens, and the UI updates with real data.

If you need code before paint: use useLayoutEffect. But 95% of cases useEffect is correct.

Q5: What is componentDidUpdate? When do you need the comparison check inside it?

Answer:

componentDidUpdate runs after every re-render — after state changes or prop changes cause a re-render.

It receives prevProps and prevState as arguments, letting you compare old and new values.

You MUST include a comparison check: if (prevProps.userId !== this.props.userId) { ... }

Without the check: componentDidUpdate fires → you fetch data → setData is called → component re-renders → componentDidUpdate fires again → infinite loop.

Hooks equivalent: useEffect(() => { fetchUser(userId); }, [userId]) — React handles the comparison automatically with the dependency array.

Q6: Why does a component in React.StrictMode mount twice in development?

Answer:

In development mode, React.StrictMode intentionally mounts, unmounts, and remounts every component.

The sequence is: Mount → run effects → Unmount (cleanup) → Remount → run effects again.

Why? To help you find bugs — specifically, effects that are missing cleanup functions. If your component breaks when mounted twice, it means your effects are not pure and you have a missing cleanup.

This only happens in development. Production builds mount once normally. StrictMode has zero performance impact in production.

If you see API calls firing twice in development — it is StrictMode exposing a missing AbortController cleanup.

Q7: What is the difference between useEffect and useLayoutEffect?

Answer:

useEffect runs AFTER the browser paints — asynchronously. The DOM is updated, the user sees the new UI, and then useEffect fires.

useLayoutEffect runs BEFORE the browser paints — synchronously. React updates the DOM but holds off painting until useLayoutEffect finishes.

Use useLayoutEffect when: you need to read DOM measurements (getBoundingClientRect) and apply changes immediately, to prevent a visual flicker.

Example: positioning a tooltip. If you use useEffect, the tooltip might flash in the wrong position for one frame. useLayoutEffect prevents this by repositioning before the user sees anything.

Rule: always start with useEffect. Switch to useLayoutEffect only if you see a visible flicker caused by DOM reads.

Q8: What is the key prop trick for forcing a remount?

Answer:

Giving a component a new key forces React to completely unmount the old instance and mount a brand new one.

React uses the key to identify component instances. When the key changes, React treats it as a completely different component — destroying the old one and creating a fresh one from scratch.

Example: <ChatWindow key={userId} userId={userId} /> — when userId changes, the key changes, ChatWindow unmounts and a fresh one mounts with clean state.

This is cleaner than writing complex useEffect logic to reset all state manually when a prop changes.

Use cases: chat windows when switching users, forms when navigating between items, profile pages, video players that should restart on source change.

Q9: What is shouldComponentUpdate? What is its hooks equivalent?

Answer:

shouldComponentUpdate is a class lifecycle method that lets you control whether a re-render should happen. It receives nextProps and nextState and should return true (allow re-render) or false (skip re-render).

It is used for performance — if you know the component does not need to update because the relevant props have not changed, you return false and React skips the expensive re-render.

React.PureComponent automatically implements shouldComponentUpdate with a shallow comparison of all props and state.

The hooks equivalent is React.memo — it wraps a function component and performs a shallow comparison of props. If props have not changed, React skips re-rendering the component.

For custom comparison logic: React.memo(Component, (prevProps, nextProps) => prevProps.id === nextProps.id)

Q10: What happens during the cleanup function — does it only run on unmount?

Answer:

No — the cleanup function runs in TWO situations, not just unmount.

Situation 1: Before the NEXT effect runs. If useEffect has a dependency that changes, React runs the cleanup from the previous effect first, then runs the new effect. This ensures you clean up after each run.

Situation 2: When the component unmounts. The final cleanup runs when the component is removed.

Example: useEffect(() => { const id = setInterval(fn, 1000); return () => clearInterval(id); }, [count]) — every time count changes, the old interval is cleared and a new one starts. On unmount, the last interval is also cleared.

This dual-cleanup behaviour is what makes useEffect so powerful and why it replaces both componentDidUpdate AND componentWillUnmount.

Q11: What happens to child components when a parent unmounts?

Answer:

When a parent component unmounts, ALL its child components unmount too — automatically, in order from deepest child to the parent.

The cleanup functions run bottom-up — deepest children first, then their parents, then the root of the unmounting tree.

This is important for debugging: if you navigate away from a page and your app throws "setState on unmounted component" — it is usually because an async operation (like a fetch) finished after the parent was already unmounted.

Fix: use AbortController in your fetch effects to cancel in-flight requests when the component tree unmounts.

You do not need to manually unmount children — React handles the entire tree automatically.

Q12: How would you explain React lifecycle to someone who has never seen it?

Answer:

A React component is like a restaurant branch.

MOUNT is the opening day. The branch sets up everything for the first time — hiring staff (initialising state), opening the doors (rendering to DOM), and the manager does the opening checklist (useEffect runs).

UPDATE is daily operations. Whenever something changes — a new menu item, a price change — the kitchen updates (re-render) but only the changed dishes are remade (Virtual DOM diff, minimal DOM updates). The daily review happens (useEffect with deps).

UNMOUNT is closing day. The manager does a closing checklist (cleanup function), hands back the keys (state destroyed), and the branch is permanently closed (removed from DOM).

I use this analogy because it maps perfectly: opening checklist = componentDidMount = useEffect([]), daily review = componentDidUpdate = useEffect([dep]), closing checklist = componentWillUnmount = cleanup function.

Common Mistakes in Lifecycle

Mistake 1: Missing cleanup — the most common memory leak

```
// ❌ WRONG — timer never stops when component unmounts
useEffect(() => {
  setInterval(() => setCount(c => c + 1), 1000);
}, []);

// ✅ CORRECT — timer stops on unmount
useEffect(() => {
  const id = setInterval(() => setCount(c => c + 1), 1000);
  return () => clearInterval(id); // cleanup = unmount
}, []);
```

Mistake 2: Missing dependency causing stale values (stale closure)

```
// ❌ WRONG — count is stale inside the interval (always 0)
useEffect(() => {
  const id = setInterval(() => {
    console.log(count); // always prints 0 — stale closure!
  }, 1000);
```

```
  return () => clearInterval(id);
}, []); // count missing from deps

// ✅ CORRECT option 1 — add count to deps
useEffect(() => {
  const id = setInterval(() => console.log(count), 1000);
  return () => clearInterval(id);
}, [count]); // re-creates interval when count changes

// ✅ CORRECT option 2 — use functional update (no dep needed)
useEffect(() => {
  const id = setInterval(() => setCount(prev => prev + 1), 1000);
  return () => clearInterval(id);
}, []); // functional update does not need count in deps
```

Mistake 3: No comparison in componentDidUpdate → infinite loop

```
// ❌ WRONG — infinite loop: update triggers fetch, fetch sets state, state
triggers update...
componentDidUpdate() {
  this.fetchUser(); // no check → fires every update → infinite!
}

// ✅ CORRECT — only fetch when userId actually changed
componentDidUpdate(prevProps) {
  if (prevProps.userId !== this.props.userId) {
    this.fetchUser(this.props.userId);
  }
}
```

Quick Reference — Chapter 5 Summary

Lifecycle	Class Method	Hooks Equivalent	Purpose
Mount	componentDidMount()	useEffect(() => {}, [])	First render: fetch data, setup listeners
Update	componentDidUpdate()	useEffect(() => {}, [dep])	React to changes: re-fetch, sync
Unmount	componentWillUnmount()	return () => {} in useEffect	Cleanup: timers, listeners, requests
Skip re-render	shouldComponentUpdate()	React.memo	Performance: avoid unnecessary renders

Before paint	—	useLayoutEffect	DOM measure before user sees
Force remount	—	key={value}	Reset component when key changes

✅ **Remember**

MOUNT = born once. useEffect(fn, []). Set up everything.
UPDATE = lives many times. useEffect(fn, [dep]). React to changes.
UNMOUNT = dies once. return () => cleanup(). Tear everything down.
Cleanup runs TWICE — before next effect AND on unmount.
StrictMode = mounts twice in dev. Normal. Find your cleanup bugs.
key trick = change key → force unmount + remount. Reset all state.
useLayoutEffect = before paint. Only for DOM measurements.
Restaurant analogy — Opening Day → Daily Ops → Closing Day.

End of Chapter 5 • Next: Chapter 6 — Event Handling & Forms

Chapter 6: Event Handling & Forms

Synthetic Events, Controlled Forms, Validation — Real Project Skills

By Syed Shaaz Akhtar | Interview Focused Guide

Forms and events are things you use in EVERY real React project — login forms, signup forms, search bars, filters, checkout pages. If you cannot build a proper form in React, you cannot pass a practical interview round. This chapter covers everything from how React handles events to building a fully validated multi-field form from scratch.

> 💡 **Tip**
> 🍔 THE ANALOGY for this entire chapter:
> Think of a React form like a RESTAURANT ORDER SYSTEM.
> — Events = Customer actions (clicks, keystrokes, selections)
> — Controlled form = Waiter writes down every change — nothing happens without the waiter knowing
> — Uncontrolled form = Customer fills form themselves — waiter only reads it at submission
> — Validation = Kitchen checks: "Is this order valid before we cook it?"
> — onSubmit = Order is placed — send it to the kitchen (server)

Topic 1: How React Events Work — Synthetic Events

In plain JavaScript you add events like this: element.addEventListener("click", handler). In React, you write events directly on JSX elements as props: onClick={handler}. But under the hood, React does something much smarter.

What is a Synthetic Event?

React does NOT attach individual event listeners to each DOM element. Instead, React attaches ONE single listener at the ROOT of your app and catches all events through a technique called **Event Delegation**. When your button is clicked, the event bubbles up to the root, React figures out which component handled it, and calls your handler with a **SyntheticEvent** object.

A **SyntheticEvent** is React's wrapper around the browser's native event. It normalises differences between browsers so your code works identically in Chrome, Firefox, Safari, and IE. The SyntheticEvent has the same API as a native event — e.target, e.preventDefault(), e.stopPropagation() — it just works consistently everywhere.

```
// Native JS — different syntax, browser inconsistencies
document.getElementById("btn").addEventListener("click", function(event) {
  console.log(event.target.value);
});

// React — camelCase, on JSX prop, SyntheticEvent passed automatically
function Button() {
  function handleClick(e) {
```

```
    // e is a SyntheticEvent — same API as native, works in all browsers
    console.log(e.target);           // the clicked element
    console.log(e.type);             // "click"
    e.preventDefault();              // works in all browsers
    e.stopPropagation();             // stops event bubbling
  }

  return <button onClick={handleClick}>Click Me</button>;
}
```

📝 **Note**

React events are camelCase: onClick, onChange, onSubmit, onFocus, onBlur, onKeyDown, onMouseEnter.

Native HTML uses lowercase: onclick, onchange, onsubmit — these do NOT work in React JSX.

SyntheticEvent is pooled in older React versions — e.persist() was needed to use it asynchronously. In React 17+, this pooling was removed. No longer needed.

Topic 2: Common Event Types — The Full List

These are the events you will use in every real project. Know them by heart.

Event Prop	Fires When	Most Common Use
onClick	Element is clicked	Buttons, links, any clickable element
onChange	Input value changes (every keystroke)	Text inputs, select, checkbox, radio
onSubmit	Form is submitted (Enter or submit button)	Form submission — always on <form> tag
onFocus	Element gains focus (user clicks/tabs into it)	Show dropdown, highlight input
onBlur	Element loses focus (user clicks away)	Validate field when user leaves it
onKeyDown	A key is pressed down	Keyboard shortcuts, detect Enter key
onKeyUp	A key is released	Trigger search after typing stops
onMouseEnter	Mouse enters element area	Show tooltip, highlight on hover
onMouseLeave	Mouse leaves element area	Hide tooltip, remove highlight
onScroll	Element is scrolled	Infinite scroll, sticky header

Topic 3: Writing Event Handlers — 4 Ways

There are four ways to write event handlers in React. Know all four because you will see them all in real code and interviews.

```
// WAY 1: Inline arrow function — simple, fine for small handlers
<button onClick={() => console.log("clicked")}>Click</button>

// WAY 2: Inline arrow function with argument — passing extra data
<button onClick={() => handleDelete(product.id)}>Delete</button>

// WAY 3: Named function defined inside component
function Form() {
  function handleSubmit(e) {
    e.preventDefault();
    console.log("Form submitted");
  }
  return <form onSubmit={handleSubmit}>...</form>;
}

// WAY 4: Named function defined outside component (if no state needed)
function handleButtonClick(e) {
  console.log("Clicked:", e.target.textContent);
}
function App() {
  return <button onClick={handleButtonClick}>Click</button>;
}

// ❌ COMMON MISTAKE — calling the function instead of passing it
<button onClick={handleClick()}>Wrong</button>  // runs on render!
<button onClick={handleClick}>Correct</button>  // runs on click
```

> ✅ **Remember**
>
> Pass the function REFERENCE — not the function CALL.
>
> onClick={handleClick} ← correct — passes the function
>
> onClick={handleClick()} ← wrong — calls it immediately on render
>
> When you need to pass arguments: onClick={() => handleDelete(id)} — the outer arrow function is the handler, it calls handleDelete(id) only when clicked.

Topic 4: Event Object — What's Inside

Every event handler receives the SyntheticEvent object as its first argument. Knowing what is inside it helps you write real forms and event handlers confidently.

```
function InputDemo() {
  function handleChange(e) {
    // e.target — the DOM element that triggered the event
    console.log(e.target);            // <input> element
```

```
    console.log(e.target.value);      // current value typed
    console.log(e.target.name);       // name attribute of input
    console.log(e.target.type);       // "text", "checkbox", "radio" etc
    console.log(e.target.checked);    // for checkboxes: true/false

    // e itself
    console.log(e.type);              // "change", "click", "submit" etc
    console.log(e.preventDefault);    // function to stop default action
    console.log(e.stopPropagation);   // function to stop bubbling
    console.log(e.nativeEvent);       // original browser event
  }

  function handleKeyDown(e) {
    console.log(e.key);               // "Enter", "Escape", "a", "ArrowUp"
    console.log(e.keyCode);           // 13 for Enter, 27 for Escape
    console.log(e.ctrlKey);           // true if Ctrl held down
    console.log(e.shiftKey);          // true if Shift held down

    if (e.key === "Enter") console.log("Enter pressed!");
    if (e.key === "Escape") console.log("Escape pressed!");
  }

  return (
    <input
      onChange={handleChange}
      onKeyDown={handleKeyDown}
    />
  );
}
```

Topic 5: Controlled vs Uncontrolled Forms

This is the most asked forms question in React interviews. You must understand both — and know WHEN to use which.

Controlled Form — React Owns the Data

In a controlled form, every input's value is stored in React state. The input's value is set by state, and every keystroke calls onChange which updates the state. React is the single source of truth.

It's like a waiter who writes down your order change by change. Nothing happens without the waiter knowing.

```
// CONTROLLED FORM — React owns every character the user types
function LoginForm() {
  const [email, setEmail]       = useState("");
  const [password, setPassword] = useState("");

  function handleSubmit(e) {
    e.preventDefault(); // prevents browser page reload
```

```
    console.log("Submitting:", { email, password });
    // React already has the values — no need to read from DOM
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}                           // controlled by state
        onChange={e => setEmail(e.target.value)} // updates state on each key
        placeholder="Email"
      />
      <input
        type="password"
        value={password}
        onChange={e => setPassword(e.target.value)}
        placeholder="Password"
      />
      <button type="submit">Login</button>
    </form>
  );
}

// At any point, email and password in state reflect exactly
// what is visible in the input boxes. Always in sync.
```

Uncontrolled Form — DOM Owns the Data

In an uncontrolled form, the DOM manages the input value itself. You use a **ref** to read the value only when you need it — usually on submit. React does not know what the user is typing until you ask.

It's like giving the customer a paper form to fill themselves. You only read it when they hand it back.

```
// UNCONTROLLED FORM — DOM owns data, ref reads it on demand
import { useRef } from "react";

function LoginForm() {
  const emailRef    = useRef(null);
  const passwordRef = useRef(null);

  function handleSubmit(e) {
    e.preventDefault();
    // Read from DOM only when needed — at submit time
    const email    = emailRef.current.value;
    const password = passwordRef.current.value;
    console.log("Submitting:", { email, password });
  }

  return (
    <form onSubmit={handleSubmit}>
```

```
      <input type="email"    ref={emailRef}    placeholder="Email" />
      <input type="password" ref={passwordRef} placeholder="Password" />
      <button type="submit">Login</button>
    </form>
  );
}
// Notice: NO useState, NO onChange, NO value prop
// React has NO idea what is typed until handleSubmit runs
```

Aspect	Controlled	Uncontrolled
Data lives in	React STATE	DOM itself
Access value	Directly from state variable	Read via ref.current.value on submit
Real-time access	Yes — every keystroke updates state	No — only when you explicitly read ref
Validation	Easy — validate on every change	Only possible on submit
Re-renders	Every keystroke causes re-render	No re-renders while typing
When to use	Most cases — validation, live feedback	Simple forms, file inputs, integrations
Code amount	More (value + onChange for each input)	Less (just a ref per input)

🎯 **Interview Tip**

Perfect answer to "What is a controlled component in React?":

"A controlled component is a form input whose value is controlled by React state. The current value comes from state via the value prop, and every change is reported back via onChange which updates the state. React becomes the single source of truth for the input's value at all times."

"I prefer controlled components for most forms because they allow real-time validation, conditional disabling of the submit button, and instant access to form data without reading the DOM."

Topic 6: Handling Multiple Inputs with One Handler

In a real form with 5-10 fields, creating a separate onChange handler for each input is repetitive. There is a clean pattern using the input's **name** attribute to update the right field with one generic handler.

```
// GENERIC HANDLER — one function handles all inputs
function RegistrationForm() {
  const [formData, setFormData] = useState({
    firstName: "",
    lastName:  "",
    email:     "",
    phone:     "",
    city:      "",
```

```
  });

  // ONE handler for ALL inputs
  // Uses e.target.name to know WHICH field changed
  // Uses e.target.value to know the NEW value
  function handleChange(e) {
    const { name, value } = e.target;
    setFormData(prev => ({
      ...prev,        // keep all other fields as-is
      [name]: value   // update only the changed field
      // [name] is computed property — uses the variable "name" as the key
    }));
  }

  function handleSubmit(e) {
    e.preventDefault();
    console.log("Form data:", formData);
  }

  return (
    <form onSubmit={handleSubmit}>
      {/* Each input has a "name" that matches the state key */}
      <input name="firstName" value={formData.firstName} onChange={handleChange}
placeholder="First Name" />
      <input name="lastName"  value={formData.lastName}  onChange={handleChange}
placeholder="Last Name" />
      <input name="email"     value={formData.email}     onChange={handleChange}
placeholder="Email" type="email" />
      <input name="phone"     value={formData.phone}     onChange={handleChange}
placeholder="Phone" />
      <input name="city"      value={formData.city}      onChange={handleChange}
placeholder="City" />
      <button type="submit">Register</button>
    </form>
  );
}
```

> 💡 **Tip**
>
> The [name]: value pattern uses JavaScript computed property names.
>
> If name = "email" and value = "shaaz@gmail.com", then { [name]: value } is { email: "shaaz@gmail.com" }.
>
> This one pattern eliminates 5 separate onChange handlers. Use it in every multi-field form.
>
> For checkboxes: use e.target.checked instead of e.target.value for booleans.

Topic 7: Handling Special Input Types

Checkbox

```
function PreferencesForm() {
```

```
  const [prefs, setPrefs] = useState({
    newsletter:    false,
    notifications: true,
    darkMode:      false,
  });

  function handleCheckbox(e) {
    const { name, checked } = e.target; // use "checked" not "value"
    setPrefs(prev => ({ ...prev, [name]: checked }));
  }

  return (
    <div>
      <label>
        <input type="checkbox" name="newsletter" checked={prefs.newsletter}
onChange={handleCheckbox} />
        Subscribe to newsletter
      </label>
      <label>
        <input type="checkbox" name="notifications"
checked={prefs.notifications} onChange={handleCheckbox} />
        Enable notifications
      </label>
    </div>
  );
}
```

Radio Buttons

```
function GenderSelect() {
  const [gender, setGender] = useState("");

  return (
    <div>
      {["Male", "Female", "Other"].map(option => (
        <label key={option}>
          <input
            type="radio"
            name="gender"
            value={option}
            checked={gender === option}      // controlled: checked if matches
state
            onChange={e => setGender(e.target.value)}
          />
          {option}
        </label>
      ))}
      <p>Selected: {gender || "None"}</p>
    </div>
  );
```

```
}
```

Select Dropdown

```
function CitySelect() {
  const [city, setCity] = useState("");

  return (
    <select value={city} onChange={e => setCity(e.target.value)}>
      <option value="">-- Select City --</option>
      <option value="delhi">Delhi</option>
      <option value="mumbai">Mumbai</option>
      <option value="bangalore">Bangalore</option>
    </select>
  );
}
```

File Input — The Exception (Always Uncontrolled)

```
// File inputs CANNOT be controlled — browser security restriction
// You cannot set the value of a file input programmatically
// Always use uncontrolled (ref) for file inputs

function FileUpload() {
  const fileRef = useRef(null);
  const [fileName, setFileName] = useState("");

  function handleFileChange(e) {
    const file = e.target.files[0];
    if (file) setFileName(file.name);
  }

  function handleSubmit() {
    const file = fileRef.current.files[0];
    const formData = new FormData();
    formData.append("profile_photo", file);
    // fetch("/api/upload", { method: "POST", body: formData })
  }

  return (
    <div>
      <input type="file" ref={fileRef} onChange={handleFileChange}
accept="image/*" />
      {fileName && <p>Selected: {fileName}</p>}
      <button onClick={handleSubmit}>Upload</button>
    </div>
  );
}
```

Topic 8: Form Validation — Complete Pattern

Validation is what separates a production form from a demo form. There are three types of validation timing — know all three.

Validation Type	When it Runs	Best For	UX
On Change	Every keystroke	Showing live character count, password strength	Immediate feedback — can be annoying if too early
On Blur	When field loses focus	Validating each field after user finishes it	Best UX — validate when user is done with a field
On Submit	When form is submitted	Final check before sending to server	Shows all errors at once — good backup

Complete Form with On-Blur Validation

```
// COMPLETE REGISTRATION FORM with full validation
function RegistrationForm() {
  const [formData, setFormData] = useState({
    name: "", email: "", password: "", confirmPassword: ""
  });

  // Tracks which fields have been "touched" (user has blurred out of them)
  const [touched, setTouched] = useState({
    name: false, email: false, password: false, confirmPassword: false
  });

  // Derive errors from formData — no separate errors state needed
  function getErrors() {
    const errors = {};

    if (!formData.name.trim())
      errors.name = "Name is required";
    else if (formData.name.trim().length < 2)
      errors.name = "Name must be at least 2 characters";

    if (!formData.email.trim())
      errors.email = "Email is required";
    else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email))
      errors.email = "Enter a valid email address";

    if (!formData.password)
```

```
      errors.password = "Password is required";
    else if (formData.password.length < 8)
      errors.password = "Password must be at least 8 characters";
    else if (!/[A-Z]/.test(formData.password))
      errors.password = "Password must have at least one uppercase letter";

    if (!formData.confirmPassword)
      errors.confirmPassword = "Please confirm your password";
    else if (formData.confirmPassword !== formData.password)
      errors.confirmPassword = "Passwords do not match";

    return errors;
  }

  const errors = getErrors(); // derived — recalculated every render
  const isFormValid = Object.keys(errors).length === 0;

  function handleChange(e) {
    const { name, value } = e.target;
    setFormData(prev => ({ ...prev, [name]: value }));
  }

  function handleBlur(e) {
    const { name } = e.target;
    setTouched(prev => ({ ...prev, [name]: true }));
  }

  function handleSubmit(e) {
    e.preventDefault();
    // Touch all fields on submit to show all errors
    setTouched({ name: true, email: true, password: true, confirmPassword:
true });
    if (isFormValid) {
      console.log("Form submitted:", formData);
      // Call API here
    }
  }

  // Helper — only show error if field was touched
  function showError(field) {
    return touched[field] && errors[field];
  }

  return (
    <form onSubmit={handleSubmit}>

      <div>
        <input
          name="name"
          value={formData.name}
          onChange={handleChange}
```

```
          onBlur={handleBlur}
          placeholder="Full Name"
          className={showError("name") ? "input-error" : ""}
        />
        {showError("name") && <p className="error">{errors.name}</p>}
      </div>

      <div>
        <input
          name="email"
          type="email"
          value={formData.email}
          onChange={handleChange}
          onBlur={handleBlur}
          placeholder="Email"
          className={showError("email") ? "input-error" : ""}
        />
        {showError("email") && <p className="error">{errors.email}</p>}
      </div>

      <div>
        <input
          name="password"
          type="password"
          value={formData.password}
          onChange={handleChange}
          onBlur={handleBlur}
          placeholder="Password"
        />
        {showError("password") && <p className="error">{errors.password}</p>}
      </div>

      <div>
        <input
          name="confirmPassword"
          type="password"
          value={formData.confirmPassword}
          onChange={handleChange}
          onBlur={handleBlur}
          placeholder="Confirm Password"
        />
        {showError("confirmPassword") && <p
className="error">{errors.confirmPassword}</p>}
      </div>

      {/* Submit button disabled until form is valid */}
      <button type="submit" disabled={!isFormValid}>
        {isFormValid ? "Create Account" : "Please fix errors"}
      </button>

    </form>
```

```
  );
}
```

> ✅ **Remember**
>
> Errors are DERIVED — calculated from formData every render. Never store errors separately in useState — they would go stale.
>
> "touched" state tracks whether the user has visited each field. Without it, errors would show on empty fields before the user even types.
>
> Touch all fields on submit — so all errors show at once if user clicks submit without filling anything.
>
> Disable submit button when form is invalid — good UX, prevents bad submissions.

Topic 9: Preventing Default Browser Behaviour

Forms in HTML have default browser behaviour: submitting a form causes a **full page reload**. In React (SPA), you never want a page reload. You must call **e.preventDefault()** in your onSubmit handler.

```
// ❌ WRONG — missing preventDefault causes page reload
function SearchForm() {
  function handleSubmit() {
    console.log("This never runs — page reloads first!");
  }
  return <form onSubmit={handleSubmit}><button>Search</button></form>;
}

// ✅ CORRECT — preventDefault stops the reload
function SearchForm() {
  function handleSubmit(e) {
    e.preventDefault();  // Must be first line in submit handler
    console.log("Now this runs correctly");
    // Do your fetch/navigation here
  }
  return <form onSubmit={handleSubmit}><button>Search</button></form>;
}

// Other common uses of preventDefault:
// Links — prevent navigation:
<a href="/old-page" onClick={e => { e.preventDefault(); navigate("/new-
page"); }}>

// Drag and drop — prevent browser default drag behaviour:
<div onDragOver={e => e.preventDefault()} onDrop={handleDrop}>
```

Topic 10: Event Bubbling and stopPropagation

In the DOM, when you click a button, the click event does not just fire on the button. It also fires on every parent element — all the way up to the document. This is called **event bubbling**.

Sometimes this causes problems — clicking a child triggers the parent's handler unintentionally. The fix is **e.stopPropagation()**.

```
// EVENT BUBBLING PROBLEM
function ProductCard() {
  function handleCardClick() {
    console.log("Card clicked — navigate to product page");
  }

  function handleDeleteClick(e) {
    // Without stopPropagation — BOTH fire when delete is clicked:
    // 1. handleDeleteClick runs
    // 2. Event bubbles up to card → handleCardClick ALSO runs
    // User gets navigated to product page when they tried to delete!

    e.stopPropagation();  // stop event bubbling to parent card
    console.log("Delete clicked — only this runs now");
  }

  return (
    <div onClick={handleCardClick} className="product-card">
      <h3>Wireless Headphones</h3>
      <p>₹2,499</p>
      {/* This button is INSIDE the card */}
      <button onClick={handleDeleteClick}>Delete</button>
    </div>
  );
}

// With stopPropagation:
// Click card body    → only handleCardClick fires ✓
// Click delete button → only handleDeleteClick fires ✓ (bubble stopped)
```

Method	What it Does	When to Use
e.preventDefault()	Stops browser default action (page reload, link nav, form submit)	Always in onSubmit, sometimes in onClick on links
e.stopPropagation()	Stops event from bubbling up to parent elements	When clicking child should NOT trigger parent handler
Both together	Stops default AND stops bubble	Drag events, context menus, custom dropdowns

Topic 11: Building a Reusable Form Input Component

In real projects, you do not write the same input + label + error pattern 10 times. You extract it into a reusable component. This is a very practical skill that impresses interviewers.

```
// Reusable FormInput component
```

```
function FormInput({ label, name, type = "text", value, onChange, onBlur, error,
touched }) {
  return (
    <div className="form-group">
      <label htmlFor={name}>{label}</label>
      <input
        id={name}
        name={name}
        type={type}
        value={value}
        onChange={onChange}
        onBlur={onBlur}
        className={touched && error ? "input-error" : "input-normal"}
      />
      {touched && error && (
        <p className="error-message">{error}</p>
      )}
    </div>
  );
}

// Now your form is CLEAN — no repeated label/input/error markup
function LoginForm() {
  const [form, setForm]       = useState({ email: "", password: "" });
  const [touched, setTouched] = useState({ email: false, password: false });

  const errors = {
    email:    !form.email ? "Required" : !/\S+@\S+/.test(form.email) ? "Invalid
email" : "",
    password: form.password.length < 6 ? "Min 6 characters" : "",
  };

  const onChange = e => setForm(p => ({ ...p, [e.target.name]:
e.target.value }));
  const onBlur   = e => setTouched(p => ({ ...p, [e.target.name]: true }));

  return (
    <form onSubmit={e => { e.preventDefault(); console.log(form); }}>

      <FormInput label="Email" name="email" type="email"
        value={form.email} onChange={onChange} onBlur={onBlur}
        error={errors.email} touched={touched.email} />

      <FormInput label="Password" name="password" type="password"
        value={form.password} onChange={onChange} onBlur={onBlur}
        error={errors.password} touched={touched.password} />

      <button type="submit">Login</button>
    </form>
  );
}
```

> 💡 **Tip**
> This reusable FormInput pattern is what real companies use. It shows you think in components, not just code.
> In a real interview project or take-home test — using this pattern will make your code stand out immediately.
> You can extend it with icons, tooltips, character counters — all inside one place.

Topic 12: Debouncing Input — Avoiding Excessive API Calls

When a user types in a search box, you do NOT want to make an API call for every single keystroke. If they type "react" — that is 5 keystrokes and 5 API calls. You want to wait until they pause typing, then call the API once. This is called **debouncing**.

```
// DEBOUNCE — wait 500ms after user stops typing, THEN call API
import { useState, useEffect } from "react";

function SearchBar() {
  const [query, setQuery]           = useState("");
  const [debouncedQuery, setDebounced] = useState("");
  const [results, setResults]       = useState([]);

  // Debounce: update debouncedQuery 500ms after query stops changing
  useEffect(() => {
    const timer = setTimeout(() => {
      setDebounced(query);
    }, 500);

    // Cleanup: if query changes before 500ms, cancel this timer
    // and start a new 500ms countdown
    return () => clearTimeout(timer);
  }, [query]);

  // Only call API when debouncedQuery changes (after 500ms pause)
  useEffect(() => {
    if (!debouncedQuery.trim()) { setResults([]); return; }
    fetch(`/api/search?q=${debouncedQuery}`))
      .then(r => r.json())
      .then(setResults);
  }, [debouncedQuery]);

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Search..."
      />
      {/* query shows every keystroke — instant UI feedback */}
```

```
      {/* API call only fires 500ms after typing stops */}
      <ul>{results.map(r => <li key={r.id}>{r.title}</li>)}</ul>
    </div>
  );
}

// Without debounce: type "react" = 5 API calls
// With debounce: type "react" quickly = 1 API call (500ms after "t")
```

✅ **Remember**

Debounce = wait for the user to STOP before acting. Think of an elevator — it waits a few seconds for more people before closing doors.

The cleanup function (clearTimeout) is what makes debounce work — it cancels the previous timer every time a new keystroke comes in.

This is also a great example of how useEffect cleanup is not just for unmount — it runs before every re-run of the effect.

Chapter 6: Interview Questions with Full Answers

Q1: What is a SyntheticEvent in React?

Answer:

A SyntheticEvent is React's cross-browser wrapper around the browser's native event. Instead of each component handling native events directly, React uses a single event listener at the root of the application and distributes events to the correct handlers — a technique called event delegation.

The SyntheticEvent has the same API as a native browser event — e.target, e.preventDefault(), e.stopPropagation(), e.type — but it works consistently across all browsers, normalising differences between Chrome, Firefox, Safari, and IE.

In React 17+, the event delegation moved from the document root to the React root element. And event pooling (where SyntheticEvent objects were reused and cleared) was removed, so you no longer need e.persist() to use events asynchronously.

Q2: What is the difference between controlled and uncontrolled components?

Answer:

A controlled component is one where React state is the single source of truth for the input's value. The value prop is set from state, and every change calls onChange which updates state. React always knows exactly what is in the input.

An uncontrolled component lets the DOM manage the input value itself. You use a ref to access the value when needed — typically only at form submission.

Use controlled components when: you need real-time validation, you need to conditionally disable the submit button, you need to transform input (like auto-capitalize), or you need the value available anywhere in the component at any time.

Use uncontrolled when: it is a simple form where you only need the value on submit, or for file inputs (which cannot be controlled at all due to browser security).

Recommendation: use controlled components in almost all cases. They give React full control and make validation straightforward.

Q3: Why must you call e.preventDefault() in a form's onSubmit?

Answer:

HTML forms have a default browser behaviour — when submitted, they send an HTTP request and cause a full page reload. In a React Single Page Application, you never want a page reload because it would destroy your entire React app state.

e.preventDefault() stops this default behaviour, giving you control to handle the submission in JavaScript instead — calling a fetch API, updating state, navigating programmatically.

Without it: your form submits, the page reloads, all state is lost, and your React app re-initialises from scratch.

e.preventDefault() should always be the first line in your onSubmit handler.

Q4: What is event bubbling? How do you stop it?

Answer:

Event bubbling means when an event fires on a DOM element, it propagates up through all parent elements, triggering any matching event listeners along the way.

Example: clicking a button inside a div fires the button's onClick first, then bubbles up and fires the div's onClick too.

This causes problems when you want only the child's handler to fire — like a delete button inside a clickable card.

To stop bubbling: call e.stopPropagation() inside the child's event handler. This prevents the event from traveling up to parent elements.

stopPropagation vs preventDefault: stopPropagation stops the event from bubbling. preventDefault stops the default browser action. They are independent — you can call either or both depending on what you need.

Q5: How do you handle multiple form inputs efficiently in React?

Answer:

Instead of creating a separate onChange handler for each input, use a single generic handler that uses e.target.name to identify which field changed.

Store all form values in a single state object: const [formData, setFormData] = useState({ name: "", email: "", password: "" })

Generic handler: function handleChange(e) { const { name, value } = e.target; setFormData(prev => ({ ...prev, [name]: value })); }

Each input needs a name attribute that matches the key in the state object: <input name="email" value={formData.email} onChange={handleChange} />

This scales beautifully — you can have 20 fields with the exact same single handler. Just add new keys to state and new inputs with matching name attributes.

Q6: How do you implement form validation in React?

Answer:

There are three approaches: on change (every keystroke), on blur (when field loses focus), and on submit.

Best practice is on-blur validation combined with on-submit safety check.

Track a "touched" state — object with boolean per field. Set field to true in onBlur. Only show error if the field is touched AND has an error. This way errors do not show on fields the user has not interacted with yet.

Errors should be DERIVED — calculated from form data directly, not stored in a separate useState. This ensures errors are always in sync with the current values.

On submit: touch all fields at once so all errors become visible, then check if form is valid before calling the API.

Disable the submit button when there are validation errors — prevents bad submissions and gives immediate visual feedback.

Q7: What is debouncing in React forms? Why and how do you implement it?

Answer:

Debouncing delays executing a function until a certain time has passed since the last call. In forms, it prevents making an API call on every single keystroke.

Why: if a user types "react" in a search box, without debounce that fires 5 API calls. With 500ms debounce it fires 1 call — only after the user pauses.

Implementation: use useEffect with a setTimeout. The cleanup function (clearTimeout) cancels the previous timer whenever the input changes. Only when 500ms passes without a new change does the final effect actually run.

The cleanup function is what makes it work. Every keystroke cancels the previous timer and starts a new 500ms countdown. Only the final pause survives the full 500ms and triggers the API call.

This is a great example of useEffect cleanup serving a purpose beyond just unmounting.

Q8: How do you handle checkbox inputs in a controlled React form?

Answer:

Checkboxes use e.target.checked (boolean) instead of e.target.value (string). This is the most common mistake with checkbox inputs.

For a single checkbox: const [isChecked, setIsChecked] = useState(false); and <input type="checkbox" checked={isChecked} onChange={e => setIsChecked(e.target.checked)} />

For multiple checkboxes (like preferences): store them in a state object where each key is the checkbox name and value is boolean.

Generic handler for multiple checkboxes: const { name, checked } = e.target; setPrefs(prev => ({ ...prev, [name]: checked }));

Note: use checked={value} not value={value} for checkboxes to make them controlled.

Q9: Why can't you control a file input in React?

Answer:

File inputs cannot be controlled because the browser prevents JavaScript from programmatically setting the value of a file input. This is a security restriction — if JavaScript could set file input values, malicious websites could silently pre-select files to steal.

The value of a file input is always read-only from JavaScript's perspective. You can read what the user selected, but you cannot set it.

For file inputs, always use uncontrolled approach with useRef: const fileRef = useRef(); and <input type="file" ref={fileRef} />. Read the file at submission: fileRef.current.files[0].

You can still react to file selection with onChange to show the filename or preview the image — you just cannot set the value.

Q10: What is the difference between onClick={handleClick} and onClick={handleClick()}?

Answer:

onClick={handleClick} passes the function reference — the function will be called when the click happens.

onClick={handleClick()} calls the function immediately when the component renders, and passes its return value (usually undefined) as the onClick handler. This means the function runs on every render — not on click.

This is one of the most common beginner mistakes in React. The component renders, handleClick() executes (might call setState, log something, make an API call), and the onClick handler receives whatever handleClick returned.

When you need to pass arguments: use onClick={() => handleDelete(id)} — the outer arrow function is the handler reference, it calls handleDelete(id) only when clicked.

Rule: if you see () at the end in JSX, the function is being CALLED, not passed.

Q11: What is React event delegation? How is it different from native event delegation?

Answer:

Event delegation is a technique where instead of attaching event listeners to every individual element, you attach one listener to a common parent and let events bubble up to it.

In native JS, developers manually attach one listener to document: document.addEventListener("click", handler) and check event.target to figure out what was clicked.

React does this automatically and more efficiently. React attaches a single event listener to the root div (not document — since React 17). All events bubble up to this root listener. React's event system then figures out which component's handler should be called and dispatches the SyntheticEvent to it.

Benefits: fewer listeners (performance), consistent cross-browser behaviour (SyntheticEvent), automatic cleanup when components unmount.

Q12: How do you build a reusable, accessible form input component in React?

Answer:

A reusable form input should accept: label, name, type, value, onChange, onBlur, error, touched as props.

It renders: a label (with htmlFor matching the input id for accessibility), the input element, and conditionally the error message (only when touched and error exists).

The label htmlFor and input id must match — this allows screen readers to associate the label with the input, and allows users to click the label to focus the input.

For accessibility: use aria-invalid={!!error} on the input when there is an error. Use aria-describedby pointing to the error message element's id.

This component is used in every form — you write it once and reuse it everywhere. The parent form manages all state, the FormInput just receives and displays.

Common Mistakes in Event Handling & Forms

Mistake 1: Calling the function instead of passing it

```
// ❌ WRONG — runs on every render
<button onClick={handleClick()}>Click</button>

// ✅ CORRECT — runs only on click
<button onClick={handleClick}>Click</button>

// ✅ CORRECT — with arguments
<button onClick={() => handleDelete(item.id)}>Delete</button>
```

Mistake 2: Forgetting e.preventDefault on form submit

```
// ❌ WRONG — page reloads on submit
function handleSubmit() {
  console.log("This never runs — page reloads first");
}

// ✅ CORRECT
function handleSubmit(e) {
  e.preventDefault();  // First line, always
  console.log("Now this runs");
}
```

Mistake 3: Using e.target.value for checkboxes

```
// ❌ WRONG — e.target.value for checkbox gives "on" or "off" string
onChange={e => setChecked(e.target.value)}

// ✅ CORRECT — use e.target.checked for booleans
onChange={e => setChecked(e.target.checked)}
```

Mistake 4: Showing validation errors before user touches the field

```
// ❌ BAD UX — error shows immediately, before user types anything
function Form() {
  const [email, setEmail] = useState("");
  const error = !email ? "Email required" : "";
  return (
    <div>
      <input value={email} onChange={e => setEmail(e.target.value)} />
      {error && <p>{error}</p>}  // Shows "Email required" immediately!
    </div>
  );
}

// ✅ GOOD UX — only show error after user has left the field
function Form() {
  const [email, setEmail]   = useState("");
  const [touched, setTouched] = useState(false);
  const error = !email ? "Email required" : "";
  return (
    <div>
      <input
        value={email}
        onChange={e => setEmail(e.target.value)}
        onBlur={() => setTouched(true)}
      />
      {touched && error && <p>{error}</p>}  // Only after blur
    </div>
  );
}
```

Quick Reference — Chapter 6 Summary

Concept	Key Point	Remember
SyntheticEvent	React's cross-browser event wrapper	Same API as native, works everywhere
Event names	camelCase in JSX	onClick, onChange, onSubmit (not onclick)
Controlled form	value + onChange = React owns data	Single source of truth in state
Uncontrolled form	ref reads DOM on demand	Only for file inputs or very simple forms
Generic handler	Use e.target.name to identify field	{ ...prev, [name]: value } pattern

Checkbox	Use e.target.checked not e.target.value	Returns boolean, not string
e.preventDefault()	Stops browser default (page reload)	First line in every onSubmit
e.stopPropagation()	Stops event bubbling to parent	When child click should not trigger parent
Validation	Errors are DERIVED, touched tracks visited	Show error only if touched && error
Debounce	Wait 500ms after last keystroke	setTimeout + clearTimeout cleanup

✅ **Remember**

e.preventDefault() — stop page reload. First line in onSubmit. Always.
e.stopPropagation() — stop parent from also firing. For nested click handlers.
value + onChange = controlled. **ref** = uncontrolled.
e.target.name = which field. **e.target.value** = new text. **e.target.checked** = checkbox boolean.
touched state = only show errors for fields user has visited (onBlur).
Errors are derived — calculate from formData, never store separately.
Debounce = setTimeout + clearTimeout cleanup = wait for pause before API call.
onClick={fn} = pass reference. **onClick={fn()}** = calling immediately. Never do second.

End of Chapter 6 • Next: Chapter 7 — Conditional Rendering & Lists

Chapter 7: Conditional Rendering & Lists

map, filter, keys, all conditional patterns — used in every React app

By Syed Shaaz Akhtar | Interview Focused Guide

If you open ANY React app — Gmail, Amazon, Swiggy, Instagram — every single screen uses conditional rendering and lists. Showing a loading spinner instead of data. Showing an error message when something fails. Rendering a list of products, messages, or notifications. These are not optional skills — they are the building blocks of every React UI.

This chapter covers every pattern you will use in real projects, every mistake beginners make, and every question interviewers ask about these topics.

> 💡 **Tip**
> 🎬 THE ANALOGY for this chapter:
> Think of conditional rendering like a FILM DIRECTOR's script.
> "If the actor is available — show them on screen. Otherwise — show their understudy."
> "If the scene is night — show the dark background. Otherwise — show the bright one."
> And lists? Think of them like a PLAYLIST.
> "Take this list of songs. For each song — create a song card. Show them all."
> React does exactly this — it follows your instructions and renders accordingly.

Topic 1: What is Conditional Rendering?

Conditional rendering means showing different UI based on a condition. If the user is logged in, show the dashboard. If not, show the login page. If data is loading, show a spinner. If an error occurred, show an error message.

React does NOT have special template directives like v-if (Vue) or *ngIf (Angular). Instead, it uses plain JavaScript inside JSX — because JSX is just JavaScript. This means all the JavaScript you already know works for conditional rendering.

Method	Syntax	Best For
if/else statement	Full if/else block before return	Complex conditions, early returns
Ternary operator	condition ? A : B	Show one of two things
&& (AND) operator	condition && <Component />	Show something or nothing
|| (OR) operator	value || fallback	Fallback/default values

switch statement	switch(status) { case ... }	Multiple exclusive conditions
Object/Map lookup	statusMap[status]	Clean alternative to long switch

Topic 2: if/else — The Classic Way

Regular if/else cannot go INSIDE JSX return directly. But you can use it BEFORE the return statement, or inside a function that returns JSX. This is the most readable approach for complex conditions.

```
// WAY 1: if/else before return — early return pattern
function UserGreeting({ user, isLoading, error }) {

  // Show loading first
  if (isLoading) {
    return <div className="spinner">Loading...</div>;
  }

  // Show error if fetch failed
  if (error) {
    return <div className="error">Error: {error}</div>;
  }

  // Show empty state if no user
  if (!user) {
    return <div>No user found.</div>;
  }

  // Happy path — show the actual content
  return (
    <div>
      <h1>Welcome, {user.name}!</h1>
      <p>Role: {user.role}</p>
    </div>
  );
}

// WAY 2: Store JSX in a variable
function StatusBadge({ status }) {
  let badge;

  if (status === "active")   badge = <span className="green">Active</span>;
  else if (status === "away") badge = <span className="yellow">Away</span>;
  else                        badge = <span className="red">Offline</span>;

  return <div>Status: {badge}</div>;
}
```

✅ **Remember**

The early return pattern is the cleanest way to handle loading, error, and empty states.
Check in this order: loading → error → empty/null → actual content.
This is called the "Guard Clause" pattern — handle all the edge cases first, then write the happy path.

Topic 3: Ternary Operator — Show A or B

The ternary operator is the most commonly used conditional in JSX. Use it when you want to show ONE of TWO things based on a condition.

```
// Basic ternary — show one of two elements
function LoginButton({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn
        ? <button>Logout</button>
        : <button>Login</button>
      }
    </div>
  );
}

// Ternary with text
<p>Status: {isOnline ? "Online" : "Offline"}</p>

// Ternary with className — dynamic styling
<button className={isActive ? "btn-active" : "btn-inactive"}>
  Click Me
</button>

// Ternary with component
<div>
  {user.isAdmin ? <AdminPanel /> : <UserPanel />}
</div>

// Nested ternary — use sparingly, gets hard to read
<p>
  {score >= 90 ? "A Grade"
    : score >= 70 ? "B Grade"
    : score >= 50 ? "C Grade"
    : "Fail"}
</p>
// Tip: if nested ternary is 3+ levels — switch to if/else or a function
```

💡 **Tip**

Ternary = always has BOTH sides: true case AND false case.
If you only want to show something OR show nothing — use && instead.

> Keep ternaries simple. If the condition or result is complex, extract it to a variable or function.

Topic 4: && Operator — Show Something or Nothing

The && (AND) operator is perfect when you want to show a component only if a condition is true — and show NOTHING if it is false. No else case needed.

How it works in JavaScript: A && B returns A if A is falsy, otherwise returns B. In React, if A is false/null/undefined, nothing is rendered. If A is true, B (your JSX) is rendered.

```
// Basic && — show badge only if count > 0
function NavBar({ cartCount, unreadMessages }) {
  return (
    <nav>
      <span>Cart
        {cartCount > 0 && (
          <span className="badge">{cartCount}</span>
        )}
      </span>

      <span>Messages
        {unreadMessages > 0 && (
          <span className="badge">{unreadMessages}</span>
        )}
      </span>
    </nav>
  );
}

// More examples
{isAdmin && <AdminMenu />}
{hasError && <ErrorBanner message={error} />}
{isLoading && <Spinner />}
{user && <p>Welcome, {user.name}</p>}
```

> 📝 **Note**
> ⚠️ THE MOST COMMON && BUG IN REACT:
> Never use a NUMBER as the condition for &&.
>
> // ❌ WRONG — renders "0" on screen when array is empty!
> {items.length && <ItemList items={items} />}
> // When items.length is 0, JavaScript evaluates 0 && ... = 0
> // React renders 0 as text on screen — visible bug!
>
> // ✅ CORRECT — convert to boolean first
> {items.length > 0 && <ItemList items={items} />}
> {!!items.length && <ItemList items={items} />}

```
{Boolean(items.length) && <ItemList items={items} />}
```

Topic 5: switch and Object Lookup — For Multiple Conditions

When you have 3 or more exclusive conditions, switch statements or object lookups are cleaner than chaining multiple ternaries.

```
// SWITCH statement — inside a function
function OrderStatus({ status }) {
  function renderStatus() {
    switch (status) {
      case "pending":    return <span className="yellow">⏳ Pending</span>;
      case "processing": return <span className="blue">🔄 Processing</span>;
      case "shipped":    return <span className="purple">🚚 Shipped</span>;
      case "delivered":  return <span className="green">✅ Delivered</span>;
      case "cancelled":  return <span className="red">❌ Cancelled</span>;
      default:           return <span className="grey">Unknown</span>;
    }
  }

  return <div>Order Status: {renderStatus()}</div>;
}

// OBJECT LOOKUP — cleaner, more declarative
const STATUS_MAP = {
  pending:    <span className="yellow">⏳ Pending</span>,
  processing: <span className="blue">🔄 Processing</span>,
  shipped:    <span className="purple">🚚 Shipped</span>,
  delivered:  <span className="green">✅ Delivered</span>,
  cancelled:  <span className="red">❌ Cancelled</span>,
};

function OrderStatus({ status }) {
  return (
    <div>
      Order Status: {STATUS_MAP[status] || <span
className="grey">Unknown</span>}
    </div>
  );
}

// Object lookup is preferred — no switch/case boilerplate
// Easier to add new statuses — just add to the object
// Can be defined outside the component — no re-creation on every render
```

Topic 6: Hiding vs Not Rendering — visibility vs null

There are two ways to "hide" something in React. They look similar but behave very differently. Understanding this is important for performance and animations.

```
// WAY 1: Conditional render — component is NOT in the DOM at all
function App() {
  const [show, setShow] = useState(false);
  return (
    <div>
      <button onClick={() => setShow(!show)}>Toggle</button>
      {show && <ExpensiveComponent />}
      {/* When show=false: ExpensiveComponent is UNMOUNTED */}
      {/* It does not exist in the DOM. State is destroyed. */}
      {/* useEffect cleanup runs. Re-mounts when show=true. */}
    </div>
  );
}

// WAY 2: CSS visibility — component stays in DOM, just invisible
function App() {
  const [show, setShow] = useState(false);
  return (
    <div>
      <button onClick={() => setShow(!show)}>Toggle</button>
      <ExpensiveComponent style={{ display: show ? "block" : "none" }} />
      {/* Component is ALWAYS in the DOM — just hidden with CSS */}
      {/* State is PRESERVED. No unmount/remount. No cleanup. */}
    </div>
  );
}
```

Approach	In DOM?	State Preserved?	Lifecycle	Use When
Conditional render (&&)	NO when false	NO — destroyed	Mounts/unmounts	Component is rarely shown, state should reset
CSS display:none	YES always	YES — preserved	Never unmounts	Component shown/hidden often, keep its state

> 💡 **Tip**
> Use CSS display:none when: a tab panel that should remember scroll position, an accordion that should remember its expanded state, an animation that needs to transition smoothly.

> Use conditional render (&&) when: a modal that should fully reset each time it opens, a form that should start fresh each time, any component where you WANT the state to reset.

Topic 7: Rendering Lists with .map()

Whenever you have an array of data and need to show each item as a UI element, you use JavaScript's **.map()** method. This is the standard way to render lists in React.

Think of it like a PLAYLIST. You have 10 songs. For each song in the playlist, create a song card. Show all 10 cards.

```
// Basic list rendering with .map()
function ProductList() {
  const products = [
    { id: 1, name: "Headphones",  price: 2499 },
    { id: 2, name: "Keyboard",    price: 1299 },
    { id: 3, name: "Mouse",       price:  799 },
  ];

  return (
    <ul>
      {products.map(product => (
        // key prop is REQUIRED on the outermost element in the map
        <li key={product.id}>
          {product.name} — ₹{product.price}
        </li>
      ))}
    </ul>
  );
}

// Rendering components from a list
function ProductList({ products }) {
  return (
    <div className="grid">
      {products.map(product => (
        <ProductCard
          key={product.id}
          name={product.name}
          price={product.price}
          image={product.image}
        />
      ))}
    </div>
  );
}
```

Topic 8: The key Prop — The Most Asked List Question

Every element in a .map() needs a **key** prop. This is React's most famous "gotcha" and is asked in virtually every interview. You must understand not just WHAT it is, but WHY React needs it.

Why Does React Need Keys?

When a list re-renders, React needs to figure out which items changed, which were added, and which were removed. Without keys, React has no way to identify each item — it just compares by position. This leads to bugs.

```
// THE PROBLEM — no keys, React compares by position
// List: [A, B, C]  →  user deletes A  →  new list: [B, C]
//
// Without keys, React compares:
//   Position 0: was A, now B → React UPDATES the first item (wrong!)
//   Position 1: was B, now C → React UPDATES the second item (wrong!)
//   Position 2: was C, now gone → React REMOVES the third item
//
// Result: React did 2 unnecessary updates + 1 removal instead of 1 removal
// In complex components — input focus lost, animations wrong, state wrong

// WITH KEYS — React compares by identity
// List: [A(key=1), B(key=2), C(key=3)]  →  delete A  →  [B(key=2), C(key=3)]
//
// React compares by key:
//   key=1 (A): was here, now gone → REMOVE A
//   key=2 (B): still here → keep, no change
//   key=3 (C): still here → keep, no change
//
// Result: 1 removal, 0 unnecessary updates. Correct!
```

Good Keys vs Bad Keys

```
// ✅ BEST KEY — unique, stable database ID
{users.map(user => <UserCard key={user.id} user={user} />)}

// ✅ GOOD KEY — any unique, stable identifier
{countries.map(c => <CountryItem key={c.code} country={c} />)}

// ⚠️ LAST RESORT — index as key (use only when list never changes order)
{staticMenuItems.map((item, index) => (
  <MenuItem key={index} item={item} />
))}

// ❌ TERRIBLE KEY — random value, changes every render
{items.map(item => <Card key={Math.random()} item={item} />)}
// This destroys React's optimisation completely.
// React unmounts and remounts EVERY item on every render!

// ❌ BAD KEY — index when list is sortable/filterable/deletable
```

```
{todos.filter(t => !t.done).map((todo, index) => (
  <TodoItem key={index} todo={todo} />
))}
// After filtering, indices change — causes wrong items to update
```

✅ Remember

Keys must be UNIQUE among siblings (not globally unique).

Keys must be STABLE — same item should always get the same key across renders.

Keys must be a STRING or NUMBER — not objects or arrays.

Best key = database ID. Second best = some unique property. Last resort = index (only for static, never-reordered lists).

Key is NOT a prop — you cannot access props.key inside the component. It is for React's internal use only.

If you need the id inside the component — pass it as a separate prop: <Card key={item.id} id={item.id} />

Topic 9: Filtering and Sorting Lists

In real apps, you almost never render the raw array. You filter it (show only matching items), sort it (order by price, date, name), or both. These are all derived values — calculated from state, never stored separately.

```
// FILTERING + SORTING + RENDERING — complete pattern
function ProductCatalog() {
  const [products] = useState([
    { id: 1, name: "Headphones",  price: 2499, category: "Audio",  inStock: true
},
    { id: 2, name: "Speaker",     price: 1999, category: "Audio",  inStock:
false },
    { id: 3, name: "Keyboard",    price: 1299, category: "Input",  inStock: true
},
    { id: 4, name: "Mouse",       price:  799, category: "Input",  inStock: true
},
    { id: 5, name: "Webcam",      price: 3499, category: "Video",  inStock: true
},
  ]);

  const [search,   setSearch]   = useState("");
  const [category, setCategory] = useState("All");
  const [sortBy,   setSortBy]   = useState("name");
  const [onlyStock,setOnlyStock]= useState(false);

  // ALL DERIVED — calculated every render, never stored in state
  const filtered = products
    .filter(p => category === "All" || p.category === category)
    .filter(p => p.name.toLowerCase().includes(search.toLowerCase()))
    .filter(p => !onlyStock || p.inStock)
    .sort((a, b) => {
      if (sortBy === "name")       return a.name.localeCompare(b.name);
```

```
      if (sortBy === "price-asc")  return a.price - b.price;
      if (sortBy === "price-desc") return b.price - a.price;
      return 0;
    });

  return (
    <div>
      {/* Controls */}
      <input value={search} onChange={e => setSearch(e.target.value)}
placeholder="Search..." />

      <select value={category} onChange={e => setCategory(e.target.value)}>
        {["All","Audio","Input","Video"].map(c => <option key={c}>{c}</option>)}
      </select>

      <select value={sortBy} onChange={e => setSortBy(e.target.value)}>
        <option value="name">Name A-Z</option>
        <option value="price-asc">Price Low-High</option>
        <option value="price-desc">Price High-Low</option>
      </select>

      <label>
        <input type="checkbox" checked={onlyStock} onChange={e =>
setOnlyStock(e.target.checked)} />
        In stock only
      </label>

      {/* Results count */}
      <p>Showing {filtered.length} of {products.length} products</p>

      {/* Empty state */}
      {filtered.length === 0 && (
        <p className="empty">No products match your filters.</p>
      )}

      {/* List */}
      <div className="grid">
        {filtered.map(product => (
          <ProductCard key={product.id} product={product} />
        ))}
      </div>
    </div>
  );
}
```

📝 Note

Never do .sort() directly on state: products.sort() mutates the original array.
Always sort a COPY: [...products].sort() or products.filter(...).sort().
.filter() and .map() return new arrays — safe to chain.

.sort() mutates in-place — always spread first if sorting the original array.

Topic 10: Empty States — Always Handle Them

A real app always handles the case where a list is empty. Showing a blank space when there are no results is bad UX. Always show an empty state message.

```
// THREE STATES every list component should handle:
// 1. Loading  2. Error  3. Empty  4. Has data

function TodoList() {
  const [todos, setTodos]     = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError]     = useState(null);

  useEffect(() => {
    fetch("/api/todos")
      .then(r => r.json())
      .then(data => { setTodos(data); setLoading(false); })
      .catch(err => { setError(err.message); setLoading(false); });
  }, []);

  // Early returns for each state
  if (loading) return <div className="spinner">Loading todos...</div>;
  if (error)   return <div className="error">Failed to load: {error}</div>;

  // Empty state — very important for good UX
  if (todos.length === 0) {
    return (
      <div className="empty-state">
        <span className="empty-icon">📋</span>
        <h3>No todos yet!</h3>
        <p>Click the button below to add your first task.</p>
        <button>Add Todo</button>
      </div>
    );
  }

  // Happy path — render the list
  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
}
```

💡 **Tip**

The 4 states every list needs: Loading → Error → Empty → Data.

Handle them in this order with early returns. The last return is always the happy path with real data.

An empty state is NOT just showing nothing. It should tell the user WHY it is empty and what they can do about it.

Topic 11: Nested Lists — Lists Inside Lists

Real apps often have nested data — a list of categories, each containing a list of products. Rendering nested lists just means nesting .map() calls.

```
// NESTED LIST — categories with products inside
function CategoryCatalog() {
  const categories = [
    {
      id: "audio",
      name: "Audio",
      products: [
        { id: 1, name: "Headphones", price: 2499 },
        { id: 2, name: "Speaker",    price: 1999 },
      ]
    },
    {
      id: "input",
      name: "Input Devices",
      products: [
        { id: 3, name: "Keyboard", price: 1299 },
        { id: 4, name: "Mouse",    price:  799 },
      ]
    },
  ];

  return (
    <div>
      {/* Outer map — one item per category */}
      {categories.map(category => (
        <section key={category.id}>   {/* key on outermost outer element */}
          <h2>{category.name}</h2>

          {/* Inner map — one item per product */}
          {category.products.map(product => (
            <div key={product.id}>    {/* key on outermost inner element */}
              <span>{product.name}</span>
              <span>₹{product.price}</span>
            </div>
          ))}
        </section>
      ))}
    </div>
  );
```

```
}

// Keys are unique within their own list — not globally.
// product.id=1 in "audio" and product.id=1 in "input" are fine
// because they are siblings within different parent groups.
```

Topic 12: Fragments in Lists — Rendering Without Wrapper Divs

Sometimes when you render a list, each item needs to render MULTIPLE elements — but you do not want to wrap them in a div (which would add unwanted HTML structure). That is where **React.Fragment** with a key comes in.

```
// PROBLEM: each item needs TWO elements — dt and dd
// But you cannot use <> </> shorthand because you need a key

const glossary = [
  { id: 1, term: "JSX",      def: "JavaScript XML — HTML-like syntax in JS" },
  { id: 2, term: "Props",    def: "Data passed from parent to child" },
  { id: 3, term: "State",    def: "Data that lives inside a component" },
];

// ❌ WRONG — extra div breaks the dl > dt+dd semantic structure
{glossary.map(item => (
  <div key={item.id}>        // unwanted wrapper div
    <dt>{item.term}</dt>
    <dd>{item.def}</dd>
  </div>
))}

// ✅ CORRECT — Fragment with key, no extra DOM element
{glossary.map(item => (
  <React.Fragment key={item.id}>  // no DOM output, but accepts key
    <dt>{item.term}</dt>
    <dd>{item.def}</dd>
  </React.Fragment>
))}

// Result: clean semantic HTML
// <dl>
//   <dt>JSX</dt>
//   <dd>JavaScript XML...</dd>
//   <dt>Props</dt>
//   <dd>Data passed...</dd>
// </dl>

// NOTE: <> </> shorthand cannot take a key prop.
// When you need key in a Fragment — use <React.Fragment key={...}>
```

Topic 13: Dynamic Classes and Styles — Conditional Styling

Conditional rendering is not just about showing/hiding components. It is also about applying different CSS classes or inline styles based on state. This is used in every real React component.

```
// CONDITIONAL CLASS — using ternary
function Tab({ label, isActive, onClick }) {
  return (
    <button
      className={isActive ? "tab tab-active" : "tab"}
      onClick={onClick}
    >
      {label}
    </button>
  );
}

// MULTIPLE CONDITIONAL CLASSES — template literal approach
function Card({ isSelected, isDisabled, isHighlighted }) {
  const classes = [
    "card",
    isSelected    && "card-selected",
    isDisabled    && "card-disabled",
    isHighlighted && "card-highlighted",
  ].filter(Boolean).join(" ");  // removes false values, joins with space

  return <div className={classes}>Content</div>;
}

// INLINE STYLES — always an object, properties in camelCase
function ProgressBar({ percentage }) {
  return (
    <div className="progress-track">
      <div
        className="progress-fill"
        style={{
          width: `${percentage}%`,
          backgroundColor: percentage === 100 ? "green" : "blue",
          transition: "width 0.3s ease",
        }}
      />
    </div>
  );
}

// POPULAR LIBRARY: clsx (used in real projects)
// npm install clsx
import clsx from "clsx";

function Button({ isActive, isDisabled, variant }) {
  return (
```

```
    <button className={clsx(
      "btn",                                  // always applied
      isActive && "btn-active",               // conditional
      isDisabled && "btn-disabled",           // conditional
      variant === "primary" && "btn-primary", // conditional
      variant === "danger"  && "btn-danger",  // conditional
    )}>
      Click
    </button>
  );
}
```

✅ **Remember**

Inline styles in React = JavaScript object with camelCase keys: backgroundColor not background-color.

style={{ }} — outer braces = JSX expression, inner braces = JavaScript object.

For complex conditional classes, the array.filter(Boolean).join(" ") pattern or clsx library is cleaner than multiple ternaries.

Inline styles override CSS classes — use them for truly dynamic values (widths, colors from data) and CSS classes for everything else.

Topic 14: Complete Example — Dashboard with Everything

Let us build a real dashboard component that combines conditional rendering, lists, filtering, empty states, and dynamic styling all together. This is the kind of component you build in every project.

```
// COMPLETE DASHBOARD — combines everything in this chapter
function NotificationsDashboard() {
  const [notifications, setNotifications] = useState([
    { id: 1, type: "order",   text: "Your order #1234 was delivered!", read:
false, time: "2m ago" },
    { id: 2, type: "promo",   text: "50% off on all electronics!",      read:
false, time: "1h ago" },
    { id: 3, type: "order",   text: "Your order #1235 was shipped.",     read:
true,  time: "3h ago" },
    { id: 4, type: "system",  text: "Scheduled maintenance at midnight", read:
true,  time: "1d ago" },
    { id: 5, type: "promo",   text: "New arrivals in Audio section!",    read:
false, time: "2d ago" },
  ]);

  const [filter, setFilter] = useState("all");    // all | unread | order |
promo | system
  const [isLoading]         = useState(false);

  const unreadCount = notifications.filter(n => !n.read).length;

  const filtered = notifications.filter(n => {
    if (filter === "all")    return true;
```

```
    if (filter === "unread") return !n.read;
    return n.type === filter;
  });

  function markAsRead(id) {
    setNotifications(prev =>
      prev.map(n => n.id === id ? { ...n, read: true } : n)
    );
  }

  function markAllRead() {
    setNotifications(prev => prev.map(n => ({ ...n, read: true })));
  }

  const TYPE_ICON = { order: "📦", promo: "🎁", system: "⚙️" };

  // Loading state
  if (isLoading) return <div>Loading notifications...</div>;

  return (
    <div className="notifications-panel">

      {/* Header with unread badge */}
      <div className="panel-header">
        <h2>
          Notifications
          {unreadCount > 0 && (
            <span className="badge">{unreadCount}</span>
          )}
        </h2>
        {unreadCount > 0 && (
          <button onClick={markAllRead}>Mark all read</button>
        )}
      </div>

      {/* Filter tabs */}
      <div className="filter-tabs">
        {["all", "unread", "order", "promo", "system"].map(tab => (
          <button
            key={tab}
            className={filter === tab ? "tab active" : "tab"}
            onClick={() => setFilter(tab)}
          >
            {tab.charAt(0).toUpperCase() + tab.slice(1)}
          </button>
        ))}
      </div>

      {/* Empty state */}
      {filtered.length === 0 ? (
        <div className="empty-state">
```

```
          <p>🔔 No {filter === "all" ? "" : filter} notifications</p>
        </div>
      ) : (
        <ul className="notification-list">
          {filtered.map(notification => (
            <li
              key={notification.id}
              className={[
                "notification-item",
                !notification.read && "unread",
              ].filter(Boolean).join(" ")}
              onClick={() => markAsRead(notification.id)}
            >
              <span className="icon">{TYPE_ICON[notification.type]}</span>
              <span className="text">{notification.text}</span>
              <span className="time">{notification.time}</span>
              {!notification.read && (
                <span className="unread-dot" />
              )}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}
```

Chapter 7: Interview Questions with Full Answers

Q1: What is conditional rendering in React? What are the different ways to do it?

Answer:

Conditional rendering means showing different UI based on a condition. React uses plain JavaScript for this — no special template directives.

1. if/else before return — best for complex conditions, loading/error/empty state handling (early return pattern).
2. Ternary operator (condition ? A : B) — best for showing one of two things inside JSX.
3. && operator (condition && <Component />) — best for showing something or nothing. No else needed.
4. switch statement — best for 3+ exclusive conditions, especially with a named function.
5. Object lookup (map[key]) — cleaner alternative to switch, easy to extend.

I use early returns for loading/error states, ternary for binary choices, and && for optional UI elements.

Q2: What is the key prop in React? Why is it important?

Answer:

The key prop is a special attribute you must provide to list items when rendering arrays. It helps React identify which items have changed, been added, or been removed.

Without keys, React compares list items by POSITION. If you delete the first item, React thinks every item shifted — it updates all remaining items unnecessarily, which is slow and causes bugs (wrong focus, wrong animations, wrong state).

With keys, React compares by IDENTITY. It knows which specific item was removed and only removes that one, leaving all others untouched.

Keys must be: unique among siblings, stable (same item always gets same key), and a string or number.

Best key = database ID. Never use Math.random() as a key — it changes every render and destroys React's optimisation completely.

Q3: Why should you not use array index as a key?

Answer:

Using array index as key causes bugs when the list can be reordered, filtered, or have items deleted.

Example: you have items [A(index 0), B(index 1), C(index 2)]. You delete A. Now the list is [B(index 0), C(index 1)]. React sees: index 0 changed from A to B, index 1 changed from B to C. React updates two items instead of removing one.

This causes: input fields showing wrong values (the state from deleted item migrates to next item), incorrect animations, wrong focus.

When is index OK? Only when: the list is static (never reordered/filtered/deleted), items have no state, and items are never reordered.

As soon as any mutation is possible — use a proper unique ID.

Q4: What is the && pitfall with numbers in React?

Answer:

When you use a number as the condition in &&, React renders the number 0 on screen instead of nothing.

This happens because: in JavaScript, 0 is falsy, so 0 && <Component /> evaluates to 0 (not false). React renders numbers as text, so 0 becomes visible text on the screen.

Example: {items.length && <ItemList />} — when items is empty, items.length is 0, and React renders "0" on screen as text.

Fix: always convert to a boolean first. Use items.length > 0 && <ItemList /> or !!items.length && <ItemList />.

This is a very common bug in real apps. Always use explicit boolean conditions with &&.

Q5: How do you render a list of items in React?

Answer:

Use JavaScript's .map() method on the array. For each element, return JSX. The result is an array of JSX elements that React renders.

Every item in the list MUST have a unique key prop on the outermost element returned from the map callback.

Example: {products.map(p => <ProductCard key={p.id} product={p} />)}

The key should be a unique, stable identifier — ideally the database ID.

You can chain .filter() before .map() to show only matching items. Always put .filter() before .map() — no point mapping items you will filter out.

Q6: What is the difference between conditional render and CSS display:none?

Answer:

Conditional render (using && or ternary returning null) completely REMOVES the component from the DOM when the condition is false. The component unmounts, its state is destroyed, lifecycle cleanup runs.

CSS display:none keeps the component in the DOM but makes it invisible. The component never unmounts, its state is preserved.

Use conditional render when: you want the component to reset its state each time it appears, it is rarely shown (saves memory by not keeping it in DOM), or it has heavy setup/cleanup (like a modal with event listeners).

Use CSS display:none when: the component should preserve its state while hidden (like a tab that remembers scroll position), it toggles frequently (avoiding repeated mount/unmount overhead), or you need CSS transitions that require the element to exist in DOM.

Q7: How do you handle loading, error, and empty states for a list?

Answer:

Use the early return / guard clause pattern with four states.

First: if (isLoading) return <Spinner /> — show loading indicator.

Second: if (error) return <ErrorMessage error={error} /> — show error details.

Third: if (items.length === 0) return <EmptyState /> — show a helpful empty message.

Last: return the actual list with .map() — the happy path.

This order ensures each state gets its own clear UI. Never try to handle all states inline with nested ternaries — it becomes unreadable quickly.

Empty states should be descriptive — tell the user WHY it is empty and what action they can take (e.g., "No items in cart. Start shopping →").

Q8: Can you use if/else directly inside JSX return?

Answer:

No — you cannot use if/else statements directly inside JSX. JSX only accepts expressions, not statements. if/else is a statement.

But you CAN use if/else in three places: (1) Before the return — write full if/else blocks and return JSX or assign to a variable. (2) Inside a function called in JSX — {renderContent()}. (3) Using ternary operator — which IS an expression: condition ? A : B.

The ternary operator is the JSX-compatible alternative to if/else when you need it inline.

For complex logic: move it outside JSX. Define a variable or function, then use the result inside JSX.

Q9: How do you apply conditional CSS classes in React?

Answer:

Three approaches depending on complexity.

For simple binary: className={isActive ? "active" : "inactive"}

For multiple conditions: build an array, filter out false values, join with spaces: [base, cond1 && "class1", cond2 && "class2"].filter(Boolean).join(" ")

For complex class logic in real projects: use the clsx library (very popular). clsx("base", isActive && "active", isDisabled && "disabled") — automatically handles falsy values.

Inline styles: use style={{ property: value }} with camelCase properties. Only for truly dynamic values from data. Everything else should be CSS classes.

Q10: What happens if you forget to add a key prop to list items?

Answer:

React shows a console warning: "Each child in a list should have a unique key prop."

Functionally, the app still works in most cases — React falls back to index-based comparison.

But it causes: performance issues (React re-renders more than necessary), and subtle bugs (incorrect state association when items are reordered or deleted).

The most visible bug: if you have inputs in a list without keys, deleting an item can cause the wrong input's value to appear in the remaining items — the state migrates to the wrong position.

Always add keys. Make it a habit. React's warning is React telling you "I cannot be efficient without this."

Q11: When would you use React.Fragment with a key?

Answer:

When each list item needs to render multiple sibling elements, but you do not want a wrapper div in the actual DOM.

The <> </> shorthand cannot accept a key prop. So when you need a key on a fragment, you must use the full <React.Fragment key={id}> syntax.

Example: rendering a definition list where each entry is a <dt> term and <dd> definition. You cannot wrap them in a div (breaks dl semantic structure). You cannot use <> (no key). Solution: <React.Fragment key={item.id}>.

This keeps your HTML semantically correct while React can still track items efficiently with keys.

Q12: How do you sort a list in React without mutating state?

Answer:

JavaScript's .sort() method mutates the array in place. Calling it directly on the state array would mutate state — which React prohibits.

Always create a copy before sorting: [...products].sort((a, b) => ...) or products.slice().sort((a, b) => ...)

In practice: sort the derived array, not the state. const sorted = [...products].sort(...). Then render sorted.map(...).

Even better: chain filter and sort together as a derived value: const displayItems = products.filter(...).sort(...). Never store sorted or filtered arrays in state — they go stale. Calculate them fresh every render.

This is the same "derived state" principle from Chapter 3 — calculate from existing state, never store separately.

Common Mistakes in Conditional Rendering & Lists

Mistake 1: The number zero && bug

```
// ❌ WRONG — renders "0" on screen when array is empty
{items.length && <ItemList items={items} />}

// ✅ CORRECT — explicit boolean comparison
{items.length > 0 && <ItemList items={items} />}
```

Mistake 2: Using Math.random() as key

```
// ❌ TERRIBLE — new random key every render = total remount every time
{items.map(item => <Card key={Math.random()} item={item} />)}

// ✅ CORRECT — stable unique ID
{items.map(item => <Card key={item.id} item={item} />)}
```

Mistake 3: Mutating state array with .sort()

```
// ❌ WRONG — .sort() mutates the original state array
const sorted = products.sort((a, b) => a.price - b.price);

// ✅ CORRECT — spread to copy first, then sort
const sorted = [...products].sort((a, b) => a.price - b.price);
```

Mistake 4: Nesting too many ternaries

```
// ❌ HARD TO READ — deeply nested ternaries
{isLoading ? <Spinner /> : error ? <Error /> : data.length === 0 ? <Empty /> :
<List data={data} />}

// ✅ CLEAN — early returns before the main return
if (isLoading) return <Spinner />;
```

```
if (error)     return <Error />;
if (!data.length) return <Empty />;
return <List data={data} />;
```

Quick Reference — Chapter 7 Summary

Concept	Syntax	Use When
Early return	if (loading) return <X>	Loading, error, empty — guard clauses
Ternary	cond ? <A /> : <B />	Show one of exactly two things
AND operator	cond && <A />	Show something or nothing
Object lookup	MAP[key] || <Default />	3+ exclusive conditions, status maps
list.map()	{items.map(i => <Card key={i.id} />)}	Render array as UI elements
key prop	key={item.id}	Every list item, use stable unique ID
Filter then map	.filter().map()	Show subset of list
Fragment with key	<React.Fragment key={id}>	Multiple siblings in list, no div wrapper
Conditional class	clsx("base", cond && "extra")	Dynamic CSS based on state
&& number bug	length > 0 && <X />	Always boolean, never raw number

✅ **Remember**

Four list states — always handle: Loading → Error → Empty → Data.

key prop — unique, stable ID. Never index (for mutable lists). Never Math.random(). Never missing.

&& with numbers — items.length && shows "0". Use items.length > 0 && instead.

Sort safely — [...array].sort() not array.sort(). Spread first.

Early return = cleanest pattern for multiple conditions. Guards first, happy path last.

Ternary = A or B. **&&** = A or nothing. **if/else** = complex logic.

CSS vs null — null removes from DOM (resets state). display:none hides but preserves state.

Fragment key — use <React.Fragment key={id}> when shorthand <> cannot accept key.

End of Chapter 7 • Next: Chapter 8 — Context API

Chapter 8: Context API

Global data without prop drilling — explained so simply you will implement it in 5 minutes

By Syed Shaaz Akhtar | Interview Focused Guide

Remember Prop Drilling from Chapter 2? Passing data through 5 components just to reach the one that needs it? Context API is the solution. It lets ANY component in your app access shared data directly — without passing it through every parent in between.

This chapter is designed so you not only understand Context — you can IMPLEMENT it from memory after reading this once. Every concept builds on the previous one. Follow the steps. Do not skip.

> 💡 **Tip**
> 📡 THE ANALOGY — remember this for the entire chapter:
> Context API is like a WIFI ROUTER in your house.
> — Without WiFi: you have to physically pass a cable from room to room to room to reach the last room. That is prop drilling.
> — With WiFi (Context): the router broadcasts the signal. ANY room can connect directly — no cables needed.
> The router = Context Provider (broadcasts data).
> Each room = Component that needs the data.
> Connecting to WiFi = useContext hook (accessing the data).
> The WiFi password = the value you put in the Provider.

Topic 1: The Problem Context Solves — Quick Recap

You already know this from Chapter 2. But let us see it one more time with a clear example — so you can feel the pain that Context removes.

```
// THE PROP DRILLING PAIN — user data needed 4 levels deep

// Level 1: App — has the logged-in user
function App() {
  const [user] = useState({ name: "Shaaz", role: "Admin", avatar: "pic.jpg" });
  return <Dashboard user={user} />;        // pass down
}

// Level 2: Dashboard — does NOT need user, just passes it
function Dashboard({ user }) {
  return <Sidebar user={user} />;          // pass down again
}

// Level 3: Sidebar — does NOT need user, just passes it
function Sidebar({ user }) {
  return <UserProfile user={user} />;      // pass down again
```

```
}

// Level 4: UserProfile — FINALLY uses user
function UserProfile({ user }) {
  return <h2>Hello, {user.name}! ({user.role})</h2>;
}

// Dashboard and Sidebar are POLLUTED with a prop they never use.
// If user shape changes — update ALL four files. Painful.
```

> ✅ **Remember**
>
> Prop drilling is NOT always bad. For 1-2 levels, just pass props normally.
>
> Prop drilling becomes PAINFUL at 3+ levels, especially when middle components do not use the data.
>
> Context removes the pain by letting components access data DIRECTLY — no middle-man.

Topic 2: The 3 Steps of Context API — Learn Once, Use Forever

Context API has exactly 3 steps. These 3 steps are the SAME every single time — whether you are sharing user data, theme, language, cart, or anything else. Learn these 3 steps and you have learned Context API.

Step	What You Do	Code	WiFi Analogy
Step 1: CREATE	Create the context	createContext()	Buy the WiFi router
Step 2: PROVIDE	Wrap components, set the value	<Context.Provider value={}>	Turn on the router, set the password
Step 3: CONSUME	Read the value in any component	useContext(Context)	Connect your device to WiFi

Topic 3: Step 1 — createContext (Create the Router)

The first step is creating the context. You do this ONCE — outside any component — and export it so other files can use it.

```
// UserContext.js — create and export the context
import { createContext } from "react";

// createContext(defaultValue)
// defaultValue = what useContext returns if there is NO Provider above it
// Usually set to null — means "no provider found"
const UserContext = createContext(null);

export default UserContext;
```

```
// That is literally it. Step 1 is done.
// createContext just creates an empty "channel" for data to flow through.
```

📝 **Note**

The default value in createContext(defaultValue) is used ONLY when a component calls useContext but there is no Provider above it in the tree.

In practice, you almost always have a Provider, so the default value is rarely used.

Setting defaultValue to null is fine. Some teams put the actual shape of the data as default for better TypeScript autocomplete.

Topic 4: Step 2 — Provider (Turn On the Router)

The Provider is the component that broadcasts the data. You wrap it around any part of your component tree. Every component INSIDE the Provider can access the data. Components OUTSIDE cannot.

```
// App.jsx — provide the context at the top level
import { useState } from "react";
import UserContext from "./UserContext";

function App() {
  const [user, setUser] = useState({
    name:   "Shaaz Akhtar",
    role:   "Admin",
    avatar: "shaaz.jpg",
    email:  "shaaz@example.com",
  });

  return (
    // Wrap your entire app (or just the part that needs this context)
    // value = what you are broadcasting — any JS value
    <UserContext.Provider value={{ user, setUser }}>
      <Dashboard />    // Dashboard does NOT receive user as prop anymore!
      <NavBar />        // NavBar can access user directly via useContext
      <Footer />        // Footer too — any component inside Provider
    </UserContext.Provider>
  );
}

// Notice: NONE of the children receive user as a prop.
// The Provider is broadcasting it. They just tune in.
```

💡 **Tip**

You can have MULTIPLE providers wrapping the same tree:

<ThemeContext.Provider value={theme}>

<UserContext.Provider value={{ user, setUser }}>

```
    <App />
  </UserContext.Provider>
</ThemeContext.Provider>
```

Each context is independent. A component can consume as many contexts as it needs.

Topic 5: Step 3 — useContext (Connect to WiFi)

The final step is consuming the context inside any component. You just call useContext and pass it the context object you created in Step 1. That is it — you get the value instantly.

```
// UserProfile.jsx — anywhere inside the Provider, even 10 levels deep
import { useContext } from "react";
import UserContext from "./UserContext";

function UserProfile() {
  // useContext(UserContext) reads the value from the nearest Provider above
  const { user } = useContext(UserContext);

  return (
    <div>
      <img src={user.avatar} alt={user.name} />
      <h2>{user.name}</h2>
      <p>Role: {user.role}</p>
    </div>
  );
}

// NavBar.jsx — also inside the Provider, completely separate part of tree
function NavBar() {
  const { user } = useContext(UserContext);
  return (
    <nav>
      <span>Welcome, {user.name}</span>
      <img src={user.avatar} width={30} />
    </nav>
  );
}

// Dashboard and Sidebar are now CLEAN — no user prop at all
function Dashboard() {
  return <Sidebar />;  // no props passed, no pollution
}

function Sidebar() {
  return <UserProfile />;  // no props passed
}
```

✅ **Remember**

The 3 steps — always the same:
1. createContext() → create the channel
2. <Context.Provider value={...}> → broadcast the data
3. useContext(Context) → receive the data

Any component inside the Provider can useContext. No matter how deep.

Components OUTSIDE the Provider get the default value (usually null).

Topic 6: The BEST Pattern — Custom Provider + Custom Hook

The 3-step approach above works. But in real projects, everyone uses a cleaner pattern that wraps the Provider in its own component and creates a custom hook for consuming it. This is the PROFESSIONAL way to use Context.

This pattern keeps all context logic in ONE file, hides the implementation details, and gives a much better developer experience. You will see this in every real codebase.

```
// UserContext.jsx — ALL context logic in one file
import { createContext, useContext, useState } from "react";

// STEP 1: Create the context (keep it private — not exported)
const UserContext = createContext(null);

// STEP 2: Custom Provider component — wraps the logic
export function UserProvider({ children }) {
  const [user, setUser] = useState(null);
  const [isLoading, setIsLoading] = useState(true);

  // Login function — updates user state
  async function login(email, password) {
    setIsLoading(true);
    const data = await fetch("/api/login", {
      method: "POST",
      body: JSON.stringify({ email, password })
    }).then(r => r.json());
    setUser(data.user);
    setIsLoading(false);
  }

  // Logout function — clears user state
  function logout() {
    setUser(null);
  }

  // Provide everything components might need
  const value = { user, isLoading, login, logout };

  return (
    <UserContext.Provider value={value}>
      {children}
```

```
    </UserContext.Provider>
  );
}

// STEP 3: Custom hook — hides useContext, gives better error message
export function useUser() {
  const context = useContext(UserContext);

  // Guard: if someone calls useUser() outside the Provider — helpful error
  if (context === null) {
    throw new Error("useUser must be used inside a <UserProvider>.");
  }

  return context;
}

// That is it. This one file handles everything.
```

```
// main.jsx — wrap your app with the Provider
import { UserProvider } from "./UserContext";

ReactDOM.createRoot(document.getElementById("root")).render(
  <UserProvider>
    <App />
  </UserProvider>
);

// NavBar.jsx — consume with the custom hook (clean!)
import { useUser } from "./UserContext";

function NavBar() {
  const { user, logout } = useUser();  // one line, clean

  if (!user) return <a href="/login">Login</a>;

  return (
    <nav>
      <span>Hello, {user.name}</span>
      <button onClick={logout}>Logout</button>
    </nav>
  );
}

// LoginPage.jsx — also consume with the custom hook
import { useUser } from "./UserContext";

function LoginPage() {
  const { login, isLoading } = useUser();
  const [email, setEmail]       = useState("");
  const [password, setPassword] = useState("");
```

```
  async function handleSubmit(e) {
    e.preventDefault();
    await login(email, password);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={email}    onChange={e => setEmail(e.target.value)}
placeholder="Email" />
      <input value={password} onChange={e => setPassword(e.target.value)}
placeholder="Password" type="password" />
      <button disabled={isLoading}>{isLoading ? "Logging in..." :
"Login"}</button>
    </form>
  );
}
```

💡 **Tip**

Why is this pattern better?

1. The context itself is PRIVATE (not exported) — only the Provider and useUser hook are public.
2. The useUser hook throws a helpful error if used outside the Provider — catches mistakes instantly.
3. All logic (login, logout, state) lives in ONE file — easy to find and change.
4. Consumers just call useUser() — they do not need to import the context object at all.

This is the pattern used in React Router, React Query, and every popular React library.

Topic 7: Real Example 1 — Theme Context (Dark / Light Mode)

Theme switching is the most classic Context API example. Every interview where they ask you to implement Context — build a theme toggle. Here is the complete implementation.

```
// ThemeContext.jsx — complete theme context in one file
import { createContext, useContext, useState } from "react";

const ThemeContext = createContext(null);

export function ThemeProvider({ children }) {
  const [theme, setTheme] = useState("light"); // "light" or "dark"

  function toggleTheme() {
    setTheme(prev => prev === "light" ? "dark" : "light");
  }

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {/* Apply theme class to a wrapper div — affects all children */}
      <div className={`app-wrapper theme-${theme}`}>
```

```
        {children}
      </div>
    </ThemeContext.Provider>
  );
}

export function useTheme() {
  const context = useContext(ThemeContext);
  if (!context) throw new Error("useTheme must be inside ThemeProvider");
  return context;
}
```

```
// main.jsx
import { ThemeProvider } from "./ThemeContext";
ReactDOM.createRoot(document.getElementById("root")).render(
  <ThemeProvider><App /></ThemeProvider>
);

// ToggleButton.jsx — anywhere in the app
import { useTheme } from "./ThemeContext";

function ThemeToggleButton() {
  const { theme, toggleTheme } = useTheme();
  return (
    <button onClick={toggleTheme}>
      {theme === "light" ? "🌙 Dark Mode" : "☀️ Light Mode"}
    </button>
  );
}

// Header.jsx — reads theme for conditional styling
function Header() {
  const { theme } = useTheme();
  return (
    <header style={{
      background: theme === "dark" ? "#1a1a1a" : "#ffffff",
      color:      theme === "dark" ? "#ffffff" : "#000000",
    }}>
      <h1>My App</h1>
      <ThemeToggleButton />
    </header>
  );
}
```

Topic 8: Real Example 2 — Cart Context (E-commerce)

Cart state is another perfect use case for Context. The cart count is shown in the NavBar. The cart contents are shown on the Cart page. When a product is added from any page — both must update. Context handles this perfectly.

```
// CartContext.jsx — complete cart context
import { createContext, useContext, useState } from "react";

const CartContext = createContext(null);

export function CartProvider({ children }) {
  const [cartItems, setCartItems] = useState([]);

  // Derived values — calculated, not stored
  const cartCount = cartItems.length;
  const cartTotal = cartItems.reduce((sum, item) => sum + item.price * item.qty,
0);

  function addToCart(product) {
    setCartItems(prev => {
      // Check if product already in cart
      const existing = prev.find(item => item.id === product.id);
      if (existing) {
        // Increase quantity
        return prev.map(item =>
          item.id === product.id ? { ...item, qty: item.qty + 1 } : item
        );
      }
      // Add new item with qty=1
      return [...prev, { ...product, qty: 1 }];
    });
  }

  function removeFromCart(productId) {
    setCartItems(prev => prev.filter(item => item.id !== productId));
  }

  function updateQty(productId, qty) {
    if (qty <= 0) { removeFromCart(productId); return; }
    setCartItems(prev =>
      prev.map(item => item.id === productId ? { ...item, qty } : item)
    );
  }

  function clearCart() {
    setCartItems([]);
  }

  return (
    <CartContext.Provider value={{
      cartItems, cartCount, cartTotal,
      addToCart, removeFromCart, updateQty, clearCart
    }}>
      {children}
    </CartContext.Provider>
```

```
  );
}

export function useCart() {
  const context = useContext(CartContext);
  if (!context) throw new Error("useCart must be inside CartProvider");
  return context;
}
```

```
// NavBar.jsx — shows cart count badge
function NavBar() {
  const { cartCount } = useCart();
  return (
    <nav>
      <span>🛒 {cartCount > 0 ? cartCount : ""}</span>
    </nav>
  );
}

// ProductCard.jsx — adds to cart
function ProductCard({ product }) {
  const { addToCart } = useCart();
  return (
    <div>
      <h3>{product.name}</h3>
      <p>₹{product.price}</p>
      <button onClick={() => addToCart(product)}>Add to Cart</button>
    </div>
  );
}

// CartPage.jsx — shows cart contents
function CartPage() {
  const { cartItems, cartTotal, removeFromCart, updateQty, clearCart } =
useCart();

  if (cartItems.length === 0) return <p>Your cart is empty.</p>;

  return (
    <div>
      {cartItems.map(item => (
        <div key={item.id}>
          <span>{item.name}</span>
          <input
            type="number"
            value={item.qty}
            onChange={e => updateQty(item.id, Number(e.target.value))}
          />
          <span>₹{item.price * item.qty}</span>
          <button onClick={() => removeFromCart(item.id)}>Remove</button>
        </div>
```

```
      ))}
      <p>Total: ₹{cartTotal}</p>
      <button onClick={clearCart}>Clear Cart</button>
    </div>
  );
}
```

Topic 9: Context Re-renders — The Important Performance Issue

This is the most important performance thing to know about Context. It is asked in almost every senior React interview.

When the value in a Context.Provider changes, EVERY component that calls useContext for that context re-renders — even if it only uses a part of the value that did not change.

```
// PROBLEM: ALL consumers re-render when ANY part of value changes
function App() {
  const [user, setUser]   = useState({ name: "Shaaz" });
  const [theme, setTheme] = useState("light");

  // BAD: combining user and theme in ONE context
  return (
    <AppContext.Provider value={{ user, setUser, theme, setTheme }}>
      <NavBar />        // uses ONLY user
      <MainContent />   // uses ONLY theme
      <Settings />      // uses both
    </AppContext.Provider>
  );
}

// When theme changes → value object changes → ALL consumers re-render
// NavBar re-renders even though it only uses user — wasted render!

// SOLUTION 1: Split into separate contexts
function App() {
  const [user, setUser]   = useState({ name: "Shaaz" });
  const [theme, setTheme] = useState("light");

  return (
    <UserContext.Provider value={{ user, setUser }}>
      <ThemeContext.Provider value={{ theme, setTheme }}>
        <NavBar />        // useUser → only re-renders when user changes
        <MainContent />   // useTheme → only re-renders when theme changes
        <Settings />      // uses both contexts independently
      </ThemeContext.Provider>
    </UserContext.Provider>
  );
}
```

```
// SOLUTION 2: useMemo to stabilise the value object
```

```
import { useMemo } from "react";

function UserProvider({ children }) {
  const [user, setUser] = useState(null);

  // Without useMemo: new object every render = all consumers re-render
  // const value = { user, setUser };  // new reference every time

  // With useMemo: same object reference unless user actually changes
  const value = useMemo(() => ({ user, setUser }), [user]);

  return (
    <UserContext.Provider value={value}>
      {children}
    </UserContext.Provider>
  );
}

// Now consumers only re-render when "user" actually changes
// Not on every parent render
```

✅ **Remember**

Rule 1: Split contexts by how often they change. User data (rarely) in one context. UI state (often) in another.

Rule 2: If the Provider's parent re-renders and re-creates the value object, all consumers re-render. Use useMemo to prevent this.

Rule 3: Context is NOT a replacement for local state. If data only one component needs — keep it local.

Rule 4: Context is NOT always better than prop drilling. For 2-3 levels — just pass props. Context adds complexity.

Topic 10: Multiple Contexts — Composing Providers

Real apps usually have multiple contexts — user, theme, cart, language. You can compose them by nesting Providers. A common pattern is creating a single "AppProviders" component that wraps all of them.

```
// providers.jsx — all providers in one place (clean pattern)
import { UserProvider }  from "./contexts/UserContext";
import { ThemeProvider } from "./contexts/ThemeContext";
import { CartProvider }  from "./contexts/CartContext";
import { LangProvider }  from "./contexts/LangContext";

// Compose all providers into one component
function AppProviders({ children }) {
  return (
    <UserProvider>
      <ThemeProvider>
```

```
        <CartProvider>
          <LangProvider>
            {children}
          </LangProvider>
        </CartProvider>
      </ThemeProvider>
    </UserProvider>
  );
}

// main.jsx — clean and simple
import { AppProviders } from "./providers";

ReactDOM.createRoot(document.getElementById("root")).render(
  <AppProviders>
    <App />
  </AppProviders>
);

// Any component can now use any context:
function CheckoutPage() {
  const { user }               = useUser();
  const { theme }              = useTheme();
  const { cartItems, cartTotal } = useCart();
  const { t }                  = useLang(); // translation function

  return (
    <div className={`page theme-${theme}`}>
      <h1>{t("checkout.title")}</h1>
      <p>{t("checkout.welcome", { name: user.name })}</p>
      <p>Total: ₹{cartTotal}</p>
    </div>
  );
}
```

Topic 11: Context vs Prop Drilling vs Redux — When to Use What

This is the most asked comparison question in React interviews. Knowing when to use each approach shows senior-level thinking.

Approach	Best For	When to Avoid	Examples
Prop Drilling	1-3 levels deep, simple parent-child	More than 3 levels, many components need it	Button variant, modal title
Context API	App-wide semi-static data, 4+ levels	Frequently changing data (causes re-renders)	Auth user, theme, language,

			cart
Redux Toolkit	Complex, frequently changing shared state	Simple apps — over-engineering	Real-time chat, complex dashboards
Zustand	Simpler global state, less boilerplate	When team already uses Redux	Shopping cart, UI state, filters

```
// DECISION FLOW — ask these questions:

// Q1: Does only ONE component need this data?
//     YES → Keep it local (useState inside that component)
//     NO  → continue to Q2

// Q2: Do only 2-3 nearby components need it?
//     YES → Lift State Up (Chapter 3 pattern)
//     NO  → continue to Q3

// Q3: Do many components at different levels need it?
//     YES → Use Context API

// Q4: Does the data change very frequently (every second/keystroke)?
//     YES → Consider Redux/Zustand (Context re-renders too much)
//     NO  → Context API is fine

// Examples:
// Is the user logged in? → Context (used everywhere, rarely changes)
// Current theme?         → Context (app-wide, changes rarely)
// Shopping cart?         → Context or Zustand (app-wide, changes often)
// A counter button?      → Local state (only that component needs it)
// Form field value?      → Local state (only that form needs it)
// Real-time stock prices?→ Redux/Zustand (changes every second)
```

Topic 12: Complete Implementation — Auth Context (Most Common Interview Ask)

The most common Context implementation you will be asked to build in an interview is an Authentication context. Here is the COMPLETE implementation you can memorise and reproduce. Every line has a comment explaining why it is there.

```
// AuthContext.jsx — complete authentication context
import { createContext, useContext, useState, useEffect } from "react";

// 1. Create context — private
const AuthContext = createContext(null);
```

```
// 2. Provider component
export function AuthProvider({ children }) {
  const [user, setUser]       = useState(null);   // null = not logged in
  const [loading, setLoading] = useState(true);   // checking auth on load

  // On first load — check if user has a saved session (e.g. token)
  useEffect(() => {
    const savedUser = localStorage.getItem("user");
    if (savedUser) {
      setUser(JSON.parse(savedUser));  // restore session
    }
    setLoading(false);
  }, []);

  // Login — sets user and saves to localStorage
  async function login(email, password) {
    const res = await fetch("/api/login", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ email, password }),
    });
    if (!res.ok) throw new Error("Invalid credentials");
    const data = await res.json();
    setUser(data.user);
    localStorage.setItem("user", JSON.stringify(data.user));
  }

  // Logout — clears user and localStorage
  function logout() {
    setUser(null);
    localStorage.removeItem("user");
  }

  // Computed — derive isLoggedIn from user state
  const isLoggedIn = user !== null;
  const isAdmin    = user?.role === "admin";

  const value = { user, loading, isLoggedIn, isAdmin, login, logout };

  // Show nothing while checking saved session (prevents flash)
  if (loading) return <div>Loading...</div>;

  return (
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
}

// 3. Custom hook — useAuth
export function useAuth() {
```

```
  const context = useContext(AuthContext);
  if (!context) throw new Error("useAuth must be inside <AuthProvider>");
  return context;
}
```

```
// Using the Auth context throughout the app:

// main.jsx
import { AuthProvider } from "./AuthContext";
ReactDOM.createRoot(document.getElementById("root")).render(
  <AuthProvider><App /></AuthProvider>
);

// NavBar.jsx
function NavBar() {
  const { isLoggedIn, user, logout } = useAuth();
  return (
    <nav>
      {isLoggedIn
        ? <><span>{user.name}</span><button onClick={logout}>Logout</button></>
        : <a href="/login">Login</a>
      }
    </nav>
  );
}

// ProtectedRoute.jsx — redirect if not logged in
import { Navigate } from "react-router-dom";

function ProtectedRoute({ children }) {
  const { isLoggedIn } = useAuth();
  if (!isLoggedIn) return <Navigate to="/login" replace />;
  return children;
}

// AdminRoute.jsx — redirect if not admin
function AdminRoute({ children }) {
  const { isAdmin } = useAuth();
  if (!isAdmin) return <Navigate to="/unauthorized" replace />;
  return children;
}

// LoginPage.jsx
function LoginPage() {
  const { login } = useAuth();
  const [email, setEmail]       = useState("");
  const [password, setPassword] = useState("");
  const [error, setError]       = useState("");

  async function handleSubmit(e) {
```

```
    e.preventDefault();
    try {
      await login(email, password);  // updates context state
      // After login — NavBar automatically shows user name
      // Because all components consuming useAuth re-render
    } catch (err) {
      setError(err.message);
    }
  }

  return (
    <form onSubmit={handleSubmit}>
      <input value={email}    onChange={e => setEmail(e.target.value)}
placeholder="Email" />
      <input value={password} onChange={e => setPassword(e.target.value)}
type="password" placeholder="Password" />
      {error && <p className="error">{error}</p>}
      <button type="submit">Login</button>
    </form>
  );
}
```

🎯 Interview Tip

This AuthContext is a COMPLETE interview answer. When asked "Build an auth system with Context API" — this is exactly what you write.

Key points to mention: createContext → Provider with login/logout functions → useAuth custom hook → localStorage for persistence → loading state to prevent flash → ProtectedRoute using isLoggedIn.

Most candidates show only the basic 3 steps. Showing localStorage persistence and ProtectedRoute will make you stand out.

Chapter 8: Interview Questions with Full Answers

Q1: What is the Context API in React? Why was it introduced?

Answer:

Context API is React's built-in solution for sharing data across the component tree without passing it as props through every level. It is used for "global" data that many components need — like the logged-in user, current theme, or language preference.

It was introduced to solve prop drilling — where data has to be passed through many intermediate components that do not need it, just to reach a deeply nested component that does.

Before Context, developers had to either drill props (messy) or use external state management libraries like Redux (complex setup) even for simple global state.

Context is now built into React and covers most use cases for global state without any extra library.

Q2: What are the 3 steps to use Context API?

Answer:

Step 1 — createContext(): Create the context outside any component. This creates the "channel" that data flows through. Export the context so other files can reference it.

Step 2 — Provider: Wrap the components that need access with <Context.Provider value={data}>. The value prop is what gets shared. Any component inside this Provider can access the value.

Step 3 — useContext(): Inside any component that needs the data, call useContext(ContextObject). It returns the current value from the nearest Provider above it in the tree.

That is the complete API. Three things: create, provide, consume. The professional pattern also adds a custom Provider component and a custom hook (like useUser) that wraps useContext for better DX.

Q3: What is the performance issue with Context API? How do you fix it?

Answer:

When the value in a Context.Provider changes, EVERY component that consumes that context with useContext re-renders — even if the specific part of the value it uses did not change.

This happens because React cannot know which part of the value a component uses. Any change to the value object triggers all consumers.

Fix 1: Split into multiple contexts by concern. User data in UserContext, theme in ThemeContext. A theme change only re-renders theme consumers, not user consumers.

Fix 2: Wrap the value in useMemo inside the Provider. This ensures a new object is only created when the actual data changes, not on every parent render.

Fix 3: For very frequently changing data — consider Zustand or Redux which have more granular subscription control.

Q4: What is the difference between Context API and prop drilling?

Answer:

Prop drilling is passing data through every level of the component tree as props — even through components that do not need it. It works but gets messy at 3+ levels.

Context API eliminates the middle-man. Data goes from the Provider directly to the consuming component, skipping all intermediate levels.

Prop drilling is explicit — you can trace data flow by following props. Context is implicit — data appears in components without being visibly passed.

Prop drilling is better for 1-3 levels (explicit, no extra setup). Context is better for 3+ levels or app-wide data.

Context is not always the winner — it hides data flow, which can make debugging harder. Only use it when prop drilling is genuinely painful.

Q5: What is the default value in createContext and when is it used?

Answer:

The default value in createContext(defaultValue) is what useContext returns when there is NO Provider above the consuming component in the tree.

In practice, this is rarely used because you almost always have a Provider. Most teams set it to null and let the custom hook throw an error if used outside the Provider.

The error in the custom hook is much more helpful than receiving null silently: if (context === null) throw new Error("useUser must be inside UserProvider")

Some teams provide an actual default shape (like an empty user object) to help with TypeScript type inference and testing.

Q6: Can you update context data from a child component?

Answer:

Yes. Include a setter function in the context value.

In the Provider, define state with useState and put both the state value AND the setState function in the value prop: value={{ user, setUser }}.

Any child that consumes the context gets both: const { user, setUser } = useContext(UserContext). It can call setUser to update the shared state.

The cleaner approach is to provide named functions instead of raw setters: value={{ user, login, logout }} — the consumer calls login() without knowing the implementation.

This is how theme toggling works: the button calls toggleTheme(), which is defined in the Provider and calls setTheme internally.

Q7: What is the difference between Context API and Redux?

Answer:

Context API is built into React. No installation needed. Best for semi-static, app-wide data like user info and theme.

Redux is an external library with more structure. It has a single global store, reducers, actions, and middleware. Best for complex state that changes frequently.

Performance: Redux has optimised subscription — components only re-render when the specific slice they use changes. Context re-renders ALL consumers when ANY part changes.

Devtools: Redux has excellent browser devtools for time-travel debugging and action replay. Context has basic React DevTools support.

Boilerplate: Context is minimal setup. Redux Toolkit reduced boilerplate significantly, but still more than Context.

Rule: use Context for auth, theme, language. Use Redux/Zustand for complex app state, real-time features, or when performance is critical.

Q8: Why should you create a custom hook for consuming context?

Answer:

Three reasons.

First — better developer experience. Instead of importing both useContext and the context object in every consumer file, they just import useUser() and call it. One import, one call.

Second — error handling. The custom hook can check if it is being used inside the Provider and throw a helpful error message if not. Without this, the component silently gets null and fails mysteriously.

Third — abstraction. The consumer does not need to know HOW the data is stored. You could change from Context to Redux internally — the useUser() API stays the same. No consumer file needs to change.

This is the professional pattern used in React Router (useNavigate, useParams), React Query (useQuery), and every well-designed Context implementation.

Q9: How does Context compare to Zustand?

Answer:

Context API is built-in, zero dependencies. Zustand is a tiny external library (around 1kb).

Context re-renders all consumers when value changes. Zustand has fine-grained subscriptions — components only re-render when their specific slice of state changes.

Context requires Provider wrapper. Zustand state is accessible anywhere without Provider.

For simple cases (auth, theme) — Context is perfect. For frequently changing state (like a cart being updated every second, or filters being changed rapidly) — Zustand performs significantly better.

In 2024-2025, Zustand has become the go-to library for global state in new React projects. Context + Zustand together is a common pattern: Context for auth/theme, Zustand for everything else.

Q10: Can you have multiple contexts in one app?

Answer:

Yes — and you should, for performance reasons.

Each context is independent. A component can consume as many contexts as it needs by calling useContext multiple times (or using multiple custom hooks).

Keep contexts focused on one concern: UserContext for auth, ThemeContext for theme, CartContext for cart. Do not combine unrelated data into one mega-context.

A common pattern is creating an AppProviders component that nests all providers, and wrapping the entire app with it in main.jsx. This keeps main.jsx clean.

When one context's value changes, only consumers of THAT context re-render. Separate contexts = more granular re-render control.

Q11: How do you persist context state across page refreshes?

Answer:

Context state lives in memory — it is reset on every page refresh. To persist it, you sync it with localStorage or sessionStorage.

In the Provider: use lazy initialisation in useState to read from localStorage on first render: useState(() => JSON.parse(localStorage.getItem("key")) || defaultValue).

Then in the update functions: whenever state changes, also update localStorage: localStorage.setItem("key", JSON.stringify(newValue)).

For auth: save the user object or token to localStorage on login, read it on app load, and clear it on logout.

For theme: save the selected theme string. On load, read it and apply the correct theme class immediately to prevent a light-to-dark flash.

Q12: Build a simple theme context from scratch. Walk me through it.

Answer:

I will do it in one file with the professional pattern.

First, createContext(null) — creates the channel.

Then, ThemeProvider component: useState("light") for current theme. toggleTheme function that flips between light and dark. Provide both through Provider value.

Then, useTheme custom hook: calls useContext(ThemeContext), throws error if null, returns the context.

In main.jsx: wrap App with ThemeProvider. Any component uses useTheme() to get theme and toggleTheme.

Extensions I can add: persist to localStorage using lazy useState init and save on every toggle. Apply theme class to a wrapper div in the Provider. Support system preference with window.matchMedia("(prefers-color-scheme: dark)") for the initial value.

Common Mistakes with Context API

Mistake 1: Putting everything in one mega-context

```
// ❌ WRONG — one context for everything = every component re-renders on any change
<AppContext.Provider value={{ user, theme, cart, language, notifications }}>

// ✅ CORRECT — separate contexts for separate concerns
<UserContext.Provider value={{ user, login, logout }}>
  <ThemeContext.Provider value={{ theme, toggleTheme }}>
    <CartContext.Provider value={{ cart, addToCart }}>
      <App />
    </CartContext.Provider>
  </ThemeContext.Provider>
</UserContext.Provider>
```

Mistake 2: Using Context for local state

```
// ❌ WRONG — isOpen is only used by one component, no need for Context
const ModalContext = createContext();
// 20 lines of Provider setup...

// ✅ CORRECT — local state is perfectly fine here
function ProductCard() {
  const [isOpen, setIsOpen] = useState(false);
  return <div>{isOpen && <Modal />}</div>;
```

```
}
```

Mistake 3: Forgetting useMemo — re-creating the value object every render

```
// ❌ BAD — new object every render = all consumers re-render needlessly
function UserProvider({ children }) {
  const [user, setUser] = useState(null);
  return <UserContext.Provider value={{ user, setUser
}}>{children}</UserContext.Provider>;
}

// ✅ BETTER — useMemo stabilises the reference
function UserProvider({ children }) {
  const [user, setUser] = useState(null);
  const value = useMemo(() => ({ user, setUser }), [user]);
  return <UserContext.Provider value={value}>{children}</UserContext.Provider>;
}
```

Mistake 4: Not handling the null default — silent failures

```
// ❌ BAD — if used outside Provider, context is null, crashes silently
export function useUser() {
  return useContext(UserContext);  // returns null — crash later
}

// ✅ GOOD — throw early with a helpful message
export function useUser() {
  const context = useContext(UserContext);
  if (context === null) {
    throw new Error("useUser must be used inside a <UserProvider>.");
  }
  return context;
}
```

Quick Reference — Chapter 8 Summary

Concept	One-liner	Code
Create context	Create the data channel	const Ctx = createContext(null)
Provider	Broadcast data to all children	<Ctx.Provider value={data}>
Consume	Receive data in any child component	const data = useContext(Ctx)

Custom hook	Wrap useContext, add error guard	function useCtx() { return useContext(Ctx) }
Custom provider	Wrap logic + Provider in one component	function CtxProvider({ children }) { ... }
Re-render issue	All consumers re-render on value change	Split contexts or use useMemo on value
Multiple contexts	Stack providers, each independent	<UserCtx><ThemeCtx><App />
Persistence	Sync with localStorage on state change	localStorage.setItem in update functions

✅ **Remember**

3 steps always: createContext → Provider → useContext.
Professional pattern: custom Provider component + custom hook (useUser, useTheme, useCart).
WiFi analogy: Router = Provider. Connecting = useContext. Password = value prop.
Re-render rule: ALL consumers re-render when value changes — split contexts to minimise this.
useMemo on value: prevents unnecessary re-renders when Provider parent re-renders.
Error guard in hook: throw if context === null — better than silent crash.
When to use Context: 3+ levels deep, many components need it, semi-static data.
When NOT to use: single component, 1-2 levels (just pass props), frequently changing data.

End of Chapter 8 • Next: Chapter 9 — React Router

Chapter 9: React Router

Routing, Navigation, Protected Routes — every pattern used in real projects

By Syed Shaaz Akhtar | Interview Focused Guide

React by itself does not handle navigation between pages. When you go from Home to Profile to Settings in a React app, the browser does NOT reload. React Router intercepts the URL change and swaps components on screen — keeping the app alive. This is what makes React a Single Page Application (SPA).

This chapter covers React Router v6 — the current version used in all new projects. Every hook, every pattern, every interview question. Build as you read.

> 💡 **Tip**
> 🗺️ THE ANALOGY for this chapter:
> React Router is like a HOTEL RECEPTIONIST.
> — The hotel = your React app (one building, never closed)
> — Each room = a page component (Home, Profile, Settings)
> — The URL = the room number (/home, /profile, /settings)
> — The receptionist = React Router (reads the URL, sends you to the right room)
> — When you go to room 204 — the hotel does NOT rebuild. You just walk to a different room.
> — That is exactly how SPA routing works. Same app, different component shown.

Topic 1: How React Router Works — The Concept

In a traditional multi-page website, clicking a link sends a request to the server which returns a completely new HTML page. The browser reloads. All JavaScript state is lost.

In a React SPA, the server serves ONE HTML file once. After that, React Router takes over. It watches the browser URL. When it changes, React Router matches the URL to a route and renders the matching component. No server requests. No reload. Instant navigation.

Aspect	Traditional Multi-Page	React SPA with Router
Navigation	Browser sends request to server	JavaScript changes URL, React swaps component
Page reload	YES — full reload every navigation	NO — no reload, app stays alive
State	Lost on every navigation	Preserved (component state survives)
Speed	Slower — wait for server response	Instant — component already loaded
URL	Maps to a real file on server	Handled by React Router in browser
Back button	Server request for previous page	React Router pops history stack

Topic 2: Setup — Installing and Configuring React Router

React Router v6 is installed as a separate package. Setup takes 3 minutes.

```
// Step 1: Install
npm install react-router-dom

// Step 2: Wrap your app in BrowserRouter (in main.jsx)
import { BrowserRouter } from "react-router-dom";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);

// Step 3: Define routes (in App.jsx)
import { Routes, Route } from "react-router-dom";
import HomePage    from "./pages/HomePage";
import AboutPage   from "./pages/AboutPage";
import ProfilePage from "./pages/ProfilePage";
import NotFound    from "./pages/NotFound";

function App() {
  return (
    <Routes>
      <Route path="/"        element={<HomePage />} />
      <Route path="/about"   element={<AboutPage />} />
      <Route path="/profile" element={<ProfilePage />} />
      <Route path="*"        element={<NotFound />} />  {/* catch-all */}
    </Routes>
  );
}

// That is it. Your app now has client-side routing.
```

📝 **Note**

BrowserRouter uses the HTML5 History API to keep the UI in sync with the URL.

Routes = the container. It looks at all Route children and renders the FIRST one that matches the current URL.

Route path="*" is the catch-all — matches any URL not matched above. Use it for 404 pages.

In React Router v6, the "exact" prop is gone — all paths match exactly by default.

Topic 3: Link and NavLink — Navigating Without Page Reload

Never use <a href="..."> for internal navigation in React. It causes a full page reload and destroys your app state. Use **Link** or **NavLink** from React Router instead.

```
import { Link, NavLink } from "react-router-dom";

// Link — basic navigation, no page reload
function Header() {
  return (
    <nav>
      <Link to="/">Home</Link>
      <Link to="/about">About</Link>
      <Link to="/profile">Profile</Link>
      <Link to="/contact">Contact</Link>
    </nav>
  );
}

// NavLink — same as Link but adds "active" class when URL matches
// Perfect for navigation menus where active item should look different
function NavBar() {
  return (
    <nav>
      {/* NavLink automatically adds className="active" when path matches */}
      <NavLink to="/"       className={({ isActive }) => isActive ? "nav-active"
: "nav-link"}>
        Home
      </NavLink>
      <NavLink to="/about"  className={({ isActive }) => isActive ? "nav-active"
: "nav-link"}>
        About
      </NavLink>
      <NavLink to="/shop"   className={({ isActive }) => isActive ? "nav-active"
: "nav-link"}>
        Shop
      </NavLink>
    </nav>
  );
}

// Shorthand — NavLink default behaviour adds class "active" automatically
// Just style .active in CSS:
<NavLink to="/about">About</NavLink>
// When on /about → renders as: <a class="active" href="/about">About</a>
```

	<a href="">	<Link to="">	<NavLink to="">
Page reload?	YES — full reload	NO — SPA navigation	NO — SPA navigation

State lost?	YES	NO	NO
Active class?	Manual only	No built-in	YES — adds "active" automatically
Use for	External links only	Internal navigation	Navigation menus with active state

Topic 4: useNavigate — Programmatic Navigation

Sometimes you need to navigate PROGRAMMATICALLY — not from a user clicking a link, but from code. After a form submission, after login, after a timeout. That is what **useNavigate** is for.

```
import { useNavigate } from "react-router-dom";
```

```
// useNavigate returns a navigate function
function LoginPage() {
  const navigate = useNavigate();
  const [email, setEmail]       = useState("");
  const [password, setPassword] = useState("");

  async function handleLogin(e) {
    e.preventDefault();
    try {
      await loginAPI(email, password);
      // After successful login — go to dashboard
      navigate("/dashboard");
    } catch (err) {
      alert("Login failed");
    }
  }

  return (
    <form onSubmit={handleLogin}>
      <input value={email} onChange={e => setEmail(e.target.value)} />
      <input value={password} type="password" onChange={e =>
setPassword(e.target.value)} />
      <button type="submit">Login</button>
    </form>
  );
}

// navigate() options:
navigate("/dashboard");          // go to /dashboard
navigate("/dashboard", { replace: true }); // go + replace history (no back
button)
navigate(-1);                    // go BACK (like browser back button)
navigate(1);                     // go FORWARD
navigate(-2);                    // go 2 steps back
```

```
// navigate with state — pass data to next page without URL params
navigate("/order-success", {
  state: { orderId: "ORD-1234", total: 2499 }
});
```

✅ **Remember**

navigate("/path") = go forward to that URL.

navigate(-1) = go back (like pressing browser back button).

navigate("/path", { replace: true }) = go but REPLACE current history entry. User cannot press back to return.

Use replace: true after login, logout, or any navigation where going back would be confusing.

Topic 5: URL Parameters — useParams

URL parameters are dynamic parts of the URL. Like /users/42 where 42 is the user ID. Or /products/headphones where "headphones" is the product slug. React Router captures these and gives them to you via **useParams**.

```
// DEFINING dynamic routes — use :paramName syntax
function App() {
  return (
    <Routes>
      <Route path="/"                 element={<HomePage />} />
      <Route path="/users"            element={<UserListPage />} />
      <Route path="/users/:userId"    element={<UserDetailPage />} />
      {/* :userId is dynamic — matches /users/1, /users/42, /users/shaaz */}

      <Route path="/products/:category/:productId" element={<ProductPage />} />
      {/* Two params — matches /products/electronics/42 */}
    </Routes>
  );
}

// USING params — useParams gives you the dynamic parts
import { useParams } from "react-router-dom";

function UserDetailPage() {
  const { userId } = useParams();  // userId = "42" (string, not number!)

  const [user, setUser] = useState(null);

  useEffect(() => {
    fetch(`/api/users/${userId}`)  // fetch this specific user
      .then(r => r.json())
      .then(setUser);
  }, [userId]);  // re-fetch when userId changes
```

```
  if (!user) return <p>Loading...</p>;

  return (
    <div>
      <h1>{user.name}</h1>
      <p>ID: {userId}</p>
    </div>
  );
}

// Multiple params:
function ProductPage() {
  const { category, productId } = useParams();
  // URL: /products/electronics/42
  // category = "electronics", productId = "42"
  return <p>{category} — Product #{productId}</p>;
}
```

> 💡 **Tip**
>
> useParams always returns STRINGS — even if the param looks like a number.
>
> Always convert when needed: const id = Number(userId) or parseInt(userId, 10).
>
> Include userId in the useEffect dependency array — if the user navigates from /users/1 to /users/2, the component does NOT remount but the param changes. The effect must re-run to fetch new data.

Topic 6: Query Strings — useSearchParams

Query strings are the ?key=value part of a URL. Like /products?category=audio&sort=price&page=2. They are used for filters, search, sorting, and pagination — things that should be shareable in the URL.

```
import { useSearchParams } from "react-router-dom";

// useSearchParams works like useState — returns [value, setter]
function ProductsPage() {
  const [searchParams, setSearchParams] = useSearchParams();

  // Reading query params
  const category = searchParams.get("category") || "all";
  const sort     = searchParams.get("sort")     || "name";
  const page     = Number(searchParams.get("page")) || 1;
  const query    = searchParams.get("q")        || "";

  // Updating query params — updates URL without reload
  function handleCategoryChange(newCategory) {
    setSearchParams({
      category: newCategory,
      sort,
      page: 1,   // reset page when category changes
```

```
      q: query,
    });
    // URL becomes: /products?category=audio&sort=name&page=1&q=
  }

  function handleSearch(e) {
    setSearchParams(prev => {
      prev.set("q", e.target.value);
      prev.set("page", "1");   // reset page on new search
      return prev;
    });
  }

  return (
    <div>
      <input value={query} onChange={handleSearch} placeholder="Search..." />

      <select value={category} onChange={e =>
handleCategoryChange(e.target.value)}>
        <option value="all">All</option>
        <option value="audio">Audio</option>
        <option value="video">Video</option>
      </select>

      <p>Showing {category} products, sorted by {sort}, page {page}</p>
    </div>
  );
}

// Bonus: navigating with search params from another component
import { Link } from "react-router-dom";
<Link to="/products?category=audio&sort=price">Audio Products</Link>
```

📝 **Note**

Query params are great for shareable UI state. If a user copies the URL /products?category=audio&sort=price and shares it, their friend sees the exact same filtered view.

This is why search, filters, and pagination should always use query params — not local state.

searchParams.get("key") returns null if the param does not exist — always provide a fallback: searchParams.get("page") || 1.

Topic 7: useLocation — Reading Current URL Info

useLocation gives you the full details of the current URL — the path, query string, and any state that was passed during navigation. Very useful for analytics, conditional logic, and reading navigation state.

```
import { useLocation } from "react-router-dom";

function ProductPage() {
```

```
  const location = useLocation();

  console.log(location.pathname);  // "/products/42"
  console.log(location.search);    // "?color=blue&size=M"
  console.log(location.hash);      // "#reviews"
  console.log(location.state);     // data passed via navigate()

  return <div>Current path: {location.pathname}</div>;
}

// READING navigation state — data passed from navigate()
function OrderSuccessPage() {
  const location = useLocation();
  const { orderId, total } = location.state || {};
  // Came from: navigate("/order-success", { state: { orderId: "ORD-1234",
total: 2499 } })

  return (
    <div>
      <h1>Order Placed!</h1>
      <p>Order ID: {orderId}</p>
      <p>Total: ₹{total}</p>
    </div>
  );
}

// COMMON USE: redirect back to original page after login
function LoginPage() {
  const location  = useLocation();
  const navigate  = useNavigate();
  const { login } = useAuth();

  // The protected route sent them here with the original URL
  const from = location.state?.from || "/dashboard";

  async function handleLogin(e) {
    e.preventDefault();
    await login(email, password);
    navigate(from, { replace: true });  // go back to where they came from
  }

  return <form onSubmit={handleLogin}>...</form>;
}
```

Topic 8: Nested Routes — The Most Powerful Feature

Nested routes let you build layouts where only PART of the page changes when navigating, while the surrounding layout stays fixed. Think of a dashboard where the sidebar and header stay but only the main content area changes.

This is one of the most impressive React Router features to demonstrate in an interview.

```
// NESTED ROUTES SETUP

// App.jsx — define parent and child routes
function App() {
  return (
    <Routes>
      <Route path="/" element={<HomePage />} />

      {/* Dashboard has nested routes */}
      {/* The DashboardLayout renders, and inside it renders the child */}
      <Route path="/dashboard" element={<DashboardLayout />}>
        <Route index        element={<DashboardHome />} />    {/* /dashboard */}
        <Route path="stats" element={<StatsPage />} />        {/*
/dashboard/stats */}
        <Route path="users" element={<UsersPage />} />        {/*
/dashboard/users */}
        <Route path="users/:userId" element={<UserDetail />} />{/*
/dashboard/users/42 */}
        <Route path="settings" element={<SettingsPage />} />  {/*
/dashboard/settings */}
      </Route>

      <Route path="*" element={<NotFoundPage />} />
    </Routes>
  );
}
```

```
// DashboardLayout.jsx — the parent layout component
import { Outlet, NavLink } from "react-router-dom";

function DashboardLayout() {
  return (
    <div className="dashboard-container">

      {/* Sidebar — stays fixed on ALL dashboard pages */}
      <aside className="sidebar">
        <NavLink to="/dashboard">         🏠 Home</NavLink>
        <NavLink to="/dashboard/stats">   📊 Stats</NavLink>
        <NavLink to="/dashboard/users">   👥 Users</NavLink>
        <NavLink to="/dashboard/settings">⚙️ Settings</NavLink>
      </aside>

      {/* Main content area — THIS IS WHERE CHILD ROUTES RENDER */}
      <main className="dashboard-main">
        <Outlet />
        {/* <Outlet /> is the placeholder — React Router puts the matching
            child route component here. It changes on navigation.
            The sidebar never changes. Only Outlet content changes. */}
      </main>
```

```
    </div>
  );
}

// When URL is /dashboard        → DashboardLayout renders, Outlet shows
DashboardHome
// When URL is /dashboard/stats  → DashboardLayout renders, Outlet shows
StatsPage
// When URL is /dashboard/users  → DashboardLayout renders, Outlet shows
UsersPage
// The sidebar NEVER re-renders. Only the Outlet content swaps.
```

```
// index route — what shows at /dashboard (the parent path itself)
function DashboardHome() {
  return <h1>Welcome to Dashboard</h1>;
}

// Normal child routes
function StatsPage()   { return <h1>Statistics</h1>; }
function UsersPage()   { return <h1>All Users</h1>;  }
function SettingsPage(){ return <h1>Settings</h1>;   }
```

✅ Remember

<Outlet /> is the KEY concept in nested routes. It is where child route components are rendered.

Think of Outlet like a "window" in the parent layout. The frame (sidebar, header) stays. Only the window content changes.

index route = what renders when the URL exactly matches the parent path. No extra segment.

Nested routes can be nested further — you can have 3 or 4 levels of nesting for complex apps.

Topic 9: Protected Routes — Auth Guard Pattern

Protected routes are routes that only logged-in users can access. If a non-logged-in user tries to go to /dashboard, they get redirected to /login. This is implemented with a wrapper component using the Auth context from Chapter 8.

```
// ProtectedRoute.jsx — the auth guard component
import { Navigate, useLocation } from "react-router-dom";
import { useAuth } from "./AuthContext";  // from Chapter 8

function ProtectedRoute({ children }) {
  const { isLoggedIn } = useAuth();
  const location = useLocation();

  if (!isLoggedIn) {
    // Redirect to /login
    // Pass current URL as state so login page can redirect back after login
    return (
      <Navigate
```

```
        to="/login"
        state={{ from: location.pathname }}  // remember where they were
        replace                              // replace history, not add to it
      />
    );
  }

  return children;  // logged in — show the page
}

// AdminRoute.jsx — only for admin users
function AdminRoute({ children }) {
  const { isLoggedIn, isAdmin } = useAuth();
  const location = useLocation();

  if (!isLoggedIn) return <Navigate to="/login" state={{ from: location.pathname
}} replace />;
  if (!isAdmin)    return <Navigate to="/unauthorized" replace />;

  return children;
}
```

```
// App.jsx — using ProtectedRoute and AdminRoute
function App() {
  return (
    <Routes>
      {/* Public routes — anyone can access */}
      <Route path="/"      element={<HomePage />} />
      <Route path="/about" element={<AboutPage />} />
      <Route path="/login" element={<LoginPage />} />

      {/* Protected routes — only logged-in users */}
      <Route path="/dashboard" element={
        <ProtectedRoute>
          <DashboardLayout />
        </ProtectedRoute>
      }>
        <Route index         element={<DashboardHome />} />
        <Route path="profile" element={<ProfilePage />} />
        <Route path="settings" element={<SettingsPage />} />
      </Route>

      {/* Admin routes — only admins */}
      <Route path="/admin" element={
        <AdminRoute>
          <AdminLayout />
        </AdminRoute>
      }>
        <Route index        element={<AdminDashboard />} />
        <Route path="users" element={<ManageUsers />} />
      </Route>
```

```
      <Route path="/unauthorized" element={<UnauthorizedPage />} />
      <Route path="*"             element={<NotFoundPage />} />
    </Routes>
  );
}
```

```
// LoginPage.jsx — redirects back to original page after login
function LoginPage() {
  const { login }  = useAuth();
  const navigate   = useNavigate();
  const location   = useLocation();

  // Where the user was trying to go before being redirected to login
  const from = location.state?.from || "/dashboard";

  async function handleLogin(email, password) {
    await login(email, password);
    navigate(from, { replace: true });  // go back to original destination
  }

  return <LoginForm onLogin={handleLogin} />;
}

// Flow:
// 1. User visits /dashboard/settings (not logged in)
// 2. ProtectedRoute redirects to /login with state: { from:
"/dashboard/settings" }
// 3. User logs in
// 4. LoginPage navigates to location.state.from = "/dashboard/settings"
// 5. User lands exactly where they wanted to be
```

🎯 **Interview Tip**

This redirect-back-after-login pattern impresses interviewers. Most candidates just redirect to /dashboard after login.

Mention: "I save the original URL in navigation state using useLocation, and after login I redirect to location.state.from instead of a hardcoded path. This gives users a better experience."

Then show the ProtectedRoute component. This demonstrates you understand useLocation, Navigate, state passing, and the AuthContext together.

Topic 10: Navigate Component vs useNavigate Hook

React Router gives you TWO ways to redirect. **Navigate component** (used in JSX) and **useNavigate hook** (used in functions). Knowing when to use which is important.

```
// Navigate COMPONENT — used inside JSX/return
// Think of it as "render a redirect"
function ProtectedRoute({ children }) {
```

```
  const { isLoggedIn } = useAuth();

  if (!isLoggedIn) {
    return <Navigate to="/login" replace />;  // returns a redirect as JSX
  }

  return children;
}

// useNavigate HOOK — used in event handlers and async code
// Think of it as "run a redirect as a function"
function LoginPage() {
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault();
    await loginAPI(email, password);
    navigate("/dashboard");  // called after async operation
  }

  return <form onSubmit={handleSubmit}>...</form>;
}

// Rule:
// JSX return → use <Navigate />
// Event handler / async code → use useNavigate()
```

Topic 11: Route Parameters vs Query Strings — When to Use Which

Both route params and query strings put data in the URL. But they serve different purposes. Knowing which to use is a sign of experience.

Aspect	Route Params (:id)	Query Strings (?key=val)
Syntax	/users/:userId	/users?id=42
Purpose	Identify a specific RESOURCE	Filter, sort, paginate, search
Required?	YES — URL breaks without it	NO — optional, have defaults
Semantic	Part of the resource identity	Modifier/filter on the resource
Examples	/products/42, /users/shaaz	?sort=price&category=audio&page=2
When URL shared?	Shows same item to everyone	Shows same filter/view to everyone

```
// Route params — the RESOURCE is different
// /users/1 and /users/2 are DIFFERENT users
<Route path="/users/:userId" element={<UserPage />} />
```

```
// Query strings — same resource, different VIEW
// /products?sort=price and /products?sort=name are SAME products, different
order
<Route path="/products" element={<ProductsPage />} />

// Combined — specific resource with query modifiers
// /users/42/orders?status=pending&page=2
// User 42 (param), only pending orders (query), page 2 (query)
<Route path="/users/:userId/orders" element={<UserOrdersPage />} />
```

Topic 12: Lazy Loading Routes — Code Splitting

In a large app, loading all pages upfront makes the initial load slow. **Lazy loading** loads each page only when the user navigates to it. React's **React.lazy** and **Suspense** make this easy with React Router.

```
import { lazy, Suspense } from "react";
import { Routes, Route } from "react-router-dom";

// Instead of: import HomePage from "./pages/HomePage"
// Use lazy() — loads the file ONLY when the route is visited
const HomePage     = lazy(() => import("./pages/HomePage"));
const AboutPage    = lazy(() => import("./pages/AboutPage"));
const DashboardPage= lazy(() => import("./pages/DashboardPage"));
const AdminPage    = lazy(() => import("./pages/AdminPage"));

// Wrap Routes in Suspense — shows fallback while page is loading
function App() {
  return (
    <Suspense fallback={<div className="page-loader">Loading...</div>}>
      <Routes>
        <Route path="/"          element={<HomePage />} />
        <Route path="/about"     element={<AboutPage />} />
        <Route path="/dashboard" element={
          <ProtectedRoute><DashboardPage /></ProtectedRoute>
        } />
        <Route path="/admin"     element={
          <AdminRoute><AdminPage /></AdminRoute>
        } />
      </Routes>
    </Suspense>
  );
}

// What happens:
// Initial load → ONLY HomePage bundle downloaded (small, fast)
// User navigates to /about → AboutPage bundle downloaded (on demand)
// User navigates to /dashboard → DashboardPage downloaded (on demand)
// Each page is a separate JavaScript chunk loaded only when needed
```

> 💡 **Tip**
> Lazy loading is a MUST for large apps. It directly improves First Contentful Paint (FCP) — a key performance metric.
> The Suspense fallback shows while the lazy component is being downloaded. Make it a nice spinner or skeleton.
> React Router + React.lazy is the most common code splitting pattern in production React apps.
> Vite automatically splits code at lazy() boundaries — no extra config needed.

Topic 13: Complete App Example — Putting it All Together

Here is a complete routing setup for a real e-commerce app. This covers all the patterns from this chapter together.

```
// main.jsx
import { BrowserRouter } from "react-router-dom";
import { AuthProvider }  from "./contexts/AuthContext";
import { CartProvider }  from "./contexts/CartContext";

ReactDOM.createRoot(document.getElementById("root")).render(
  <BrowserRouter>
    <AuthProvider>
      <CartProvider>
        <App />
      </CartProvider>
    </AuthProvider>
  </BrowserRouter>
);

// App.jsx — complete routing for an e-commerce app
import { lazy, Suspense } from "react";
import { Routes, Route } from "react-router-dom";
import Layout          from "./components/Layout";
import ProtectedRoute  from "./components/ProtectedRoute";
import AdminRoute      from "./components/AdminRoute";

// Lazy load all pages
const Home          = lazy(() => import("./pages/Home"));
const ProductList   = lazy(() => import("./pages/ProductList"));
const ProductDetail = lazy(() => import("./pages/ProductDetail"));
const Cart          = lazy(() => import("./pages/Cart"));
const Login         = lazy(() => import("./pages/Login"));
const Register      = lazy(() => import("./pages/Register"));
const Dashboard     = lazy(() => import("./pages/Dashboard"));
const Orders        = lazy(() => import("./pages/Orders"));
const Profile       = lazy(() => import("./pages/Profile"));
const AdminPanel    = lazy(() => import("./pages/AdminPanel"));
const NotFound      = lazy(() => import("./pages/NotFound"));
```

```
function App() {
  return (
    <Suspense fallback={<div>Loading page...</div>}>
      <Routes>

        {/* Main layout wraps all pages */}
        <Route element={<Layout />}>

          {/* Public routes */}
          <Route index              element={<Home />} />
          <Route path="/products"   element={<ProductList />} />
          <Route path="/products/:productId" element={<ProductDetail />} />
          <Route path="/cart"       element={<Cart />} />
          <Route path="/login"      element={<Login />} />
          <Route path="/register"   element={<Register />} />

          {/* Protected routes — logged in users only */}
          <Route path="/account" element={<ProtectedRoute><Dashboard
/></ProtectedRoute>}>
            <Route index           element={<Dashboard />} />
            <Route path="orders"   element={<Orders />} />
            <Route path="profile"  element={<Profile />} />
          </Route>

          {/* Admin routes */}
          <Route path="/admin" element={<AdminRoute><AdminPanel /></AdminRoute>}
/>

          {/* 404 */}
          <Route path="*" element={<NotFound />} />
        </Route>

      </Routes>
    </Suspense>
  );
}

// Layout.jsx — navbar + footer always visible, Outlet in middle
function Layout() {
  return (
    <div>
      <NavBar />
      <main><Outlet /></main>
      <Footer />
    </div>
  );
}
```

Chapter 9: Interview Questions with Full Answers

Q1: What is React Router? Why do we need it?

Answer:

React Router is a third-party library (react-router-dom) that enables client-side routing in React SPAs. It synchronises the browser URL with React components — when the URL changes, React Router renders the matching component without a page reload.

We need it because React itself has no routing. Without React Router, navigating between sections would either cause a full page reload (losing all state) or require complex manual URL parsing.

React Router intercepts navigation, prevents the browser default behaviour (which would reload the page), updates the URL using the History API, matches the new URL to defined routes, and renders the matching component.

The result: instant navigation, preserved state, proper browser history (back/forward buttons work), and shareable URLs.

Q2: What is the difference between Link and NavLink?

Answer:

Both Link and NavLink render an anchor tag that navigates without page reload. The difference is in active state handling.

Link is basic — it navigates to the given path. No automatic styling based on current URL.

NavLink is Link with awareness of the current URL. It automatically applies an "active" class (or custom className via function) when the current URL matches the "to" prop. Perfect for navigation menus where the active item should look different.

NavLink className receives a function: className={({ isActive }) => isActive ? "active" : ""}. This gives full control over active styling.

Both are infinitely better than <a href=""> for internal navigation. The anchor tag causes a full page reload which destroys all React state.

Q3: What is the difference between useNavigate and the Navigate component?

Answer:

Both navigate to a new URL, but used in different contexts.

Navigate is a React component — you return it from render/JSX. When React renders a Navigate component, it immediately redirects to the specified path. Used for conditional redirects in render logic: if (!isLoggedIn) return <Navigate to="/login" replace />.

useNavigate is a hook that returns a navigate function — called imperatively in event handlers and async code. Used after async operations: await loginAPI(); navigate("/dashboard").

Rule of thumb: if you need to redirect as part of what renders (conditional), use Navigate component. If you need to redirect as part of what happens (after a button click, after an API call), use useNavigate.

Q4: What is useParams? How do you use it?

Answer:

useParams is a hook that returns the URL parameters from the current route. URL parameters are dynamic segments defined with : in the route path.

Example: if the route is /users/:userId and the current URL is /users/42, useParams returns { userId: "42" }.

Important: useParams always returns strings. Convert to number when needed: const id = Number(userId).

Include the param in useEffect dependencies: useEffect(() => { fetchUser(userId); }, [userId]). This ensures data re-fetches when navigation changes from /users/1 to /users/2 — the component stays mounted but the param changes.

Multiple params are supported: route /products/:category/:id gives both from useParams.

Q5: What is the Outlet component? When do you use it?

Answer:

Outlet is a placeholder component that renders the matched child route component inside a parent layout.

You use it in layout components — components that define the common structure (like sidebar + header) that should persist across multiple child routes. Put <Outlet /> where you want the child content to appear.

Example: DashboardLayout has a sidebar and header. In the centre, it renders <Outlet />. When URL is /dashboard/stats, React Router renders StatsPage in the Outlet. When URL is /dashboard/users, UsersPage renders in the Outlet. The sidebar never re-renders.

Outlet is what makes nested routing powerful. It enables true "layouts" where structure persists and only the content area changes.

Q6: How do you implement protected routes in React Router?

Answer:

Create a ProtectedRoute wrapper component that checks authentication status and either renders the children or redirects to login.

The component uses the auth context (useAuth) to get isLoggedIn. If logged in, it returns children. If not, it returns <Navigate to="/login" replace />.

Best practice: also save the current URL in navigation state so the user can be redirected back after login. Pass state={{ from: location.pathname }} on the Navigate. In the login page, read this with useLocation and navigate to location.state?.from after successful login.

For role-based routes (admin only), create a separate AdminRoute that checks both isLoggedIn and isAdmin, redirecting to different pages for each failure case.

Q7: What is the difference between route params and query strings?

Answer:

Route params (:id) are part of the URL path. They identify a specific resource. The URL is invalid without them. Example: /users/42 — 42 is the user ID, required.

Query strings (?key=value) are appended to the URL. They are optional modifiers — filters, sorts, search terms, pagination. Example: /products?sort=price&category=audio.

Use route params for: identifying a specific resource (user, product, post), navigating to a unique item, breadcrumb paths.

Use query strings for: search, filters, sorting, pagination, view preferences — anything that modifies how a resource is displayed but does not change which resource is shown.

They can be combined: /users/42/orders?status=pending shows user 42's orders (params) filtered to pending (query).

Q8: What is useSearchParams? How is it different from useParams?

Answer:

useSearchParams is for reading and updating query string parameters (?key=value in the URL). It works like useState — returns [searchParams, setSearchParams].

useParams is for reading URL path parameters (:paramName in the route path).

useSearchParams: searchParams.get("sort") reads the sort param. setSearchParams({ sort: "price", page: 2 }) updates the URL query string without navigation.

useParams: const { userId } = useParams() reads :userId from the path.

A key advantage of useSearchParams: changing query params is shareable — users can bookmark/share the exact filter view.

Q9: What is lazy loading in React Router? How do you implement it?

Answer:

Lazy loading means loading page components on demand — only when the user navigates to that route — instead of loading everything upfront.

This reduces the initial bundle size dramatically. Instead of loading a 500KB bundle, the user gets a 50KB bundle for the first page, and additional chunks load as needed.

Implementation: replace static imports with React.lazy(() => import("./pages/Page")). Wrap your Routes in Suspense with a fallback. Suspense shows the fallback while the chunk is being downloaded.

React Router works seamlessly with React.lazy. Vite automatically splits the bundle at lazy() boundaries — no extra webpack configuration needed.

This directly improves Core Web Vitals, especially LCP (Largest Contentful Paint) and TTI (Time to Interactive) — metrics that affect both user experience and SEO.

Q10: What is an index route in React Router v6?

Answer:

An index route is the default child route that renders when the parent route's path matches exactly — with no additional path segment.

Defined with the "index" prop instead of a path: <Route index element={<DashboardHome />} />

Example: parent route is /dashboard. Index route renders at exactly /dashboard. Other child routes render at /dashboard/stats, /dashboard/users, etc.

Without an index route, navigating to /dashboard would render the DashboardLayout but the Outlet would be empty.

Think of the index route as the "home" of that section — what shows when you arrive at the parent path itself.

Q11: How does the browser back button work with React Router?

Answer:

React Router uses the browser's History API (pushState, replaceState, popstate) to manage navigation without page reloads.

Every time you navigate with Link or useNavigate, React Router pushes a new entry to the browser history stack. The URL changes, the matching component renders.

When the user clicks the browser back button, the browser fires a popstate event. React Router listens for this event, reads the previous URL from the history stack, and renders the matching component for that URL — all without any server request.

navigate("/path", { replace: true }) uses replaceState instead of pushState — it replaces the current entry instead of adding a new one. This means the back button skips that navigation. Use it after login/logout where going back would be confusing.

navigate(-1) programmatically triggers the back button behaviour. navigate(-2) goes back two steps.

Q12: What is the difference between BrowserRouter and HashRouter?

Answer:

BrowserRouter uses the HTML5 History API. URLs look clean: myapp.com/dashboard. Navigation uses pushState/popstate. This is the standard choice for modern apps.

HashRouter uses the URL hash for routing. URLs look like: myapp.com/#/dashboard. Navigation changes the hash part of the URL, which never triggers a server request.

The key difference: with BrowserRouter, if a user directly visits myapp.com/dashboard, the server must be configured to serve the React app for any path (not just /). Otherwise, the server returns 404 — the path /dashboard does not exist as a file.

With HashRouter, the server always serves the index.html (because the hash is never sent to the server). No special server configuration needed.

Use BrowserRouter for all modern apps deployed on properly configured servers (Nginx, Vercel, Netlify all handle this). Use HashRouter only for static file hosting with no server config access.

Common Mistakes with React Router

Mistake 1: Using <a href> instead of <Link>

```
// ❌ WRONG — causes full page reload, all state lost
<a href="/dashboard">Go to Dashboard</a>

// ✅ CORRECT — SPA navigation, no reload
<Link to="/dashboard">Go to Dashboard</Link>
```

Mistake 2: Forgetting useParams is always a string

```
// ❌ WRONG — userId is string "42", not number 42
const { userId } = useParams();
fetch(`/api/users?id=${userId}`)  // fine for string interpolation
if (userId === 42) { ... }        // WRONG — "42" !== 42

// ✅ CORRECT — convert explicitly
const { userId } = useParams();
const id = Number(userId);
if (id === 42) { ... }  // works correctly
```

Mistake 3: Missing userId in useEffect dependencies

```
// ❌ WRONG — navigating from /users/1 to /users/2 does NOT refetch
const { userId } = useParams();
useEffect(() => {
  fetchUser(userId);
}, []);  // no deps — runs once and never again

// ✅ CORRECT — refetches when userId changes
useEffect(() => {
  fetchUser(userId);
}, [userId]);  // re-runs when URL param changes
```

Mistake 4: Using navigate() to navigate to external URLs

```
// ❌ WRONG — navigate() is for INTERNAL routes only
navigate("https://google.com");  // does not work correctly

// ✅ CORRECT — use window.location for external URLs
window.location.href = "https://google.com";
// Or open in new tab:
window.open("https://google.com", "_blank");

// For external links in JSX — use <a> (not <Link>)
<a href="https://google.com" target="_blank" rel="noopener noreferrer">
  Visit Google
</a>
```

Quick Reference — Chapter 9 Summary

Concept	What It Does	Code
BrowserRouter	Enables routing in your app	<BrowserRouter> in main.jsx

Routes + Route	Define URL-to-component mapping	<Route path="/x" element={<X />} />
Link	Navigate without page reload	<Link to="/path">Click</Link>
NavLink	Link that knows if it's active	className={({ isActive }) => ...}
useNavigate	Programmatic navigation in code	const nav = useNavigate(); nav("/path")
useParams	Read :param from URL path	const { id } = useParams()
useSearchParams	Read/write ?query=string	const [p, setP] = useSearchParams()
useLocation	Full current URL info + state	const loc = useLocation()
Outlet	Where child routes render in parent	<Outlet /> in layout component
Navigate	Redirect in JSX/render	<Navigate to="/login" replace />
ProtectedRoute	Auth guard for private pages	if (!isLoggedIn) return <Navigate />
lazy + Suspense	Load page only when visited	const P = lazy(() => import("./P"))

✅ **Remember**

Hotel analogy — Router = Receptionist. URL = Room number. Component = Room.
<Link> not <a> — always use Link for internal navigation. <a> causes page reload.
useNavigate — go to path, go back, replace history. Use after async operations.
useParams — always strings. Always in useEffect deps. Convert if number needed.
useSearchParams — for filters, search, sort, pagination. Shareable URL state.
Outlet — placeholder in layout. Child route renders here. Layout never re-renders.
ProtectedRoute — check auth → redirect to login with state.from → redirect back after login.
lazy() — code splitting. Each page loads on demand. Wrap in Suspense with fallback.
replace: true — replaces history entry. Back button skips it. Use after login/logout.

End of Chapter 9 • Next: Chapter 10 — Redux Toolkit

Chapter 10: Redux Toolkit

Global State Management — explained so clearly you will never forget it

By Syed Shaaz Akhtar | Interview Focused Guide

Redux sounds scary. Most developers hear "Redux" and think of hundreds of files, complex setup, and confusing patterns. That was OLD Redux. Redux Toolkit (RTK) is the modern way — same power, 80% less code. And once you truly understand the core idea, it is surprisingly simple.

This chapter teaches Redux Toolkit from absolute zero. We build understanding layer by layer — analogy first, concept second, code third. By the end of this chapter you will be able to implement Redux Toolkit from memory and explain it confidently in any interview.

> 💡 **Tip**
>
> 🏦 THE ANALOGY — remember this for the ENTIRE chapter:
> Redux is like a BANK.
>
> — The BANK VAULT = Store (one place where ALL the money/data lives)
> — Your ACCOUNT BALANCE = State (the actual data inside the vault)
> — A TRANSACTION SLIP = Action (a written request: "I want to withdraw ₹500")
> — The BANK TELLER = Reducer (reads your slip, updates the vault)
> — SUBMITTING the slip = Dispatch (sending your action to the store)
> — CHECKING your balance = useSelector (reading data from the store)
> — DEPARTMENT in the bank = Slice (one section that manages one type of account)
>
> You never touch the vault directly. You submit slips. The teller handles everything.
> This is exactly how Redux works. Keep this analogy in your head.

Topic 1: Why Redux? The Problem It Solves

You already know Context API from Chapter 8. So why would you need Redux? They both manage global state. The key differences show up in specific situations.

Situation	Context API	Redux Toolkit
Simple global state (theme, user)	✅ Perfect — simple setup	❌ Overkill
State changes frequently	⚠️ All consumers re-render	✅ Fine-grained subscriptions
Complex state logic	⚠️ Logic spreads everywhere	✅ Centralised in slices
Multiple developers, large team	⚠️ Easy to misuse	✅ Strict patterns, predictable

Debugging — time-travel	❌ Basic DevTools only	✅ Redux DevTools — see every action
Async operations (API calls)	⚠️ Manual useEffect patterns	✅ createAsyncThunk built-in
Performance critical app	⚠️ Can cause unnecessary renders	✅ Only subscribed parts re-render

Simple rule: Use Context for auth, theme, language — data that rarely changes and is consumed by many. Use Redux for complex, frequently changing, interdependent state — shopping carts, dashboards, real-time apps with multiple developers.

Topic 2: The 5 Core Concepts — Learn These First

Before writing any code, understand these 5 concepts. Everything in Redux is built on them. The bank analogy maps perfectly.

Concept	Bank Analogy	What It Is	Simple Definition
Store	The Bank Vault	Single object holding ALL state	One central place where all data lives
State	Account Balance	The actual data inside the store	The current values — count: 5, user: {...}
Action	Transaction Slip	Plain object describing what happened	{ type: "counter/increment", payload: 1 }
Reducer	Bank Teller	Pure function: (state, action) => newState	Takes old state + action → returns new state
Dispatch	Submitting the slip	Function to send an action to the store	dispatch(increment()) — triggers the reducer

```
// The complete Redux data flow — read this carefully
```

```
// 1. STATE lives in the store
store = { counter: { value: 0 }, user: { name: "Shaaz" } }

// 2. COMPONENT reads state with useSelector
const count = useSelector(state => state.counter.value);  // count = 0

// 3. USER does something → component dispatches an ACTION
dispatch({ type: "counter/increment" });
// "I want to increment the counter" — the transaction slip

// 4. REDUCER receives action, returns new state (never mutates)
function counterReducer(state, action) {
  if (action.type === "counter/increment") {
    return { ...state, value: state.value + 1 };  // new state: { value: 1 }
```

```
  }
  return state;
}

// 5. STORE updates → all useSelector subscribers re-render with new value
// count is now 1 → component re-renders showing 1

// THE GOLDEN RULE: Data flows in ONE DIRECTION
// Action → Reducer → Store → UI → Action → Reducer → ...
```

✅ **Remember**

One-way data flow is the most important principle in Redux. Data NEVER flows backward.
You can NEVER directly modify state. You must dispatch an action.
The reducer MUST return a NEW state object — never mutate the existing one.
The store is the single source of truth for ALL global state in your app.

Topic 3: Redux Toolkit Setup — 4 Steps

Old Redux required 10+ files and dozens of lines of boilerplate. Redux Toolkit collapses all of that into 4 simple steps. Here they are.

```
// STEP 1: Install
npm install @reduxjs/toolkit react-redux

// @reduxjs/toolkit  — the modern Redux library
// react-redux       — connects Redux to React components
```

```
// STEP 2: Create a Slice (src/features/counter/counterSlice.js)
// A slice = one section of state + its reducers
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",          // name of this slice
  initialState: { value: 0 }, // starting state
  reducers: {               // functions that update state
    increment: (state) => { state.value += 1; },  // RTK uses Immer — you CAN
mutate here!
    decrement: (state) => { state.value -= 1; },
    incrementByAmount: (state, action) => { state.value += action.payload; },
    reset: (state) => { state.value = 0; },
  },
});

// createSlice automatically generates action creators for each reducer
export const { increment, decrement, incrementByAmount, reset } =
counterSlice.actions;

// Export the reducer to add to the store
```

```
export default counterSlice.reducer;
```

```
// STEP 3: Create the Store (src/app/store.js)
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../features/counter/counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer,    // "counter" = key in state
    // Add more slices here as your app grows:
    // user: userReducer,
    // cart: cartReducer,
    // products: productsReducer,
  },
});

// store.getState() now returns:
// { counter: { value: 0 }, user: {...}, cart: {...} }
```

```
// STEP 4: Provide the store to your React app (main.jsx)
import { Provider } from "react-redux";
import { store }    from "./app/store";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);

// Provider makes the Redux store available to ALL components
// Works exactly like Context Provider — wraps the entire app
// Any component can now use useSelector and useDispatch
```

💡 **Tip**

That is the COMPLETE setup. 4 steps. Most tutorials make this seem harder.

1. npm install → 2. createSlice → 3. configureStore → 4. Provider

Memorise this flow. In an interview you should be able to set up Redux Toolkit in under 5 minutes.

Topic 4: useSelector — Reading from the Store

useSelector is the hook for READING data from the Redux store. You pass it a selector function — a function that picks which piece of the state you want. The component re-renders ONLY when that specific piece changes.

```
import { useSelector } from "react-redux";

// useSelector(selectorFunction)
// selectorFunction receives the ENTIRE state, you return the piece you need
```

```
function CounterDisplay() {
  // Pick just the counter value — re-renders only when counter.value changes
  const count = useSelector(state => state.counter.value);

  return <h1>Count: {count}</h1>;
}

// Multiple useSelector calls — each subscribes independently
function UserProfile() {
  const name   = useSelector(state => state.user.name);
  const avatar = useSelector(state => state.user.avatar);
  const role   = useSelector(state => state.user.role);

  return (
    <div>
      <img src={avatar} />
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}

// Derived data — compute inside selector
function CartSummary() {
  // Selector can compute derived values
  const itemCount = useSelector(state => state.cart.items.length);
  const total     = useSelector(state =>
    state.cart.items.reduce((sum, item) => sum + item.price * item.qty, 0)
  );

  return <p>{itemCount} items — Total: ₹{total}</p>;
}
```

✅ **Remember**

useSelector is like checking your bank balance. You are READING, not changing anything.

The selector function must be a PURE function — same state always returns same result.

Component re-renders ONLY when the selector's return value changes — not on every store update.

For expensive computations in selectors, use createSelector (from reselect) to memoize them.

Topic 5: useDispatch — Sending Actions to the Store

useDispatch gives you the dispatch function. You call it with an action to tell Redux "this thing happened". The reducer handles the rest.

```
import { useDispatch } from "react-redux";
import { increment, decrement, incrementByAmount, reset } from "./counterSlice";
```

```
function CounterControls() {
  const dispatch = useDispatch();
  // dispatch is like the bank counter — you submit your transaction slip here

  return (
    <div>
      {/* dispatch(actionCreator()) — action creator generates the action object
*/}
      <button onClick={() => dispatch(increment())}>
        +1
      </button>

      <button onClick={() => dispatch(decrement())}>
        -1
      </button>

      {/* payload — extra data sent with the action */}
      <button onClick={() => dispatch(incrementByAmount(5))}>
        +5
      </button>

      <button onClick={() => dispatch(reset())}>
        Reset
      </button>
    </div>
  );
}

// What dispatch(increment()) actually sends to the reducer:
// { type: "counter/increment" }

// What dispatch(incrementByAmount(5)) sends:
// { type: "counter/incrementByAmount", payload: 5 }
// RTK automatically wraps your argument as "payload"
```

Topic 6: The Magic of Immer — Why You CAN "Mutate" in RTK

This confuses many developers. The rule in React is "never mutate state". But in Redux Toolkit reducers, you WRITE code that looks like mutation. Why?

Redux Toolkit uses a library called **Immer** internally. Immer wraps your reducer and watches every "mutation" you write. It creates a brand new state object with those changes applied. You write simple mutation code — Immer produces an immutable update behind the scenes.

```
// OLD Redux (without Immer) — immutable update boilerplate
function oldReducer(state = initialState, action) {
  switch (action.type) {
    case "ADD_ITEM":
      return {
```

```
        ...state,
        items: [
          ...state.items,
          action.payload
        ]
      };  // verbose spread syntax to avoid mutation
    default:
      return state;
  }
}

// RTK WITH Immer — write simple mutation code, Immer handles immutability
const cartSlice = createSlice({
  name: "cart",
  initialState: { items: [], total: 0 },
  reducers: {
    addItem: (state, action) => {
      state.items.push(action.payload);  // looks like mutation — but Immer
makes it immutable
      state.total += action.payload.price;
    },
    removeItem: (state, action) => {
      // filter returns new array — also fine in RTK
      state.items = state.items.filter(item => item.id !== action.payload);
    },
    updateQty: (state, action) => {
      const item = state.items.find(i => i.id === action.payload.id);
      if (item) item.qty = action.payload.qty;  // direct property assignment —
Immer handles it
    },
    clearCart: (state) => {
      state.items = [];
      state.total = 0;
    },
  },
});

// IMPORTANT RULE: You can EITHER mutate state OR return a new state — not both
// ✅ Mutate:  state.value += 1
// ✅ Return:  return { ...state, value: state.value + 1 }
// ❌ Both:   state.value += 1; return state;  // DO NOT DO THIS
```

📝 Note

Immer only works INSIDE createSlice reducers. Outside them (like in useEffect or event handlers), you still cannot mutate state directly.

You can use either mutation style OR immutable return style inside RTK reducers — pick one per function.

Immer works with arrays (push, pop, splice) and objects (direct assignment). Use whatever reads most clearly.

Topic 7: Complete Example — Counter App

Let us build a complete counter app step by step. This is the "Hello World" of Redux. After this, you will understand the full flow.

```
// src/features/counter/counterSlice.js
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: {
    value: 0,
    step:  1,    // how much to add per click
  },
  reducers: {
    increment:        (state) => { state.value += state.step; },
    decrement:        (state) => { state.value -= state.step; },
    reset:            (state) => { state.value = 0; },
    setStep:          (state, action) => { state.step = action.payload; },
    incrementByAmount:(state, action) => { state.value += action.payload; },
  },
});

export const { increment, decrement, reset, setStep, incrementByAmount } =
counterSlice.actions;
export default counterSlice.reducer;
```

```
// src/app/store.js
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../features/counter/counterSlice";

export const store = configureStore({
  reducer: { counter: counterReducer },
});
```

```
// src/features/counter/Counter.jsx
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement, reset, setStep } from "./counterSlice";

function Counter() {
  // READ from store
  const value = useSelector(state => state.counter.value);
  const step  = useSelector(state => state.counter.step);

  // WRITE to store
  const dispatch = useDispatch();

  return (
    <div>
      <h1>Count: {value}</h1>
```

```
      <div>
        <button onClick={() => dispatch(decrement())}>-</button>
        <button onClick={() => dispatch(increment())}>+</button>
        <button onClick={() => dispatch(reset())}>Reset</button>
      </div>

      <div>
        <label>Step: </label>
        <input
          type="number"
          value={step}
          onChange={e => dispatch(setStep(Number(e.target.value)))}
        />
      </div>
    </div>
  );
}

export default Counter;

// main.jsx
import { Provider } from "react-redux";
import { store }    from "./app/store";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <Counter />
  </Provider>
);

// COMPLETE FLOW when user clicks "+"
// 1. dispatch(increment()) called
// 2. Action { type: "counter/increment" } sent to store
// 3. counterReducer runs: state.value += state.step
// 4. Store updates: { counter: { value: 1, step: 1 } }
// 5. useSelector detects value changed → Counter re-renders
// 6. count shows "1" on screen
```

Topic 8: createAsyncThunk — Handling API Calls

Real apps need to fetch data from APIs. Redux Toolkit has a built-in utility called **createAsyncThunk** for handling async operations. It automatically generates pending, fulfilled, and rejected actions so you can track loading and error states.

The Problem Without createAsyncThunk

Without it, you would manually dispatch loading/success/error actions and handle async logic outside Redux. With createAsyncThunk, the entire async flow is handled in one place.

```
// src/features/products/productsSlice.js
```

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

// createAsyncThunk("actionName", async function)
// The async function must return the data on success
// Redux Toolkit handles the rest automatically
export const fetchProducts = createAsyncThunk(
  "products/fetchAll",      // action type prefix
  async ({ category, page } = {}) => {
    const params = new URLSearchParams({ category, page }).toString();
    const response = await fetch(`/api/products?${params}`);
    if (!response.ok) throw new Error("Failed to fetch products");
    return await response.json();   // this becomes action.payload in
"fulfilled"
  }
);

export const createProduct = createAsyncThunk(
  "products/create",
  async (productData) => {
    const response = await fetch("/api/products", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(productData),
    });
    if (!response.ok) throw new Error("Failed to create product");
    return await response.json();
  }
);

const productsSlice = createSlice({
  name: "products",
  initialState: {
    items:   [],
    loading: false,
    error:   null,
    total:   0,
  },
  reducers: {
    // Synchronous reducers — for local state changes
    clearError: (state) => { state.error = null; },
    removeProduct: (state, action) => {
      state.items = state.items.filter(p => p.id !== action.payload);
    },
  },
  // extraReducers handles async thunk lifecycle actions
  extraReducers: (builder) => {

    // fetchProducts lifecycle
    builder
      .addCase(fetchProducts.pending, (state) => {
        state.loading = true;
```

```
        state.error   = null;
        // "pending" = fetch started — show spinner
      })
      .addCase(fetchProducts.fulfilled, (state, action) => {
        state.loading = false;
        state.items   = action.payload.products;
        state.total   = action.payload.total;
        // "fulfilled" = fetch succeeded — action.payload = response data
      })
      .addCase(fetchProducts.rejected, (state, action) => {
        state.loading = false;
        state.error   = action.error.message;
        // "rejected" = fetch failed — action.error.message = error
      })

    // createProduct lifecycle
      .addCase(createProduct.fulfilled, (state, action) => {
        state.items.push(action.payload);  // add new product to list
      });
  },
});

export const { clearError, removeProduct } = productsSlice.actions;
export default productsSlice.reducer;
```

```
// ProductsPage.jsx — using the async thunk
import { useEffect }            from "react";
import { useSelector, useDispatch } from "react-redux";
import { fetchProducts }        from "./productsSlice";

function ProductsPage() {
  const dispatch  = useDispatch();
  const { items, loading, error } = useSelector(state => state.products);

  // Dispatch the thunk on mount — it handles pending/fulfilled/rejected
  useEffect(() => {
    dispatch(fetchProducts({ category: "all", page: 1 }));
  }, [dispatch]);

  if (loading) return <div>Loading products...</div>;
  if (error)   return <div>Error: {error}</div>;

  return (
    <div>
      <p>{items.length} products found</p>
      <div className="grid">
        {items.map(product => (
          <ProductCard key={product.id} product={product} />
        ))}
      </div>
```

```
    </div>
  );
}

// What happens when dispatch(fetchProducts()) is called:
// 1. fetchProducts.pending fires → loading: true
// 2. API call runs...
// 3a. Success → fetchProducts.fulfilled fires → items updated, loading: false
// 3b. Error   → fetchProducts.rejected fires → error set, loading: false
```

✅ **Remember**

createAsyncThunk generates 3 action types automatically: actionName.pending, actionName.fulfilled, actionName.rejected.

pending = started loading. fulfilled = success, payload = returned data. rejected = failed, error.message = error.

Handle all 3 in extraReducers for complete loading/error UX.

The thunk function's return value automatically becomes action.payload in the fulfilled case.

If the thunk throws an error, it automatically goes to rejected case with error.message.

Topic 9: Real Example — Complete Cart with Redux Toolkit

Here is a complete, production-quality cart implementation using Redux Toolkit. This demonstrates everything: slice, selectors, dispatch, and multiple components sharing state.

```
// src/features/cart/cartSlice.js
import { createSlice } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart",
  initialState: {
    items:     [],
    isOpen:    false,
    coupon:    null,
    discount:  0,
  },
  reducers: {
    addItem: (state, action) => {
      const existing = state.items.find(i => i.id === action.payload.id);
      if (existing) {
        existing.qty += 1;   // Immer — direct mutation is fine here
      } else {
        state.items.push({ ...action.payload, qty: 1 });
      }
    },
    removeItem: (state, action) => {
      state.items = state.items.filter(i => i.id !== action.payload);
    },
    updateQty: (state, action) => {
```

```
      const { id, qty } = action.payload;
      if (qty <= 0) {
        state.items = state.items.filter(i => i.id !== id);
      } else {
        const item = state.items.find(i => i.id === id);
        if (item) item.qty = qty;
      }
    },
    clearCart:    (state) => { state.items = []; state.coupon = null;
state.discount = 0; },
    toggleCart:   (state) => { state.isOpen = !state.isOpen; },
    applyCoupon:  (state, action) => {
      state.coupon   = action.payload.code;
      state.discount = action.payload.discount;
    },
    removeCoupon: (state) => { state.coupon = null; state.discount = 0; },
  },
});

export const { addItem, removeItem, updateQty, clearCart, toggleCart,
applyCoupon, removeCoupon } = cartSlice.actions;

// SELECTORS — reusable functions to compute derived data
// Define them here, import them wherever needed
export const selectCartItems     = state => state.cart.items;
export const selectCartCount     = state => state.cart.items.length;
export const selectCartIsOpen    = state => state.cart.isOpen;
export const selectCartSubtotal  = state =>
  state.cart.items.reduce((sum, i) => sum + i.price * i.qty, 0);
export const selectCartDiscount  = state => state.cart.discount;
export const selectCartTotal     = state => {
  const subtotal = state.cart.items.reduce((sum, i) => sum + i.price * i.qty,
0);
  return subtotal - state.cart.discount;
};

export default cartSlice.reducer;
```

```
// NavBar.jsx — shows cart badge
import { useSelector, useDispatch } from "react-redux";
import { selectCartCount, toggleCart } from "../features/cart/cartSlice";

function NavBar() {
  const cartCount = useSelector(selectCartCount);
  const dispatch  = useDispatch();

  return (
    <nav>
      <button onClick={() => dispatch(toggleCart())}>
        🛒 {cartCount > 0 && <span className="badge">{cartCount}</span>}
      </button>
```

```
    </nav>
  );
}

// ProductCard.jsx — adds items to cart
import { useDispatch } from "react-redux";
import { addItem } from "../features/cart/cartSlice";

function ProductCard({ product }) {
  const dispatch = useDispatch();

  return (
    <div>
      <h3>{product.name}</h3>
      <p>₹{product.price}</p>
      <button onClick={() => dispatch(addItem(product))}>
        Add to Cart
      </button>
    </div>
  );
}

// CartDrawer.jsx — shows cart contents
import { useSelector, useDispatch } from "react-redux";
import { selectCartItems, selectCartSubtotal, selectCartTotal, selectCartIsOpen,
         removeItem, updateQty, clearCart, toggleCart } from
"../features/cart/cartSlice";

function CartDrawer() {
  const dispatch  = useDispatch();
  const isOpen    = useSelector(selectCartIsOpen);
  const items     = useSelector(selectCartItems);
  const subtotal  = useSelector(selectCartSubtotal);
  const total     = useSelector(selectCartTotal);

  if (!isOpen) return null;

  return (
    <div className="cart-drawer">
      <div className="cart-header">
        <h2>Your Cart ({items.length})</h2>
        <button onClick={() => dispatch(toggleCart())}>x</button>
      </div>

      {items.length === 0 ? (
        <p>Your cart is empty</p>
      ) : (
        <ul>
          {items.map(item => (
            <li key={item.id}>
              <span>{item.name}</span>
```

```
              <input
                type="number" min="1"
                value={item.qty}
                onChange={e => dispatch(updateQty({ id: item.id, qty:
Number(e.target.value) }))}
              />
              <span>₹{item.price * item.qty}</span>
              <button onClick={() =>
dispatch(removeItem(item.id))}>Remove</button>
            </li>
          ))}
        </ul>
      )}

      <div className="cart-footer">
        <p>Subtotal: ₹{subtotal}</p>
        <p><strong>Total: ₹{total}</strong></p>
        <button onClick={() => dispatch(clearCart())}>Clear Cart</button>
        <button>Checkout</button>
      </div>
    </div>
  );
}
```

Topic 10: File Structure — The Feature-Based Pattern

How you organise your Redux files matters in a team. The industry standard is the **feature-based** (or "ducks") pattern — everything for one feature in one folder.

```
// RECOMMENDED: Feature-Based Structure
src/
├── app/
│   └── store.js              // configureStore — all reducers combined here
│
├── features/                 // each feature has its own folder
│   ├── auth/
│   │   ├── authSlice.js      // createSlice + createAsyncThunk for auth
│   │   ├── LoginPage.jsx     // page component
│   │   └── authSelectors.js  // (optional) named selector functions
│   │
│   ├── products/
│   │   ├── productsSlice.js  // state, reducers, async thunks
│   │   ├── ProductList.jsx
│   │   ├── ProductCard.jsx
│   │   └── ProductDetail.jsx
│   │
│   ├── cart/
│   │   ├── cartSlice.js
│   │   ├── CartDrawer.jsx
```

```
│   │   └── CartItem.jsx
│   │
│   └── user/
│       ├── userSlice.js
│       └── UserProfile.jsx
│
└── main.jsx                  // Provider + store

// Each feature is self-contained.
// To delete a feature — delete the folder. Clean.
// To add a feature — add a folder + register reducer in store.js.
```

> 💡 **Tip**
>
> In interviews, mentioning this structure shows you have worked on real, team-based projects.
>
> The key principle: "one folder per feature, one slice per feature, import into store.js".
>
> As the app grows, you keep adding features folders. store.js stays clean — just imports and combines.

Topic 11: Selectors — The Right Way to Read State

Selectors are functions that extract and compute data from the store. Instead of writing state.products.items.filter(...) everywhere, you write it once as a selector and reuse it.

```
// WITHOUT selectors — logic repeated everywhere, hard to maintain

// Component A:
const inStockCount = useSelector(s => s.products.items.filter(p =>
p.inStock).length);

// Component B:
const inStockItems = useSelector(s => s.products.items.filter(p => p.inStock));

// If the data shape changes — update EVERY component. Bad.

// WITH selectors — define once, use everywhere
// productsSlice.js — export named selectors
export const selectAllProducts     = state => state.products.items;
export const selectProductsLoading = state => state.products.loading;
export const selectProductsError   = state => state.products.error;
export const selectInStockProducts = state =>
  state.products.items.filter(p => p.inStock);
export const selectProductById     = (id) => state =>
  state.products.items.find(p => p.id === id);
export const selectProductsByCategory = (cat) => state =>
  state.products.items.filter(p => p.category === cat);

// Component A:
const inStockCount = useSelector(s => selectInStockProducts(s).length);
```

```
// Component B:
const inStockItems = useSelector(selectInStockProducts);

// Component C — parameterised selector:
const product = useSelector(selectProductById(42));

// If data shape changes — fix ONE selector. All components get the fix.
```

```
// MEMOISED SELECTORS with createSelector (from reselect)
// Prevents re-computation when unrelated state changes
import { createSelector } from "@reduxjs/toolkit";  // re-exported from RTK

const selectProducts  = state => state.products.items;
const selectSearchQuery = state => state.filters.searchQuery;

// createSelector(inputSelectors[], resultFunction)
// resultFunction only re-runs when inputs ACTUALLY change
export const selectFilteredProducts = createSelector(
  [selectProducts, selectSearchQuery],
  (products, query) => {
    console.log("Computing filtered products...");
    // This only runs when products OR searchQuery changes
    // Not on every single store update
    return products.filter(p =>
      p.name.toLowerCase().includes(query.toLowerCase())
    );
  }
);

// Usage:
const filtered = useSelector(selectFilteredProducts);
```

Topic 12: Redux DevTools — Your Debugging Superpower

Redux DevTools is a browser extension (Chrome/Firefox) that gives you an X-ray view into your Redux store. It is the single biggest advantage Redux has over Context.

```
// Install Redux DevTools Extension in Chrome from Chrome Web Store
// Then run your React app — the DevTools panel appears in Chrome DevTools

// What Redux DevTools shows you:

// 1. ACTION LOG — every action dispatched, in order
//    ✓ counter/increment
//    ✓ cart/addItem
//    ✓ products/fetchAll/pending
//    ✓ products/fetchAll/fulfilled
//    ✓ user/login/fulfilled
```

```
// 2. STATE DIFF — exactly what changed in state after each action
//    Before: { counter: { value: 4 } }
//    After:  { counter: { value: 5 } }

// 3. TIME TRAVEL — click any past action to see app state at that moment
//    Step back through your app's history like a video scrubber

// 4. ACTION PAYLOAD — inspect exactly what data each action carried

// 5. SKIP/REPLAY — skip actions to test different flows

// configureStore sets up DevTools automatically in development
// No extra configuration needed — just install the browser extension
export const store = configureStore({
  reducer: { ... },
  // devTools: process.env.NODE_ENV !== "production"  // auto-handled by RTK
});
```

🎯 **Interview Tip**

"What debugging tools do you use with Redux?"

"Redux DevTools extension. It shows every dispatched action in order, the exact state diff after each action, and lets me time-travel — clicking any action in history shows the exact app state at that moment. This is invaluable when debugging complex state bugs — I can see exactly which action caused an unexpected state change."

"This is something Context API cannot do — there's no equivalent time-travel debugging for Context."

Topic 13: Redux vs Context API — Complete Comparison

This is the most asked comparison in senior interviews. Know this thoroughly.

Aspect	Context API	Redux Toolkit
Setup	createContext + Provider (~10 lines)	createSlice + configureStore + Provider (~25 lines)
Boilerplate	Very little	Moderate (RTK reduces old Redux boilerplate by 80%)
Re-renders	ALL consumers re-render on any change	Only subscribed slice re-renders
Debugging	React DevTools (basic)	Redux DevTools (time-travel, action log)
Async	Manual useEffect patterns	createAsyncThunk (built-in)
Middleware	Not supported	redux-thunk (default), logger, etc.

Learning curve	Low	Moderate
Best for	Auth, theme, language, preferences	Complex state, large teams, real-time apps
Testing	Context testing with mock Provider	Reducers are pure functions — easy unit test
Bundle size	0kb (built-in)	~13kb (RTK min+gzip)

Chapter 10: Interview Questions with Full Answers

Q1: What is Redux? Why do we use it?

Answer:

Redux is a predictable state management library. It stores all global state in a single JavaScript object called the store. State can only be updated by dispatching actions — plain objects that describe what happened. Reducers are pure functions that handle those actions and return new state.

We use Redux when: multiple components need the same state and it changes frequently, state transitions are complex and need to be predictable, we need powerful debugging tools like time-travel, or we are working on a large team where consistent patterns are critical.

Redux Toolkit is the modern way to write Redux — it reduces boilerplate dramatically while keeping all the power. It includes createSlice, createAsyncThunk, and configureStore.

For simpler global state (auth, theme), Context API is sufficient. Redux shines for complex, frequently changing, deeply interconnected state.

Q2: What is the Redux data flow? Explain the complete cycle.

Answer:

Redux uses a strict one-way data flow.

1. State lives in the store — one central JavaScript object holding all global state.
2. Components read state with useSelector. They do not get the whole store — just the slice they need.
3. User interaction triggers an action — components call dispatch(actionCreator()).
4. Dispatch sends the action to all reducers.
5. The matching reducer handles the action and returns new state. Reducers must be pure — no side effects, no mutation.
6. Store updates with the new state.
7. useSelector detects the relevant change and the subscribing components re-render with the new data.

This cycle repeats. Data always flows: Event → Action → Reducer → Store → UI.

Q3: What is createSlice? What does it do?

Answer:

createSlice is the core function of Redux Toolkit. It takes a name, initial state, and a reducers object — and returns a slice object containing action creators and the reducer function.

Before RTK, you had to manually write action type constants, action creator functions, and a reducer with switch/case — typically across 3 separate files. createSlice collapses all of that into one clean object.

It automatically generates action creator functions with the correct type strings (name/reducerName), and uses Immer internally so you can write mutation-style code in reducers without actually mutating state.

You export the actions (for dispatching) and the reducer (for configureStore). That is the complete slice API.

Q4: What is createAsyncThunk? How does it work?

Answer:

createAsyncThunk is Redux Toolkit's solution for handling asynchronous operations like API calls. It takes an action name prefix and an async function, and automatically generates three lifecycle actions: pending, fulfilled, and rejected.

When you dispatch the thunk, it runs the async function. If the function succeeds, fulfilled is dispatched with the return value as payload. If it throws, rejected is dispatched with the error.

You handle these three cases in extraReducers using builder.addCase(). This gives you clean, predictable loading/error state management without any manual action creators.

The async function receives two arguments: the argument you passed to dispatch, and a thunkAPI object that includes dispatch, getState, and rejectWithValue for custom error handling.

Q5: Why can you "mutate" state inside Redux Toolkit reducers?

Answer:

In Redux Toolkit, you can write mutation-style code inside createSlice reducers because RTK uses a library called Immer under the hood.

Immer wraps your reducer in a Proxy. It records every "mutation" you make to the draft state. When your reducer finishes, Immer creates a brand new immutable state object with all those changes applied. Your actual state is never mutated.

This means state.count += 1 inside a reducer is safe — Immer converts it to { ...state, count: state.count + 1 } automatically.

Important: this ONLY works inside createSlice reducers. Everywhere else in your app (components, effects, custom hooks), the rule still stands — never mutate state directly.

Q6: What is the difference between reducers and extraReducers in createSlice?

Answer:

reducers handles synchronous actions that belong to this slice. Actions created here are automatically exported as action creators by createSlice.

extraReducers handles actions that come from OUTSIDE this slice — typically from createAsyncThunk or from other slices. You use a builder callback to add cases.

Example: if a products slice has a fetchProducts thunk, the three lifecycle actions (pending, fulfilled, rejected) are handled in extraReducers — because they are generated outside the slice definition.

The builder API in extraReducers: builder.addCase(actionCreator, reducerFn) for specific actions. addMatcher for matching multiple actions by pattern. addDefaultCase as a fallback.

Q7: What are selectors? Why should you use them?

Answer:

Selectors are functions that receive the entire Redux state and return a specific piece of data. They encapsulate the shape of your state.

Without selectors, you scatter state.products.items.filter(...) logic across all components. If your state shape changes, you update every component — painful.

With named selectors, you define the logic once in the slice file and import it everywhere. If the shape changes, you fix the selector — all components get the fix for free.

createSelector from RTK (re-exported from reselect) creates memoized selectors that only recompute when their input data actually changes. This prevents unnecessary recalculations and re-renders.

Naming convention: selector functions start with "select": selectCartItems, selectFilteredProducts, selectUserById.

Q8: What is the difference between useSelector and useDispatch?

Answer:

useSelector is for READING state from the Redux store. You pass a selector function that picks the piece of state you need. The component re-renders when that specific value changes.

useDispatch is for WRITING — it returns the dispatch function. You call dispatch(actionCreator()) to send an action to the store.

Reading analogy: useSelector is checking your bank balance. useDispatch is submitting a transaction.

useSelector can be called multiple times in one component — each call independently subscribes to its own piece of state.

Never do heavy computation directly in useSelector without memoization — it runs on every render. Use createSelector for expensive derivations.

Q9: What is Redux Toolkit vs old Redux?

Answer:

Old Redux required separate files for action type constants, action creators, and reducers. Setting up a single feature needed 50+ lines across 3+ files and was extremely repetitive.

Redux Toolkit introduces createSlice which combines constants, action creators, and reducer into one call. createAsyncThunk handles async without manual loading state management. configureStore automatically sets up Redux DevTools and thunk middleware.

RTK reduces boilerplate by roughly 80% while keeping Redux's core principles: single store, one-way data flow, pure reducers.

RTK also uses Immer for immutable updates and re-exports createSelector for memoized selectors.

Today, the official Redux team recommends RTK for ALL new Redux projects. Writing old-style Redux is considered bad practice.

Q10: How do multiple slices combine into one store?

Answer:

Each slice manages a specific piece of state. configureStore combines them using the reducer property — an object where each key becomes a key in the global state and the value is that slice's reducer.

Example: { reducer: { counter: counterReducer, cart: cartReducer, user: userReducer } } creates state = { counter: {...}, cart: {...}, user: {...} }.

When an action is dispatched, Redux passes it to ALL reducers. Each reducer checks if the action type matches any of its cases. Only the matching slice updates its state. Other slices return their current state unchanged.

This is why slice names and action types are prefixed (counter/increment, cart/addItem) — to avoid conflicts between slices.

Q11: Can you update one slice's state from another slice's action?

Answer:

Yes — using extraReducers.

If cart's clearCart action should also reset a "checkout" slice's state, the checkout slice can listen for the cart/clearCart action in its extraReducers.

Example: when a user logs out, you might want to clear the cart, clear the user profile, and reset filters all at once. Each of those slices can listen for the auth/logout action in their extraReducers.

This cross-slice coordination is one of Redux's strengths over Context — in Context, you would need the auth provider to import and call cart and filter update functions, creating tight coupling.

In Redux, each slice independently decides which external actions to react to — loose coupling.

Q12: When would you choose Redux over Context API?

Answer:

Choose Redux when: the app is large with many developers — Redux enforces patterns that prevent chaos. State changes happen frequently — Redux subscriptions are more efficient than Context re-renders. You need async state management — createAsyncThunk is cleaner than manual useEffect patterns.

Choose Redux when: you need debugging power — time-travel debugging in Redux DevTools is unmatched. State logic is complex with many transitions — pure reducer functions are easier to reason about and test. Multiple slices of state are interdependent — cross-slice actions in extraReducers handle this cleanly.

Stick with Context when: it is a small/medium app. The global state is simple (auth user, theme, language). State changes rarely. Team is small and familiar with React patterns.

> Real answer: in a large production app, use BOTH. Context for auth/theme/preferences (simple, rarely changes), Redux for products/cart/filters/notifications (complex, frequently changes).

Common Mistakes with Redux Toolkit

Mistake 1: Mutating state outside createSlice reducers

```
// ❌ WRONG — directly mutating Redux state in a component
const items = useSelector(state => state.cart.items);
items.push(newItem);  // mutating state directly — Redux never knows!

// ✅ CORRECT — dispatch an action, let the reducer handle it
dispatch(addItem(newItem));
```

Mistake 2: Both mutating AND returning in a reducer

```
// ❌ WRONG — mutation AND return together confuses Immer
addItem: (state, action) => {
  state.items.push(action.payload);  // mutate
  return state;                       // also return — Immer error!
}

// ✅ CORRECT — either mutate OR return, not both
addItem: (state, action) => {
  state.items.push(action.payload);  // just mutate (Immer handles it)
}
// OR
addItem: (state, action) => {
  return { ...state, items: [...state.items, action.payload] };  // just return
}
```

Mistake 3: Not handling all 3 thunk states (pending/fulfilled/rejected)

```
// ❌ BAD — only handling fulfilled, no loading/error states
extraReducers: (builder) => {
  builder.addCase(fetchProducts.fulfilled, (state, action) => {
    state.items = action.payload;  // data updates but no loading or error
handling
  });
}

// ✅ COMPLETE — all 3 states handled
extraReducers: (builder) => {
  builder
    .addCase(fetchProducts.pending,   (state)         => { state.loading = true;
state.error = null; })
```

```
    .addCase(fetchProducts.fulfilled, (state, action) => { state.loading = false; state.items = action.payload; })
    .addCase(fetchProducts.rejected,  (state, action) => { state.loading = false; state.error = action.error.message; })
}
```

Mistake 4: Putting ALL state in Redux — over-engineering

```
// ❌ WRONG — UI-only local state in Redux is overkill
const isDropdownOpen = useSelector(state => state.ui.isDropdownOpen);
dispatch(setDropdownOpen(true));  // ridiculous for local UI state

// ✅ CORRECT — local state stays local
const [isOpen, setIsOpen] = useState(false);
// Only put state in Redux if 2+ unrelated components need to share it
```

Quick Reference — Chapter 10 Summary

Concept	What It Is	Code
createSlice	Create state + reducers + actions in one call	createSlice({ name, initialState, reducers })
configureStore	Create the store combining all slices	configureStore({ reducer: { name: reducer } })
Provider	Make store available to all components	<Provider store={store}>
useSelector	Read data from store	useSelector(state => state.slice.value)
useDispatch	Get dispatch function	const dispatch = useDispatch()
action creator	Auto-generated function to create actions	dispatch(increment()) or dispatch(addItem(data))
extraReducers	Handle async thunk + cross-slice actions	builder.addCase(thunk.fulfilled, fn)
createAsyncThunk	Async action with pending/fulfilled/rejected	createAsyncThunk("name", async fn)
createSelector	Memoised derived selector	createSelector([inputs], resultFn)
Immer	Safe mutation inside reducers	state.items.push(x) — just works

✅ **Remember**

Bank analogy — Store=Vault, State=Balance, Action=Slip, Reducer=Teller, Dispatch=Submit slip.
4 setup steps: npm install → createSlice → configureStore → Provider.
One-way flow: Dispatch → Action → Reducer → Store → UI.
useSelector = read. **useDispatch** = write. Simple.
Immer: mutate inside createSlice reducers only. Either mutate OR return — never both.
createAsyncThunk: 3 cases always — pending (loading), fulfilled (data), rejected (error).
Selectors: define once in slice file, import everywhere. createSelector for expensive ones.
Feature folders: one folder per feature, one slice per feature, register in store.js.
Redux DevTools: time-travel debugging — the biggest advantage over Context.
When to use: large team, complex state, frequent updates, async heavy, debugging critical.

End of Chapter 10 • Next: Chapter 11 — Performance Optimization

Chapter 11: Performance Optimization

React.memo, useMemo, useCallback, Lazy Loading, Virtualization — make your app blazing fast

By Syed Shaaz Akhtar | Interview Focused Guide

A React app that works is good. A React app that is FAST is what gets you hired. Performance optimization separates junior developers from seniors. This chapter covers every optimization technique used in real production apps — when to use them, why they work, and just as importantly, when NOT to use them.

The golden rule of optimization: measure first, optimize second. Never optimize blindly. A poorly placed useMemo can make things SLOWER. Read this chapter and you will know exactly what to do and when.

> 💡 **Tip**
>
> 🏎️ THE ANALOGY for this chapter:
>
> Think of React performance like a CAR ENGINE.
>
> — Every re-render = engine firing a piston. Some pistons MUST fire. Some fire for no reason — waste of fuel.
>
> — React.memo = a valve that stops unnecessary piston fires for child components.
>
> — useMemo = a turbo that caches a computed result so the engine does not recompute it every time.
>
> — useCallback = a fuel injector that keeps function references stable so the engine recognises them.
>
> — Lazy Loading = only loading engine parts you actually need right now.
>
> — Virtualization = only showing the part of a list that fits in the viewport — not rendering 10,000 rows.
>
> Your goal: make every render count. Stop the wasted ones.

Topic 1: How React Renders — The Foundation

Before optimizing, you must understand WHEN and WHY React re-renders. You cannot fix what you do not understand.

What Causes a Re-render?

- **State changes** — when useState setter is called with a new value
- **Props change** — when parent passes different props to a child
- **Context changes** — when a consumed context value changes
- **Parent re-renders** — by default, when a parent re-renders, ALL its children re-render too

That last point is the most important. The default React behaviour is: parent re-renders → all children re-render. Even if their props did not change. This is called **wasted renders**.

```
// THE PROBLEM: Parent re-renders cause ALL children to re-render
```

```
function Parent() {
  const [count, setCount] = useState(0);
  // count changes → Parent re-renders
  // → ExpensiveChild ALSO re-renders even though its props did NOT change

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Click: {count}</button>
      <ExpensiveChild name="Shaaz" />
      {/* name="Shaaz" never changes but ExpensiveChild still re-renders on
every count change */}
    </div>
  );
}

function ExpensiveChild({ name }) {
  console.log("ExpensiveChild rendered");  // logs on EVERY parent re-render
  return <div>Hello, {name}</div>;
}
```

> 📝 **Note**
>
> Re-renders are NOT always bad. React is fast. Most re-renders are fine.
>
> Only optimize when you have a REAL performance problem — measured with React DevTools Profiler.
>
> Premature optimization is the root of all evil. Adding React.memo and useMemo everywhere without need adds complexity and can actually slow things down.

Topic 2: React.memo — Preventing Unnecessary Child Re-renders

React.memo is a Higher Order Component (HOC) that wraps a function component. It performs a shallow comparison of the previous and new props. If the props have not changed, React SKIPS re-rendering the component and reuses the last rendered output.

Think of it as a smart valve. Before firing the piston, it checks: "Did the input change? No? Then don't fire."

```
// WITHOUT React.memo — re-renders every time parent re-renders
function ProductCard({ product }) {
  console.log("ProductCard rendered:", product.name);
  return (
    <div className="card">
      <h3>{product.name}</h3>
      <p>₹{product.price}</p>
    </div>
  );
}

// WITH React.memo — only re-renders when product prop actually changes
const ProductCard = React.memo(function ProductCard({ product }) {
```

```
  console.log("ProductCard rendered:", product.name);
  return (
    <div className="card">
      <h3>{product.name}</h3>
      <p>₹{product.price}</p>
    </div>
  );
});

// Now rendering 100 ProductCards with the same product data:
// WITHOUT memo: parent re-render → all 100 ProductCards re-render
// WITH memo: parent re-render → 0 ProductCards re-render (props same)

// Only the cards whose product data actually changed will re-render.
```

```
// React.memo with custom comparison function
// When you need more control than shallow comparison
const ProductCard = React.memo(
  function ProductCard({ product, onAddToCart }) {
    return (
      <div>
        <h3>{product.name}</h3>
        <button onClick={() => onAddToCart(product.id)}>Add</button>
      </div>
    );
  },
  // Custom compare: only re-render if id or price changed
  // (prevProps, nextProps) => true means "they are equal — skip re-render"
  // (prevProps, nextProps) => false means "they changed — please re-render"
  (prevProps, nextProps) => {
    return (
      prevProps.product.id    === nextProps.product.id &&
      prevProps.product.price === nextProps.product.price
    );
  }
);
```

✅ Remember

React.memo does SHALLOW comparison. { name: "Shaaz" } === { name: "Shaaz" } is FALSE in JS (different object references). React.memo compares property values — it sees name is same, price is same → skip render.

React.memo FAILS when a new function or object is created every parent render and passed as prop — even if content is same. Fix with useCallback (functions) and useMemo (objects).

When to use: components that render often but receive same props most of the time. Expensive components (complex UI, charts). List items rendered 50+ times.

When NOT to use: simple components that are cheap to render. Components whose props change almost every render anyway.

Topic 3: useMemo — Caching Expensive Computed Values

useMemo caches the RESULT of a function call. It only re-runs the function when its dependencies change. Use it for expensive calculations that should not run on every render.

```
// WITHOUT useMemo — filterProducts runs on EVERY render
function ProductList({ products, searchQuery, sortBy }) {
  // This runs even if products/searchQuery/sortBy did NOT change
  // — like when an unrelated state changes and causes a re-render
  const processedProducts = products
    .filter(p => p.name.toLowerCase().includes(searchQuery.toLowerCase()))
    .sort((a, b) => {
      if (sortBy === "price") return a.price - b.price;
      return a.name.localeCompare(b.name);
    });

  return <div>{processedProducts.map(p => <ProductCard key={p.id} product={p}
/>)}</div>;
}

// WITH useMemo — only re-computes when deps actually change
function ProductList({ products, searchQuery, sortBy }) {
  const processedProducts = useMemo(() => {
    console.log("Filtering and sorting...");  // only when deps change
    return products
      .filter(p => p.name.toLowerCase().includes(searchQuery.toLowerCase()))
      .sort((a, b) => {
        if (sortBy === "price") return a.price - b.price;
        return a.name.localeCompare(b.name);
      });
  }, [products, searchQuery, sortBy]);  // only recalculate when these change

  return <div>{processedProducts.map(p => <ProductCard key={p.id} product={p}
/>)}</div>;
}
```

```
// More useMemo examples

// 1. Expensive statistics calculation
function Analytics({ transactions }) {
  const stats = useMemo(() => {
    const total    = transactions.reduce((s, t) => s + t.amount, 0);
    const avg      = total / transactions.length;
    const max      = Math.max(...transactions.map(t => t.amount));
    const byMonth  = transactions.reduce((acc, t) => {
      const m = t.date.slice(0, 7);
      acc[m] = (acc[m] || 0) + t.amount;
      return acc;
    }, {});
    return { total, avg, max, byMonth };
  }, [transactions]);  // recalculate only when transactions change
```

```
  return <div>Total: ₹{stats.total} | Avg: ₹{stats.avg}</div>;
}

// 2. Stable object reference for child props
function Parent({ userId }) {
  // Without useMemo: new config object every render → child always re-renders
  const config = useMemo(() => ({
    userId,
    theme:    "dark",
    pageSize: 20,
  }), [userId]);  // only new object when userId changes

  return <MemoizedChild config={config} />;
}
```

> 💡 **Tip**
>
> useMemo is NOT free. It has overhead: storing the cached value, comparing dependencies on every render.
>
> Only use useMemo when: the computation is genuinely expensive (filter/sort of large arrays, complex math), OR you need a stable object reference for React.memo to work correctly.
>
> Do NOT use useMemo for: simple calculations (a + b, string concat), primitive values (numbers, strings — they are already stable), components that rarely re-render.
>
> Test: remove useMemo and measure. If app feels slow — add it back. If no difference — leave it out.

Topic 4: useCallback — Stable Function References

useCallback returns a memoized (cached) version of a function. The function only gets a new reference when its dependencies change. This is essential when passing callback functions as props to React.memo components.

Why Functions Break React.memo

```
// THE PROBLEM: Functions are recreated on every render
function Parent() {
  const [count, setCount] = useState(0);
  const [name,  setName]  = useState("Shaaz");

  // handleDelete is a NEW function on every render — new reference in memory
  const handleDelete = (id) => {
    console.log("Delete", id);
  };

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
      <MemoizedProductList onDelete={handleDelete} />
```

```
      {/* Even though ProductList is memoized, it re-renders on every count
change */}
      {/* Because handleDelete is a NEW function reference every render */}
      {/* React.memo sees "prop changed" → re-renders */}
    </div>
  );
}

// THE FIX: useCallback gives handleDelete a stable reference
function Parent() {
  const [count, setCount] = useState(0);

  // handleDelete is the SAME function reference across renders
  // Only creates a new one if dependencies change (empty [] = never)
  const handleDelete = useCallback((id) => {
    console.log("Delete", id);
  }, []);  // empty deps = same function forever

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
      <MemoizedProductList onDelete={handleDelete} />
      {/* Now React.memo works correctly — onDelete is same reference */}
      {/* ProductList does NOT re-render when count changes */}
    </div>
  );
}

const MemoizedProductList = React.memo(function ProductList({ onDelete }) {
  console.log("ProductList rendered");  // only logs when onDelete reference
changes
  return <div>...</div>;
});
```

```
// useCallback with dependencies
function SearchPage() {
  const [query,    setQuery]    = useState("");
  const [category, setCategory] = useState("all");

  // This function needs query and category — list them as dependencies
  // New function created only when query OR category changes
  const handleSearch = useCallback(() => {
    fetchProducts({ query, category });
  }, [query, category]);

  // handleSearch is stable as long as query and category are unchanged
  return (
    <div>
      <SearchInput onSearch={handleSearch} />
      <CategoryFilter onChange={setCategory} />
    </div>
```

```
  );
}
```

Hook	What it Caches	Returns	Use When
useMemo	Result of calling a function	The computed VALUE	Expensive filter/sort/calc, stable objects for memo
useCallback	The function itself	The FUNCTION reference	Callbacks passed to React.memo child components

✅ **Remember**

useMemo = cache the VALUE. useCallback = cache the FUNCTION.

useMemo(() => fn(), deps) is equivalent to useCallback(fn, deps) — same concept, different return.

useCallback is ONLY useful when passing the function to a React.memo child. If the child is not memoized, a stable function reference does nothing.

Rule: React.memo + useCallback always go together. One without the other is usually pointless.

Topic 5: The Optimization Trinity — memo + useMemo + useCallback Together

These three tools work together as a system. Understanding how they complement each other is what separates senior developers from juniors. Here is the complete pattern.

```
// COMPLETE OPTIMIZATION PATTERN
// Parent with expensive child list — optimized correctly

function ProductCatalog() {
  const [products]     = useState(LARGE_PRODUCT_LIST);  // 500 products
  const [searchQuery,  setSearchQuery]  = useState("");
  const [cartCount,    setCartCount]    = useState(0);
  const [wishlistCount,setWishlistCount]= useState(0);

  // 1. useMemo — expensive filter + sort, only rerun when search changes
  //    cartCount or wishlistCount changes should NOT trigger refiltering
  const filteredProducts = useMemo(() =>
    products.filter(p =>
      p.name.toLowerCase().includes(searchQuery.toLowerCase())
    ),
    [products, searchQuery]  // NOT cartCount — intentionally excluded
  );

  // 2. useCallback — stable function for the memoized child
  //    Without this: addToCart new reference → ProductCard re-renders every
click
  const handleAddToCart = useCallback((productId) => {
    setCartCount(c => c + 1);
```

```
    // add to cart logic...
  }, []);  // no deps — setCartCount is stable

  const handleAddToWishlist = useCallback((productId) => {
    setWishlistCount(w => w + 1);
    // add to wishlist logic...
  }, []);

  return (
    <div>
      <p>Cart: {cartCount} | Wishlist: {wishlistCount}</p>
      <input value={searchQuery} onChange={e =>
setSearchQuery(e.target.value)} />
      <p>{filteredProducts.length} products</p>

      {filteredProducts.map(product => (
        // 3. React.memo — ProductCard only re-renders when its own props change
        //    Since handleAddToCart and handleAddToWishlist are stable
(useCallback),
        //    ProductCards only re-render when their specific product data
changes
        <ProductCard
          key={product.id}
          product={product}
          onAddToCart={handleAddToCart}
          onAddToWishlist={handleAddToWishlist}
        />
      ))}
    </div>
  );
}

// 3. React.memo on the child
const ProductCard = React.memo(function ProductCard({ product, onAddToCart,
onAddToWishlist }) {
  console.log("ProductCard rendered:", product.name);
  return (
    <div>
      <h3>{product.name}</h3>
      <p>₹{product.price}</p>
      <button onClick={() => onAddToCart(product.id)}>Cart</button>
      <button onClick={() => onAddToWishlist(product.id)}>Wishlist</button>
    </div>
  );
});

// BEFORE optimization: cartCount click → ALL 500 ProductCards re-render
// AFTER optimization: cartCount click → 0 ProductCards re-render
//   (filteredProducts is memoized, callbacks are stable, ProductCard is memo'd)
```

Topic 6: Code Splitting and Lazy Loading

When your React app builds, all components go into one JavaScript bundle. Even pages the user will never visit. Code splitting breaks this bundle into smaller chunks that load on demand. **Lazy loading** loads components only when they are needed.

```
import { lazy, Suspense } from "react";
```

```
// Static import — ALL of these load upfront (increases initial bundle size)
// import HomePage    from "./pages/HomePage";
// import AdminPage   from "./pages/AdminPage";
// import ReportsPage from "./pages/ReportsPage";

// Lazy imports — each page loads ONLY when the route is visited
const HomePage    = lazy(() => import("./pages/HomePage"));
const AboutPage   = lazy(() => import("./pages/AboutPage"));
const AdminPage   = lazy(() => import("./pages/AdminPage"));
const ReportsPage = lazy(() => import("./pages/ReportsPage"));

// Suspense shows fallback while chunk is being downloaded
function App() {
  return (
    <Suspense fallback={<PageLoader />}>
      <Routes>
        <Route path="/"        element={<HomePage />} />
        <Route path="/about"   element={<AboutPage />} />
        <Route path="/admin"   element={<AdminPage />} />
        <Route path="/reports" element={<ReportsPage />} />
      </Routes>
    </Suspense>
  );
}

// You can also lazy load heavy components within a page
const HeavyChart    = lazy(() => import("./components/HeavyChart"));
const RichTextEditor= lazy(() => import("./components/RichTextEditor"));

function Dashboard() {
  const [showChart, setShowChart] = useState(false);

  return (
    <div>
      <button onClick={() => setShowChart(true)}>Show Chart</button>
      {showChart && (
        <Suspense fallback={<div>Loading chart...</div>}>
          <HeavyChart />
          {/* Chart JS library only loads when this button is clicked */}
        </Suspense>
      )}
    </div>
  );
```

```
}
```

> 💡 **Tip**
>
> Code splitting is one of the highest-impact optimizations. A 2MB bundle becomes a 200KB initial load + on-demand chunks.
>
> Vite and Create React App (webpack) automatically create separate chunks at every lazy() boundary.
>
> Good candidates for lazy loading: admin pages (most users never visit), heavy editor/chart libraries, rarely-used modals with large dependencies, PDF viewers, video players.
>
> Do NOT lazy load: the main landing page, components needed immediately on first render.

Topic 7: List Virtualization — Rendering 10,000 Rows Efficiently

If you have a list of 10,000 items and render all of them, React puts 10,000 DOM nodes on the page. The browser struggles to manage them — scrolling becomes laggy, memory spikes.

Virtualization (also called "windowing") solves this by only rendering the items VISIBLE in the viewport. If your list shows 15 items at a time, only 15-20 DOM nodes exist — regardless of whether the list has 100 or 100,000 items.

```
// Install react-window (most popular virtualization library)
npm install react-window

import { FixedSizeList } from "react-window";

// WITHOUT virtualization — renders all 10,000 items
function BigList({ items }) {
  return (
    <div>
      {items.map(item => (       // 10,000 DOM nodes — browser struggles
        <div key={item.id} style={{ height: 50 }}>
          {item.name}
        </div>
      ))}
    </div>
  );
}

// WITH react-window — only renders visible items (~15)
function BigList({ items }) {
  // Row component — receives index and style from react-window
  const Row = ({ index, style }) => (
    <div style={style}>  {/* style MUST be applied — controls positioning */}
      <div key={items[index].id}>
        {items[index].name}
      </div>
    </div>
  );
```

```
  return (
    <FixedSizeList
      height={600}           // visible height of the list container
      itemCount={items.length} // total number of items
      itemSize={50}          // height of each row in pixels
      width="100%"
    >
      {Row}
    </FixedSizeList>
  );
}

// For variable height items — use VariableSizeList instead
// For grid layouts — use FixedSizeGrid or VariableSizeGrid

// Result:
// 10,000 items in the list
// Only ~15 DOM nodes rendered at any time
// Scrolling stays at 60fps regardless of list size
```

📝 **Note**

When to use virtualization: lists with 100+ items where scrolling performance matters.

For small lists (under 50 items) — regular map() is fine. Virtualization adds complexity.

react-window is the lightweight choice. react-virtual (TanStack Virtual) is the modern alternative with more flexibility.

Important: each row MUST apply the style prop from react-window — it contains the absolute positioning that makes virtualization work.

Topic 8: Avoiding Re-renders — Practical Patterns

Beyond the big three tools (memo, useMemo, useCallback), there are simple code patterns that naturally prevent unnecessary re-renders. These are free — no libraries needed.

Pattern 1: Move State Down

```
// BAD: State high up causes entire tree to re-render
function App() {
  const [isMenuOpen, setIsMenuOpen] = useState(false);
  // Toggling menu → App re-renders → ALL children re-render
  // Even if most children have nothing to do with the menu

  return (
    <div>
      <ExpensiveDashboard />      // re-renders on menu toggle — wasted!
      <ExpensiveAnalytics />      // re-renders on menu toggle — wasted!
      <NavBar onToggle={() => setIsMenuOpen(o => !o)} isOpen={isMenuOpen} />
    </div>
  );
```

```
}

// GOOD: Move state down to where it is actually used
function App() {
  return (
    <div>
      <ExpensiveDashboard />  // never re-renders when menu toggles
      <ExpensiveAnalytics />  // never re-renders when menu toggles
      <NavBar />              // manages its own menu state internally
    </div>
  );
}

function NavBar() {
  const [isMenuOpen, setIsMenuOpen] = useState(false);  // state lives HERE
  // Toggling menu → only NavBar re-renders — correct!
  return <nav>{isMenuOpen && <Menu />}</nav>;
}
```

Pattern 2: Lift Content Up (Children as Props)

```
// BAD: Slow component wraps fast component — both re-render together
function SlowParent() {
  const [count, setCount] = useState(0);
  // count changes → SlowParent re-renders → FastChild ALSO re-renders

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>{count}</button>
      <FastChild />  // should not re-render, but does
    </div>
  );
}

// GOOD: Pass children from outside — they belong to a DIFFERENT parent
function SlowParent({ children }) {
  const [count, setCount] = useState(0);
  // count changes → SlowParent re-renders → children do NOT re-render
  // Because children come from OUTSIDE SlowParent — their parent did not re-
render

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>{count}</button>
      {children}  // FastChild is rendered by the outer parent, not by
SlowParent
    </div>
  );
}
```

```
// Usage — FastChild is defined outside SlowParent, not inside
function App() {
  return (
    <SlowParent>
      <FastChild />  // FastChild belongs to App, not SlowParent
    </SlowParent>
  );
}
```

Pattern 3: Stable Initial State

```
// BAD: Creating new object/array references on every render
function Component() {
  // ❌ New array reference every render — causes unnecessary effects/memos to
refire
  const [filters, setFilters] = useState(["new", "sale"]);
  const config = { pageSize: 20, sortBy: "date" };  // ❌ New object every
render

  useEffect(() => {
    fetchData(config);
  }, [config]);  // config is new every render → effect runs every render!
}

// GOOD: Define stable references outside the component
const DEFAULT_FILTERS = ["new", "sale"];  // defined ONCE outside component
const API_CONFIG = { pageSize: 20, sortBy: "date" };  // stable reference

function Component() {
  const [filters, setFilters] = useState(DEFAULT_FILTERS);

  useEffect(() => {
    fetchData(API_CONFIG);
  }, []);  // config is stable — effect runs once
}
```

Topic 9: Debouncing and Throttling — Controlling Expensive Operations

Some operations should not run on every single event — like searching on every keystroke or tracking scroll position on every scroll event. Debouncing and throttling control HOW OFTEN these run.

```
// DEBOUNCE — wait until user STOPS doing the action
// "Wait 500ms after last keystroke, THEN search"
function SearchInput({ onSearch }) {
  const [query, setQuery] = useState("");

  useEffect(() => {
    const timer = setTimeout(() => {
```

```
      onSearch(query);  // only fires 500ms after typing stops
    }, 500);

    return () => clearTimeout(timer);  // cancel if user types again before
500ms
  }, [query, onSearch]);

  return <input value={query} onChange={e => setQuery(e.target.value)} />;
}

// THROTTLE — run at most once per time period
// "Run at most once per 200ms, no matter how fast scroll fires"
function ScrollTracker() {
  const [scrollY, setScrollY] = useState(0);
  const lastRunRef = useRef(0);

  useEffect(() => {
    function handleScroll() {
      const now = Date.now();
      if (now - lastRunRef.current < 200) return;  // throttle to 200ms
      lastRunRef.current = now;
      setScrollY(window.scrollY);
    }

    window.addEventListener("scroll", handleScroll);
    return () => window.removeEventListener("scroll", handleScroll);
  }, []);

  return <p>Scroll Y: {scrollY}</p>;
}
```

Technique	How it Works	Use When
Debounce	Wait until action STOPS, then run once	Search input, form auto-save, resize handler
Throttle	Run at most once per time period, then allow	Scroll tracking, real-time game events, API rate limits

Topic 10: Image Optimization

Images are often the heaviest assets on a page. Slow images = slow app. These simple techniques can dramatically improve perceived performance.

```
// 1. LAZY LOAD IMAGES — only load when in viewport
function ProductImage({ src, alt }) {
  return (
    <img
      src={src}
      alt={alt}
```

```
      loading="lazy"     // browser-native lazy loading — zero JS needed
      decoding="async"   // decode in background, does not block render
    />
  );
}

// 2. USE WebP FORMAT — 30% smaller than JPEG/PNG with same quality
<picture>
  <source srcSet="/image.webp" type="image/webp" />
  <img src="/image.jpg" alt="Product" />  // fallback for old browsers
</picture>

// 3. SHOW SKELETON while image loads — better UX than blank space
function ProductCard({ product }) {
  const [loaded, setLoaded] = useState(false);

  return (
    <div>
      {!loaded && <div className="skeleton" />}
      <img
        src={product.image}
        alt={product.name}
        loading="lazy"
        onLoad={() => setLoaded(true)}
        style={{ display: loaded ? "block" : "none" }}
      />
    </div>
  );
}

// 4. CORRECT SIZES — never load 1200px image for 200px thumbnail
// Use CDN/image service to serve correctly sized images:
<img
  src={`https://cdn.example.com/products/${id}?w=400&h=400&format=webp`}
  alt={name}
  width={200}    // always specify dimensions — prevents layout shift (CLS)
  height={200}
/>
```

Topic 11: React DevTools Profiler — Measure Before You Optimize

The React DevTools Profiler is a browser extension feature that shows you EXACTLY which components re-rendered, how long each render took, and why they re-rendered. This is your primary tool for identifying real performance bottlenecks.

```
// HOW TO USE REACT DEVTOOLS PROFILER:

// 1. Install "React Developer Tools" extension in Chrome/Firefox
```

```
// 2. Open Chrome DevTools → go to "Profiler" tab

// 3. Click the red record button, interact with your app, stop recording

// 4. What you see:
//    — Flame graph: shows ALL components that rendered, color = time taken
//    — Yellow/orange = slow renders
//    — Each bar shows: component name, render time, render count
//    — Click any bar to see WHY it re-rendered (prop change, state change,
parent)

// 5. Key things to look for:
//    — Components re-rendering too often (high render count)
//    — Components taking too long per render (yellow/orange in flame graph)
//    — Large component trees re-rendering when only a leaf changed

// Enable "Highlight updates" in React DevTools settings:
// → Every component that re-renders gets a colored highlight on screen
// → Blue = re-rendered. You can see unnecessary re-renders visually.

// THE RULE: Only optimize what the Profiler shows is actually slow.
// Do not guess. Do not add React.memo everywhere.
// Measure → Identify bottleneck → Fix → Measure again.
```

💡 **Tip**

React DevTools Profiler is free, built-in, and incredibly powerful.

The "why did this render?" feature in Profiler shows exactly which prop or state caused each re-render.

Only after using the Profiler and identifying a real slow component should you reach for React.memo, useMemo, or useCallback.

Most React apps never need these optimizations. The Profiler will tell you if yours does.

Topic 12: useTransition and useDeferredValue — React 18 Concurrency

React 18 introduced Concurrent Mode features that let you mark certain state updates as non-urgent. This prevents the UI from freezing when doing expensive operations.

```
// useTransition — mark a state update as non-urgent
import { useState, useTransition } from "react";

function SearchPage() {
  const [query,     setQuery]     = useState("");
  const [results,   setResults]   = useState([]);
  const [isPending, startTransition] = useTransition();

  function handleSearch(e) {
    const value = e.target.value;
```

```
    // Urgent: update input immediately (user sees their typing)
    setQuery(value);

    // Non-urgent: heavy filtering can wait — marked as "transition"
    // React may interrupt this if a more urgent update comes in
    startTransition(() => {
      const filtered = LARGE_LIST.filter(item =>
        item.name.toLowerCase().includes(value.toLowerCase())
      );
      setResults(filtered);
    });
  }

  return (
    <div>
      <input value={query} onChange={handleSearch} placeholder="Search..." />
      {isPending && <span>Updating results...</span>}
      <ResultsList results={results} />
    </div>
  );
}

// Without useTransition: typing is BLOCKED while filtering 10,000 items
// With useTransition: typing is always instant; results update when React has
time
```

```
// useDeferredValue — defer an expensive derived computation
import { useState, useDeferredValue } from "react";

function ProductSearch() {
  const [query, setQuery] = useState("");

  // Deferred value — "use the fresh value when React has time,
  //                    use stale value in the meantime"
  const deferredQuery = useDeferredValue(query);

  // ExpensiveResults re-renders with fresh query when React is free
  // Input stays responsive because it uses the immediate "query"
  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Search..."
      />
      {/* This component gets the deferred (possibly stale) value */}
      {/* It does not block the input from being responsive */}
      <ExpensiveResults query={deferredQuery} />
    </div>
  );
```

```
}
```

Feature	useTransition	useDeferredValue
Controls	Marks a state UPDATE as non-urgent	Marks a VALUE as non-urgent to derive from
API	Wraps setState in startTransition()	Wraps the input value
isPending	YES — gives you a loading flag	NO — no loading indicator built-in
Use when	You control the state setter	You receive a value from outside (prop, parent)

Topic 13: Performance Checklist — Complete

Here is the complete checklist to run through when optimizing a React app. Go through it in order — highest impact first.

Priority	Technique	Impact	Effort
1 — Highest	Code splitting / lazy loading	Massive — reduces initial bundle	Low
2	Image optimization (WebP, lazy load)	Very High — images are heaviest assets	Low
3	Virtualize long lists (react-window)	High for lists >100 items	Medium
4	Debounce expensive event handlers	High for search/scroll	Low
5	Move state down / lift content up	Medium — prevents cascading re-renders	Low
6	React.memo on expensive child components	Medium — targeted re-render prevention	Medium
7	useCallback for memoized child callbacks	Medium — works with React.memo	Low
8	useMemo for expensive calculations	Low-Medium — only when measured slow	Low
9	useTransition / useDeferredValue	Medium for complex UIs	Low
10 — Last	createSelector for Redux selectors	Low for most apps	Low

Chapter 11: Interview Questions with Full Answers

Q1: What is React.memo? When should you use it?

Answer:

React.memo is a Higher Order Component that wraps a function component and memoizes it. Before re-rendering, it performs a shallow comparison of previous and new props. If props are equal, it skips re-rendering and returns the cached output.

By default in React, when a parent re-renders, all its children re-render too — even if their props did not change. React.memo prevents this wasted render for a specific child.

Use it when: the component renders often, its props rarely change, the component is expensive to render (complex UI, lots of calculations). Common use cases: list items in large lists, complex chart components, table rows.

Do NOT use it blindly. React.memo has overhead — storing cached output, comparing props on every parent render. For simple cheap components, the memo overhead can exceed the render cost.

Also: React.memo only works if the props are referentially stable. If a new function or object is created every parent render as a prop, memo fails. Fix with useCallback and useMemo.

Q2: What is the difference between useMemo and useCallback?

Answer:

Both memoize something to prevent unnecessary recalculation, but they memoize different things.

useMemo memoizes a COMPUTED VALUE — the result of calling a function. It re-runs the function and returns a new value only when dependencies change. Use for expensive calculations: filtering large arrays, computing statistics, building complex objects.

useCallback memoizes a FUNCTION REFERENCE — it returns the same function object across renders until dependencies change. Use when passing callbacks to React.memo children, where a new function reference every render would break memoization.

Simple way to remember: useMemo(() => a + b, deps) returns the SUM. useCallback(() => doSomething(), deps) returns the FUNCTION.

useCallback(fn, deps) is equivalent to useMemo(() => fn, deps). Same concept, different syntax, different return type.

Q3: Why does React.memo sometimes not work?

Answer:

React.memo does a shallow comparison of props. It breaks when props contain non-primitive values (objects, arrays, functions) that are created fresh on every parent render — because they are new references even if their content is the same.

Example: if a parent passes <Child config={{ size: 10 }} />, a new config object is created every render. React.memo sees a different reference and re-renders Child even though content is identical.

Fix for objects/arrays: wrap with useMemo in the parent so the same reference is passed.

Fix for functions: wrap with useCallback in the parent so the same function reference is passed.

The combination of React.memo on the child + useCallback/useMemo on the parent is the complete solution.

Q4: What is code splitting and how do you implement it in React?

Answer:

Code splitting is the technique of breaking your JavaScript bundle into smaller chunks that load on demand, instead of loading everything upfront. This dramatically reduces the initial load time.

In React, implement it with React.lazy() and Suspense. Replace static imports with lazy imports: const HomePage = lazy(() => import("./pages/HomePage")). Wrap in Suspense with a fallback component shown while the chunk loads.

Vite and webpack automatically create separate JavaScript files at every lazy() boundary. When the user navigates to /admin, the admin chunk downloads — not before.

High-impact candidates for code splitting: routes/pages (each page is a separate chunk), heavy libraries like chart.js or PDF viewers, admin panels that most users never visit, rich text editors.

Core Web Vitals impact: code splitting directly improves LCP (Largest Contentful Paint) and TTI (Time to Interactive) — metrics that affect both user experience and SEO ranking.

Q5: What is virtualization? When would you use it?

Answer:

Virtualization (or "windowing") renders only the items currently visible in the viewport, not the entire list. As the user scrolls, items are swapped in and out — but the number of DOM nodes stays constant.

Without virtualization, a list of 10,000 items creates 10,000 DOM nodes. The browser must layout, paint, and manage all of them — causing sluggish scrolling and high memory usage.

With react-window, a 10,000-item list maintains only ~15 DOM nodes (the visible ones). Scrolling is always 60fps regardless of list size.

Use when: list has more than 100-200 items, scrolling performance is noticeably slow, items are complex to render.

The key API: FixedSizeList for equal-height rows (simplest), VariableSizeList for different heights.

Q6: What is the "move state down" optimization pattern?

Answer:

Moving state down means keeping state as close as possible to the component that actually uses it — instead of lifting it to a high-level parent unnecessarily.

When state lives in a parent, every state change re-renders the parent AND all its children. If only one child uses that state, all the others re-render for nothing.

Moving the state down into the component that uses it means only that component re-renders on state change. All its siblings are unaffected.

Example: a toggle button for a menu. If the menu state is in App, every App render re-renders the entire page. If the state is in NavBar (which contains the menu), only NavBar re-renders on toggle.

This is a zero-cost optimization — no extra libraries, no useCallback/useMemo needed. Just structure your state correctly.

Q7: What is the difference between debounce and throttle?

Answer:

Both limit how often a function runs in response to repeated events, but differently.

Debounce waits until the action STOPS before running. If a user types quickly, the function runs only once — 500ms after the last keystroke. The function "bounces" until you stop calling it.

Throttle runs at most once per time period regardless of how many events fire. If set to 200ms, it runs at 0ms, 200ms, 400ms — no matter how many scroll events occur in between.

Use debounce for: search input (wait for user to finish typing), form auto-save, resize handler.

Use throttle for: scroll position tracking, real-time game events, sending periodic analytics.

In React, implement debounce with setTimeout + clearTimeout in useEffect. For production, use lodash.debounce or lodash.throttle.

Q8: What is useTransition in React 18?

Answer:

useTransition is a React 18 hook that lets you mark certain state updates as non-urgent. It returns [isPending, startTransition].

Wrap expensive state updates inside startTransition(). React treats these as low-priority and will interrupt them if a more urgent update (like user typing) needs to happen.

Without it: if filtering 10,000 items takes 300ms, the UI freezes for 300ms during typing — because React completes the filter before updating the input.

With startTransition: the input updates instantly (urgent). The filter runs in the background (non-urgent) and React shows the latest results when it has time.

isPending is true while the transition is in progress — use it to show a loading indicator.

The key insight: React can now work on multiple state updates at different priorities — this is Concurrent Mode.

Q9: How do you measure React performance?

Answer:

Three main tools.

React DevTools Profiler: records renders, shows a flame graph of which components rendered, how long each took, and why they rendered. The "why did this render?" feature is invaluable. Install the React Developer Tools browser extension to access it.

Chrome Performance tab: captures a full performance trace including JavaScript execution, layout, paint. Shows frame rate, long tasks, total blocking time.

Core Web Vitals in Chrome Lighthouse: LCP (Largest Contentful Paint), FID (First Input Delay), CLS (Cumulative Layout Shift). These directly measure user experience and affect SEO.

Process: profile first → identify the bottleneck → apply the right optimization → profile again to confirm improvement. Never optimize without measuring first.

Q10: What is the risk of over-using React.memo, useMemo, and useCallback?

Answer:

These tools have overhead. React.memo stores the previous render output and compares props on every render. useMemo stores the cached value and compares dependencies. useCallback stores the function. All of this costs memory and CPU.

For cheap components and simple values, this overhead can be greater than the cost of just re-rendering.

Over-using them makes code harder to read and reason about. A component wrapped in React.memo with a useMemo value and three useCallback props is significantly more complex than just letting React re-render normally.

They can also give false confidence. If applied wrong (missing dependencies, wrong comparison) they introduce bugs instead of fixing them.

Rule: profile first. If the Profiler shows a specific component is slow — use the tools. If the app feels fast — leave it alone.

Q11: What is image lazy loading and why does it matter?

Answer:

Image lazy loading means images are not downloaded until they are near the viewport. Instead of loading all images on page load, each image loads only when the user is about to scroll to it.

Why it matters: images are typically the heaviest assets. A page with 20 product images that all load upfront wastes bandwidth downloading images the user may never see. Lazy loading reduces initial page weight significantly.

Native implementation: <img loading="lazy"> — browser-native, zero JavaScript needed, supported in all modern browsers.

Additionally: specify image dimensions (width/height) to prevent Cumulative Layout Shift (CLS) — where content jumps when images load. Use WebP format for 30% smaller file size with the same quality. Serve correctly sized images from a CDN — do not load a 1200px image for a 200px thumbnail.

Q12: What is the complete order of performance optimizations you would apply to a slow React app?

Answer:

I follow impact-first order.

1. Code splitting with React.lazy — reduces initial bundle. Biggest impact for the least effort.
2. Image optimization — add loading="lazy", convert to WebP, correct sizing. Images are usually the heaviest assets.
3. Virtualize long lists with react-window — if the app has lists of 100+ items.
4. Debounce heavy event handlers — search inputs, resize listeners.
5. Move state down — restructure to avoid unnecessary parent re-renders.
6. React.memo + useCallback on specific identified bottlenecks — ONLY after Profiler shows these components are re-rendering unnecessarily.
7. useMemo for expensive calculations — ONLY where Profiler shows recalculation is costly.

I always profile before and after each change to confirm the improvement is real.

Common Mistakes in Performance Optimization

Mistake 1: Wrapping everything in React.memo and useMemo without profiling

```
// ❌ WRONG — adding memo/useMemo blindly everywhere
const Name = React.memo(({ name }) => <span>{name}</span>);
const doubled = useMemo(() => count * 2, [count]);  // count * 2 is instant!

// ✅ CORRECT — profile first, then add where Profiler shows it helps
const Name = ({ name }) => <span>{name}</span>;  // cheap, no memo needed
const doubled = count * 2;  // instant calculation, no memo needed
```

Mistake 2: Using useCallback without React.memo on the child

```
// ❌ USELESS — useCallback without React.memo does nothing
function Parent() {
  const handleClick = useCallback(() => doSomething(), []);
  return <Child onClick={handleClick} />;  // Child is NOT memoized!
}
// Child re-renders anyway — useCallback served no purpose

// ✅ CORRECT — both together
function Parent() {
  const handleClick = useCallback(() => doSomething(), []);
  return <MemoizedChild onClick={handleClick} />;
}
const MemoizedChild = React.memo(({ onClick }) => <button
onClick={onClick}>Click</button>);
```

Mistake 3: Sorting state array directly (mutation)

```
// ❌ WRONG — .sort() mutates state array
const sortedProducts = products.sort((a, b) => a.price - b.price);

// ✅ CORRECT — spread to copy, then sort
const sortedProducts = useMemo(
  () => [...products].sort((a, b) => a.price - b.price),
  [products]
);
```

Mistake 4: Creating new objects/functions in JSX props

```
// ❌ WRONG — new object AND function created every render — breaks memo
<MemoizedChild
  config={{ size: 10, theme: "dark" }}  // new object every render
```

```
  onAction={() => handleAction("buy")}   // new function every render
/>

// ✅ CORRECT — stable references
const config   = useMemo(() => ({ size: 10, theme: "dark" }), []);
const onAction = useCallback(() => handleAction("buy"), []);
<MemoizedChild config={config} onAction={onAction} />
```

Quick Reference — Chapter 11 Summary

Tool	What It Does	When To Use
React.memo	Skip re-render if props unchanged (shallow compare)	Expensive children that render with same props often
useMemo	Cache expensive computed value	Filter/sort large arrays, avoid new object refs
useCallback	Cache function reference	Callbacks passed to React.memo children
React.lazy	Load component only when needed	Routes, heavy components, admin pages
Suspense	Show fallback while lazy component loads	Always with React.lazy
react-window	Render only visible list items	Lists with 100+ items
Debounce	Run after action stops	Search input, form auto-save
Throttle	Run at most once per period	Scroll, resize, real-time events
useTransition	Mark state update as non-urgent	Filtering while keeping input responsive
useDeferredValue	Defer a value for expensive derivations	Received value that drives expensive render
Profiler	Measure actual render time and reasons	ALWAYS — before any optimization

> ✅ **Remember**
> **Car engine analogy** — every render = piston fire. Stop wasted fires.
> **MEASURE FIRST** — Profiler → bottleneck → fix → measure again. Never guess.
> **React.memo** = valve. Stops child re-rendering when props did not change.

useMemo = cache VALUE. **useCallback** = cache FUNCTION. Always need both together with memo.

React.memo + useCallback always go together. One without the other is usually useless.

Move state down = free optimization. No libraries. Just good structure.

Code splitting = biggest bang for buck. Lazy load pages and heavy components.

Virtualization = never render what user cannot see. react-window for 100+ item lists.

Debounce = wait for stop. **Throttle** = run at most once per period.

Do NOT over-optimize. Adding useMemo to simple math is slower than not having it.

End of Chapter 11 • Next: Chapter 12 — Custom Hooks

Chapter 12: Custom Hooks

Extract, reuse, and share logic across components — the professional way

By Syed Shaaz Akhtar | Interview Focused Guide

Custom hooks are one of the most powerful features in React. They let you extract stateful logic from a component into a reusable function. Once you understand custom hooks, you will write cleaner components, avoid duplication, and think about React in a completely different — more professional — way.

If you look at any real React codebase — React Router's useNavigate, React Query's useQuery, SWR's useSWR, every popular library — they are all built using custom hooks. Learning to write them is what separates mid-level from senior developers.

> 💡 **Tip**
> 🔧 THE ANALOGY for this chapter:
> Custom hooks are like POWER TOOLS in a workshop.
> — Without custom hooks: every project, you wire up the motor, assemble the blade, set the voltage... manually. Every time. 50 lines of setup for the same drill.
> — With custom hooks: you pick up the drill. It is ready. You just use it.
> A custom hook = a packaged, reusable tool.
> You build it ONCE. You use it in any project, any component, any team.
> The drill does not share its battery with other drills (logic is shared, state is NOT).
> Each component that uses the hook gets its OWN independent state — like giving each worker their own drill.

Topic 1: What is a Custom Hook?

A custom hook is a JavaScript function whose name starts with "use" and which calls other React hooks inside it. That is the complete definition. No magic, no special API — just a function.

It lets you extract stateful logic from a component so it can be reused in multiple components. The key insight: the logic is shared, but the state is NOT shared. Every component that calls the hook gets its own independent state.

```
// WITHOUT custom hook — same logic copy-pasted in two components

// Component A — online status logic
function UserProfile() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);

  useEffect(() => {
    const handleOnline  = () => setIsOnline(true);
    const handleOffline = () => setIsOnline(false);
    window.addEventListener("online",  handleOnline);
    window.addEventListener("offline", handleOffline);
```

```
    return () => {
      window.removeEventListener("online",  handleOnline);
      window.removeEventListener("offline", handleOffline);
    };
  }, []);

  return <p>{isOnline ? "Online" : "Offline"}</p>;
}

// Component B — SAME logic, copy-pasted
function NetworkStatusBar() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);

  useEffect(() => {       // identical useEffect — duplicated code
    const handleOnline  = () => setIsOnline(true);
    const handleOffline = () => setIsOnline(false);
    window.addEventListener("online",  handleOnline);
    window.addEventListener("offline", handleOffline);
    return () => {
      window.removeEventListener("online",  handleOnline);
      window.removeEventListener("offline", handleOffline);
    };
  }, []);

  return <div className={isOnline ? "green" : "red"}>Network Status</div>;
}

// WITH custom hook — extract the logic once

// useOnlineStatus.js
function useOnlineStatus() {
  const [isOnline, setIsOnline] = useState(navigator.onLine);

  useEffect(() => {
    const handleOnline  = () => setIsOnline(true);
    const handleOffline = () => setIsOnline(false);
    window.addEventListener("online",  handleOnline);
    window.addEventListener("offline", handleOffline);
    return () => {
      window.removeEventListener("online",  handleOnline);
      window.removeEventListener("offline", handleOffline);
    };
  }, []);

  return isOnline;  // return what the component needs
}

// Now both components are CLEAN
function UserProfile() {
  const isOnline = useOnlineStatus();  // one line!
  return <p>{isOnline ? "Online" : "Offline"}</p>;
```

```
}

function NetworkStatusBar() {
  const isOnline = useOnlineStatus();  // same hook, independent state
  return <div className={isOnline ? "green" : "red"}>Network Status</div>;
}
```

> ✅ **Remember**
> 3 rules for custom hooks:
> 1. Name MUST start with "use" — useOnlineStatus, useFetch, useLocalStorage.
> 2. Can call other hooks (useState, useEffect, useRef, other custom hooks).
> 3. Cannot be called inside conditions, loops, or regular functions — same rules as built-in hooks.
> Each component calling the hook gets INDEPENDENT state — they do not share values.
> The logic is shared. The state instances are separate.

Topic 2: The Anatomy of a Custom Hook

Every custom hook follows a predictable structure. Understanding the parts makes writing them easy.

```
//  useSomething.js
//  ┌──────────────────────────────────────────────────────────────┐
//  │ 1. Imports — any hooks this custom hook uses                 │
//  │ 2. Function definition — starts with "use", takes params     │
//  │ 3. Internal state — useState, useRef etc                     │
//  │ 4. Effects — useEffect for side effects/subscriptions        │
//  │ 5. Functions — logic exposed to the consumer                 │
//  │ 6. Return — the values/functions the component needs         │
//  └──────────────────────────────────────────────────────────────┘

// Anatomy example:
import { useState, useEffect, useCallback } from "react";  // 1. Imports

function useWindowSize() {                                   // 2. Function
  const [size, setSize] = useState({                         // 3. State
    width:  window.innerWidth,
    height: window.innerHeight,
  });

  const handleResize = useCallback(() => {                   // 5. Functions
    setSize({
      width:  window.innerWidth,
      height: window.innerHeight,
    });
  }, []);

  useEffect(() => {                                          // 4. Effects
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
```

```
  }, [handleResize]);

  return size;                                                    // 6. Return
}

// Usage — clean, readable, no setup noise
function ResponsiveLayout() {
  const { width, height } = useWindowSize();
  return <p>Window: {width} x {height}</p>;
}
```

Topic 3: useFetch — The Most Classic Custom Hook

The data-fetching hook is the most commonly written custom hook. Every real app fetches data. Instead of writing loading/error/data state + useEffect in every component that fetches, you extract it into useFetch.

```
// useFetch.js — complete, production-ready data fetching hook
import { useState, useEffect, useCallback, useRef } from "react";

function useFetch(url) {
  const [data,    setData]    = useState(null);
  const [loading, setLoading] = useState(true);
  const [error,   setError]   = useState(null);

  // useRef to track if component is still mounted
  // Prevents setState after unmount (memory leak warning)
  const mountedRef = useRef(true);

  const fetchData = useCallback(async () => {
    try {
      setLoading(true);
      setError(null);

      const controller = new AbortController();
      const response   = await fetch(url, { signal: controller.signal });

      if (!response.ok) throw new Error(`HTTP ${response.status}: $
{response.statusText}`);

      const json = await response.json();

      // Only update state if component is still mounted
      if (mountedRef.current) {
        setData(json);
        setLoading(false);
      }

      return () => controller.abort();  // cleanup — cancel request on unmount
```

```
    } catch (err) {
      if (err.name === "AbortError") return;  // ignore cancelled requests
      if (mountedRef.current) {
        setError(err.message);
        setLoading(false);
      }
    }
  }, [url]);

  useEffect(() => {
    mountedRef.current = true;
    fetchData();
    return () => { mountedRef.current = false; };  // cleanup on unmount
  }, [fetchData]);

  // Expose refetch so consumer can manually re-trigger
  return { data, loading, error, refetch: fetchData };
}
```

```
export default useFetch;
```

```
// Using useFetch — components are now super clean
```

```
// ProductList.jsx
function ProductList() {
  const { data: products, loading, error, refetch } = useFetch("/api/products");

  if (loading) return <Spinner />;
  if (error)   return <Error message={error} onRetry={refetch} />;
  if (!products?.length) return <EmptyState />;

  return (
    <div>
      <button onClick={refetch}>Refresh</button>
      {products.map(p => <ProductCard key={p.id} product={p} />)}
    </div>
  );
}

// UserProfile.jsx — same hook, completely independent data/loading/error state
function UserProfile({ userId }) {
  const { data: user, loading, error } = useFetch(`/api/users/${userId}`);

  if (loading) return <ProfileSkeleton />;
  if (error)   return <p>Could not load profile</p>;

  return <div><h1>{user.name}</h1><p>{user.email}</p></div>;
}

// Same hook instance, independent state:
```

```
// ProductList has its OWN loading/data/error
// UserProfile has its OWN loading/data/error
// They never interfere with each other
```

Topic 4: useLocalStorage — Persisting State Across Refreshes

localStorage persists data across page refreshes. But wiring useState with localStorage manually every time is repetitive. This hook makes it feel exactly like useState — but the value survives a browser refresh.

```
// useLocalStorage.js
import { useState, useEffect, useCallback } from "react";

function useLocalStorage(key, initialValue) {
  // Lazy initializer — reads from localStorage ONCE on first render
  const [storedValue, setStoredValue] = useState(() => {
    try {
      const item = localStorage.getItem(key);
      // Parse stored JSON or return initialValue
      return item !== null ? JSON.parse(item) : initialValue;
    } catch {
      return initialValue;
    }
  });

  // setValue: update state AND persist to localStorage
  const setValue = useCallback((value) => {
    try {
      // Support functional updates: setValue(prev => prev + 1)
      const valueToStore = value instanceof Function ? value(storedValue) :
value;
      setStoredValue(valueToStore);
      localStorage.setItem(key, JSON.stringify(valueToStore));
    } catch (err) {
      console.error("useLocalStorage error:", err);
    }
  }, [key, storedValue]);

  // removeValue: clear from state and localStorage
  const removeValue = useCallback(() => {
    setStoredValue(initialValue);
    localStorage.removeItem(key);
  }, [key, initialValue]);

  // API identical to useState — drop-in replacement
  return [storedValue, setValue, removeValue];
}

export default useLocalStorage;
```

```
// Usage — works exactly like useState but survives page refresh

// Theme preference
function App() {
  const [theme, setTheme] = useLocalStorage("theme", "light");

  return (
    <div className={`app theme-${theme}`}>
      <button onClick={() => setTheme(t => t === "light" ? "dark" : "light")}>
        Toggle Theme
      </button>
    </div>
  );
}

// Cart persistence
function usePersistedCart() {
  const [cart, setCart, clearCart] = useLocalStorage("cart", []);

  const addItem = useCallback((item) => {
    setCart(prev => {
      const exists = prev.find(i => i.id === item.id);
      if (exists) return prev.map(i => i.id === item.id ? { ...i, qty: i.qty + 1
} : i);
      return [...prev, { ...item, qty: 1 }];
    });
  }, [setCart]);

  return { cart, addItem, clearCart };
}

// User language preference
function LanguageSelector() {
  const [lang, setLang] = useLocalStorage("lang", "en");
  return (
    <select value={lang} onChange={e => setLang(e.target.value)}>
      <option value="en">English</option>
      <option value="hi">Hindi</option>
      <option value="mr">Marathi</option>
    </select>
  );
}
```

Topic 5: useDebounce — Clean Debouncing in Any Component

From Chapter 11, you know debouncing prevents excessive API calls. The useDebounce hook makes it reusable — one clean hook you can drop into any component that needs it.

```
// useDebounce.js
```

```
import { useState, useEffect } from "react";

function useDebounce(value, delay = 500) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    // Set a timer to update debouncedValue after delay
    const timer = setTimeout(() => {
      setDebouncedValue(value);
    }, delay);

    // Cleanup: cancel the timer if value changes before delay expires
    // This is the core of debounce — only the LAST value after a pause survives
    return () => clearTimeout(timer);
  }, [value, delay]);

  return debouncedValue;
}

export default useDebounce;

// Usage — any component that needs search/debounce
function SearchBar() {
  const [query,         setQuery]   = useState("");
  const [results,       setResults] = useState([]);
  const debouncedQuery = useDebounce(query, 400);

  // This effect only runs when debouncedQuery changes (after 400ms pause)
  useEffect(() => {
    if (!debouncedQuery.trim()) { setResults([]); return; }
    fetch(`/api/search?q=${debouncedQuery}`))
      .then(r => r.json())
      .then(setResults);
  }, [debouncedQuery]);

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Search..."
      />
      {results.map(r => <div key={r.id}>{r.title}</div>)}
    </div>
  );
}
```

Topic 6: useForm — The Most Useful Custom Hook You Will Ever Write

Almost every app has forms. The same useState + validation + touched + handleChange + handleBlur + handleSubmit logic repeats in every form component. useForm extracts all of it. Write it once, use it everywhere.

```
// useForm.js — complete generic form hook
import { useState, useCallback } from "react";

function useForm(initialValues, validate) {
  const [values,  setValues]  = useState(initialValues);
  const [touched, setTouched] = useState({}); // which fields user has visited
  const [isSubmitting, setIsSubmitting] = useState(false);

  // Derive errors from current values — never stored separately
  const errors = validate ? validate(values) : {};
  const isValid = Object.keys(errors).length === 0;

  // Generic change handler — works for ALL input types
  const handleChange = useCallback((e) => {
    const { name, value, type, checked } = e.target;
    setValues(prev => ({
      ...prev,
      [name]: type === "checkbox" ? checked : value,
    }));
  }, []);

  // Generic blur handler — marks field as touched
  const handleBlur = useCallback((e) => {
    const { name } = e.target;
    setTouched(prev => ({ ...prev, [name]: true }));
  }, []);

  // Generic submit handler
  const handleSubmit = useCallback((onSubmit) => async (e) => {
    e.preventDefault();
    // Touch ALL fields to show all errors
    const allTouched = Object.keys(values).reduce((acc, key) => {
      acc[key] = true; return acc;
    }, {});
    setTouched(allTouched);

    if (!isValid) return;

    setIsSubmitting(true);
    try {
      await onSubmit(values);
    } finally {
      setIsSubmitting(false);
    }
  }, [values, isValid]);

  // Reset form to initial values
```

```
  const reset = useCallback(() => {
    setValues(initialValues);
    setTouched({});
  }, [initialValues]);

  // Helper: should we show this field's error?
  const showError = useCallback((field) => {
    return touched[field] && errors[field];
  }, [touched, errors]);

  return {
    values, errors, touched, isValid, isSubmitting,
    handleChange, handleBlur, handleSubmit, reset, showError,
    // Shorthand to spread onto any input
    getFieldProps: (name) => ({
      name,
      value: values[name],
      onChange: handleChange,
      onBlur:   handleBlur,
    }),
  };
}

export default useForm;
```

```
// RegisterPage.jsx — using useForm
import useForm from "../hooks/useForm";

// Validation function — pure function, easy to test
function validateRegister(values) {
  const errors = {};
  if (!values.name.trim())         errors.name     = "Name is required";
  else if (values.name.length < 2) errors.name     = "Min 2 characters";
  if (!values.email)               errors.email    = "Email is required";
  else if (!/\S+@\S+/.test(values.email)) errors.email = "Invalid email";
  if (values.password.length < 8)  errors.password = "Min 8 characters";
  if (values.confirm !== values.password) errors.confirm = "Passwords do not
match";
  return errors;
}

function RegisterPage() {
  const {
    values, errors, isValid, isSubmitting,
    handleSubmit, showError, getFieldProps,
  } = useForm(
    { name: "", email: "", password: "", confirm: "" },
    validateRegister
  );

  async function onSubmit(formValues) {
```

```
    const res = await fetch("/api/register", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(formValues),
    });
    if (res.ok) navigate("/dashboard");
  }

  return (
    <form onSubmit={handleSubmit(onSubmit)}>

      {/* getFieldProps spreads name, value, onChange, onBlur in one line */}
      <input {...getFieldProps("name")} placeholder="Full Name" />
      {showError("name") && <p className="error">{errors.name}</p>}

      <input {...getFieldProps("email")} type="email" placeholder="Email" />
      {showError("email") && <p className="error">{errors.email}</p>}

      <input {...getFieldProps("password")} type="password"
placeholder="Password" />
      {showError("password") && <p className="error">{errors.password}</p>}

      <input {...getFieldProps("confirm")} type="password" placeholder="Confirm
Password" />
      {showError("confirm") && <p className="error">{errors.confirm}</p>}

      <button type="submit" disabled={!isValid || isSubmitting}>
        {isSubmitting ? "Creating account..." : "Register"}
      </button>
    </form>
  );
}
```

🎯 Interview Tip

useForm is a showstopper in interviews. When asked "how do you handle forms in React?", showing a custom useForm hook that handles state, validation, touched, submit, and reset — all in one reusable function — signals senior-level thinking.

Mention: "I extract form logic into a useForm hook to avoid duplicating state management across every form in the app. It exposes getFieldProps for quick wiring, showError for conditional error display, and isSubmitting to disable the button during API calls."

Topic 7: useIntersectionObserver — Infinite Scroll & Lazy Loading

The IntersectionObserver API lets you detect when an element enters or leaves the viewport. It powers infinite scroll, lazy-loading images, and "load more" triggers. Wrapping it in a hook makes it trivially easy to use.

```
// useIntersectionObserver.js
import { useState, useEffect, useRef } from "react";
```

```
function useIntersectionObserver(options = {}) {
  const [isIntersecting, setIsIntersecting] = useState(false);
  const ref = useRef(null);

  useEffect(() => {
    const element = ref.current;
    if (!element) return;

    const observer = new IntersectionObserver(([entry]) => {
      setIsIntersecting(entry.isIntersecting);
    }, {
      threshold: options.threshold || 0.1,   // 10% visible triggers
      rootMargin: options.rootMargin || "0px",
    });

    observer.observe(element);
    return () => observer.disconnect();  // cleanup
  }, [options.threshold, options.rootMargin]);

  return [ref, isIntersecting];
}

// USAGE 1: Infinite scroll — load more when sentinel element is visible
function ProductFeed() {
  const [products, setProducts] = useState([]);
  const [page,     setPage]     = useState(1);
  const [hasMore,  setHasMore]  = useState(true);

  // Attach ref to the "bottom sentinel" element
  const [sentinelRef, isVisible] = useIntersectionObserver();

  useEffect(() => {
    if (isVisible && hasMore) {
      setPage(p => p + 1);  // when sentinel enters viewport — load next page
    }
  }, [isVisible, hasMore]);

  useEffect(() => {
    fetch(`/api/products?page=${page}`))
      .then(r => r.json())
      .then(data => {
        setProducts(prev => [...prev, ...data.items]);
        setHasMore(data.hasMore);
      });
  }, [page]);

  return (
    <div>
      {products.map(p => <ProductCard key={p.id} product={p} />)}
      {/* Sentinel element at the bottom — triggers load when visible */}
```

```
      {hasMore && <div ref={sentinelRef}>Loading more...</div>}
      {!hasMore && <p>All products loaded.</p>}
    </div>
  );
}

// USAGE 2: Animate elements on scroll-into-view
function AnimatedSection({ children }) {
  const [ref, isVisible] = useIntersectionObserver({ threshold: 0.2 });

  return (
    <section
      ref={ref}
      className={isVisible ? "fade-in" : "invisible"}
    >
      {children}
    </section>
  );
}
```

Topic 8: usePrevious — Accessing the Previous Value

React does not give you the previous value of a state or prop by default. But sometimes you need it — to compare old and new values, animate transitions, or track changes. useRef makes this straightforward.

```
// usePrevious.js
import { useRef, useEffect } from "react";

function usePrevious(value) {
  const ref = useRef(undefined);

  // After every render — save the current value into ref
  // ref.current is NOT updated during the current render
  // So inside this render: ref.current = the PREVIOUS render's value
  useEffect(() => {
    ref.current = value;
  });

  return ref.current;  // returns what value was in the PREVIOUS render
}

// Usage examples

// 1. Tracking score changes
function ScoreBoard({ score }) {
  const prevScore = usePrevious(score);

  return (
```

```
    <div>
      <h1>Score: {score}</h1>
      {prevScore !== undefined && (
        <p>
          {score > prevScore ? "▲ Up" : "▼ Down"} from {prevScore}
        </p>
      )}
    </div>
  );
}

// 2. Detecting prop changes for side effects
function UserDetail({ userId }) {
  const prevUserId = usePrevious(userId);

  useEffect(() => {
    if (prevUserId && prevUserId !== userId) {
      console.log(`User changed from ${prevUserId} to ${userId}`);
      // do something specific when user changes
    }
  }, [userId, prevUserId]);

  return <div>User ID: {userId}</div>;
}
```

Topic 9: useClickOutside — Closing Dropdowns and Modals

A classic UI pattern: click outside a dropdown or modal to close it. This logic is almost identical every time you need it — perfect for a custom hook.

```
// useClickOutside.js
import { useEffect, useRef } from "react";

function useClickOutside(handler) {
  const ref = useRef(null);

  useEffect(() => {
    function handleClick(event) {
      // If click is OUTSIDE the ref element — call handler
      if (ref.current && !ref.current.contains(event.target)) {
        handler();
      }
    }

    // mousedown catches click before it fires — feels more responsive
    document.addEventListener("mousedown", handleClick);
    return () => document.removeEventListener("mousedown", handleClick);
  }, [handler]);
```

```
  return ref;
}

// Dropdown.jsx
function Dropdown({ options, onSelect }) {
  const [isOpen, setIsOpen] = useState(false);

  // Attach ref to dropdown container — clicks outside this close it
  const dropdownRef = useClickOutside(() => setIsOpen(false));

  return (
    <div ref={dropdownRef} className="dropdown">
      <button onClick={() => setIsOpen(o => !o)}>
        Options ▼
      </button>

      {isOpen && (
        <ul className="dropdown-menu">
          {options.map(opt => (
            <li key={opt.value} onClick={() => { onSelect(opt);
setIsOpen(false); }}>
              {opt.label}
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}

// Modal.jsx — same hook, different use
function Modal({ onClose, children }) {
  const modalRef = useClickOutside(onClose);

  return (
    <div className="modal-overlay">
      <div ref={modalRef} className="modal-content">
        {children}
      </div>
    </div>
  );
}
```

Topic 10: useMediaQuery — Responsive Components in JavaScript

Sometimes CSS media queries are not enough — you need to know the screen size IN your component logic. useMediaQuery gives you a boolean that tells you whether a media query matches, and updates automatically when the screen resizes.

```
// useMediaQuery.js
import { useState, useEffect } from "react";

function useMediaQuery(query) {
  const [matches, setMatches] = useState(() =>
    window.matchMedia(query).matches  // initial value
  );

  useEffect(() => {
    const mediaQuery = window.matchMedia(query);
    const handleChange = (e) => setMatches(e.matches);

    mediaQuery.addEventListener("change", handleChange);
    return () => mediaQuery.removeEventListener("change", handleChange);
  }, [query]);

  return matches;
}

// Build specific breakpoint hooks on top
const useIsMobile  = () => useMediaQuery("(max-width: 768px)");
const useIsTablet  = () => useMediaQuery("(max-width: 1024px)");
const useIsDarkMode= () => useMediaQuery("(prefers-color-scheme: dark)");

// Usage
function NavBar() {
  const isMobile = useIsMobile();

  return (
    <nav>
      {isMobile
        ? <HamburgerMenu />      // mobile layout
        : <DesktopNavLinks />   // desktop layout
      }
    </nav>
  );
}

function App() {
  const prefersDark = useIsDarkMode();
  // Auto-apply dark theme based on system preference
  return <div className={prefersDark ? "dark" : "light"}>...</div>;
}
```

Topic 11: useAsync — The Complete Async State Machine

Any async operation has three states: idle, loading, success, and error. Instead of repeating this pattern everywhere, useAsync provides a clean, reusable async state machine you can use for any async function.

```
// useAsync.js — generic async state machine
import { useState, useCallback, useRef } from "react";

function useAsync(asyncFn, immediate = false) {
  const [status,  setStatus]  = useState(immediate ? "loading" : "idle");
  const [data,    setData]    = useState(null);
  const [error,   setError]   = useState(null);
  const mountedRef = useRef(true);

  const execute = useCallback(async (...args) => {
    setStatus("loading");
    setError(null);
    try {
      const result = await asyncFn(...args);
      if (mountedRef.current) {
        setData(result);
        setStatus("success");
      }
      return result;
    } catch (err) {
      if (mountedRef.current) {
        setError(err.message);
        setStatus("error");
      }
      throw err;
    }
  }, [asyncFn]);

  // Optional: run immediately on mount
  useEffect(() => {
    mountedRef.current = true;
    if (immediate) execute();
    return () => { mountedRef.current = false; };
  }, [immediate]);  // note: execute intentionally not in deps here

  return {
    execute,
    status,
    data,
    error,
    isIdle:    status === "idle",
    isLoading: status === "loading",
    isSuccess: status === "success",
    isError:   status === "error",
  };
}

// Usage examples

// 1. Manual trigger (button click)
```

```
function DeleteButton({ productId, onDeleted }) {
  const { execute, isLoading, isError, error } = useAsync(
    () => fetch(`/api/products/${productId}`, { method: "DELETE" })
  );

  async function handleDelete() {
    await execute();
    onDeleted(productId);
  }

  return (
    <div>
      <button onClick={handleDelete} disabled={isLoading}>
        {isLoading ? "Deleting..." : "Delete"}
      </button>
      {isError && <p className="error">{error}</p>}
    </div>
  );
}
```

Topic 12: Composing Custom Hooks — Hooks Using Hooks

Custom hooks can use other custom hooks. This is where real power comes in — you build layers of abstractions, each one clean and focused. This is the same "composition" principle that makes React components powerful, applied to hooks.

```
// useProducts.js — composes useFetch, useDebounce, and useLocalStorage
import useFetch         from "./useFetch";
import useDebounce      from "./useDebounce";
import useLocalStorage  from "./useLocalStorage";

function useProducts() {
  const [searchQuery,  setSearchQuery]  = useState("");
  const [category,     setCategory]     = useState("all");
  const [sortBy,       setSortBy]       = useLocalStorage("productSort",
"name");
  // ↑ user's sort preference is REMEMBERED across sessions

  const debouncedQuery = useDebounce(searchQuery, 400);
  // ↑ only triggers API after user pauses typing

  // Build the API URL from current filters
  const apiUrl = useMemo(() => {
    const params = new URLSearchParams({
      q:        debouncedQuery,
      category: category,
      sort:     sortBy,
    });
    return `/api/products?${params}`;
```

```
  }, [debouncedQuery, category, sortBy]);

  const { data, loading, error, refetch } = useFetch(apiUrl);
  // ↑ fetches whenever URL changes

  return {
    products:    data?.products || [],
    total:       data?.total    || 0,
    loading,
    error,
    refetch,
    // Controls
    searchQuery,  setSearchQuery,
    category,     setCategory,
    sortBy,       setSortBy,
  };
}

// ProductsPage.jsx — insanely clean because all logic is in the hook
function ProductsPage() {
  const {
    products, total, loading, error,
    searchQuery, setSearchQuery,
    category,    setCategory,
    sortBy,      setSortBy,
  } = useProducts();

  if (loading) return <ProductGridSkeleton />;
  if (error)   return <ErrorMessage message={error} />;

  return (
    <div>
      <input value={searchQuery} onChange={e =>
setSearchQuery(e.target.value)} />
      <CategoryFilter value={category} onChange={setCategory} />
      <SortSelector value={sortBy} onChange={setSortBy} />
      <p>{total} products</p>
      <div className="grid">
        {products.map(p => <ProductCard key={p.id} product={p} />)}
      </div>
    </div>
  );
}

// The component has ZERO logic — only rendering.
// ALL logic lives in the hook.
// This separation makes components easy to read and hooks easy to test.
```

> 💡 **Tip**
> This is the IDEAL component-hook relationship: component = pure rendering. hook = all logic.

> When a component is more than ~50 lines, ask: "which logic can I extract into a custom hook?"
> Custom hooks can be tested independently of any UI — just test the hook. No need to render a component to test the logic.
> This is how React Query, SWR, and every great library is built — deep composition of small focused hooks.

Topic 13: The Complete Custom Hooks Library — Reference

Here is a quick reference of the most commonly written and asked-about custom hooks. Memorise what each does and when to reach for it.

Hook	What It Does	Key Return
useFetch(url)	Fetch data with loading/error/data state	{ data, loading, error, refetch }
useLocalStorage(k,v)	useState that persists to localStorage	[value, setValue, removeValue]
useDebounce(val, delay)	Debounce any value by a given delay	debouncedValue
useForm(init, validate)	All form state — change, blur, submit, errors	{ values, errors, handleChange, ... }
useOnlineStatus()	Listen to online/offline network events	isOnline (boolean)
useWindowSize()	Track window width and height	{ width, height }
useMediaQuery(query)	Match a CSS media query in JS	matches (boolean)
useClickOutside(fn)	Call fn when user clicks outside the ref element	ref
usePrevious(value)	Access the previous render's value	previousValue
useIntersection(opts)	Detect when element enters viewport	[ref, isIntersecting]
useAsync(fn)	Generic async state machine	{ execute, isLoading, isError, data }
useInterval(fn, delay)	Run a function on a set interval, cleanly	void (runs automatically)
useTimeout(fn, delay)	Run a function once after a delay, cleanly	void (runs automatically)

Chapter 12: Interview Questions with Full Answers

Q1: What is a custom hook? What are the rules?

Answer:

A custom hook is a JavaScript function whose name starts with "use" and which calls other React hooks inside it. There is no special syntax or API — it is just a function.

Rules: (1) The name MUST start with "use" — this tells React (and eslint-plugin-react-hooks) to enforce hook rules on it. (2) Can call any hooks — built-in or other custom hooks. (3) Must follow the Rules of Hooks — only called at the top level of a component or another hook, never inside conditionals, loops, or nested functions.

The key purpose: extract stateful logic from components so it can be reused across multiple components. The logic is shared — but each component that calls the hook gets its own independent state instances.

Q2: How is a custom hook different from a utility function?

Answer:

A utility function is a plain JavaScript function that does not use React hooks. It can be called anywhere — including inside conditionals and loops.

A custom hook calls React hooks (useState, useEffect, etc.) inside it. Because of this, it must follow the Rules of Hooks — only called at the top level, only from components or other hooks.

Example: a function that formats a date string is a utility function — no hooks inside. A function that fetches data, stores it in state, and handles loading/error is a custom hook — it uses useState and useEffect.

The naming convention (starting with "use") is not just style — eslint-plugin-react-hooks uses it to enforce hook rules on your custom functions.

Q3: Does calling the same custom hook in two components share state between them?

Answer:

No — this is one of the most important things to understand about custom hooks.

When you call useOnlineStatus() in two different components, each component gets its own completely independent instance of the hook's state. The isOnline state in Component A is not connected to the isOnline state in Component B.

What IS shared: the logic. The addEventListener/removeEventListener code runs in both components independently. Both respond to online/offline events. But their state variables are separate.

If you actually want to share state between components, use Context API (Chapter 8) or Redux (Chapter 10). Custom hooks share LOGIC, not STATE.

Q4: When should you extract logic into a custom hook?

Answer:

Four signals that logic should become a custom hook:

1. Duplication — the same useState + useEffect pattern appears in 2 or more components. Extract it.
2. Complexity — a component has 50+ lines where most lines are setup logic rather than JSX. Extract the logic.
3. Testability — logic mixed with rendering is hard to unit test. A hook can be tested independently with renderHook from @testing-library/react.
4. Naming — if you can give the extracted logic a clear "use" name that describes what it does (useFetch, useForm, useOnlineStatus), it is probably hook-worthy.

Do NOT extract prematurely. Wait until the duplication or complexity is real. Premature abstraction is as bad as premature optimization.

Q5: What is the benefit of custom hooks over component composition for logic reuse?

Answer:

Component composition reuses UI structure. Custom hooks reuse stateful logic without any UI.

If you want multiple components to share the same visual structure, extract a component. If you want multiple components to share the same state/effect behaviour without caring about how they look, extract a custom hook.

Custom hooks separate concerns: the hook owns all the data-fetching/state logic. The component owns all the rendering. This separation makes both easier to read, change, and test.

Custom hooks work across completely unrelated parts of the UI. A useFetch hook can be used in a product card, a user profile, a notification panel — no shared ancestor or visual relationship needed.

Q6: How would you build a useFetch custom hook?

Answer:

useFetch takes a URL, manages loading/error/data state, and handles cleanup.

State: useState for data (null), loading (true), and error (null).

Effect: useEffect that runs when URL changes. Inside: fetch the URL, on success set data and loading false, on error set error and loading false. Cleanup: AbortController to cancel in-flight requests when component unmounts or URL changes.

Return: { data, loading, error, refetch } — refetch lets the component manually trigger a re-fetch.

Include a mounted ref to prevent calling setState after unmount, which causes a memory leak warning.

This is one of the most commonly asked practical implementation questions in React interviews.

Q7: Can custom hooks call other custom hooks?

Answer:

Yes — and this is one of their biggest strengths. Custom hooks compose just like components.

Example: useProducts can call useDebounce (to delay search), useFetch (to fetch data), and useLocalStorage (to persist the sort preference). Each of these hooks is independently tested and reusable. useProducts just orchestrates them together.

This composability allows you to build very complex behaviour from small, focused, single-responsibility hooks — exactly like building complex UIs from small components.

The React team calls this "extracting a custom hook" — you progressively extract and compose until each hook has one clear job.

Q8: How would you implement useLocalStorage?

Answer:

useLocalStorage(key, initialValue) should work exactly like useState — accepting a key name and initial value, returning [value, setValue].

The useState initializer should be lazy (a function) to read from localStorage only once on mount: useState(() => localStorage.getItem(key) ? JSON.parse(localStorage.getItem(key)) : initialValue).

setValue should update both React state AND localStorage: call the regular setState and also localStorage.setItem(key, JSON.stringify(newValue)).

Support functional updates: if the new value is a function, call it with the current value first — this mirrors useState's behaviour.

Handle errors with try/catch — localStorage can throw in private browsing mode or when storage is full.

Q9: What is the relationship between custom hooks and testing?

Answer:

Custom hooks dramatically improve testability. Logic mixed with JSX is hard to test — you have to render a full component to test the logic.

A custom hook can be tested with renderHook from @testing-library/react. You test just the hook — no component, no DOM rendering needed.

Example: testing useFetch means calling renderHook(() => useFetch("/api/users")), mocking the fetch, and asserting that data/loading/error states update correctly.

This is a major advantage of the "component = rendering, hook = logic" pattern. The hook has clear inputs and outputs. It is a pure test of the logic, not the UI.

Q10: How do you handle cleanup in custom hooks?

Answer:

Cleanup works the same way as in useEffect — return a cleanup function from useEffect inside the hook.

Common cleanups in custom hooks: event listeners (removeEventListener), subscriptions (unsubscribe), timers (clearTimeout/clearInterval), abort controllers for fetch (controller.abort()), IntersectionObserver (observer.disconnect()).

When a component using the hook unmounts, React runs the cleanup function. This prevents memory leaks and stale callbacks.

Using a mountedRef (useRef(true) set to false in cleanup) prevents calling setState after unmount — which causes a "Can't perform a React state update on an unmounted component" warning.

Q11: What are some real-world custom hooks you have written or used?

Answer:

useFetch — data fetching with loading/error/data state. Most commonly used in any app that hits an API.

useForm — form state management with validation, touched tracking, submit handling. Replaces Formik for simpler forms.

useLocalStorage — persistent state. Used for theme, language, draft content, user preferences.

useDebounce — debouncing search inputs or any rapidly changing value.

useClickOutside — close dropdowns/modals on outside click. Used in virtually every UI with overlays.

useIntersectionObserver — infinite scroll and scroll-triggered animations.

useMediaQuery — responsive JavaScript logic when CSS alone is not enough.

usePrevious — accessing previous value for comparisons and animations.

useAsync — generic state machine for any async operation.

Q12: How does useForm custom hook work and what does it return?

Answer:

useForm takes two arguments: initialValues (an object with field names and starting values) and a validate function (takes values, returns errors object).

Internally it maintains: values state (the current form data), touched state (which fields the user has visited), and isSubmitting boolean.

It derives errors from the validate function on every render — errors are never stored in state, always computed fresh.

It returns: values, errors, touched, isValid, isSubmitting, handleChange (updates field by name), handleBlur (marks field touched), handleSubmit (touches all fields, validates, calls your onSubmit function), reset (clears form), showError (helper: only show error if touched AND error), getFieldProps (spreads name/value/onChange/onBlur onto any input).

Usage: const { getFieldProps, showError, errors, handleSubmit } = useForm(initial, validate).

Common Mistakes with Custom Hooks

Mistake 1: Not starting the name with "use"

```
// ❌ WRONG — React will not apply hook rules to this function
function fetchUserData(userId) {
  const [user, setUser] = useState(null);  // useState inside non-hook function
  // eslint-plugin-react-hooks will not warn if rules are violated here
}

// ✅ CORRECT — starts with "use", all hook rules enforced
function useUserData(userId) {
  const [user, setUser] = useState(null);
}
```

Mistake 2: Thinking hooks share state between components

```
// MISCONCEPTION — "both components use the same hook, so they share state"
function ComponentA() {
  const { count } = useCounter();  // count = 0 (ComponentA's own instance)
}

function ComponentB() {
  const { count } = useCounter();  // count = 0 (ComponentB's OWN instance —
SEPARATE)
}

// ComponentA incrementing its count does NOT affect ComponentB's count.
// If you want shared state — use Context or Redux.
// Custom hooks share LOGIC, not STATE.
```

Mistake 3: Calling a hook conditionally

```
// ❌ WRONG — calling hook inside a condition
function Profile({ userId }) {
  if (!userId) return null;  // early return
  const { data } = useFetch(`/api/users/${userId}`);  // hook AFTER early return
  // React will error: hooks must be called in the same order every render
}

// ✅ CORRECT — hook always called, condition handled inside or in return
function Profile({ userId }) {
  const { data } = useFetch(userId ? `/api/users/${userId}` : null);
  if (!userId) return null;
  return <div>{data?.name}</div>;
}
```

Mistake 4: Doing too much in one hook

```
// ❌ BAD — one hook does everything
function useEverything() {
  const [user,     setUser]     = useState(null);
  const [products, setProducts] = useState([]);
  const [cart,     setCart]     = useState([]);
  const [theme,    setTheme]    = useState("light");
  // ... 200 more lines
}

// ✅ GOOD — each hook has one clear responsibility
function useUser()     { /* only user logic     */ }
function useProducts() { /* only products logic */ }
function useCart()     { /* only cart logic     */ }
function useTheme()    { /* only theme logic    */ }

// Compose them where needed
```

```
function CheckoutPage() {
  const { user }     = useUser();
  const { cart }     = useCart();
  const { products } = useProducts();
}
```

Quick Reference — Chapter 12 Summary

Concept	Key Point	Remember
Custom hook definition	Function starting with "use" that calls hooks	useOnlineStatus, useFetch, useForm
State sharing	Each component gets INDEPENDENT state	Logic shared, state NOT shared
When to extract	2+ components same logic, or component too complex	Duplication or >50 lines of setup
Composition	Hooks can call other hooks	useProducts → useFetch + useDebounce
Cleanup	Return cleanup from useEffect inside the hook	Same as normal useEffect cleanup
Testing	Test with renderHook — no component needed	Hooks are easier to test than mixed code
Rules	Same as built-in hooks — top level only	No hooks inside conditions or loops

✅ **Remember**

Power tool analogy — build once, use everywhere. Each user gets their own tool (state).
3 rules: name starts with "use", can call hooks, follow hook order rules.
Logic shared, state NOT shared — most important concept. Use Context/Redux to share state.
useFetch — loading/error/data + AbortController cleanup. Classic interview hook.
useLocalStorage — useState that survives refresh. Lazy init reads from storage.
useDebounce — setTimeout + clearTimeout. Returns debounced value.
useForm — values, errors, touched, handleChange, handleBlur, handleSubmit, getFieldProps.
useClickOutside — ref + document mousedown listener. Closes dropdowns and modals.
Composition — hooks calling hooks. Build complex behaviour from simple focused hooks.
Component = rendering only. Hook = all logic. The ideal separation.

End of Chapter 12 • Next: Chapter 13 — API Calls & Async Patterns

Chapter 13: API Calls & Async Patterns

fetch, axios, async/await, loading states, error handling, race conditions — complete real-world patterns

By Syed Shaaz Akhtar | Interview Focused Guide

Every real React app talks to a server. Login, loading products, saving data, uploading files — all of it goes through API calls. This chapter covers everything you need to handle async operations professionally: the right way to fetch data, handle every failure case, prevent race conditions, and build patterns that scale to large applications.

The patterns in this chapter are what interviewers check when they ask you to "build a product listing page" or "implement a login flow". Get these right and you will stand out from 90% of candidates.

> 💡 **Tip**
> 📞 THE ANALOGY for this chapter:
> An API call is like ORDERING FOOD ON ZOMATO.
> — You place the order = you dispatch a fetch() request
> — "Order placed" screen = loading state (pending)
> — Food arrives = success state (data received)
> — Restaurant cancelled = error state (API failed)
> — You cancel before delivery = AbortController (request cancelled)
> — You order twice by mistake = race condition (two requests in flight)
> — Estimated time = you cannot block UI while waiting — async/await
> You NEVER stare at the door waiting. You go live your life (UI stays responsive).
> When food arrives (data comes), your phone buzzes (state updates, component re-renders).

Topic 1: Async JavaScript — The Foundation

Before React-specific patterns, you must deeply understand the JavaScript primitives that all async operations are built on. These come up in every interview.

Callbacks → Promises → async/await

```
// CALLBACKS (old way) — deeply nested, hard to read
getUser(userId, function(err, user) {
  if (err) { handleError(err); return; }
  getOrders(user.id, function(err, orders) {
    if (err) { handleError(err); return; }
    getProductDetails(orders[0].productId, function(err, product) {
      if (err) { handleError(err); return; }
      console.log(product);  // callback hell — hard to maintain
    });
  });
});
```

```
// PROMISES — chainable, better error handling
getUser(userId)
  .then(user   => getOrders(user.id))
  .then(orders => getProductDetails(orders[0].productId))
  .then(product => console.log(product))
  .catch(err    => handleError(err));  // one catch for all errors

// ASYNC/AWAIT — reads like synchronous code, still async under the hood
async function loadUserOrder(userId) {
  try {
    const user    = await getUser(userId);
    const orders  = await getOrders(user.id);
    const product = await getProductDetails(orders[0].productId);
    console.log(product);
  } catch (err) {
    handleError(err);
  }
}
// async/await is syntactic sugar over Promises.
// await pauses execution of THIS function, but does NOT block the browser.
// Other code keeps running while waiting.
```

```
// KEY CONCEPTS you must know

// 1. Promise states: pending → fulfilled OR rejected
const promise = new Promise((resolve, reject) => {
  setTimeout(() => resolve("data"), 1000);  // fulfilled after 1 second
});

// 2. Promise.all — run multiple promises IN PARALLEL, wait for ALL
const [user, products, orders] = await Promise.all([
  fetch("/api/user"),
  fetch("/api/products"),
  fetch("/api/orders"),
]);
// All 3 requests fire simultaneously — much faster than sequential awaits

// 3. Promise.allSettled — like .all but does NOT reject on first failure
const results = await Promise.allSettled([
  fetch("/api/user"),
  fetch("/api/products"),  // even if this fails...
  fetch("/api/orders"),    // ...this still completes
]);
results.forEach(r => {
  if (r.status === "fulfilled") console.log(r.value);
  if (r.status === "rejected")  console.log(r.reason);
});

// 4. Promise.race — resolves/rejects as soon as the FIRST promise settles
```

```
const fastest = await Promise.race([
  fetch("/api/server-1"),
  fetch("/api/server-2"),
]);
// Useful for: request timeouts, fallback servers
```

Topic 2: fetch() — The Browser's Native API

fetch() is the browser's built-in function for making HTTP requests. It returns a Promise. Understanding how it works is essential before using any library on top of it.

```
// Basic GET request
async function getProducts() {
  const response = await fetch("/api/products");
  // fetch() ONLY rejects on NETWORK failure (no internet, server unreachable)
  // It does NOT reject on HTTP error codes like 404, 500!
  // You MUST check response.ok

  if (!response.ok) {
    throw new Error(`HTTP Error: ${response.status} ${response.statusText}`);
  }

  const data = await response.json();  // parse JSON body
  return data;
}

// POST request — sending data to server
async function createProduct(productData) {
  const response = await fetch("/api/products", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",       // tell server you are sending
JSON
      "Authorization": `Bearer ${getToken()}`,   // auth token
    },
    body: JSON.stringify(productData),  // JS object → JSON string
  });

  if (!response.ok) throw new Error("Failed to create product");
  return await response.json();
}

// PUT — full update
async function updateProduct(id, data) {
  const response = await fetch(`/api/products/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });
```

```
  if (!response.ok) throw new Error("Failed to update");
  return await response.json();
}

// PATCH — partial update
async function patchProduct(id, partialData) {
  const response = await fetch(`/api/products/${id}`, {
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(partialData),
  });
  if (!response.ok) throw new Error("Failed to patch");
  return await response.json();
}

// DELETE
async function deleteProduct(id) {
  const response = await fetch(`/api/products/${id}`, { method: "DELETE" });
  if (!response.ok) throw new Error("Failed to delete");
  // Some DELETE endpoints return 204 No Content — no body to parse
  return response.status === 204 ? null : await response.json();
}
```

✅ **Remember**

fetch() NEVER throws on 404 or 500 — those are valid HTTP responses.
Always check response.ok (true if status 200-299). Throw if not ok.
response.json() also returns a Promise — you need to await it.
response.text() for plain text. response.blob() for files/images.
Content-Type: application/json header is REQUIRED for POST/PUT/PATCH with JSON body.

Topic 3: Axios — The Professional Alternative

Axios is a popular HTTP library that wraps fetch with a cleaner API. It automatically parses JSON, throws on HTTP errors, supports request/response interceptors, and works in both browser and Node.js. Most production React apps use Axios.

```
npm install axios

// src/services/api.js — create a configured Axios instance
import axios from "axios";

const api = axios.create({
  baseURL: "https://api.example.com",  // prepended to every request URL
  timeout: 10000,                       // 10 seconds before timeout error
  headers: {
    "Content-Type": "application/json",
  },
});
```

```
// REQUEST INTERCEPTOR — runs before every request
// Perfect for attaching auth tokens automatically
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;  // must return modified config
  },
  (error) => Promise.reject(error)
);

// RESPONSE INTERCEPTOR — runs after every response
// Perfect for global error handling
api.interceptors.response.use(
  (response) => response,  // pass through successful responses
  (error) => {
    // Handle specific HTTP errors globally
    if (error.response?.status === 401) {
      // Token expired — log out user
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    if (error.response?.status === 403) {
      // Forbidden — redirect to unauthorised page
      window.location.href = "/unauthorized";
    }
    return Promise.reject(error);  // let component handle other errors
  }
);

export default api;
```

```
// Using the configured axios instance
import api from "./api";

// GET — response.data is already parsed JSON (no .json() needed)
const getProducts = async (params) => {
  const response = await api.get("/products", { params });
  // params = { category: "audio", page: 2 } → appended as query string
  return response.data;
};

// POST
const createProduct = async (data) => {
  const response = await api.post("/products", data);
  return response.data;
};
```

```
// PUT
const updateProduct = async (id, data) => {
  const response = await api.put(`/products/${id}`, data);
  return response.data;
};

// DELETE
const deleteProduct = async (id) => {
  const response = await api.delete(`/products/${id}`);
  return response.data;
};

// Axios automatically throws on 4xx/5xx — no need to check response.ok
// error.response.data = server error body
// error.response.status = HTTP status code
// error.message = "Request failed with status code 404"
```

Feature	fetch()	Axios
Built-in	YES — no install	NO — npm install axios
JSON parsing	Manual — await response.json()	Automatic — response.data
HTTP errors	Must check response.ok manually	Auto-throws on 4xx/5xx
Interceptors	Manual — wrap fetch in function	Built-in request/response interceptors
Timeout	Manual — Promise.race with timeout	Built-in — timeout: 10000
Cancel request	AbortController	AbortController (same)
Request config	Verbose object	Cleaner API
Node.js support	Node 18+	YES — all versions
Bundle size	0kb (built-in)	~14kb gzipped

Topic 4: The Complete Data Fetching Pattern in React

Combining everything from the previous topics — here is the complete, production-ready pattern for fetching data in a React component. This is what you write in a real app and what interviewers want to see.

```
// COMPLETE PATTERN — handling all states correctly
function ProductsPage() {
  const [products, setProducts] = useState([]);
  const [loading,  setLoading]  = useState(true);
```

```
  const [error,    setError]    = useState(null);

  useEffect(() => {
    // 1. AbortController — cancel request if component unmounts
    const controller = new AbortController();

    async function fetchProducts() {
      try {
        setLoading(true);
        setError(null);

        const response = await fetch("/api/products", {
          signal: controller.signal,  // link to abort controller
        });

        if (!response.ok) {
          throw new Error(`Failed: ${response.status} ${response.statusText}`);
        }

        const data = await response.json();
        setProducts(data.products);
      } catch (err) {
        // Ignore abort errors — they are intentional (component unmounted)
        if (err.name !== "AbortError") {
          setError(err.message);
        }
      } finally {
        setLoading(false);
      }
    }

    fetchProducts();

    // 2. Cleanup — cancel the request when component unmounts
    return () => controller.abort();
  }, []);  // [] = run once on mount

  // 3. Handle all states
  if (loading) return <ProductGridSkeleton />;
  if (error)   return <ErrorMessage message={error} onRetry={() =>
setError(null)} />;
  if (products.length === 0) return <EmptyState />;

  return (
    <div className="grid">
      {products.map(p => (
        <ProductCard key={p.id} product={p} />
      ))}
    </div>
  );
```

```
}
```

> 📝 **Note**
>
> Why define the async function INSIDE useEffect and call it — instead of making useEffect async?
>
> useEffect cannot be async directly because async functions return a Promise, and useEffect's return value must be a cleanup function or undefined — not a Promise.
>
> Solution: define an async function inside, call it immediately, return cleanup normally.
>
> The finally block sets loading to false whether the request succeeded or failed.

Topic 5: Race Conditions — The Most Dangerous Bug in Async React

A **race condition** happens when multiple async operations are in flight and they complete in an unexpected order. This is a real bug that causes users to see wrong data — and most developers do not even know it exists.

```
// THE RACE CONDITION SCENARIO:
// User is on a page that loads user details by userId from URL param

function UserDetail() {
  const { userId } = useParams();
  const [user, setUser] = useState(null);

  useEffect(() => {
    async function fetchUser() {
      const response = await fetch(`/api/users/${userId}`);
      const data     = await response.json();
      setUser(data);  // ← DANGEROUS
    }
    fetchUser();
  }, [userId]);

  return <div>{user?.name}</div>;
}

// THE RACE CONDITION:
// 1. User visits /users/1  → fetch("/api/users/1") starts (slow, takes 800ms)
// 2. User quickly clicks to /users/2 → fetch("/api/users/2") starts (fast,
takes 200ms)
// 3. Request 2 finishes first → setUser(user2) → shows "User 2" ✓
// 4. Request 1 finishes LATE → setUser(user1) → shows "User 1" on /users/2 ✗
// USER SEES WRONG DATA! User 1's data shows while URL says /users/2
```

```
// FIX 1 — AbortController (preferred, cancels the network request)
useEffect(() => {
  const controller = new AbortController();

  async function fetchUser() {
    try {
```

```
      const response = await fetch(`/api/users/${userId}`, {
        signal: controller.signal,
      });
      const data = await response.json();
      setUser(data);
    } catch (err) {
      if (err.name !== "AbortError") setError(err.message);
      // AbortError = request was intentionally cancelled — not an error
    }
  }

  fetchUser();

  // When userId changes: cleanup cancels the PREVIOUS request
  // before the new request starts
  return () => controller.abort();
}, [userId]);

// Now: user navigates from /users/1 to /users/2
// → cleanup runs → request for user 1 is CANCELLED
// → new request for user 2 starts
// → only user 2's data can arrive — race condition impossible

// FIX 2 — Ignore stale results with a flag (simpler, but does not cancel
network request)
useEffect(() => {
  let isCancelled = false;  // flag for THIS specific effect run

  async function fetchUser() {
    const response = await fetch(`/api/users/${userId}`);
    const data     = await response.json();

    // Only update state if THIS effect is still the current one
    if (!isCancelled) setUser(data);
  }

  fetchUser();
  return () => { isCancelled = true; };  // cleanup: mark as stale
}, [userId]);
// Network request still completes — just silently ignores the result
```

🎯 Interview Tip

Race conditions are an ADVANCED topic that most candidates do not know about.

When asked "how do you handle API calls in useEffect?" — mentioning race conditions and AbortController immediately signals senior-level knowledge.

Say: "I always use AbortController in useEffect to cancel in-flight requests when the component unmounts or dependencies change. This prevents race conditions where stale data overwrites newer data."

Topic 6: Error Handling — Every Case Covered

Professional error handling means handling EVERY failure case gracefully — not just showing "Error occurred." Users need context. Your app needs to recover.

```
// ERROR TYPES you need to handle:

// 1. Network error — no internet, server down
//    fetch() throws TypeError: Failed to fetch

// 2. HTTP error — server returned 4xx/5xx
//    fetch() does NOT throw — you must check response.ok
//    Axios DOES throw — error.response.status

// 3. Parse error — response body is not valid JSON
//    response.json() throws SyntaxError

// 4. Timeout — request took too long
//    Must implement manually with fetch or use Axios timeout option

// 5. Abort — request was cancelled (component unmounted)
//    AbortError — should be silently ignored

// COMPLETE ERROR HANDLER:
async function fetchWithFullErrorHandling(url) {
  const controller = new AbortController();
  const timeoutId  = setTimeout(() => controller.abort(), 10000); // 10s timeout

  try {
    const response = await fetch(url, { signal: controller.signal });
    clearTimeout(timeoutId);

    if (!response.ok) {
      // Try to get error message from server
      let serverMessage = "";
      try { serverMessage = (await response.json()).message; } catch {}

      if (response.status === 401) throw new Error("Please log in to continue");
      if (response.status === 403) throw new Error("You do not have
permission");
      if (response.status === 404) throw new Error("Resource not found");
      if (response.status >= 500)  throw new Error(serverMessage || "Server
error. Please try again.");
      throw new Error(serverMessage || `Error ${response.status}`);
    }

    return await response.json();
  } catch (err) {
    if (err.name === "AbortError") {
      throw new Error("Request timed out. Please check your connection.");
    }
```

```
    throw err;  // rethrow for component to handle
  }
}
```

```
// USER-FRIENDLY error display in components
function ProductsPage() {
  const [products, setProducts] = useState([]);
  const [loading,  setLoading]  = useState(true);
  const [error,    setError]    = useState(null);

  async function loadProducts() {
    try {
      setLoading(true);
      setError(null);
      const data = await fetchWithFullErrorHandling("/api/products");
      setProducts(data.products);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  if (error) {
    return (
      <div className="error-container">
        <p>⚠️ {error}</p>
        <button onClick={loadProducts}>Try Again</button>
        {/* Give user a way to recover — retry button */}
      </div>
    );
  }

  // ...rest of render
}
```

Topic 7: Optimistic Updates — Instant UI Without Waiting for Server

Optimistic updates mean updating the UI IMMEDIATELY when a user takes an action — before the server confirms it. If the server succeeds, great. If it fails, you roll back. This makes apps feel instantaneous.

Think of liking a post on Instagram. The heart fills instantly when you tap. Instagram does not wait for the server to confirm. If the network request fails, it reverts. That's an optimistic update.

```
// OPTIMISTIC DELETE — item disappears instantly
function TodoList() {
  const [todos, setTodos] = useState(initialTodos);

  async function deleteTodo(id) {
```

```
    // 1. Save the previous state for rollback
    const previousTodos = todos;

    // 2. Optimistically remove from UI immediately
    setTodos(todos => todos.filter(t => t.id !== id));

    try {
      // 3. Tell the server (happens in background)
      await fetch(`/api/todos/${id}`, { method: "DELETE" });
      // Server confirmed — nothing to do, UI already updated
    } catch (err) {
      // 4. Server failed — ROLLBACK to previous state
      setTodos(previousTodos);
      alert("Failed to delete. Please try again.");
    }
  }

  return (
    <ul>
      {todos.map(todo => (
        <li key={todo.id}>
          {todo.text}
          <button onClick={() => deleteTodo(todo.id)}>Delete</button>
        </li>
      ))}
    </ul>
  );
}

// OPTIMISTIC LIKE — toggle like count instantly
function Post({ post }) {
  const [liked,     setLiked]     = useState(post.likedByMe);
  const [likeCount, setLikeCount] = useState(post.likeCount);

  async function handleLike() {
    // Optimistic update
    const wasLiked   = liked;
    const prevCount  = likeCount;
    setLiked(!liked);
    setLikeCount(c => liked ? c - 1 : c + 1);

    try {
      await fetch(`/api/posts/${post.id}/like`, {
        method: liked ? "DELETE" : "POST"
      });
    } catch {
      // Rollback on failure
      setLiked(wasLiked);
      setLikeCount(prevCount);
    }
  }
```

```
  return (
    <div>
      <button onClick={handleLike}>
        {liked ? "❤️" : "🤍"} {likeCount}
      </button>
    </div>
  );
}
```

> ✅ **Remember**
>
> Optimistic update pattern: save previous → update UI → await server → rollback on failure.
>
> Always save previous state BEFORE the optimistic update — you need it for rollback.
>
> This pattern is used by every major app: Instagram likes, Twitter retweets, Gmail star/archive, Notion edits.
>
> React Query and SWR have built-in optimistic update support — they handle the rollback automatically.

Topic 8: Pagination — Complete Implementation

Almost every list-based page needs pagination or infinite scroll. Here are both patterns implemented cleanly.

Classic Pagination (Page Numbers)

```
function ProductsPage() {
  const [products,    setProducts]    = useState([]);
  const [currentPage, setCurrentPage] = useState(1);
  const [totalPages,  setTotalPages]  = useState(1);
  const [loading,     setLoading]     = useState(false);
  const ITEMS_PER_PAGE = 12;

  useEffect(() => {
    const controller = new AbortController();

    async function fetchPage() {
      try {
        setLoading(true);
        const res  = await fetch(
          `/api/products?page=${currentPage}&limit=${ITEMS_PER_PAGE}`,
          { signal: controller.signal }
        );
        const data = await res.json();
        setProducts(data.products);
        setTotalPages(Math.ceil(data.total / ITEMS_PER_PAGE));
      } catch (err) {
        if (err.name !== "AbortError") console.error(err);
      } finally {
```

```
        setLoading(false);
      }
    }

    fetchPage();
    return () => controller.abort();
  }, [currentPage]);  // re-fetch when page changes

  return (
    <div>
      {loading ? <Spinner /> : (
        <div className="grid">
          {products.map(p => <ProductCard key={p.id} product={p} />)}
        </div>
      )}

      {/* Pagination controls */}
      <div className="pagination">
        <button
          onClick={() => setCurrentPage(p => p - 1)}
          disabled={currentPage === 1}
        >
          ← Prev
        </button>

        <span>Page {currentPage} of {totalPages}</span>

        <button
          onClick={() => setCurrentPage(p => p + 1)}
          disabled={currentPage === totalPages}
        >
          Next →
        </button>
      </div>
    </div>
  );
}
```

Infinite Scroll (Load More)

```
function InfiniteProductFeed() {
  const [products, setProducts] = useState([]);
  const [page,     setPage]     = useState(1);
  const [hasMore,  setHasMore]  = useState(true);
  const [loading,  setLoading]  = useState(false);

  // Intersection observer on the sentinel element
  const sentinelRef = useRef(null);
```

```
  useEffect(() => {
    if (!hasMore || loading) return;

    const observer = new IntersectionObserver(([entry]) => {
      if (entry.isIntersecting) setPage(p => p + 1);
    }, { threshold: 0.1 });

    if (sentinelRef.current) observer.observe(sentinelRef.current);
    return () => observer.disconnect();
  }, [hasMore, loading]);

  useEffect(() => {
    async function loadMore() {
      setLoading(true);
      const res  = await fetch(`/api/products?page=${page}&limit=12`);
      const data = await res.json();
      setProducts(prev => [...prev, ...data.products]);  // APPEND new items
      setHasMore(data.hasMore);
      setLoading(false);
    }
    loadMore();
  }, [page]);

  return (
    <div>
      <div className="grid">
        {products.map(p => <ProductCard key={p.id} product={p} />)}
      </div>

      {loading && <Spinner />}
      {/* Sentinel — entering viewport triggers next page load */}
      {hasMore && <div ref={sentinelRef} />}
      {!hasMore && <p>All products loaded.</p>}
    </div>
  );
}
```

Topic 9: File Uploads — Form Data and Progress

File uploads are a common requirement — profile photos, product images, documents. They use FormData (not JSON) and can show upload progress.

```
function FileUploadForm() {
  const [file,     setFile]     = useState(null);
  const [progress, setProgress] = useState(0);
  const [uploading,setUploading]= useState(false);
  const [error,    setError]    = useState(null);
  const [uploaded, setUploaded] = useState(null);
```

```
function handleFileChange(e) {
  const selected = e.target.files[0];
  if (!selected) return;

  // Validate file type and size before upload
  const allowedTypes = ["image/jpeg", "image/png", "image/webp"];
  if (!allowedTypes.includes(selected.type)) {
    setError("Only JPG, PNG, WebP allowed");
    return;
  }
  if (selected.size > 5 * 1024 * 1024) {  // 5MB limit
    setError("File must be under 5MB");
    return;
  }
  setFile(selected);
  setError(null);
}

async function handleUpload() {
  if (!file) return;

  const formData = new FormData();
  formData.append("photo", file);
  formData.append("userId", "123");  // additional fields

  setUploading(true);
  setProgress(0);

  // Use XMLHttpRequest for progress — fetch does not support progress events
  const xhr = new XMLHttpRequest();

  xhr.upload.addEventListener("progress", (e) => {
    if (e.lengthComputable) {
      const pct = Math.round((e.loaded / e.total) * 100);
      setProgress(pct);
    }
  });

  xhr.addEventListener("load", () => {
    const response = JSON.parse(xhr.responseText);
    setUploaded(response.url);
    setUploading(false);
  });

  xhr.addEventListener("error", () => {
    setError("Upload failed. Please try again.");
    setUploading(false);
  });

  xhr.open("POST", "/api/upload");
  xhr.send(formData);
```

```
  }

  return (
    <div>
      <input type="file" accept="image/*" onChange={handleFileChange} />
      {file && <p>Selected: {file.name} ({(file.size/1024).toFixed(1)}KB)</p>}
      {error && <p className="error">{error}</p>}

      {file && !uploading && !uploaded && (
        <button onClick={handleUpload}>Upload</button>
      )}

      {uploading && (
        <div>
          <div className="progress-bar" style={{ width: `${progress}%` }} />
          <p>Uploading... {progress}%</p>
        </div>
      )}

      {uploaded && <img src={uploaded} alt="Uploaded" />}
    </div>
  );
}
```

Topic 10: Polling — Periodic Data Refreshing

Some data needs to stay fresh without the user manually refreshing — live scores, order status, dashboard metrics. **Polling** checks the server at regular intervals.

```
// usePolling.js — reusable polling hook
import { useEffect, useCallback, useRef } from "react";

function usePolling(callback, interval = 5000, enabled = true) {
  const savedCallback = useRef(callback);

  // Keep ref current without re-creating the effect
  useEffect(() => { savedCallback.current = callback; }, [callback]);

  useEffect(() => {
    if (!enabled) return;

    // Run immediately on first mount
    savedCallback.current();

    // Then run on interval
    const id = setInterval(() => savedCallback.current(), interval);
    return () => clearInterval(id);  // cleanup on unmount
  }, [interval, enabled]);
}
```

```
// USAGE — order status polling
function OrderStatus({ orderId }) {
  const [order, setOrder] = useState(null);

  async function fetchStatus() {
    const res  = await fetch(`/api/orders/${orderId}`);
    const data = await res.json();
    setOrder(data);
  }

  // Poll every 5 seconds, but stop when order is delivered
  const isComplete = order?.status === "delivered" || order?.status ===
"cancelled";
  usePolling(fetchStatus, 5000, !isComplete);

  return (
    <div>
      <h2>Order #{orderId}</h2>
      <p>Status: {order?.status || "Loading..."}</p>
      {isComplete && <p>✅ Final — polling stopped</p>}
    </div>
  );
}

// Also consider: pause polling when tab is not visible
useEffect(() => {
  const handleVisibility = () => setIsVisible(!document.hidden);
  document.addEventListener("visibilitychange", handleVisibility);
  return () => document.removeEventListener("visibilitychange",
handleVisibility);
}, []);
```

```
usePolling(fetchStatus, 5000, isVisible && !isComplete);
```

Topic 11: React Query — The Professional Data Fetching Library

TanStack Query (formerly React Query) is the industry standard for server state management in React. It handles caching, background refetching, loading/error states, pagination, infinite scroll, and optimistic updates — all with minimal code. Understanding it shows you are up to date with the ecosystem.

```
npm install @tanstack/react-query
```

```
// main.jsx — wrap app with QueryClientProvider
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
```

```
      staleTime: 1000 * 60 * 5,   // data stays fresh for 5 minutes
      gcTime:    1000 * 60 * 10,   // cache kept for 10 minutes
      retry: 2,                     // retry failed requests 2 times
    },
  },
});

ReactDOM.createRoot(document.getElementById("root")).render(
  <QueryClientProvider client={queryClient}>
    <App />
  </QueryClientProvider>
);
```

```
// USING useQuery — replace ALL your manual fetch + useState + useEffect
import { useQuery, useMutation, useQueryClient } from "@tanstack/react-query";

// Simple fetch function (no async state management needed)
const fetchProducts = async () => {
  const res = await fetch("/api/products");
  if (!res.ok) throw new Error("Failed to fetch products");
  return res.json();
};

function ProductsPage() {
  const { data, isLoading, isError, error, refetch } = useQuery({
    queryKey: ["products"],    // cache key — unique identifier for this data
    queryFn:  fetchProducts,   // your fetch function
  });
  // That is IT. React Query handles:
  // ✓ Loading state (isLoading)
  // ✓ Error state (isError, error)
  // ✓ Caching (same key = cached data reused)
  // ✓ Background refetch (stale-while-revalidate)
  // ✓ Retry on failure
  // ✓ Refetch on window focus

  if (isLoading) return <Spinner />;
  if (isError)   return <p>Error: {error.message}</p>;

  return (
    <div>
      <button onClick={refetch}>Refresh</button>
      {data.products.map(p => <ProductCard key={p.id} product={p} />)}
    </div>
  );
}

// QUERY WITH PARAMS — query key includes dynamic values
function ProductDetail({ productId }) {
  const { data: product } = useQuery({
```

```
    queryKey: ["product", productId],  // cached separately per productId
    queryFn:  () => fetch(`/api/products/${productId}`).then(r => r.json()),
    enabled:  !!productId,             // only run when productId exists
  });
  return <div>{product?.name}</div>;
}
```

```
// useMutation — for POST/PUT/DELETE operations with optimistic updates
function DeleteButton({ productId }) {
  const queryClient = useQueryClient();

  const deleteMutation = useMutation({
    mutationFn: (id) => fetch(`/api/products/${id}`, { method: "DELETE" }),

    // Optimistic update — instant UI change before server confirms
    onMutate: async (deletedId) => {
      await queryClient.cancelQueries({ queryKey: ["products"] });
      const previousProducts = queryClient.getQueryData(["products"]);

      // Optimistically remove from cache
      queryClient.setQueryData(["products"], (old) =>
        old.products.filter(p => p.id !== deletedId)
      );

      return { previousProducts };  // saved for rollback
    },

    // Rollback on error
    onError: (err, variables, context) => {
      queryClient.setQueryData(["products"], context.previousProducts);
    },

    // Refetch after mutation settles
    onSettled: () => {
      queryClient.invalidateQueries({ queryKey: ["products"] });
    },
  });

  return (
    <button
      onClick={() => deleteMutation.mutate(productId)}
      disabled={deleteMutation.isPending}
    >
      {deleteMutation.isPending ? "Deleting..." : "Delete"}
    </button>
  );
}
```

💡 **Tip**

React Query eliminates the need for 80% of useEffect data fetching code.

The queryKey is the cache key. Same key = same cached data. Different key (e.g. including productId) = separate cache entry.

invalidateQueries tells React Query to refetch that data on next access — use after mutations.

React Query also includes the React Query DevTools — a floating panel showing all cached queries.

Alternative: SWR (from Vercel) is similar but lighter. React Query is more feature-complete.

Topic 12: API Service Layer — Organising Your API Calls

In a real project, you do not scatter fetch() calls across components. You build an **API service layer** — a set of functions that encapsulate all server communication. Components call service functions, not fetch directly.

```
// src/services/productService.js — all product-related API calls
import api from "./api";  // your configured axios instance

export const productService = {
  getAll: async (params) => {
    const { data } = await api.get("/products", { params });
    return data;
  },

  getById: async (id) => {
    const { data } = await api.get(`/products/${id}`);
    return data;
  },

  create: async (productData) => {
    const { data } = await api.post("/products", productData);
    return data;
  },

  update: async (id, productData) => {
    const { data } = await api.put(`/products/${id}`, productData);
    return data;
  },

  delete: async (id) => {
    await api.delete(`/products/${id}`);
  },

  uploadImage: async (id, file) => {
    const formData = new FormData();
    formData.append("image", file);
    const { data } = await api.post(`/products/${id}/image`, formData, {
      headers: { "Content-Type": "multipart/form-data" },
    });
    return data;
  },
```

```
};

// src/services/authService.js
export const authService = {
  login: async (email, password) => {
    const { data } = await api.post("/auth/login", { email, password });
    localStorage.setItem("token", data.token);
    return data.user;
  },

  logout: async () => {
    await api.post("/auth/logout");
    localStorage.removeItem("token");
  },

  getProfile: async () => {
    const { data } = await api.get("/auth/me");
    return data;
  },
};

// Component — uses service, knows nothing about HTTP
function ProductsPage() {
  const { data } = useQuery({
    queryKey: ["products"],
    queryFn:  () => productService.getAll(),  // clean, readable
  });
  // If the API URL changes — update productService.js only
  // Component never changes
}
```

Chapter 13: Interview Questions with Full Answers

Q1: Why can't useEffect be async? How do you use async code inside it?

Answer:

useEffect cannot be async because async functions return a Promise. useEffect's return value is supposed to be either a cleanup function or undefined. Returning a Promise from useEffect could cause unexpected behaviour — React would receive a Promise where it expects a function.

The solution is to define an async function INSIDE the useEffect and call it immediately.

Pattern: useEffect(() => { async function load() { ... } load(); return () => cleanup(); }, [deps]);

Never do: useEffect(async () => { ... }, [deps]) — this works but causes a warning and makes cleanup harder.

The inner async function lets you use await naturally, while the outer effect function stays synchronous and can return a proper cleanup function.

Q2: What is a race condition in React? How do you prevent it?

Answer:

A race condition in React occurs when multiple async operations (like API calls triggered by dependency changes) complete in an unexpected order, causing stale data to overwrite newer data.

Example: user navigates from /users/1 to /users/2 quickly. Request 1 is slow. Request 2 is fast. Request 2 completes first (shows User 2). Request 1 completes second and overwrites — showing User 1 on the /users/2 page.

Fix with AbortController: create a controller in useEffect, pass its signal to fetch, and abort in the cleanup function. When dependencies change, the cleanup runs and cancels the previous in-flight request before the new one starts.

Alternative fix: use a cancelled flag — a boolean that is set to true in cleanup. After await, check if cancelled before calling setState.

React Query eliminates race conditions automatically — it tracks query keys and ignores responses from stale requests.

Q3: What is the difference between fetch() and Axios?

Answer:

fetch is browser-native — zero install. Axios is a library (14kb gzipped) that wraps XMLHttpRequest.

Key differences: fetch does not throw on HTTP errors (404, 500) — you must check response.ok manually. Axios automatically throws on 4xx/5xx errors. fetch requires manual JSON parsing (await response.json()). Axios auto-parses JSON into response.data.

Axios supports request/response interceptors — run code before every request (attach token) or after every response (handle 401 globally). fetch has no built-in interceptor — you must wrap it manually.

Axios supports a timeout option. fetch requires a manual Promise.race with setTimeout or AbortController timer.

For most production apps: use Axios. For simple apps or when minimising bundle size: fetch is fine with a small wrapper function.

Q4: What is an optimistic update? How do you implement it?

Answer:

An optimistic update means updating the UI immediately when a user takes an action — before the server confirms the action — then rolling back if the server rejects it.

The pattern: (1) save the current state for potential rollback. (2) Apply the change to local state immediately. (3) Send the request to the server. (4) If success — nothing to do. (5) If failure — restore the saved state and show an error.

This makes UI feel instantaneous. Users do not wait 500ms for a delete or like to visually respond.

Real examples: Instagram likes, Twitter retweets, Gmail archive, Notion drag-and-drop.

React Query's useMutation has built-in onMutate (optimistic), onError (rollback), and onSettled (refetch after) callbacks that handle this pattern cleanly.

Q5: What is Promise.all? When would you use it over sequential awaits?

Answer:

Promise.all takes an array of promises and returns a single promise that resolves when ALL of them resolve, or rejects as soon as any one rejects.

Sequential awaits: const user = await getUser(); const orders = await getOrders(); — these run one after the other. If each takes 500ms, total = 1000ms.

Promise.all: const [user, orders] = await Promise.all([getUser(), getOrders()]); — both run simultaneously. Total = ~500ms (the slower one).

Use Promise.all when the requests are INDEPENDENT — they do not depend on each other's results. Load a user profile: get the user AND their orders AND their preferences all at once.

Do NOT use Promise.all when they are SEQUENTIAL — when you need one result to make the next request. Get user → then use user.id to get orders. These must be sequential.

Also know: Promise.allSettled runs all and gives results for each, regardless of failures. Promise.race resolves with the first to settle.

Q6: How do you handle authentication tokens with Axios?

Answer:

The cleanest approach is an Axios interceptor that automatically attaches the token to every outgoing request.

Create a custom Axios instance with axios.create() and add a request interceptor: api.interceptors.request.use(config => { const token = localStorage.getItem("token"); if (token) config.headers.Authorization = "Bearer " + token; return config; }).

Add a response interceptor for token expiry: api.interceptors.response.use(null, error => { if (error.response?.status === 401) { localStorage.removeItem("token"); window.location.href = "/login"; } return Promise.reject(error); }).

This means every component using the api instance automatically gets the token attached — no manual token handling in components.

For refresh tokens: the 401 interceptor can attempt to refresh the token before redirecting, so long-lived sessions remain seamless.

Q7: What is React Query and why would you use it?

Answer:

React Query (TanStack Query) is a server state management library. It handles fetching, caching, background refetching, error/loading states, and cache invalidation with minimal code.

With plain useEffect: every data-fetching component needs useState for data/loading/error, a useEffect with cleanup, and careful dependency management. This is repeated across the entire app.

With React Query: useQuery({ queryKey, queryFn }) gives you data, isLoading, and isError automatically. The same query key in different components shares cached data — no duplicate requests.

Key benefits: automatic caching (same key = shared cache), background refetch when data goes stale, refetch on window focus, built-in retry logic, devtools, and pagination/infinite scroll support.

It separates server state (data from API) from client state (UI state) — a clean architectural distinction.

Q8: How do you implement retry logic for failed API calls?

Answer:

Simple manual retry: catch the error, increment a retry counter, and re-call the function if retries remain.

For useEffect-based fetching: track retryCount in state. In the catch block: if (retryCount < 3) setRetryCount(r => r + 1) — incrementing this triggers the useEffect to re-run.

Add exponential backoff: wait 1s on first retry, 2s on second, 4s on third — prevents hammering a struggling server.

With React Query: built-in retry option. useQuery({ ..., retry: 3, retryDelay: (attemptIndex) => Math.min(1000 * 2 ** attemptIndex, 30000) }).

Only retry on network errors or 5xx server errors. Do NOT retry on 4xx client errors (400 bad request, 404 not found) — retrying will not fix a bad request or missing resource.

Q9: What is AbortController and when do you use it?

Answer:

AbortController is a browser API that lets you cancel fetch requests (and other async operations) programmatically.

Usage: const controller = new AbortController(); pass controller.signal to fetch options. Call controller.abort() to cancel the request.

In React: create an AbortController in useEffect. Pass its signal to fetch. In the cleanup function, call controller.abort(). This cancels the request when: the component unmounts (memory leak prevention) OR when a dependency changes and a new request is about to start (race condition prevention).

When a request is aborted, fetch throws an AbortError. Always check err.name !== "AbortError" before setting error state — abort errors are intentional, not failures.

Without AbortController: components can try to setState after unmounting ("Can't perform a React state update on an unmounted component") and race conditions can occur.

Q10: How do you implement pagination in React?

Answer:

Track currentPage in state. Include page as a useEffect dependency. When page changes, fetch the new page of data and replace the list.

The API response should include total count (to calculate total pages) or a hasNextPage boolean.

For the UI: prev/next buttons that decrement/increment currentPage, with disabled state at boundaries. Optionally show page number buttons.

Include currentPage in the URL as a query param (useSearchParams) so users can share/bookmark specific pages and the browser back button works correctly.

For infinite scroll: instead of replacing the list, APPEND new items to it. Use IntersectionObserver to detect when the user scrolls to the bottom and trigger the next page load.

React Query has useInfiniteQuery built-in for infinite scroll with proper cursor/offset management.

Q11: What is the API service layer pattern?

Answer:

The API service layer is a set of dedicated functions or modules that encapsulate all HTTP communication. Components call service functions instead of making fetch/axios calls directly.

Benefits: Single responsibility — components do rendering, services do HTTP. If the API changes (URL, authentication, data format) — update the service, not every component. Testable — mock the service in tests instead of mocking fetch. Reusable — multiple components share the same service function.

Structure: one service per resource (productService.js, authService.js, orderService.js). Each exports async functions (getAll, getById, create, update, delete).

Combined with React Query: queryFn: () => productService.getAll() — the component has no HTTP knowledge at all.

Q12: How do you handle file uploads in React?

Answer:

File inputs cannot be controlled in React — always use uncontrolled with useRef.

For uploads: create a FormData object, append the file and any additional fields, and send with fetch or Axios.

Validate before uploading: check file.type for allowed MIME types and file.size against your limit.

For upload progress: use XMLHttpRequest instead of fetch (fetch does not support upload progress events). Listen to xhr.upload.progress events, calculate percentage, update state.

With Axios: axios.post(url, formData, { onUploadProgress: e => setProgress(Math.round((e.loaded * 100) / e.total)) }). Axios supports upload progress out of the box.

Always show a progress indicator for uploads larger than a few KB — users need feedback during the wait.

Common Mistakes in API Calls

Mistake 1: Async useEffect — making the callback itself async

```
// ❌ WRONG — async useEffect causes issues with cleanup
useEffect(async () => {
  const data = await fetch("/api/products").then(r => r.json());
  setProducts(data);
  // Cannot properly return a cleanup function from async function
}, []);

// ✅ CORRECT — define async inside, call it
useEffect(() => {
  async function load() {
    const data = await fetch("/api/products").then(r => r.json());
```

```
    setProducts(data);
  }
  load();
  return () => { /* cleanup */ };
}, []);
```

Mistake 2: Not checking response.ok with fetch()

```
// ❌ WRONG — 404 and 500 do not throw, silently returns undefined data
const response = await fetch("/api/user/999");
const data = await response.json();  // might be { error: "Not Found" }
setUser(data);  // sets user to error object!

// ✅ CORRECT — always check response.ok
const response = await fetch("/api/user/999");
if (!response.ok) throw new Error(`HTTP ${response.status}`);
const data = await response.json();
setUser(data);
```

Mistake 3: Missing AbortController — memory leak and race conditions

```
// ❌ WRONG — no cleanup, race condition possible
useEffect(() => {
  fetch(`/api/users/${userId}`).then(r => r.json()).then(setUser);
}, [userId]);

// ✅ CORRECT — AbortController prevents both issues
useEffect(() => {
  const controller = new AbortController();
  fetch(`/api/users/${userId}`, { signal: controller.signal })
    .then(r => r.json()).then(setUser)
    .catch(err => { if (err.name !== "AbortError") setError(err.message); });
  return () => controller.abort();
}, [userId]);
```

Mistake 4: Sequential awaits when requests are independent

```
// ❌ SLOW — 3 requests run one after another (total ~1500ms)
const user     = await fetch("/api/user");
const products = await fetch("/api/products");
const orders   = await fetch("/api/orders");

// ✅ FAST — 3 requests run in parallel (total ~500ms)
const [user, products, orders] = await Promise.all([
  fetch("/api/user").then(r => r.json()),
  fetch("/api/products").then(r => r.json()),
  fetch("/api/orders").then(r => r.json()),
```

```
]);
```

Quick Reference — Chapter 13 Summary

Concept	What To Remember	Key Code
fetch() + async/await	Check response.ok — it never throws on 4xx/5xx	if (!res.ok) throw new Error(res.status)
Axios	Auto JSON, auto-throws, interceptors built-in	api.interceptors.request.use(config => ...)
useEffect + async	Define async fn inside, call it, return cleanup	const load = async () => {...}; load();
AbortController	Cancel requests on unmount and dep change	controller.abort() in cleanup
Race conditions	AbortController or isCancelled flag in cleanup	if (!isCancelled) setState(data)
Error handling	try/catch + finally. Ignore AbortError	if (err.name !== "AbortError") setError(...)
Optimistic update	Save prev → update UI → await → rollback on fail	const prev = state; setState(optimistic); try { await API } catch { setState(prev) }
Promise.all	Run independent requests in parallel	await Promise.all([req1, req2, req3])
Pagination	currentPage in state + useEffect dep	useEffect with [currentPage] dep
React Query	queryKey, queryFn — handles everything else	useQuery({ queryKey, queryFn })
API service layer	All HTTP calls in one file per resource	productService.getAll(), authService.login()

✅ **Remember**

Zomato analogy — place order = fetch. Loading = pending. Food arrives = success. Cancel = AbortController.

fetch never throws on 404/500 — ALWAYS check response.ok. This is the most common mistake.

useEffect cannot be async — define async inside, call it, return cleanup normally.

AbortController — always in useEffect with deps. Prevents race conditions AND memory leaks.

Optimistic updates — save previous → update UI immediately → rollback on failure.

Promise.all — independent requests in parallel. Sequential only when one depends on the other.

React Query — replace manual fetch + useState + useEffect with useQuery. Cache, retry, refetch built-in.

API service layer — components never import fetch or axios directly. Services own all HTTP.

Interceptors — attach auth token in request interceptor. Handle 401 globally in response interceptor.

End of Chapter 13 • Next: Chapter 14 — Advanced Topics (Error Boundaries, Portals, Suspense)

Chapter 14: Advanced Topics

Error Boundaries, Portals, Suspense, Reconciliation, Forwarding Refs, HOCs — senior-level React

By Syed Shaaz Akhtar | Interview Focused Guide

This chapter covers the advanced concepts that separate senior React developers from mid-level ones. These topics are not used in every component — but they are asked in every senior React interview. Understand them deeply and you will be able to answer confidently, explain clearly, and know when to actually reach for them.

Each topic here solves a specific class of problem that nothing else can. Error Boundaries catch uncaught errors. Portals escape the DOM hierarchy. Suspense manages loading boundaries declaratively. Reconciliation explains why React is fast. Forwarding Refs bridge the gap between parent and DOM. HOCs compose behaviour. Learn them all.

> 💡 **Tip**
>
> 🏗️ THE ANALOGY for this chapter:
>
> Think of an advanced React app as a MULTI-FLOOR BUILDING.
>
> — Error Boundaries = Fire safety doors. If one room catches fire (throws error), the door contains it. Rest of the building is fine.
>
> — Portals = Emergency exits. A door inside room 301 can lead OUTSIDE the building — bypassing all floors. Modals render outside the main app tree.
>
> — Suspense = "Please wait" sign on a door. While the room is being prepared (loading), visitors wait at the sign. Once ready, they walk in.
>
> — Reconciliation = The building's renovation crew. They only replace the parts that changed — not tear down and rebuild from scratch.
>
> — Forwarding Refs = Giving a visitor a direct key to a specific room, bypassing reception.
>
> — HOCs = A room decorator. Takes a plain room, adds furniture, returns a furnished room.

Topic 1: Error Boundaries — Containing JavaScript Errors

When a React component throws an error during rendering, the ENTIRE app crashes and shows a blank white screen. Error Boundaries are special components that catch these errors and display a fallback UI instead — keeping the rest of the app alive.

The Problem Without Error Boundaries

```
// Without Error Boundary — one broken component = white screen for entire app
function BrokenWidget() {
  throw new Error("Something went wrong in this widget!");
  return <div>I never render</div>;
}

// App crashes completely — user sees a blank white screen
// Header, Footer, Sidebar, every other component — all gone
```

```
// All because of ONE broken widget

// With Error Boundary — only the broken widget shows error, rest is fine
<ErrorBoundary fallback={<p>Widget failed to load</p>}>
  <BrokenWidget />
</ErrorBoundary>
// Header still shows ✓  Footer still shows ✓  Other widgets still show ✓
// Only <BrokenWidget /> is replaced with the fallback message
```

Writing an Error Boundary

Error Boundaries MUST be class components. They use two lifecycle methods that have no hook equivalents: getDerivedStateFromError and componentDidCatch. This is the ONE case in modern React where you still need a class component.

```
// ErrorBoundary.jsx — reusable error boundary
import { Component } from "react";

class ErrorBoundary extends Component {
  constructor(props) {
    super(props);
    this.state = {
      hasError: false,
      error:    null,
      errorInfo: null,
    };
  }

  // getDerivedStateFromError — called when a descendant throws
  // Updates state so the next render shows the fallback UI
  // Must be a STATIC method
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }

  // componentDidCatch — called after getDerivedStateFromError
  // Use for logging the error to a service (Sentry, DataDog, etc.)
  componentDidCatch(error, errorInfo) {
    // errorInfo.componentStack = which components caused the error
    console.error("Error caught by boundary:", error, errorInfo);
    // logErrorToService(error, errorInfo.componentStack);
  }

  render() {
    if (this.state.hasError) {
      // Show fallback UI — either a prop or default message
      if (this.props.fallback) {
        return this.props.fallback;
      }
      return (
        <div className="error-boundary-fallback">
```

```
          <h2>Something went wrong.</h2>
          <p>{this.state.error?.message}</p>
          <button onClick={() => this.setState({ hasError: false, error:
null })}>
            Try Again
          </button>
        </div>
      );
    }

    return this.props.children;
  }
}

export default ErrorBoundary;
```

```
// Usage — wrap sections you want to isolate

// 1. Wrap the whole app — catch anything not caught below
function App() {
  return (
    <ErrorBoundary>
      <Router>
        <Routes>...</Routes>
      </Router>
    </ErrorBoundary>
  );
}

// 2. Wrap individual sections — isolate widgets from each other
function Dashboard() {
  return (
    <div className="dashboard">
      <ErrorBoundary fallback={<p>Stats failed to load</p>}>
        <StatsWidget />     {/* if this throws, only this section fails */}
      </ErrorBoundary>

      <ErrorBoundary fallback={<p>Chart failed to load</p>}>
        <RevenueChart />    {/* completely isolated from StatsWidget */}
      </ErrorBoundary>

      <ErrorBoundary fallback={<p>Notifications unavailable</p>}>
        <NotificationsPanel />
      </ErrorBoundary>
    </div>
  );
}

// 3. Per-route boundary — entire page fails gracefully
function AppRouter() {
  return (
```

```
    <Routes>
      <Route path="/dashboard" element={
        <ErrorBoundary fallback={<PageError />}>
          <DashboardPage />
        </ErrorBoundary>
      } />
    </Routes>
  );
}
```

✅ **Remember**

Error Boundaries ONLY catch errors in: rendering, lifecycle methods, constructors of child components.

They do NOT catch: async errors (inside setTimeout, fetch callbacks), event handler errors, errors in the boundary itself.

For async errors — use try/catch inside your async functions and manage error state manually.

For event handler errors — wrap in try/catch, no boundary needed.

In development React shows the error overlay — in production the boundary fallback shows.

Use react-error-boundary package for a hook-friendly version with useErrorBoundary hook.

Topic 2: Portals — Rendering Outside the Component Tree

Normally, a React component renders inside its parent's DOM node. **Portals** let you render a component's output to a DIFFERENT DOM node — anywhere in the document — while keeping it in the React component tree logically.

This solves a real CSS problem: modals, tooltips, and dropdowns need to visually appear above all other content. But if they are trapped inside a parent with overflow: hidden or a low z-index, they get clipped.

```
// THE PROBLEM — modal trapped inside parent with overflow: hidden
<div style={{ overflow: "hidden", position: "relative" }}>
  <Modal />   {/* Modal is visually clipped — it cannot escape this container
*/}
</div>

// WITH PORTAL — modal renders directly in document.body
// It is OUTSIDE the overflow: hidden container in the DOM
// But still INSIDE the React tree — events still bubble up correctly

// Modal.jsx — using createPortal
import { createPortal } from "react-dom";

function Modal({ isOpen, onClose, children }) {
  if (!isOpen) return null;

  // createPortal(whatToRender, whereToRenderIt)
  return createPortal(
    // The JSX to render
```

```
    <div className="modal-overlay" onClick={onClose}>
      <div
        className="modal-content"
        onClick={e => e.stopPropagation()}  // prevent overlay click from
closing
      >
        <button className="modal-close" onClick={onClose}>×</button>
        {children}
      </div>
    </div>,
    // Where to render it — directly in body, no parent CSS restrictions
    document.body
  );
}

// Usage — modal renders in document.body regardless of parent CSS
function ProductCard({ product }) {
  const [showModal, setShowModal] = useState(false);

  return (
    <div style={{ overflow: "hidden" }}>  {/* would normally trap a child modal
*/}
      <button onClick={() => setShowModal(true)}>Quick View</button>

      <Modal isOpen={showModal} onClose={() => setShowModal(false)}>
        <h2>{product.name}</h2>
        <p>{product.description}</p>
        <button>Add to Cart</button>
      </Modal>
      {/* Modal renders in document.body — completely escapes overflow: hidden
*/}
    </div>
  );
}
```

📝 **Note**

The portal renders OUTSIDE the parent in the DOM but remains INSIDE the React component tree.

This means: context still works (portal component can access parent's context), events still bubble through the React tree (not the DOM tree), and React's key/ref system still works.

Common use cases: modals, tooltips, dropdown menus, toast notifications, right-click context menus.

You can create a dedicated in index.html and portal to that instead of document.body.

Topic 3: Suspense — Declarative Loading States

Suspense lets you show a loading fallback while waiting for something to be ready — a lazy-loaded component, async data, or any resource. Instead of managing loading states manually with useState, you declare them at the boundary level.

Suspense with Lazy Loading (React 16+)

```
import { lazy, Suspense } from "react";

// Lazy load a heavy component — only loads when rendered
const HeavyDashboard = lazy(() => import("./pages/HeavyDashboard"));
const AdminPanel     = lazy(() => import("./pages/AdminPanel"));

function App() {
  return (
    // Suspense shows the fallback while lazy components are loading
    <Suspense fallback={<div className="page-loading">Loading page...</div>}>
      <Routes>
        <Route path="/dashboard" element={<HeavyDashboard />} />
        <Route path="/admin"     element={<AdminPanel />} />
      </Routes>
    </Suspense>
  );
}

// Nested Suspense — different fallbacks for different sections
function DashboardPage() {
  const StatsPanel   = lazy(() => import("./StatsPanel"));
  const ChartSection = lazy(() => import("./ChartSection"));

  return (
    <div>
      <h1>Dashboard</h1>

      <Suspense fallback={<StatsSkeleton />}>
        <StatsPanel />       {/* shows StatsSkeleton while loading */}
      </Suspense>

      <Suspense fallback={<ChartSkeleton />}>
        <ChartSection />     {/* shows ChartSkeleton while loading */}
      </Suspense>
    </div>
  );
}
```

Suspense with Data Fetching (React 18 — use() hook)

```
// React 18 introduced the use() hook for data fetching with Suspense
// use() suspends the component while the promise is pending

import { use, Suspense } from "react";
```

```
// Fetch function returns a promise
function fetchUser(id) {
  return fetch(`/api/users/${id}`).then(r => r.json());
}

// UserCard "suspends" while the promise is pending
// Suspense boundary above shows fallback during suspension
function UserCard({ userId }) {
  const user = use(fetchUser(userId));  // suspends if promise is pending
  return <div><h2>{user.name}</h2><p>{user.email}</p></div>;
}

// Parent wraps in Suspense and ErrorBoundary
function UserPage({ userId }) {
  return (
    <ErrorBoundary fallback={<p>Failed to load user</p>}>
      <Suspense fallback={<UserSkeleton />}>
        <UserCard userId={userId} />
        {/* While UserCard is suspended — UserSkeleton shows */}
        {/* When data arrives — UserCard renders */}
        {/* If fetch throws — ErrorBoundary catches it */}
      </Suspense>
    </ErrorBoundary>
  );
}

// The Suspense + ErrorBoundary pair is THE pattern for data + loading + errors
// <ErrorBoundary>
//   <Suspense fallback={<Loading />}>
//     <DataComponent />
//   </Suspense>
// </ErrorBoundary>
```

💡 **Tip**

Suspense and ErrorBoundary are designed to work TOGETHER.

Suspense = "show this while loading". ErrorBoundary = "show this if it fails".

Always pair them: <ErrorBoundary><Suspense fallback={<Skeleton />}><Component /></Suspense></ErrorBoundary>

The fallback prop of Suspense should be a skeleton or spinner — not a full page.

Multiple Suspense boundaries = granular loading UI. Each section gets its own skeleton.

Topic 4: Reconciliation — How React Updates the DOM

Reconciliation is the algorithm React uses to compare the previous and new virtual DOM trees and figure out the minimal set of DOM changes to make. Understanding this tells you WHY keys matter, WHY component position matters, and WHY React is fast.

The Virtual DOM

```
// The Virtual DOM is a lightweight JavaScript object representation of the real DOM
// React keeps two copies:
//   1. The current tree (what is on screen)
//   2. The new tree (result of the latest render)

// When state changes:
// 1. React creates a NEW virtual DOM tree
// 2. DIFF — compare new tree with previous tree (reconciliation)
// 3. PATCH — apply ONLY the differences to the real DOM

// WHY this matters for performance:
// DOM operations are SLOW. JavaScript is FAST.
// Instead of rebuilding the DOM from scratch on every state change,
// React calculates the minimal changes in JavaScript, then applies them.

// Simplified example:
// Previous VDOM:  <ul><li>Apple</li><li>Banana</li></ul>
// New VDOM:       <ul><li>Apple</li><li>Banana</li><li>Cherry</li></ul>
// Diff:           Add <li>Cherry</li> to the end
// DOM Operation:  ONE insertBefore — nothing else touched
```

The Diffing Algorithm — Rules React Follows

```
// RULE 1: Different element types = destroy and rebuild from scratch
// Changing a div to a span destroys all children and remounts

// Previous: <div><Counter /></div>
// New:      <span><Counter /></span>
// Result:   Counter is UNMOUNTED and REMOUNTED — its state is LOST


// RULE 2: Same element type = update only changed attributes
// className, style, event handlers updated in-place

// Previous: <div className="old" onClick={fn1}>text</div>
// New:      <div className="new" onClick={fn2}>text</div>
// Result:   className and onClick updated — no remount, no state loss


// RULE 3: Children — use keys to identify which item is which
// WITHOUT keys — React compares by position
// Adding to the START without keys:

// Previous: [A, B, C]
// New:      [X, A, B, C]  — X inserted at position 0
// Without keys: React sees position 0 changed (was A, now X)
//               Updates position 0, 1, 2, 3 — mutates all 4
//               A, B, C lose their state (they were "updated", not "moved")
```

```
// WITH keys — React uses keys to track identity
// <li key="A">A</li>  <li key="B">B</li>  <li key="C">C</li>
// New: <li key="X">X</li>  <li key="A">A</li>  <li key="B">B</li>
// React sees: X is NEW, A/B/C are the SAME — just moved
// Inserts X, leaves A/B/C in place — their state is PRESERVED

// This is WHY keys are so important — they are identity cards for React
```

```
// PRACTICAL IMPLICATIONS

// 1. Component type changes reset state
// If condition changes which component type renders — state is lost
function Form({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn
        ? <UserForm />     // mounting UserForm resets its internal state
        : <GuestForm />    // React sees different type at position 0
      }
    </div>
  );
}

// 2. Same position = preserved state (surprising behaviour)
function App() {
  const [isPlayerA, setIsPlayerA] = useState(true);
  return (
    <div>
      {isPlayerA
        ? <Counter person="PlayerA" />  // same type, same position
        : <Counter person="PlayerB" />  // React REUSES the Counter — state
persists!
      }
      {/* Counter's internal count is NOT reset when switching A→B */}
      {/* Fix: add a key to force remount on person change */}
      {isPlayerA
        ? <Counter key="PlayerA" person="PlayerA" />  // key forces separate
instances
        : <Counter key="PlayerB" person="PlayerB" />  // different key = remount
= state reset
      }
    </div>
  );
}

// 3. Using key to force component reset
// Sometimes you WANT to reset state — use a key change
function ProfilePage({ userId }) {
  return <Profile key={userId} />;
  // When userId changes → key changes → Profile UNMOUNTS and REMOUNTS
  // All state inside Profile is reset — correct behaviour for profile switching
```

```
}
```

> ✅ **Remember**
>
> Virtual DOM = React's lightweight JS copy of the real DOM. Diffs happen in JS (fast), then real DOM is patched (minimal operations).
>
> Different element type at same position = full remount. Same type = update in-place.
>
> Keys are identity cards for list items. Without keys, React uses position. With keys, React tracks by identity.
>
> Key change forces remount — use this intentionally to reset component state.
>
> Component position in JSX matters. Moving a component in JSX can cause unexpected state resets.

Topic 5: forwardRef — Accessing Child Component's DOM Node

Normally, **ref** only works on DOM elements (div, input, button). If you try to put a ref on a custom component, React ignores it. **forwardRef** is the API that lets a component pass a ref down to a specific DOM node inside it.

```
// PROBLEM: ref on a custom component does not work
function TextInput({ placeholder }) {
  return <input placeholder={placeholder} />;
}

function Parent() {
  const inputRef = useRef(null);

  // ❌ DOES NOT WORK — ref is NOT forwarded to the inner <input>
  return <TextInput ref={inputRef} placeholder="Search..." />;
  // inputRef.current = null — useless
}

// FIX: forwardRef — explicitly forward the ref to the DOM node
import { forwardRef } from "react";

const TextInput = forwardRef(function TextInput({ placeholder, className }, ref)
{
  // ref is the SECOND argument — pass it to the DOM element you want to expose
  return (
    <input
      ref={ref}           // forward to the actual DOM input element
      placeholder={placeholder}
      className={`text-input ${className}`}
    />
  );
});

// Now ref works correctly
function SearchPage() {
```

```
  const inputRef = useRef(null);

  function focusInput() {
    inputRef.current.focus();   // direct DOM method on the inner <input>
  }

  function selectAll() {
    inputRef.current.select();
  }

  return (
    <div>
      <TextInput ref={inputRef} placeholder="Search products..." />
      <button onClick={focusInput}>Focus</button>
      <button onClick={selectAll}>Select All</button>
    </div>
  );
}
```

useImperativeHandle — Controlling What the Ref Exposes

```
// useImperativeHandle lets you customise what ref.current contains
// Instead of exposing the full DOM node, expose only specific methods
import { forwardRef, useRef, useImperativeHandle } from "react";

const VideoPlayer = forwardRef(function VideoPlayer({ src }, ref) {
  const videoRef = useRef(null);

  // Define what the parent can do through the ref
  useImperativeHandle(ref, () => ({
    play:  () => videoRef.current.play(),
    pause: () => videoRef.current.pause(),
    seek:  (time) => { videoRef.current.currentTime = time; },
    // Do NOT expose the full video element — only these specific methods
    // Parent cannot do videoRef.current.innerHTML = "" (good for safety)
  }));

  return <video ref={videoRef} src={src} />;
});

// Parent uses the controlled interface
function VideoControls() {
  const playerRef = useRef(null);

  return (
    <div>
      <VideoPlayer ref={playerRef} src="/video.mp4" />
      <button onClick={() => playerRef.current.play()}>▶️ Play</button>
      <button onClick={() => playerRef.current.pause()}>⏸️ Pause</button>
      <button onClick={() => playerRef.current.seek(30)}>⏩ Skip 30s</button>
```

```
    </div>
  );
}
```

> 📝 **Note**
>
> Use forwardRef when: building a design system with reusable inputs, building third-party components that others ref, integrating with animation libraries that need DOM access.
>
> Use useImperativeHandle with forwardRef to expose only a clean API rather than the raw DOM node.
>
> React 19 simplifies this — ref can be passed as a regular prop without forwardRef. But forwardRef is still needed in React 17/18 which most projects use.

Topic 6: Higher-Order Components (HOC)

A **Higher-Order Component** is a function that takes a component and returns a NEW component with additional capabilities. HOCs are a pattern for reusing component logic — they are like component decorators.

HOCs were the main code-reuse pattern before hooks. Today, custom hooks are usually preferred. But HOCs still appear in many codebases and some libraries use them heavily (React Redux's connect, withRouter).

```
// HOC PATTERN: takes a component, returns an enhanced component
// Naming convention: "with" prefix — withAuth, withLoading, withTheme

// withLoading HOC — adds a loading spinner to any component
function withLoading(WrappedComponent) {
  // Return a new component that wraps WrappedComponent
  return function WithLoading({ isLoading, ...props }) {
    if (isLoading) return <Spinner />;
    return <WrappedComponent {...props} />;
    // ...props passes all other props to the wrapped component
  };
}

// Usage:
const ProductListWithLoading = withLoading(ProductList);

function App() {
  const { products, loading } = useFetch("/api/products");
  return (
    <ProductListWithLoading
      isLoading={loading}  // consumed by HOC
      products={products}  // passed through to ProductList
    />
  );
}

// withAuth HOC — protects a component from unauthenticated access
```

```
function withAuth(WrappedComponent) {
  return function WithAuth(props) {
    const { isLoggedIn, isLoading } = useAuth();

    if (isLoading)    return <Spinner />;
    if (!isLoggedIn)  return <Navigate to="/login" replace />;

    return <WrappedComponent {...props} />;
  };
}

// Protect any page component
const ProtectedDashboard = withAuth(DashboardPage);
const ProtectedProfile   = withAuth(ProfilePage);

// withErrorBoundary HOC — wraps any component in an error boundary
function withErrorBoundary(WrappedComponent, fallback) {
  return function WithErrorBoundary(props) {
    return (
      <ErrorBoundary fallback={fallback}>
        <WrappedComponent {...props} />
      </ErrorBoundary>
    );
  };
}

// HOC composition — apply multiple HOCs
const SafeDashboard = withErrorBoundary(withAuth(withLoading(DashboardPage)),
<p>Error</p>);
```

Aspect	HOC Pattern	Custom Hook Pattern
Code Location	Wraps component — wrapper function	Inside component — hook function
Returns	A new COMPONENT	State values and functions
Composition	Wrap: withA(withB(withC(Component)))	Call: const a = useA(); const b = useB();
Prop passing	Injects props — can cause "prop drilling"	Returns values directly — no prop injection
Readability	Wrapper nesting can get complex	Linear calls, easier to trace
Modern usage	Legacy code, library APIs (connect, Router)	Preferred for new code — simpler
Debug names	Anonymous in DevTools without displayName	Clear in DevTools as useHookName

Topic 7: React.Children — Manipulating Children Props

Components receive children via props. React provides utilities to work with children programmatically — counting them, mapping over them, cloning them with extra props.

```
// React.Children utilities

// 1. React.Children.count — count children (handles arrays, null, undefined)
function TabList({ children }) {
  const count = React.Children.count(children);
  return (
    <div>
      <p>{count} tabs</p>
      <div className="tab-list">{children}</div>
    </div>
  );
}

// 2. React.Children.map — map over children (handles single vs array)
function RadioGroup({ children, onChange }) {
  return (
    <fieldset>
      {React.Children.map(children, (child, index) =>
        // React.cloneElement — clone a child with added/overridden props
        React.cloneElement(child, {
          name: "radio-group",
          onChange: onChange,
          key: index,
          // child's own props are preserved, these are added/overridden
        })
      )}
    </fieldset>
  );
}

// Usage — RadioGroup injects name and onChange into each Radio child
<RadioGroup onChange={handleChange}>
  <Radio value="a">Option A</Radio>
  <Radio value="b">Option B</Radio>
  <Radio value="c">Option C</Radio>
</RadioGroup>

// 3. React.Children.toArray — converts children to a real array
//    Useful when you need array methods like filter or sort
function TabBar({ children }) {
  const tabs = React.Children.toArray(children);
  const visibleTabs = tabs.filter(tab => !tab.props.hidden);
  return <div>{visibleTabs}</div>;
}
```

Topic 8: Compound Components Pattern

The **Compound Components** pattern creates a set of components that work together as a single cohesive unit — sharing implicit state through Context. Think of how HTML's <select> and <option> work together. You do not need to pass any shared state as props.

```
// COMPOUND COMPONENTS — Tab system example
// The Tabs, TabList, Tab, TabPanel components work together seamlessly

import { createContext, useContext, useState } from "react";

const TabsContext = createContext(null);

// Root component — manages shared state
function Tabs({ children, defaultTab = 0 }) {
  const [activeTab, setActiveTab] = useState(defaultTab);
  return (
    <TabsContext.Provider value={{ activeTab, setActiveTab }}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

// Tab trigger component — reads and writes active tab
function Tab({ index, children }) {
  const { activeTab, setActiveTab } = useContext(TabsContext);
  return (
    <button
      className={activeTab === index ? "tab active" : "tab"}
      onClick={() => setActiveTab(index)}
    >
      {children}
    </button>
  );
}

// Panel component — shows when its index matches activeTab
function TabPanel({ index, children }) {
  const { activeTab } = useContext(TabsContext);
  if (activeTab !== index) return null;
  return <div className="tab-panel">{children}</div>;
}

// Attach sub-components to parent
Tabs.Tab      = Tab;
Tabs.TabPanel = TabPanel;

// USAGE — extremely clean, no prop drilling
function ProductPage() {
  return (
    <Tabs defaultTab={0}>
```

```
      <div className="tab-list">
        <Tabs.Tab index={0}>Details</Tabs.Tab>
        <Tabs.Tab index={1}>Reviews</Tabs.Tab>
        <Tabs.Tab index={2}>Shipping</Tabs.Tab>
      </div>

      <Tabs.TabPanel index={0}><ProductDetails /></Tabs.TabPanel>
      <Tabs.TabPanel index={1}><ProductReviews /></Tabs.TabPanel>
      <Tabs.TabPanel index={2}><ShippingInfo /></Tabs.TabPanel>
    </Tabs>
  );
}
// Tab state is shared via Context — no prop drilling between sibling tabs and
panels
```

Topic 9: Render Props Pattern

The **Render Props** pattern is a technique for sharing logic by passing a function as a prop. The function tells the component what to render. This was a predecessor to custom hooks and is still seen in many libraries.

```
// Render Props — the "render" prop is a function that returns JSX

// MouseTracker — shares mouse position via render prop
function MouseTracker({ render }) {
  const [position, setPosition] = useState({ x: 0, y: 0 });

  function handleMouseMove(e) {
    setPosition({ x: e.clientX, y: e.clientY });
  }

  return (
    <div onMouseMove={handleMouseMove} style={{ height: "100vh" }}>
      {render(position)}  {/* call the render prop with the shared state */}
    </div>
  );
}

// Usage — the parent decides what to RENDER with the shared state
function App() {
  return (
    <MouseTracker
      render={({ x, y }) => (  // parent controls rendering
        <p>Mouse at: {x}, {y}</p>
      )}
    />
  );
}
```

```
// children-as-a-function — same pattern using children prop
function DataLoader({ url, children }) {
  const { data, loading, error } = useFetch(url);
  return children({ data, loading, error });
}

<DataLoader url="/api/products">
  {({ data, loading, error }) => {
    if (loading) return <Spinner />;
    if (error)   return <p>Error</p>;
    return data.map(p => <ProductCard key={p.id} product={p} />);
  }}
</DataLoader>

// Custom hooks replaced render props for most use cases
// But render props still appear in: React Router, Formik, React Query
// Understanding them helps you work with these libraries
```

Topic 10: StrictMode — Catching Bugs in Development

React.StrictMode is a development tool that helps you find potential problems in your application. It intentionally invokes certain functions TWICE to surface impure behaviour, and warns about deprecated APIs.

```
// main.jsx — wrap your app in StrictMode
import { StrictMode } from "react";

ReactDOM.createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);

// WHAT StrictMode does:

// 1. Renders components TWICE in development
//    If your component has side effects in the render phase
//    (like modifying globals), double render catches it
//    First render = discarded. Second render = used.
//    If they give different results — you have a bug

// 2. Runs effects TWICE (React 18)
//    mounts → runs effect → UNMOUNTS (cleanup) → REMOUNTS → runs effect again
//    This verifies that your effect cleanup is correct
//    If your feature breaks on second mount — your cleanup is incomplete

// 3. Warns about deprecated APIs:
//    findDOMNode, legacy context API, componentWillMount, etc.
```

```
// StrictMode is development-only. In production builds it has ZERO overhead.
// The double-render and double-effect is invisible to users.

// COMMON STRICTMODE TRAP:
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("mounted");
    // In StrictMode this logs "mounted" TWICE in React 18
    // Each mount is followed by unmount + remount
    return () => console.log("unmounted");
  }, []);
  // This is EXPECTED — it is StrictMode testing your cleanup
  // Not a bug in your code
}
```

Topic 11: React Fiber — The Reconciliation Engine

React Fiber is the internal reimplementation of React's reconciliation algorithm introduced in React 16. While you never interact with Fiber directly, understanding it explains how React achieves concurrent features like Suspense, useTransition, and time-slicing.

```
// Before Fiber (React 15 and earlier):
// Reconciliation was SYNCHRONOUS and RECURSIVE
// Once React started updating a component tree — it could not stop
// A complex update could block the browser's main thread for 100ms+
// During that time: no user input, no animations, UI feels frozen

// Fiber changes this with an incremental rendering model:

// Key Fiber concepts:

// 1. UNITS OF WORK — each Fiber node represents one component
//    React processes one fiber at a time instead of the whole tree at once

// 2. INTERRUPTIBLE — between processing each fiber, React can check:
//    "Is there a more urgent update? (user typed, clicked)"
//    If yes — pause current work, handle urgent update, resume
//    This is Concurrent Mode

// 3. PRIORITY LEVELS — different updates have different priorities:
//    Urgent: user typing, clicking → process immediately
//    Normal: data fetching results → process when browser is free
//    Low:    analytics, non-visible updates → process last

// This is what enables:
//  • useTransition — mark state updates as low priority
//  • useDeferredValue — defer processing of a value
```

```
//  • Suspense — pause rendering while data loads
//  • Concurrent rendering — multiple renders in progress simultaneously

// You do not write Fiber code. But knowing it explains:
// WHY useTransition keeps UI responsive during heavy updates
// WHY effects run twice in StrictMode (Fiber tests mount/unmount cycles)
// HOW React handles Suspense (Fiber throws special promises)
```

🎯 **Interview Tip**

Interviewers rarely ask you to explain Fiber in detail. But mentioning it earns senior credibility.

"React Fiber is the internal engine that makes concurrent rendering possible. It breaks rendering work into units that can be paused and resumed, which is what enables useTransition to keep the UI responsive during heavy updates and Suspense to pause rendering while awaiting data."

That one sentence shows you understand the architecture, not just the API surface.

Topic 12: Complete Pattern — Building with All Advanced Topics Together

Here is how all these advanced topics work together in a production dashboard. This is the complete, professional architecture.

```
// main.jsx — StrictMode + QueryClient + Providers
import { StrictMode } from "react";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

const queryClient = new QueryClient();

ReactDOM.createRoot(document.getElementById("root")).render(
  <StrictMode>
    <QueryClientProvider client={queryClient}>
      <AuthProvider>
        <ThemeProvider>
          <BrowserRouter>
            {/* Top-level Error Boundary — catches anything not caught below */}
            <ErrorBoundary fallback={<AppCrashPage />}>
              <App />
            </ErrorBoundary>
          </BrowserRouter>
        </ThemeProvider>
      </AuthProvider>
    </QueryClientProvider>
  </StrictMode>
);

// App.jsx — lazy routes with Suspense + per-route ErrorBoundary
const DashboardPage = lazy(() => import("./pages/DashboardPage"));
const AdminPage     = lazy(() => import("./pages/AdminPage"));

function App() {
```

```
  return (
    <Routes>
      <Route
        path="/dashboard"
        element={
          <ErrorBoundary fallback={<PageError />}>
            <Suspense fallback={<PageSkeleton />}>
              <DashboardPage />
            </Suspense>
          </ErrorBoundary>
        }
      />
    </Routes>
  );
}

// DashboardPage.jsx — compound components + forwardRef + portals
function DashboardPage() {
  const searchRef = useRef(null);
  const [showModal, setShowModal] = useState(false);

  return (
    <div>
      {/* forwardRef component — can focus programmatically */}
      <SearchInput ref={searchRef} placeholder="Search..." />
      <button onClick={() => searchRef.current.focus()}>Search</button>

      {/* Compound components — clean tab interface */}
      <Tabs defaultTab={0}>
        <Tabs.Tab index={0}>Overview</Tabs.Tab>
        <Tabs.Tab index={1}>Analytics</Tabs.Tab>
        <Tabs.Tab index={2}>Settings</Tabs.Tab>

        {/* Per-widget Error Boundaries — isolated failure */}
        <Tabs.TabPanel index={0}>
          <ErrorBoundary fallback={<p>Overview failed</p>}>
            <OverviewPanel />
          </ErrorBoundary>
        </Tabs.TabPanel>

        <Tabs.TabPanel index={1}>
          <ErrorBoundary fallback={<p>Chart error</p>}>
            <Suspense fallback={<ChartSkeleton />}>
              <AnalyticsChart />  {/* lazy loaded heavy chart library */}
            </Suspense>
          </ErrorBoundary>
        </Tabs.TabPanel>
      </Tabs>

      {/* Portal — modal renders in document.body, escapes any CSS */}
      <Modal isOpen={showModal} onClose={() => setShowModal(false)}>
```

```
        <UserDetailsForm />
      </Modal>
    </div>
  );
}
```

Chapter 14: Interview Questions with Full Answers

Q1: What is an Error Boundary? How do you implement one?

Answer:

An Error Boundary is a React component that catches JavaScript errors in its child component tree, logs them, and displays a fallback UI instead of crashing the whole application.

Implementation: Error Boundaries must be class components with two special lifecycle methods. getDerivedStateFromError(error) is called during the render phase when a descendant throws — it updates state to trigger fallback rendering. componentDidCatch(error, errorInfo) is called after the error is caught — use it to log to an error monitoring service.

In the render method: check this.state.hasError. If true, render the fallback. Otherwise render this.props.children.

What Error Boundaries do NOT catch: errors inside event handlers, async code (setTimeout, Promises), errors in the boundary itself, server-side rendering errors.

Use react-error-boundary package for a simpler API with useErrorBoundary hook for functional component usage.

Q2: What is a React Portal? When would you use it?

Answer:

A Portal renders a component's JSX output to a different DOM node than its parent — while keeping it inside the React component tree logically.

Created with: createPortal(children, domNode) from react-dom.

Why needed: CSS properties like overflow: hidden, z-index stacking contexts, and position: fixed can trap child elements visually. If a modal is inside a scrollable container with overflow hidden, it gets clipped. Rendering to document.body via a portal escapes all parent CSS constraints.

Despite rendering elsewhere in the DOM, portals remain in the React tree: events bubble correctly through React's tree, Context still works (portal child can access parent context), key/ref system is unaffected.

Use cases: modals, tooltips, dropdown menus, toast notifications, any overlay that must appear above all other content.

Q3: What is React Reconciliation? How does the diffing algorithm work?

Answer:

Reconciliation is the algorithm React uses to compare the new virtual DOM tree with the previous one and determine the minimal set of real DOM changes needed.

When state or props change, React creates a new virtual DOM tree. It diffs this against the current tree. The diff finds additions, deletions, and changes. Only the changed parts are applied to the real DOM.

Key diffing rules: if elements have DIFFERENT types at the same position, React tears down the old tree and builds fresh. If they have the SAME type, React updates attributes in place without remounting.

For lists: React compares by position without keys. With keys, React tracks elements by identity. Keys prevent unnecessary remounts when items are reordered, and preserve state correctly.

React's diffing is O(n) — linear time — achieved by the two assumptions: different element types produce different trees, and keys uniquely identify siblings.

Q4: What is forwardRef? When do you need it?

Answer:

forwardRef is a React API that allows a component to receive a ref prop and pass it down to a DOM element inside its render.

By default, React does not pass ref through to custom components — ref on a custom component is silently ignored. forwardRef explicitly adds support for it.

Usage: wrap the component with React.forwardRef(function Component(props, ref) { ... }). The ref becomes the second argument. Pass it to the specific DOM element you want to expose.

Common use cases: design system input components (FancyInput, SearchBar) where the parent needs to call .focus() or .value, animation libraries that manipulate DOM nodes directly, third-party integrations that expect a DOM ref.

Combine with useImperativeHandle to expose only a specific controlled API through the ref, rather than the full DOM node.

Q5: What is the difference between an HOC and a custom hook?

Answer:

Both HOCs and custom hooks are patterns for reusing logic. The key difference is what they return and where they live.

A Higher-Order Component is a function that takes a component and returns a NEW component. It adds behaviour by wrapping. The logic lives outside the component, injected as props. Example: const ProtectedComponent = withAuth(MyComponent).

A custom hook is a function called inside a component that returns state values and functions. The logic lives inside the component via the hook call. Example: const { user } = useAuth() called inside MyComponent.

HOCs can cause "wrapper hell" when stacking multiple HOCs. Custom hooks compose linearly — call useA(), useB(), useC() in sequence. Custom hooks are more readable in component code and show clearly in React DevTools.

HOCs are still used by libraries (React Redux connect, withRouter). For new code, prefer custom hooks.

Q6: What does React.StrictMode do?

Answer:

React.StrictMode is a development-only tool that helps identify potential problems. It has zero effect on production builds.

Key behaviours in development: renders components twice — this catches side effects in the render phase. If double-rendering changes behaviour, something in the render phase is impure. In React 18, also mounts and unmounts effects twice — this ensures your effect cleanup is correct.

It also logs warnings for: deprecated lifecycle methods, legacy string ref usage, legacy context API, findDOMNode usage.

The double-rendering explains why you see console.log inside render running twice in development — this is intentional and expected. Do not remove StrictMode to hide this.

StrictMode is the reason the React docs say "useEffect fires twice in development" — it is testing your cleanup. If your feature breaks on second mount, fix the cleanup.

Q7: What is Suspense? What can it suspend on?

Answer:

Suspense is a React component that displays a fallback UI while its children are "suspended" — waiting for something to be ready.

It suspends on: (1) Lazy-loaded components — React.lazy(() => import()) suspends until the JavaScript chunk is downloaded. This is the most common and well-supported use case. (2) Data fetching in React 18 — when using the use() hook with a Promise, Suspense catches the suspension and shows the fallback.

The key mental model: when a component needs something that is not ready yet, it throws a special Promise. Suspense catches this throw, shows the fallback, and re-renders when the Promise resolves.

Suspense should always be paired with ErrorBoundary: Suspense handles the loading state. ErrorBoundary handles the error state. Together they declaratively manage async UI: loading → ready → error.

In React 18, Suspense also works with streaming server-side rendering — the server can send the shell immediately and stream content sections as they complete.

Q8: What is the Compound Components pattern?

Answer:

Compound Components is a pattern where a set of components work together as a unit, sharing implicit state through React Context. The user composes them declaratively without managing the shared state themselves.

Classic HTML example: <select> and <option> are compound components. Option does not receive "isSelected" as a prop — select manages that state internally and option reads it.

React implementation: the parent component holds state in Context. Child sub-components read from that Context. The API design attaches sub-components as properties of the parent — Tabs.Tab, Tabs.Panel.

Benefits: clean usage API (no prop drilling between siblings), flexible composition, internal state hidden from users of the component.

Used in: Material UI (Accordion, AccordionSummary, AccordionDetails), Reach UI, Headless UI, and any well-designed component library.

Q9: What is React Fiber?

Answer:

React Fiber is the internal reimplementation of React's reconciliation engine introduced in React 16. It replaced the original synchronous stack-based reconciler.

The core problem Fiber solves: the old reconciler could not pause work mid-way. Once it started updating a component tree, it ran to completion blocking the browser for potentially hundreds of milliseconds.

Fiber breaks reconciliation into units of work (one fiber = one component). Between units, React checks if there is more urgent work (user typing, clicking). If yes, it pauses and handles the urgent work first.

This enables Concurrent Mode features: useTransition (low-priority state updates), useDeferredValue (deferred value processing), Suspense (pause rendering for async data), streaming SSR.

You never write Fiber directly. But understanding it explains WHY concurrent features work and why StrictMode double-invokes effects.

Q10: Why does a component sometimes lose state when its position in the tree changes?

Answer:

React preserves component state when the component stays at the same position in the tree with the same type. It destroys and resets state when either of those conditions changes.

When you conditionally render A or B at the same position and both are the same component type, React REUSES the component instance — preserving its state. This surprises developers who expect the state to reset.

Fix: add a key prop to explicitly identity them as different instances. Different keys at the same position force React to unmount the old and mount new, resetting all state.

Example: switching between Player A and Player B profiles should reset the score. Give each <Profile key={playerName} /> — different key = new instance = fresh state.

This is also the intentional use of key to reset forms: change the key and all form state resets, instead of manually clearing each field.

Q11: What is the Render Props pattern? How does it compare to custom hooks?

Answer:

Render Props is a pattern where a component accepts a function prop that it calls with some data, and that function returns JSX. The component provides the logic; the parent controls the rendering.

Example: <MouseTracker render={({ x, y }) => Mouse: {x}, {y} } />

The children-as-a-function variant uses children instead: <DataLoader url="/api/..."><({ data }) => <List items={data} />}</DataLoader>

Custom hooks largely replace render props for most use cases — they are simpler and avoid JSX nesting. const { x, y } = useMousePosition() inside your component achieves the same thing with no wrapper.

But render props still appear in libraries like React Query (for certain rendering patterns), and understanding them helps you work with legacy codebases and certain advanced patterns.

Q12: How do Error Boundaries and Suspense work together?

Answer:

Error Boundaries and Suspense are designed as complements. Suspense handles the loading state; Error Boundaries handle the error state.

When a component with data fetching suspends, Suspense shows the fallback. When the data arrives, the component renders. If the fetch throws an error instead, Suspense does not catch it — the nearest Error Boundary does.

The canonical pattern pairs them: <ErrorBoundary fallback={<ErrorUI />}><Suspense fallback={<Skeleton />}><DataComponent /></Suspense></ErrorBoundary>.

This gives you three states declaratively: Suspense fallback while loading, normal rendering when data arrives, ErrorBoundary fallback if it fails. No manual loading/error state needed in the child component.

This pattern aligns with the react-query philosophy of separating success rendering from loading/error handling at the boundary level.

Common Mistakes in Advanced React

Mistake 1: Using Error Boundaries for async errors

```
// ❌ WRONG — Error Boundary does NOT catch this
function Component() {
  useEffect(() => {
    fetchData()  // async — error happens outside render phase
      .catch(err => { throw err; });  // Error Boundary will NOT catch this
  }, []);
}

// ✅ CORRECT — handle async errors in state
function Component() {
  const [error, setError] = useState(null);
  useEffect(() => {
    fetchData().catch(err => setError(err.message));
  }, []);
  if (error) return <p>Error: {error}</p>;
}
```

Mistake 2: Forgetting that same component type preserves state

```
// ❌ UNEXPECTED — Counter state does NOT reset on person change
function Game({ isPlayerA }) {
  return isPlayerA ? <Counter /> : <Counter />;  // same type, same position
  // React reuses the component — score is preserved across player switch!
}

// ✅ CORRECT — use key to force separate instances
```

```
function Game({ isPlayerA }) {
  return isPlayerA
    ? <Counter key="A" />  // key A → separate instance
    : <Counter key="B" />; // key B → separate instance, fresh state
}
```

Mistake 3: Overusing HOCs when a custom hook is simpler

```
// ❌ COMPLEX — wrapper hell
export default withAuth(withLoading(withErrorBoundary(withTheme(MyComponent))));

// ✅ SIMPLER — custom hooks compose linearly
function MyComponent() {
  const { user }   = useAuth();
  const { theme }  = useTheme();
  // Loading and error handled inline or with ErrorBoundary/Suspense
  return <div className={`theme-${theme}`}>Hello {user.name}</div>;
}
```

Mistake 4: Using Portals when CSS alone would solve it

```
// ❌ OVER-ENGINEERED for a simple tooltip
function Tooltip({ text }) {
  return createPortal(<div className="tooltip">{text}</div>, document.body);
  // Most tooltips do not need portals
}

// ✅ CSS FIXED POSITION works for most tooltips
function Tooltip({ text }) {
  return <div className="tooltip" style={{ position: "fixed" }}>{text}</div>;
  // Only use Portal when fixed positioning is insufficient (z-index issues,
etc.)
}
```

Quick Reference — Chapter 14 Summary

Concept	What It Solves	Key API
Error Boundary	Contain render errors — show fallback, keep app alive	getDerivedStateFromError + componentDidCatch
Portal	Render outside parent DOM — escape CSS constraints	createPortal(jsx, domNode)
Suspense	Declarative loading state for	<Suspense fallback={<Skeleton />}>

	lazy + async	
Reconciliation	Efficient DOM updates — virtual DOM diffing	Keys = identity cards for list items
forwardRef	Pass ref to inner DOM element of custom component	forwardRef((props, ref) =>)
useImperativeHandle	Control what ref.current exposes to parent	useImperativeHandle(ref, () => ({ play, pause }))
HOC	Wrap component to add behaviour (legacy pattern)	function withX(Component) { return function Enhanced... }
Compound Components	Group of components sharing implicit Context state	Parent holds state, children consume via Context
Render Props	Share logic via function prop (pre-hooks pattern)	render={({ data }) => <JSX />}
StrictMode	Catch impure renders and incomplete cleanup in dev	<StrictMode> in main.jsx
React Fiber	Interruptible reconciliation enabling concurrent features	Internal engine — explains useTransition, Suspense

✅ **Remember**

Building analogy: Error Boundary=fire door, Portal=emergency exit, Suspense=please wait sign.
Error Boundary: class component only. Catches render errors. NOT async errors.
Portal: renders elsewhere in DOM, stays in React tree. Events + Context still work.
Suspense: pairs with ErrorBoundary. Loading=Suspense, Error=ErrorBoundary.
Reconciliation: virtual DOM diffing. Same type=update. Different type=remount.
Keys: identity cards. Different key=force remount (use to reset form/score).
forwardRef: expose inner DOM via ref. Add useImperativeHandle for controlled API.
HOC vs hooks: HOC wraps and returns component. Hook returns values. Prefer hooks for new code.
Compound Components: parent holds Context state, children consume it. No prop drilling.
StrictMode: double render and double effect in dev. Expected. Tests cleanup correctness.
Fiber: makes concurrency possible. Explains useTransition, Suspense, StrictMode double-effects.

End of Chapter 14 • Next: Chapter 15 — React 18 Features & Final Review

Chapter 15: React 18 Features & Final Review

Concurrent Mode, Automatic Batching, Transitions, New Hooks — and the complete guide summary

By Syed Shaaz Akhtar | Interview Focused Guide

React 18 was the biggest update since React 16 (which brought Hooks). It introduced Concurrent Mode as the default, automatic batching, new hooks, and streaming SSR. Interviewers at top companies specifically ask "what do you know about React 18?" — this chapter gives you everything you need to answer confidently.

This chapter also serves as the final review of the entire guide — connecting all 15 chapters into a complete picture of professional React development.

> 💡 **Tip**
>
> 🚀 THE ANALOGY for React 18:
>
> Imagine a RESTAURANT KITCHEN upgrading from one chef to a SMART KITCHEN.
>
> — Old React: one chef, one order at a time — finishes fully before starting next.
>
> — React 18 Concurrent: smart kitchen — can PAUSE cooking a slow dish, serve a quick order, then resume. Everything feels faster.
>
> — Automatic Batching: instead of washing one dish after each meal, the kitchen collects all dirty dishes and washes them together — one re-render for multiple state changes.
>
> — useTransition: marking the slow dish as "low priority" — keep serving fast snacks (urgent UI) while the biryani (heavy update) cooks in the background.
>
> — Suspense (improved): the "Please wait" sign now works with ALL async operations — not just lazy loading.
>
> — Streaming SSR: the kitchen sends appetisers immediately while the main course is still being prepared.

Topic 1: How to Upgrade to React 18

React 18 requires ONE change to your entry point: switching from ReactDOM.render() to createRoot(). Everything else in your app continues to work — React 18 is fully backward compatible.

```
// REACT 17 — old API
import ReactDOM from "react-dom";

ReactDOM.render(
  <App />,
  document.getElementById("root")
);

// REACT 18 — new API (createRoot)
```

```
import { createRoot } from "react-dom/client";

const root = createRoot(document.getElementById("root"));
root.render(<App />);

// That single change unlocks ALL React 18 features:
// ✓ Automatic batching
// ✓ Concurrent rendering
// ✓ useTransition, useDeferredValue
// ✓ Improved Suspense
// ✓ Streaming SSR

// With StrictMode (recommended):
import { StrictMode }  from "react";
import { createRoot }  from "react-dom/client";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

✅ **Remember**

React 18 is backward compatible — your existing components work without any changes.
The ONLY required change is createRoot() instead of ReactDOM.render().
Without createRoot(), React 18 runs in "legacy mode" — you lose all new features.
Vite's React template already uses createRoot() by default.

Topic 2: Automatic Batching — Fewer Re-renders, Free Performance

Batching means grouping multiple state updates into a single re-render. React 17 batched updates inside React event handlers, but NOT inside async code (setTimeout, Promises, native event listeners). React 18 batches ALL state updates automatically — everywhere.

```
// REACT 17 BEHAVIOUR

// Inside React event handler — BATCHED (1 re-render)
function handleClick() {
  setCount(c => c + 1);   // ┐
  setName("Shaaz");        //  ├─ batched → 1 re-render
  setIsOpen(true);         // ┘
}

// Inside async / Promise / setTimeout — NOT batched in React 17 (3 re-renders)
async function handleSubmit() {
  const data = await fetch("/api/submit");
  setLoading(false);       // re-render 1
  setData(data);           // re-render 2
```

```
  setSuccess(true);        // re-render 3  ← 3 separate renders in React 17
}

// REACT 18 BEHAVIOUR — AUTOMATIC BATCHING EVERYWHERE

// Same async code — now BATCHED (1 re-render)
async function handleSubmit() {
  const data = await fetch("/api/submit");
  setLoading(false);       // ┐
  setData(data);           //  ├─ all batched → 1 re-render  ✓
  setSuccess(true);        // ┘
}

// setTimeout — also batched in React 18
setTimeout(() => {
  setCount(c => c + 1);   // ┐
  setFlag(true);           //  ├─ 1 re-render in React 18  ✓
}, 1000);                  // ┘

// Native event listeners — also batched in React 18
document.addEventListener("click", () => {
  setA(1);                 // ┐
  setB(2);                 //  ├─ 1 re-render in React 18  ✓
});                        // ┘

// OPT OUT — if you specifically need separate renders (rare)
import { flushSync } from "react-dom";

flushSync(() => setCount(c => c + 1));  // forces immediate re-render
flushSync(() => setName("Shaaz"));       // another immediate re-render
// Use flushSync only when you need to read updated DOM immediately after
setState
```

> 📝 **Note**
>
> Automatic batching is a FREE performance improvement. Your code runs the same — React just re-renders fewer times.
>
> flushSync is an escape hatch for rare cases where you need to read a DOM measurement immediately after a state update.
>
> React 18 batching never batches across different user interaction events — each click/keypress gets its own flush.

Topic 3: Concurrent Mode — React's New Default

Concurrent Mode is not a feature you turn on — it IS React 18's default rendering model when you use createRoot(). It means React can now work on multiple renders simultaneously, pause and resume work, and prioritise urgent updates over background ones.

```
// BEFORE Concurrent Mode (React 17 and earlier)
// Rendering was SYNCHRONOUS — once started, it must finish
```

```
// Problem: heavy state update → UI freezes for the entire computation

// User types in a search box → triggers filtering of 10,000 items
// React starts the filter render... cannot stop...
// Browser input is frozen for 200ms while filter completes
// User sees stutter/lag every keystroke

// WITH Concurrent Mode (React 18)
// Rendering is INTERRUPTIBLE
// React can pause the filter render when a MORE URGENT update arrives
// (like the next keystroke)

// The user's input ALWAYS gets processed immediately
// The filtered results update in the background when React has time
// Zero stutter. Instant input response.

// WHAT Concurrent Mode enables:
// 1. useTransition     — mark state updates as "non-urgent"
// 2. useDeferredValue  — defer a value's processing to background
// 3. Suspense (data)   — pause renders while awaiting data
// 4. Streaming SSR     — stream HTML progressively from server

// KEY MENTAL MODEL:
// Concurrent Mode = React has TWO priorities
// URGENT: user input, typing, clicking    → process immediately
// NON-URGENT: filtering, heavy renders    → process when free
// useTransition and useDeferredValue let you mark which is which
```

Topic 4: useTransition — Non-Urgent State Updates

useTransition lets you mark a state update as a "transition" — non-urgent. React will render urgent updates first and process the transition when the browser is free. This keeps the UI responsive during heavy operations.

```
import { useState, useTransition } from "react";

function SearchPage() {
  const [query,   setQuery]   = useState("");
  const [results, setResults] = useState(ALL_ITEMS);  // 10,000 items
  const [isPending, startTransition] = useTransition();

  function handleChange(e) {
    const value = e.target.value;

    // URGENT — update the input instantly
    // User must see their typing without any delay
    setQuery(value);

    // NON-URGENT — filtering 10,000 items can wait
```

```
    // React processes this only when the browser is free
    startTransition(() => {
      const filtered = ALL_ITEMS.filter(item =>
        item.toLowerCase().includes(value.toLowerCase())
      );
      setResults(filtered);
    });
  }

  return (
    <div>
      <input
        value={query}
        onChange={handleChange}
        placeholder="Search 10,000 items..."
      />

      {/* isPending = true while transition is processing */}
      {isPending
        ? <p style={{ opacity: 0.5 }}>Updating results...</p>
        : <p>{results.length} results</p>
      }

      <ul>
        {results.map(item => <li key={item}>{item}</li>)}
      </ul>
    </div>
  );
}

// WITHOUT useTransition: typing lags because filter blocks the input
// WITH useTransition: typing is always instant, results update in background
```

```
// REAL WORLD USE CASES for useTransition

// 1. Tab switching — keep current tab visible while new tab loads
function TabContainer() {
  const [activeTab,   setActiveTab]   = useState("home");
  const [isPending,   startTransition] = useTransition();

  function switchTab(tab) {
    startTransition(() => {
      setActiveTab(tab);  // switching tab is non-urgent
    });
  }

  return (
    <div>
      <nav>
        {["home", "products", "about"].map(tab => (
```

```
          <button
            key={tab}
            onClick={() => switchTab(tab)}
            style={{ opacity: isPending ? 0.7 : 1 }}
          >
            {tab}
          </button>
        ))}
      </nav>
      {/* Previous tab stays visible until new tab is ready */}
      <TabContent tab={activeTab} />
    </div>
  );
}

// 2. Page navigation — show previous page while next page loads
function App() {
  const [page, setPage]                = useState("home");
  const [isPending, startTransition]   = useTransition();

  function navigate(newPage) {
    startTransition(() => setPage(newPage));
  }

  return (
    <div style={{ opacity: isPending ? 0.6 : 1 }}>
      <NavBar onNavigate={navigate} />
      <PageContent page={page} />
    </div>
  );
}
```

Topic 5: useDeferredValue — Deferring a Value

useDeferredValue is similar to useTransition but for VALUES rather than state setters. It gives you a "deferred" copy of a value that updates in the background — ideal when you receive a value from a parent or a prop and need to defer its expensive processing.

```
import { useState, useDeferredValue, memo } from "react";

function SearchPage() {
  const [query, setQuery] = useState("");

  // deferredQuery lags BEHIND query intentionally
  // query updates immediately on every keystroke (urgent)
  // deferredQuery updates only when the browser is free (non-urgent)
  const deferredQuery = useDeferredValue(query);

  return (
```

```
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
      />

      {/* Show a "stale" indicator while deferred value is catching up */}
      <div style={{ opacity: deferredQuery !== query ? 0.5 : 1 }}>
        {/* This component re-renders with the DEFERRED query */}
        {/* It does not block the input field from being responsive */}
        <SearchResults query={deferredQuery} />
      </div>
    </div>
  );
}

// SearchResults is expensive — wrap in memo so it only re-renders
// when deferredQuery actually changes (not on every input keystroke)
const SearchResults = memo(function SearchResults({ query }) {
  // Simulate expensive computation
  const results = ALL_ITEMS.filter(item =>
    item.toLowerCase().includes(query.toLowerCase())
  );
  return <ul>{results.map(r => <li key={r}>{r}</li>)}</ul>;
});

// KEY: memo + useDeferredValue work together
// Without memo: SearchResults re-renders on every query change anyway
// With memo: SearchResults only re-renders when deferredQuery changes
```

Feature	useTransition	useDeferredValue
What you control	The state UPDATE that happens	The VALUE that is expensive to process
How it works	Wrap setState in startTransition()	Wrap the value with useDeferredValue()
isPending flag	YES — built in	NO — compute manually (deferred !== current)
Best for	When YOU own the state setter	When you RECEIVE a value as prop or from parent
Mental model	"This update is low priority"	"Use stale value until React is free"

Topic 6: useId — Stable Unique IDs for Accessibility

useId generates a unique, stable ID that is consistent between server and client renders. Use it to link form labels with inputs, or ARIA attributes — anywhere you need a unique ID that must match between renders.

```
import { useId } from "react";

// PROBLEM without useId — using Math.random() breaks hydration
function InputField({ label }) {
  const id = `field-${Math.random()}`;  // ❌ Different on server vs client
  // React shows a hydration mismatch warning
  return (
    <div>
      <label htmlFor={id}>{label}</label>
      <input id={id} />
    </div>
  );
}

// SOLUTION — useId generates a stable, unique ID
function InputField({ label, type = "text" }) {
  const id = useId();  // ":r0:", ":r1:", etc. — stable across renders

  return (
    <div>
      <label htmlFor={id}>{label}</label>
      <input id={id} type={type} />
    </div>
  );
}

// Multiple IDs from one useId call — use prefix pattern
function FormField({ label }) {
  const id = useId();
  // prefix-based IDs from a single hook call
  const inputId   = `${id}-input`;
  const helpTextId = `${id}-help`;

  return (
    <div>
      <label htmlFor={inputId}>{label}</label>
      <input id={inputId} aria-describedby={helpTextId} />
      <p id={helpTextId}>Enter your full name as on ID card</p>
    </div>
  );
}

// Every instance of InputField gets a different ID automatically
<InputField label="First Name" />  // id = ":r0:"
<InputField label="Last Name" />   // id = ":r1:"
<InputField label="Email" />       // id = ":r2:"
```

Topic 7: useSyncExternalStore — Subscribing to External Stores

useSyncExternalStore is designed for library authors and advanced use cases. It provides a safe, concurrent-mode-compatible way to subscribe to external data sources — browser APIs (localStorage, window.innerWidth, online status) or third-party state stores outside React.

```
import { useSyncExternalStore } from "react";

// useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?)
// subscribe: a function that adds a listener and returns cleanup
// getSnapshot: returns the current value synchronously

// Example 1: Online status (better than useEffect approach)
function useOnlineStatus() {
  return useSyncExternalStore(
    // subscribe — called to attach listener
    (callback) => {
      window.addEventListener("online",  callback);
      window.addEventListener("offline", callback);
      return () => {
        window.removeEventListener("online",  callback);
        window.removeEventListener("offline", callback);
      };
    },
    // getSnapshot — returns current value synchronously
    () => navigator.onLine,
    // getServerSnapshot — for SSR (server always reports online)
    () => true
  );
}

// Example 2: Window dimensions
function useWindowWidth() {
  return useSyncExternalStore(
    (callback) => {
      window.addEventListener("resize", callback);
      return () => window.removeEventListener("resize", callback);
    },
    () => window.innerWidth,
    () => 1200  // SSR default
  );
}

// Usage
function StatusBar() {
  const isOnline = useOnlineStatus();
  const width    = useWindowWidth();

  return (
```

```
    <div>
      <span>{isOnline ? "🟢 Online" : "🔴 Offline"}</span>
      <span>Width: {width}px</span>
    </div>
  );
}
```

> 📝 **Note**
>
> useSyncExternalStore is primarily for LIBRARY AUTHORS building state management tools or browser API wrappers.
>
> For most app developers: custom hooks with useEffect + useState work fine for browser API subscriptions.
>
> The key advantage: useSyncExternalStore is guaranteed to work correctly in Concurrent Mode — no tearing (where different components see different snapshots of the same external data during a single render).
>
> If you are interviewing at a company that builds React libraries or infrastructure — know this hook deeply.

Topic 8: useInsertionEffect — CSS-in-JS Libraries

useInsertionEffect fires synchronously BEFORE any DOM mutations. It exists specifically for CSS-in-JS libraries (styled-components, Emotion) to inject styles before the browser paints — preventing flash of unstyled content. App developers almost never use this directly.

```
// useInsertionEffect fires in this order:
// useInsertionEffect → DOM mutations → useLayoutEffect → useEffect

// For CSS-in-JS library internals:
function useCSS(rule) {
  useInsertionEffect(() => {
    // Inject <style> tag before browser paints
    if (!isInserted.has(rule)) {
      isInserted.add(rule);
      document.head.appendChild(createStyleSheet(rule));
    }
  });
  return rule;
}

// Effect firing order summary (React 18):
// Render phase:     Component function runs, returns JSX
// Commit phase 1:   useInsertionEffect  (before DOM mutations)
// Commit phase 2:   DOM updated         (mutations applied)
// Commit phase 3:   useLayoutEffect     (synchronous, before paint)
// Commit phase 4:   Browser paints      (user sees update)
// Commit phase 5:   useEffect           (asynchronous, after paint)
```

Topic 9: Improved Suspense — Siblings Don't Need to Wait

React 18 improved Suspense significantly. In React 17, if one sibling suspended, ALL siblings had to wait for it to resolve before ANY of them could show. React 18 fixes this — each suspended component shows its fallback independently.

```
// REACT 17 SUSPENSE — siblings all wait for slowest
// If UserProfile suspends for 3 seconds:
// BOTH UserProfile AND Timeline are hidden for 3 seconds
<Suspense fallback={<Skeleton />}>
  <UserProfile />     {/* suspends for 3s */}
  <Timeline />        {/* ready instantly, but hidden waiting for UserProfile
*/}
</Suspense>

// REACT 18 SUSPENSE — each suspends independently
// UserProfile and Timeline each have their own boundary
<Suspense fallback={<ProfileSkeleton />}>
  <UserProfile />     {/* shows skeleton for 3s, then renders */}
</Suspense>

<Suspense fallback={<TimelineSkeleton />}>
  <Timeline />        {/* shows skeleton briefly, renders quickly */}
</Suspense>
// Users see Timeline immediately — they do not wait for UserProfile

// SELECTIVE HYDRATION — React 18 SSR feature
// On the server: HTML streams in progressively
// React prioritises hydrating the component the user INTERACTS with first
// User clicks a button before full hydration?
// React hydrates that component first — click is handled correctly

// SuspenseList — controls the ORDER siblings reveal (experimental)
<SuspenseList revealOrder="forwards" tail="collapsed">
  <Suspense fallback={<Skeleton />}><ComponentA /></Suspense>
  <Suspense fallback={<Skeleton />}><ComponentB /></Suspense>
  <Suspense fallback={<Skeleton />}><ComponentC /></Suspense>
</SuspenseList>
// revealOrder="forwards" — A appears first, then B, then C (even if C loads
first)
// tail="collapsed" — only show one skeleton at a time
```

Topic 10: Streaming Server-Side Rendering

React 18's Streaming SSR lets servers send HTML to the browser in chunks — streaming the shell immediately and filling in slower sections as they complete. The user sees content faster and the page becomes interactive sooner.

```
// Traditional SSR (React 17):
// 1. Server fetches ALL data                  (e.g. 800ms)
```

```
// 2. Renders entire HTML string              (e.g. 100ms)
// 3. Sends complete HTML to browser          (user sees something after 900ms)
// 4. Browser downloads React JS bundle       (e.g. 300ms)
// 5. React hydrates entire page at once      (user can interact after 1200ms)

// React 18 Streaming SSR:
// 1. Server renders SHELL instantly → streams to browser  (user sees layout in
50ms)
// 2. Slow data fetches run in background
// 3. As each section completes → streams HTML chunk to browser
// 4. Browser hydrates each section AS IT ARRIVES (selective hydration)
// 5. User can interact with ready sections BEFORE whole page hydrates

// Implementation with Next.js App Router (React 18):
// app/dashboard/page.jsx
import { Suspense } from "react";

export default function DashboardPage() {
  return (
    <main>
      {/* Shell renders immediately */}
      <h1>Dashboard</h1>
      <Nav />

      {/* Suspense boundary — server streams this when ready */}
      <Suspense fallback={<StatsSkeleton />}>
        <Stats />       {/* awaits fast DB query */}
      </Suspense>

      {/* This streams even if Stats is still loading */}
      <Suspense fallback={<ChartSkeleton />}>
        <RevenueChart />  {/* awaits slow analytics query */}
      </Suspense>
    </main>
  );
}

// Stats data comes in at 200ms → streams its HTML → shown
// RevenueChart comes in at 800ms → streams when ready
// User sees the Stats section 600ms EARLIER than without streaming
```

💡 Tip

Streaming SSR is mainly relevant if you use Next.js 13+ App Router or Remix.

For interviews at companies using Next.js — mention Streaming SSR and Selective Hydration. These are React 18's biggest SSR improvements.

Core insight: the browser does not wait for the slowest data fetch anymore. The fastest sections arrive first.

Topic 11: React 18 — Complete Feature Summary Table

Feature	What It Does	How to Use
createRoot()	Enables all React 18 features	Replace ReactDOM.render() with createRoot().render()
Automatic Batching	Groups ALL state updates into one re-render	FREE — happens automatically with createRoot
Concurrent Mode	Interruptible rendering — urgent work first	FREE — enabled by createRoot
useTransition	Mark setState as non-urgent (low priority)	const [pending, startTransition] = useTransition()
useDeferredValue	Defer a value's expensive processing	const deferred = useDeferredValue(value)
useId	Stable unique IDs for accessibility	const id = useId()
useSyncExternalStore	Subscribe to external stores safely	useSyncExternalStore(subscribe, getSnapshot)
useInsertionEffect	Inject styles before DOM mutations (CSS-in-JS)	Same API as useEffect — fires earliest
Improved Suspense	Siblings suspend independently (no more waiting)	Use multiple <Suspense> boundaries
Streaming SSR	Stream HTML in chunks — faster TTFB and TTI	Use Suspense boundaries in server components
flushSync	Force synchronous render (escape batching)	flushSync(() => setState(...))

Topic 12: The Complete React 18 Interview Questions

Q1: What are the key changes in React 18?

Answer:

React 18 introduced several major improvements centred around Concurrent Mode becoming the default.

createRoot() replaces ReactDOM.render() — this single change unlocks all new features. Without it, React 18 runs in legacy mode.

Automatic batching now applies to ALL state updates — including inside Promises, setTimeout, and native event listeners. Previously only React event handlers were batched.

Concurrent Mode is now the default rendering model. React can pause, resume, and prioritise rendering work.

New hooks: useTransition (mark updates as non-urgent), useDeferredValue (defer a value), useId (stable unique IDs), useSyncExternalStore (external store subscription), useInsertionEffect (CSS-in-JS).

Improved Suspense — sibling components no longer wait for each other. Streaming SSR with selective hydration — server streams HTML progressively, React hydrates interacted components first.

Q2: What is the difference between useTransition and useDeferredValue?

Answer:

Both mark work as low-priority for Concurrent Mode, but they apply at different points.

useTransition wraps a setState call — you control which STATE UPDATE is non-urgent. Returns [isPending, startTransition]. Example: startTransition(() => setResults(filter(data))).

useDeferredValue wraps a VALUE — you mark a received value as something whose expensive derived rendering should be deferred. Example: const deferredQuery = useDeferredValue(query). No isPending returned — detect staleness by comparing deferred !== current.

Rule: if you own the setState — use useTransition. If you receive a value from outside (prop, parent state) — use useDeferredValue.

Both should be combined with React.memo on the expensive child component — otherwise the component re-renders on every value change anyway, defeating the purpose.

Q3: What is Automatic Batching in React 18?

Answer:

Batching means grouping multiple state updates into a single re-render. React 17 already batched updates inside React event handlers, but not in async code like Promises and setTimeout.

React 18 extends batching to ALL state updates everywhere — inside async functions, setTimeout, native event listeners, and any other context.

Practical impact: after an await, if you set three state values, React previously triggered three re-renders. In React 18, they are batched into one.

This is a free performance improvement — your code does not change. React just re-renders less often.

Escape hatch: flushSync() forces an immediate synchronous re-render for rare cases where you need to read an updated DOM measurement right after setState.

Q4: What is Concurrent Mode?

Answer:

Concurrent Mode is React 18's new rendering model that makes rendering interruptible. In the old model, once React started rendering a component tree it had to finish — blocking the browser for the entire duration.

In Concurrent Mode, React breaks rendering into units of work. Between each unit, it checks if there is more urgent work — like a user typing or clicking. If so, it pauses the current work, handles the urgent update, then resumes.

This means UI interactions are always responsive, even if React is in the middle of a heavy render.

Concurrent Mode is enabled by using createRoot(). It powers useTransition, useDeferredValue, improved Suspense, and streaming SSR.

The React team calls the effect "time-slicing" — React slices rendering work across multiple frames instead of blocking for one long frame.

Q5: What is useId and why was it introduced?

Answer:

useId generates a stable unique ID that is the same between server and client renders.

Before useId, developers used Math.random() or incrementing counters to generate IDs for form labels and ARIA attributes. These broke server-side rendering — the server would generate a different ID than the client, causing hydration mismatch warnings.

useId solves this by using a deterministic algorithm based on the component's position in the React tree — the same position generates the same ID on both server and client.

Primary use case: linking form <label> to <input> with htmlFor/id pairs, and ARIA attributes like aria-describedby and aria-labelledby.

Do not use useId as a database key or list key — it is for accessibility IDs only.

🎯 FINAL REVIEW — The Complete 15-Chapter Guide Summary

You have covered every topic a React interviewer can throw at you. Here is the complete picture — all 15 chapters in one final reference.

Chapter	Topic	Core Concept to Remember
1	Introduction to React	Component tree, JSX, Virtual DOM, SPA. React = UI library not framework.
2	Props & Prop Drilling	Props = read-only. Prop drilling = passing through middle components. Fix with Context/Redux.
3	Lifting State Up	Shared state lives in lowest common ancestor. Sibling communication via parent.
4	React Hooks	useState, useEffect, useRef, useReducer, useContext. Rules: top-level only, functions/hooks only.
5	Component Lifecycle	Mount → Update → Unmount. useEffect covers all 3. Always return cleanup.
6	Events & Forms	Controlled components = React owns value. e.preventDefault() for forms.
7	Conditional Rendering &	&&, ternary, early return. Keys = identity cards — never use

	Lists	index.
8	Context API	createContext → Provider → useContext. 3 steps. Logic shared. State shared too (unlike hooks).
9	React Router	BrowserRouter, Routes, Route. useNavigate, useParams, useSearchParams. Outlet for nested.
10	Redux Toolkit	Bank analogy. createSlice → configureStore → Provider. dispatch → action → reducer → store → UI.
11	Performance Optimization	Measure first. React.memo, useMemo, useCallback trinity. Lazy load, virtualize, debounce.
12	Custom Hooks	Logic shared, state NOT shared. useFetch, useLocalStorage, useForm, useDebounce.
13	API Calls & Async	fetch never throws on 4xx/5xx. AbortController prevents race conditions. Optimistic updates.
14	Advanced Topics	Error Boundary=fire door, Portal=escape CSS, Reconciliation=virtual DOM diff, forwardRef, HOC.
15	React 18	createRoot unlocks all. Automatic batching free. useTransition=non-urgent. Concurrent Mode default.

The 20 Most Asked React Interview Questions — Quick Answers

```
1.  What is React?
    JS library for building UIs using reusable components. Uses virtual DOM for
efficient updates.

2.  Virtual DOM vs Real DOM?
    VDOM = JS object. React diffs VDOMs, applies minimal changes to real DOM.
Faster than direct DOM manipulation.

3.  What are props vs state?
    Props = read-only data from parent. State = mutable data owned by the
component.

4.  What are React Hooks?
    Functions (use*) for state and lifecycle in function components. useState,
useEffect, etc.

5.  What is useEffect?
    Runs side effects after render. Covers mount/update/unmount. Return cleanup
function.

6.  What is useState?
    Declares reactive state. Returns [value, setter]. Setter triggers re-render.
```

```
7.  What is Context API?
    Share data without prop drilling. createContext → Provider → useContext. 3
steps.

8.  What is lifting state up?
    Move shared state to lowest common ancestor. Pass down via props, pass
callbacks for updates.

9.  What is Redux Toolkit?
    Predictable global state. createSlice + configureStore + Provider. dispatch
→ action → reducer.

10. What is the key prop?
    Identity for list items. Tells React which item is which across renders.
Never use array index.

11. What is React.memo?
    Memoize component — skip re-render if props unchanged (shallow compare).

12. What is useMemo vs useCallback?
    useMemo = cache VALUE. useCallback = cache FUNCTION. Both need React.memo to
be useful.

13. What is code splitting?
    React.lazy + Suspense. Load components on demand. Reduces initial bundle
size.

14. What is a custom hook?
    Function starting with "use" that calls React hooks. Shares logic not state.

15. What is prop drilling?
    Passing props through many levels just to reach a deep child. Fix with
Context or Redux.

16. What is an Error Boundary?
    Class component that catches render errors in children. Shows fallback
instead of white screen.

17. What is a Portal?
    createPortal renders JSX to a different DOM node. Escapes parent CSS
constraints. Used for modals.

18. What is reconciliation?
    React's diff algorithm. Compares old and new VDOM. Applies minimal DOM
changes.

19. What is useTransition (React 18)?
    Mark setState as non-urgent. React processes urgent updates first. Keeps UI
responsive.

20. What is Concurrent Mode?
```

```
    React 18's default. Interruptible rendering. Urgent work processed first.
Enabled by createRoot().
```

Final Tips for React Interviews

💡 **Tip**

💡 INTERVIEW SUCCESS FORMULA:

1. ANALOGY FIRST — Always lead with a real-world analogy before code.
   "Redux is like a bank — state in a vault, you submit transaction slips"
   Analogies show deep understanding. Code alone shows you read documentation.

2. KNOW THE "WHY" — Not just what a feature does, but WHY it exists.
   "Error Boundaries exist because one uncaught render error crashes the entire app"
   "AbortController exists to prevent race conditions and memory leaks in useEffect"

3. MENTION TRADEOFFS — Senior developers know when NOT to use something.
   "React.memo helps when props rarely change, but adds overhead for cheap components"
   "Redux is overkill for simple auth state — Context is enough there"

4. CONNECT TOPICS — Show you understand how they relate.
   "useCallback + React.memo work together — one without the other is usually pointless"
   "Error Boundary + Suspense are designed as a pair — loading and error states respectively"

5. USE REAL EXAMPLES — Name real products and patterns.
   "Optimistic updates are how Instagram handles likes"
   "Virtual lists are how WhatsApp renders thousands of messages"
   "Code splitting is why Gmail loads fast on first visit"

✅ **Remember**

React 18 magic word: createRoot() — say it first. It unlocks everything.
Automatic Batching: FREE — all state updates batched everywhere. No code change needed.
useTransition: you own the setter → startTransition(() => setState(...))
useDeferredValue: you receive a value → const d = useDeferredValue(val)
useId: stable IDs for SSR. For labels and ARIA only — not list keys.
Concurrent Mode: rendering is now interruptible. Urgent = typing/clicking. Non-urgent = heavy renders.
Streaming SSR: send HTML shell immediately. Stream sections as data arrives.
Final advice: Every concept in this guide has a WHY. Master the WHY and the HOW follows naturally.

🎉 Congratulations!

You have completed the React JS Interview Guide — all 15 chapters.

From JSX and Props to Concurrent Mode and Streaming SSR — you now have the complete picture.

Go build great things. 🚀

How to Approach Any Machine Coding Question

A battle-tested framework so you can solve ANY React problem — even ones you have never seen before

By Syed Shaaz Akhtar | Interview Focused Guide

Most candidates fail machine coding rounds not because they do not know React — but because they do not have a system. They stare at the problem, panic, and start typing random code. The interviewer watches this and thinks: "This person cannot think under pressure."

This section gives you THE system. A step-by-step approach that works for every single React problem — whether it is a counter or a Kanban board. Follow this framework every time and you will always have something smart to say, always have a plan, and always produce clean working code.

> 💡 **Tip**
> 🧠 THE CORE INSIGHT:
> The interviewer does not care if you finish the problem.
> They care HOW you think.
>
> A candidate who explains their approach clearly, structures components well,
> and handles edge cases — but runs out of time — scores HIGHER than a candidate
> who silently rushes to a working solution with messy code.
>
> THINK OUT LOUD. PLAN BEFORE YOU CODE. NARRATE WHAT YOU ARE DOING.
> These three habits alone will double your score.

The 5-Phase Framework — Every Question, Every Time

No matter what the problem is — a simple toggle or a complex dashboard — you go through these same 5 phases. This makes you predictable (to yourself) and professional (to the interviewer).

Phase	Name	Time	What You Do
1	Understand	2–3 min	Read carefully, ask clarifying questions, repeat the problem back
2	Break Down	3–5 min	List components, identify state, map data flow
3	Plan	2–3 min	Decide state shape, identify tricky parts, choose patterns
4	Build	30–50 min	Code top-down: shell → children → logic → edge cases

5	Review	3–5 min	Test edge cases, clean up, talk through improvements

Phase 1 — Understand the Problem (2–3 minutes)

Before writing a single character of code, you must fully understand what you are building. This sounds obvious — but under interview pressure, people skip this and start coding the wrong thing.

The 5 Questions to Ask Yourself (or the Interviewer)

```
// For EVERY problem, run through these 5 questions:

// Q1: What is the CORE feature?
//     Strip away everything else. What is the ONE thing this does?
//     "A todo list that adds and removes items."
//     "A search box that filters a product list."

// Q2: What data is involved?
//     What shape does the data have?
//     "A list of todo objects: { id, text, done }"
//     "A list of products: { id, name, price, category }"
//     "A user object: { id, name, email, avatar }"

// Q3: What can the USER DO?
//     List all interactions as action verbs.
//     "User can: ADD a todo, TOGGLE done, DELETE, FILTER by status."
//     "User can: TYPE in search box, SELECT category, CLICK sort."

// Q4: What are the STATES the UI can be in?
//     Think: loading, empty, filled, error, success, editing...
//     "Loading (fetching), Empty (no results), Filled (list shows), Error."

// Q5: Any CONSTRAINTS or EDGE CASES mentioned?
//     "Cannot add empty todo." "Counter cannot go below 0."
//     "Only one accordion open at a time." "Max file size 5MB."

// THEN: Repeat the problem back to the interviewer.
// "So I need to build X, where the user can do A, B, C.
//  The data looks like Y. I need to handle Z edge case. Is that right?"
// This shows understanding AND gives them a chance to correct you early.
```

> ✅ **Remember**
> Asking clarifying questions is NOT a weakness — it is a senior trait.
> Senior developers always clarify requirements before building.
> Say: "Before I start coding, let me make sure I understand the requirements."
> This one habit makes you look 10x more professional immediately.

Phase 2 — Break Down into Components (3–5 minutes)

The most important skill in React is knowing how to split a UI into components. Interviewers watch this closely. Before coding, draw (or write) your component tree.

The Component-Splitting Rules

```
// RULE 1: One responsibility per component
// If you can describe a component in one sentence without "and" — good size.
// "Shows a single product card" ✓
// "Shows product card AND manages the cart AND handles checkout" ✗ — too big

// RULE 2: Split when you see repetition
// If you are writing the same JSX structure 3 times — extract a component.
// 3 product cards → <ProductCard />> component
// 3 filter buttons → <FilterButton />> component

// RULE 3: Split when a section has its own state
// A search bar has its own input state → SearchBar component
// A modal has its own open/close state → Modal component

// RULE 4: Split when a section gets complex (>50 lines)
// If a section of your JSX is getting long — it probably deserves its own file.

// EXAMPLE — Todo App breakdown:
//
//  App
//  ├── Header (title, maybe global actions)
//  ├── TodoForm (the add-todo input + button)
//  ├── FilterBar (All / Active / Completed tabs)
//  ├── TodoList (the ul, maps over todos)
//  |   └── TodoItem (single todo: checkbox + text + delete button)
//  └── Footer (count of remaining)
//
// Write this tree BEFORE coding. Then build top-down.
```

Identifying State — The Key Question

```
// For each piece of data, ask:
// "WHO needs this? Which components read it? Which write it?"

// LOCAL STATE → keep inside the component that owns it
// Only ONE component uses this data. No sibling needs it.
// Example: isDropdownOpen — only the Dropdown component cares.
// Example: inputValue in SearchBar — stays in SearchBar.

// LIFTED STATE → move to lowest common ancestor
// Two or more SIBLING components need the same data.
// Example: todos[] — both TodoList and Footer need it → goes in App.
// Example: searchQuery — SearchBar writes it, ProductList reads it → goes in
App.
```

```
// CONTEXT / REDUX → for data needed everywhere
// Deep tree, many components need it, or it changes app-wide behaviour.
// Example: currentUser, theme, language, cartItems.

// THE QUESTION TO ASK:
// "If I put this state here and a sibling also needs it — would I have to
//  pass it through unnecessary middle components?"
// If YES → lift it up.
// If NO → keep it local.
```

Phase 3 — Plan the State Shape (2–3 minutes)

Once you know WHAT state you need and WHERE it lives, spend two minutes deciding exactly WHAT SHAPE the state should be. This decision affects everything that comes after.

State Shape Rules

```
// RULE 1: Don't store what you can COMPUTE (derived state)
// ❌ BAD — storing both todos AND filteredTodos
const [todos,          setTodos]          = useState([]);
const [filteredTodos,  setFilteredTodos]  = useState([]);
// filteredTodos is derived from todos + filter. Never store it.

// ✅ GOOD — compute it
const [todos,  setTodos]  = useState([]);
const [filter, setFilter] = useState("all");
const filteredTodos = todos.filter(/* based on filter */);
// filteredTodos auto-updates whenever todos or filter changes. No sync needed.

// RULE 2: Flat state over nested
// ❌ BAD
const [user, setUser] = useState({ profile: { name: "", email: "" }, settings:
{} });
// Updating nested state is complex: setUser(u => ({ ...u, profile:
{ ...u.profile, name: "X" } }))

// ✅ BETTER for simple cases — flat
const [name,  setName]  = useState("");
const [email, setEmail] = useState("");

// RULE 3: Use useReducer when you have 3+ related state values
// If you find yourself doing 3+ useState calls that always change together,
// it is a sign to switch to useReducer.
// Example: { loading, error, data } — classic useReducer candidate

// RULE 4: Model list items as objects with IDs
// ❌ BAD — list of strings
const [todos, setTodos] = useState(["Buy milk", "Read book"]);
// How do you toggle one? You have to use index — fragile.
```

```
// ✅ GOOD — list of objects with stable IDs
const [todos, setTodos] = useState([
  { id: 1, text: "Buy milk", done: false },
  { id: 2, text: "Read book", done: false },
]);
// Now toggle: setTodos(prev => prev.map(t => t.id === 1 ? {...t, done: !t.done}
: t))
```

Phase 4 — Build (The Actual Coding)

Now you code. But HOW you code matters as much as WHAT you produce. Follow this sequence every time.

The Build Sequence — Top Down

```
// STEP 1: Write the App shell — just structure, no logic yet
function App() {
  return (
    <div>
      <Header />
      <TodoForm />
      <FilterBar />
      <TodoList />
      <Footer />
    </div>
  );
}
// This compiles. You can see the structure immediately.
// Interviewer can see your plan.

// STEP 2: Stub out child components — just render something
function TodoForm()  { return <div>Form goes here</div>; }
function TodoList()  { return <div>List goes here</div>; }
function FilterBar() { return <div>Filters go here</div>; }
// App renders without errors. Foundation is set.

// STEP 3: Add state to App — just declarations first
function App() {
  const [todos,  setTodos]  = useState([]);
  const [filter, setFilter] = useState("all");
  const [input,  setInput]  = useState("");
  // ...return same JSX
}

// STEP 4: Wire up the MOST IMPORTANT feature first
// In a todo app: adding a todo. In a search app: the search working.
// Get the happy path working. Then add edge cases.

// STEP 5: Pass props down, connect events
```

```
// Add the data and handlers to each component.

// STEP 6: Add edge cases LAST
// Empty state message, disabled buttons, input validation.
// Only add these after the core works.
```

The 4 Lines You Should Always Write First

```
// These 4 things should be in EVERY component you write:

// 1. Meaningful component name
function ProductCard({ product }) {   // not "Card", not "Comp", not "Thing"

// 2. Destructure props immediately
  const { id, name, price, category } = product;  // clear what data you use

// 3. Handle the empty/loading state first
  if (!product) return null;

// 4. Return clean JSX with semantic HTML
  return (
    <article>      {/* not <div> everywhere */}
      <h3>{name}</h3>
      <p>₹{price}</p>
    </article>
  );
}

// These 4 habits signal: "This person writes production code."
```

Phase 5 — Review and Extend (3–5 minutes)

When you finish (or run out of time), do NOT just stop. This phase is where you score BONUS points by showing senior-level thinking.

```
// CHECKLIST — run through this after coding:

// ✓ Empty state
//   Does the UI look right when the list is empty?
//   "No todos yet. Add one above!" — not a blank space.

// ✓ Loading state
//   If you fetch data, is there a spinner/skeleton while it loads?

// ✓ Error state
//   If the API fails, does the user see a helpful message + retry option?

// ✓ Input validation
//   Can the user submit an empty form? Add a guard.
```

```
//   Can the user type spaces only? Use .trim().

// ✓ Keys on lists
//   Every .map() has a key prop? Using id, not index?

// ✓ Cleanup in useEffect
//   If you use useEffect with subscriptions/timers/fetch — return cleanup.

// ✓ Button disabled states
//   Submit disabled when loading? Delete disabled when empty list?

// THEN — proactively mention improvements:
// "Given more time I would:"
// 1. Extract the fetch logic into a useFetch custom hook
// 2. Add optimistic updates so delete feels instant
// 3. Add a loading skeleton instead of a spinner
// 4. Persist the data to localStorage so it survives a refresh
// 5. Add keyboard shortcut — Enter to submit the form

// This list shows you THINK about production quality.
// You do not need to build them — just name them confidently.
```

The Universal Pattern for ANY React Problem

Every React machine coding problem you will ever see fits into one of 6 categories. Here is exactly how to approach each one.

Category 1 — List + CRUD (Todo, Notes, Cart)

```
// ALWAYS start with this state shape:
const [items, setItems] = useState([]);
// items = array of objects with unique id

// 4 operations, same pattern every time:

// ADD
setItems(prev => [...prev, { id: Date.now(), ...newItemData }]);

// UPDATE / TOGGLE
setItems(prev => prev.map(item =>
  item.id === targetId ? { ...item, ...changes } : item
));

// DELETE
setItems(prev => prev.filter(item => item.id !== targetId));

// DERIVED — filter / sort / count
```

```
const filteredItems = useMemo(() => {
  return items.filter(/* condition */).sort(/* if needed */);
}, [items, filter, sortBy]);

// EDGE CASES to handle:
// - Empty list message
// - Duplicate prevention (check if item already exists before adding)
// - Input trim() guard (no empty/whitespace-only submissions)
```

Category 2 — Search + Filter + Sort

```
// State:
const [query,    setQuery]    = useState("");
const [category, setCategory] = useState("all");
const [sortBy,   setSortBy]   = useState("default");
// NOTE: never store filteredResults in state — always compute it

// The pipeline — always in this ORDER: filter → search → sort
const results = useMemo(() => {
  let list = [...allItems];  // spread to avoid mutating original

  // 1. Filter by category first (reduces the list fastest)
  if (category !== "all") {
    list = list.filter(item => item.category === category);
  }

  // 2. Filter by search query
  if (query.trim()) {
    const q = query.toLowerCase();
    list = list.filter(item => item.name.toLowerCase().includes(q));
  }

  // 3. Sort last (sort a smaller list = faster)
  if (sortBy === "price-asc")  return list.sort((a, b) => a.price - b.price);
  if (sortBy === "price-desc") return list.sort((a, b) => b.price - a.price);
  return list;
}, [query, category, sortBy]);

// ALWAYS show result count:
<p>{results.length} results found</p>

// If search has API call → add useDebounce(query, 400)
// to prevent API call on every keystroke
```

Category 3 — Data Fetching + Display

```
// State pattern — always these 3:
const [data,    setData]    = useState(null);
```

```
const [loading, setLoading] = useState(true);
const [error,   setError]   = useState(null);

// The COMPLETE fetch template — memorise this:
useEffect(() => {
  const controller = new AbortController();  // ALWAYS include

  async function fetchData() {
    try {
      setLoading(true);
      setError(null);

      const res = await fetch("/api/data", { signal: controller.signal });
      if (!res.ok) throw new Error(`Error ${res.status}`);
      const json = await res.json();
      setData(json);

    } catch (err) {
      if (err.name !== "AbortError") setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  fetchData();
  return () => controller.abort();  // ALWAYS include cleanup
}, [/* deps */]);

// Render — always handle all 3 states:
if (loading) return <Skeleton />;     // not just "Loading..."
if (error)   return <ErrorMessage message={error} onRetry={refetch} />;
if (!data?.length) return <EmptyState />;
return <DataDisplay data={data} />;
```

Category 4 — Forms + Validation

```
// State pattern for a form:
const [values,  setValues]  = useState({ name: "", email: "", password: "" });
const [touched, setTouched] = useState({});
const [loading, setLoading] = useState(false);

// Derive errors — NEVER store in state:
const errors = validate(values);  // pure function → returns {} or { field:
"message" }
const isValid = Object.keys(errors).length === 0;

// Generic change handler — one function for ALL inputs:
const handleChange = (e) => {
  const { name, value, type, checked } = e.target;
```

```
  setValues(prev => ({
    ...prev,
    [name]: type === "checkbox" ? checked : value,
  }));
};

// Blur handler — marks field as visited (show error after user leaves field):
const handleBlur = (e) => {
  setTouched(prev => ({ ...prev, [e.target.name]: true }));
};

// Submit handler:
const handleSubmit = async (e) => {
  e.preventDefault();  // ALWAYS first line
  if (!isValid) {
    // Touch all fields to show all errors at once
    setTouched(Object.keys(values).reduce((acc, k) => ({ ...acc, [k]: true }),
{}));
    return;
  }
  setLoading(true);
  try { await submitToAPI(values); }
  finally { setLoading(false); }
};

// Show error only if field was touched AND has error:
{touched.email && errors.email && <p className="error">{errors.email}</p>}
```

Category 5 — UI Components (Modal, Accordion, Tabs, Rating)

```
// ALWAYS identify: what is the "which one is active" state?
```

```
// Modal — boolean open/close:
const [isOpen, setIsOpen] = useState(false);
// open:  <button onClick={() => setIsOpen(true)}>
// close: onClose={() => setIsOpen(false)} — pass to modal
// overlay click closes: <div onClick={onClose}> on the backdrop
// PORTAL: wrap modal content in createPortal(jsx, document.body)
//         so it escapes any overflow: hidden parent

// Accordion — single open index:
const [openIndex, setOpenIndex] = useState(null);
// toggle: setOpenIndex(prev => prev === i ? null : i)
// isOpen: openIndex === index
// animation: maxHeight trick (cannot animate height: auto)
//   style={{ maxHeight: isOpen ? 500 : 0, overflow: "hidden", transition: "max-
height 0.3s" }}

// Tabs — active tab key or index:
const [activeTab, setActiveTab] = useState("overview");
```

```
// highlight: className={activeTab === tab ? "active" : ""}
// show panel: {activeTab === "overview" && <OverviewPanel />}

// Star Rating — selected + hovered:
const [selected, setSelected] = useState(0);
const [hovered,  setHovered]  = useState(0);
const display = hovered || selected;  // hovered takes priority
// fill star: star <= display ? filled color : empty color

// Dropdown — open + selected value:
const [isOpen,  setIsOpen]  = useState(false);
const [value,   setValue]   = useState(null);
// close on outside click: useClickOutside hook
```

Category 6 — Multi-Step / Wizard

```
// State — always useReducer for multi-step (too complex for useState):
const [state, dispatch] = useReducer(reducer, {
  step:      0,
  values:    { /* all form fields */ },
  errors:    {},
  submitted: false,
});

// Key actions in the reducer:
// CHANGE — update a field
// NEXT   — validate current step, advance if valid
// BACK   — go to previous step
// SUBMIT — mark as submitted
// RESET  — back to initial state

// Validation gate inside the reducer:
case "NEXT": {
  const errors = validateStep(state.step, state.values);
  if (Object.keys(errors).length > 0) return { ...state, errors };
  return { ...state, step: state.step + 1, errors: {} };
}

// Progress bar:
const progress = (step / totalSteps) * 100;
<div style={{ width: `${progress}%`, transition: "width 0.3s" }} />

// Step indicator — show ✓ for completed, number for future:
{steps.map((s, i) => (
  <span key={i} style={{ color: i < step ? "green" : i === step ? "blue" :
"gray" }}>
    {i < step ? "✓" : i + 1}. {s}
  </span>
))}
```

The Confidence Toolkit — Phrases That Make You Sound Senior

What you SAY during the interview matters as much as what you code. Here are exact phrases to use at each stage. Practice saying these out loud until they feel natural.

```
// ── WHEN YOU FIRST READ THE PROBLEM ──────────────────────────────
"Before I start coding, let me make sure I understand the requirements."
"Can I ask a few clarifying questions?"
"So the core feature is X, and the user can do A, B, C — is that right?"
"Should I use real API data or mock it for now?"
"Do you want me to handle mobile/responsive layout or is desktop fine?"

// ── WHEN PLANNING COMPONENTS ─────────────────────────────────────
"Let me first think about how to break this into components."
"I am thinking the App component holds the state, and passes it down to..."
"This piece of state needs to be lifted up because both X and Y need it."
"I will keep this state local to the component since nothing else needs it."

// ── WHEN WRITING STATE ───────────────────────────────────────────
"I will model the list items as objects with unique IDs — cleaner for updates."
"I am using useReducer here because there are multiple related state fields."
"This is derived state — I will compute it instead of storing it separately."
"I am putting the filter logic in useMemo so it only recomputes when needed."

// ── WHEN WRITING EFFECTS ─────────────────────────────────────────
"I am adding an AbortController here to prevent race conditions."
"The cleanup function cancels the request when the component unmounts."
"I am using a dependency array of [userId] so this refetches when the ID
changes."

// ── WHEN YOU HIT A BUG ───────────────────────────────────────────
"Let me think through this... [pause]. Ah, I see — I need to use a functional
  update here because the new state depends on the previous state."
"That is because I am mutating the array directly. Let me spread it instead."
"I think the issue is the dependency array — let me add X to it."

// ── WHEN RUNNING OUT OF TIME ─────────────────────────────────────
"I have the core feature working. Given more time, I would also:"
"1. Extract this logic into a useFetch custom hook"
"2. Add a loading skeleton instead of a plain spinner"
"3. Handle the error state with a retry button"
"4. Persist this to localStorage so data survives a refresh"
"5. Add proper TypeScript types for all props and state"
```

The 10 Most Common Patterns — Memorise These

These 10 code patterns appear in almost every machine coding round. If you can write each one from memory in under 30 seconds, you will never get stuck.

```
// 1. TOGGLE BOOLEAN
setIsOpen(prev => !prev);

// 2. ADD ITEM TO LIST
setItems(prev => [...prev, { id: Date.now(), ...newItem }]);

// 3. UPDATE ITEM IN LIST
setItems(prev => prev.map(item =>
  item.id === id ? { ...item, ...changes } : item
));

// 4. DELETE ITEM FROM LIST
setItems(prev => prev.filter(item => item.id !== id));

// 5. GENERIC INPUT CHANGE HANDLER
const handleChange = e => {
  const { name, value, type, checked } = e.target;
  setForm(prev => ({ ...prev, [name]: type === "checkbox" ? checked : value }));
};

// 6. FETCH WITH LOADING + ERROR
const [data, setData]       = useState(null);
const [loading, setLoading] = useState(true);
const [error, setError]     = useState(null);

// 7. DEBOUNCE A VALUE
const debouncedQuery = useDebounce(query, 400);
useEffect(() => { /* API call */ }, [debouncedQuery]);

// 8. CLOSE ON OUTSIDE CLICK
const ref = useClickOutside(() => setIsOpen(false));
<div ref={ref}>...</div>

// 9. DERIVED / COMPUTED VALUE
const filtered = useMemo(() => items.filter(...), [items, filter]);

// 10. FORM SUBMIT GUARD
const handleSubmit = e => {
  e.preventDefault();
  if (!input.trim()) return;
  /* proceed */;
  setInput("");
};
```

Mental Checklist — Before You Submit Your Solution

✅ **Remember**

- ☐ Does every .map() have a key prop using a unique id (not index)?
- ☐ Does every form have e.preventDefault() as the first line of onSubmit?
- ☐ Is there an empty state when the list is empty?
- ☐ Is there a loading state when data is being fetched?
- ☐ Is there an error state with a retry button?
- ☐ Are inputs validated before form submission? (.trim() check)
- ☐ Is the button disabled when loading / when input is empty?
- ☐ Are you computing derived state instead of storing it?
- ☐ Are you using functional updates (prev =>) when new state depends on old?
- ☐ Is useEffect cleanup handled (AbortController / removeEventListener)?
- ☐ Are component and variable names descriptive and clear?
- ☐ Have you talked through at least 2 improvements you would make with more time?

💡 **Tip**

🎯 THE FINAL MINDSET:

"I have a framework. I know how to break any problem down.
I know exactly what state I need and where it goes.
I know how to handle every edge case.
I know what to say when I get stuck.
I know how to impress even when I run out of time."

That is the mindset. Now you have the system to back it up.
Go build. Go practice. Go ace the round.

🏆 **You are ready. The framework is yours. Now go build.**

Chapter 16: Machine Coding Round

10 real interview projects — Basic to Senior level — step by step with every concept practical

By Syed Shaaz Akhtar | Interview Focused Guide

Machine coding rounds are where interviews are WON or LOST. The interviewer gives you a problem and watches you code live for 45–90 minutes. They are not just checking if you get the right answer — they are watching HOW you think, how you structure code, how you handle edge cases, and whether your React fundamentals are solid.

This chapter has 10 real projects arranged in increasing difficulty. Each project isolates specific concepts. Build every single one yourself before your interview. The last three projects are senior-level — completing them confidently will set you apart from 95% of candidates.

> 💡 **Tip**
> 🏆 HOW TO USE THIS CHAPTER:
> 1. Read the problem. Close the book. Build it from scratch.
> 2. Then read the solution and compare.
> 3. Note every gap between your code and the solution.
> 4. Build it AGAIN — from memory — the next day.
>
> Machine coding is a SKILL, not knowledge. Knowledge comes from reading.
> Skill comes from building. Build each project at least twice.
>
> INTERVIEWER CHECKLIST — what they watch:
> — Do you break the problem into components before writing any code?
> — Do you manage state correctly — right level, right structure?
> — Do you handle loading, error, and empty states?
> — Do you write clean, readable component names and variable names?
> — Do you think about edge cases (empty input, double click, network failure)?
> — Do you use keys correctly on lists?

Project 1 — Counter App [Level: Beginner]

Problem Statement

Build a counter with increment, decrement, and reset buttons. The counter cannot go below 0. Add a step input so the user can change how much to increment/decrement.

Concepts Covered

useState, event handlers, conditional logic, controlled input.

```
// Counter.jsx
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
  const [step,  setStep]  = useState(1);

  const increment = () => setCount(c => c + step);
  const decrement = () => setCount(c => Math.max(0, c - step));  // cannot go
below 0
  const reset     = () => setCount(0);

  return (
    <div style={{ textAlign: "center", padding: 40 }}>
      <h1 style={{ fontSize: 64 }}>{count}</h1>

      <div>
        <button onClick={decrement} disabled={count === 0}>−</button>
        <button onClick={reset}>Reset</button>
        <button onClick={increment}>+</button>
      </div>

      <div style={{ marginTop: 20 }}>
        <label>Step: </label>
        <input
          type="number"
          min="1"
          value={step}
          onChange={e => setStep(Math.max(1, Number(e.target.value)))}
          style={{ width: 60 }}
        />
      </div>
    </div>
  );
}

export default Counter;

// WHAT TO NOTICE:
// 1. Functional update: setCount(c => c + step) — safe with React batching
// 2. Math.max(0, ...) — prevents negative values without a separate if-check
// 3. disabled={count === 0} — correct UX, prevents confusing state
// 4. Math.max(1, ...) on step input — prevents step of 0 or negative
```

Project 2 — Todo List [Level: Beginner-Medium]

Problem Statement

Build a todo list. User can add todos, mark them complete, delete them, and filter by All / Active / Completed. Show a count of remaining active todos.

Concepts Covered

useState with arrays, CRUD operations, derived state, controlled form, list rendering with keys, conditional rendering.

```
// TodoApp.jsx
import { useState } from "react";

let nextId = 1;

function TodoApp() {
  const [todos,  setTodos]  = useState([]);
  const [input,  setInput]  = useState("");
  const [filter, setFilter] = useState("all"); // "all" | "active" | "completed"

  // ADD
  function handleAdd(e) {
    e.preventDefault();
    if (!input.trim()) return;  // guard: no empty todos
    setTodos(prev => [...prev, { id: nextId++, text: input.trim(), done:
false }]);
    setInput("");
  }

  // TOGGLE
  function handleToggle(id) {
    setTodos(prev => prev.map(t => t.id === id ? { ...t, done: !t.done } : t));
  }

  // DELETE
  function handleDelete(id) {
    setTodos(prev => prev.filter(t => t.id !== id));
  }

  // DERIVED STATE — compute from todos, do not store in separate state
  const filtered = todos.filter(t => {
    if (filter === "active")    return !t.done;
    if (filter === "completed") return t.done;
    return true;
  });
  const activeCount = todos.filter(t => !t.done).length;

  return (
    <div style={{ maxWidth: 480, margin: "40px auto" }}>
      <h1>Todo List</h1>

      {/* ADD FORM */}
```

```
      <form onSubmit={handleAdd} style={{ display: "flex", gap: 8 }}>
        <input
          value={input}
          onChange={e => setInput(e.target.value)}
          placeholder="What needs to be done?"
          style={{ flex: 1 }}
        />
        <button type="submit">Add</button>
      </form>

      {/* FILTER TABS */}
      <div style={{ marginTop: 12, display: "flex", gap: 8 }}>
        {["all", "active", "completed"].map(f => (
          <button
            key={f}
            onClick={() => setFilter(f)}
            style={{ fontWeight: filter === f ? "bold" : "normal" }}
          >
            {f}
          </button>
        ))}
      </div>

      {/* LIST */}
      {filtered.length === 0
        ? <p style={{ color: "#999" }}>No todos here.</p>
        : (
          <ul style={{ listStyle: "none", padding: 0 }}>
            {filtered.map(todo => (
              <li key={todo.id} style={{ display: "flex", alignItems: "center",
gap: 8, padding: "8px 0" }}>
                <input
                  type="checkbox"
                  checked={todo.done}
                  onChange={() => handleToggle(todo.id)}
                />
                <span style={{ flex: 1, textDecoration: todo.done ? "line-
through" : "none", color: todo.done ? "#aaa" : "inherit" }}>
                  {todo.text}
                </span>
                <button onClick={() => handleDelete(todo.id)}>×</button>
              </li>
            ))}
          </ul>
        )
      }

      {/* FOOTER */}
      <p style={{ color: "#777", fontSize: 13 }}>{activeCount}
item{activeCount !== 1 ? "s" : ""} left</p>
    </div>
```

```
  );
}

export default TodoApp;

// WHAT TO NOTICE:
// 1. nextId outside component — avoids recreating on every render
// 2. Derived state — filtered and activeCount computed from todos, not stored
// 3. Functional updates (prev =>) — correct when new state depends on old
// 4. input.trim() guard — prevents whitespace-only todos
// 5. Empty state message — professional UX detail
```

Project 3 — Search + Filter + Sort [Level: Medium]

Problem Statement

Given a list of 20 products (name, price, category), build a UI with: live search by name, category filter dropdown, sort by price (low→high / high→low), and show total count of visible results.

Concepts Covered

Multiple useState, derived computed values, useMemo for performance, controlled inputs, chained array operations.

```
// products.js — mock data
export const PRODUCTS = [
  { id: 1,  name: "Wireless Headphones", price: 2999,  category: "audio"    },
  { id: 2,  name: "Mechanical Keyboard",  price: 4500,  category: "computing"},
  { id: 3,  name: "USB-C Hub",            price: 1200,  category: "computing"},
  { id: 4,  name: "Desk Lamp",            price: 899,   category: "lighting" },
  { id: 5,  name: "Bluetooth Speaker",    price: 1800,  category: "audio"    },
  { id: 6,  name: "Webcam HD",            price: 3200,  category: "computing"},
  { id: 7,  name: "Standing Desk",        price: 15000, category: "furniture"},
  { id: 8,  name: "Monitor Arm",          price: 2200,  category: "furniture"},
  { id: 9,  name: "LED Strip",            price: 450,   category: "lighting" },
  { id: 10, name: "Noise Cancelling Buds",price: 5999,  category: "audio"    },
];
```

```
// ProductCatalog.jsx
import { useState, useMemo } from "react";
import { PRODUCTS } from "./products";

const CATEGORIES = ["all", ...new Set(PRODUCTS.map(p => p.category))];

function ProductCatalog() {
  const [search,   setSearch]   = useState("");
  const [category, setCategory] = useState("all");
  const [sortBy,   setSortBy]   = useState("default");
```

```
  // useMemo — only recompute when filters change
  const results = useMemo(() => {
    let list = PRODUCTS;

    // 1. Filter by category
    if (category !== "all") {
      list = list.filter(p => p.category === category);
    }

    // 2. Filter by search query
    if (search.trim()) {
      const q = search.toLowerCase();
      list = list.filter(p => p.name.toLowerCase().includes(q));
    }

    // 3. Sort
    if (sortBy === "price-asc")  return [...list].sort((a, b) => a.price -
b.price);
    if (sortBy === "price-desc") return [...list].sort((a, b) => b.price -
a.price);
    return list;
  }, [search, category, sortBy]);

  return (
    <div style={{ maxWidth: 700, margin: "40px auto", padding: "0 16px" }}>
      <h1>Product Catalog</h1>

      {/* CONTROLS */}
      <div style={{ display: "flex", gap: 12, flexWrap: "wrap", marginBottom: 16
}}>
        <input
          value={search}
          onChange={e => setSearch(e.target.value)}
          placeholder="Search products..."
          style={{ flex: 1, minWidth: 180 }}
        />

        <select value={category} onChange={e => setCategory(e.target.value)}>
          {CATEGORIES.map(c => (
            <option key={c} value={c}>{c === "all" ? "All Categories" :
c}</option>
          ))}
        </select>

        <select value={sortBy} onChange={e => setSortBy(e.target.value)}>
          <option value="default">Default</option>
          <option value="price-asc">Price: Low to High</option>
          <option value="price-desc">Price: High to Low</option>
        </select>
      </div>
```

```
      <p style={{ color: "#666" }}>{results.length} product{results.length !== 1
? "s" : ""} found</p>

      {/* RESULTS */}
      {results.length === 0
        ? <p>No products match your filters.</p>
        : (
          <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fill,
minmax(200px, 1fr))", gap: 16 }}>
            {results.map(product => (
              <div key={product.id} style={{ border: "1px solid #e0e0e0",
borderRadius: 8, padding: 16 }}>
                <h3 style={{ margin: "0 0 8px", fontSize:
15 }}>{product.name}</h3>
                <p style={{ margin: "0 0 4px", color: "#666", fontSize:
13 }}>{product.category}</p>
                <p style={{ margin: 0, fontWeight: "bold", color: "#1a73e8" }}>₹
{product.price.toLocaleString()}</p>
              </div>
            ))}
          </div>
        )
      }
    </div>
  );
}

export default ProductCatalog;

// WHAT TO NOTICE:
// 1. useMemo — all 3 operations (filter-category, filter-search, sort) in one
memo
// 2. [...list].sort() — spread to avoid mutating the original array
// 3. Chained operations: category filter → search filter → sort
// 4. CATEGORIES derived outside component — stable reference, not recreated
// 5. Results count — always show user how many items match
```

Project 4 — Star Rating Component [Level: Medium]

Problem Statement

Build a reusable StarRating component. On hover, stars fill up to the hovered star. On click, the rating is set. Show the current selected rating below. The component must work as both controlled and uncontrolled.

Concepts Covered

Mouse events (onMouseEnter, onMouseLeave), useState for hover state, reusable component design, props vs internal state.

```
// StarRating.jsx — reusable controlled/uncontrolled component
import { useState } from "react";

function StarRating({ totalStars = 5, value = 0, onChange }) {
  const [hovered, setHovered] = useState(0);  // which star is hovered (0 =
none)

  // Show hovered stars if hovering, else show selected value
  const displayValue = hovered || value;

  return (
    <div style={{ display: "inline-flex", gap: 4, cursor: "pointer" }}>
      {Array.from({ length: totalStars }, (_, i) => i + 1).map(star => (
        <span
          key={star}
          onMouseEnter={() => setHovered(star)}
          onMouseLeave={() => setHovered(0)}
          onClick={() => onChange?.(star)}  // optional chaining — works
uncontrolled too
          style={{
            fontSize: 32,
            color: star <= displayValue ? "#f5a623" : "#ccc",
            transition: "color 0.1s",
            userSelect: "none",
          }}
        >
          ★
        </span>
      ))}
    </div>
  );
}

// ProductReview.jsx — using StarRating
function ProductReview() {
  const [rating,  setRating]  = useState(0);
  const [review,  setReview]  = useState("");
  const [submitted, setSubmitted] = useState(false);

  function handleSubmit(e) {
    e.preventDefault();
    if (rating === 0) { alert("Please select a star rating"); return; }
    setSubmitted(true);
  }

  if (submitted) {
    return (
      <div>
```

```
        <h2>Thank you for your review!</h2>
        <StarRating value={rating} />
        <p>{review}</p>
      </div>
    );
  }

  return (
    <form onSubmit={handleSubmit} style={{ maxWidth: 400 }}>
      <h2>Rate this product</h2>
      <StarRating value={rating} onChange={setRating} />
      {rating > 0 && <p>You selected {rating} star{rating > 1 ? "s" : ""}</p>}
      <br />
      <textarea
        value={review}
        onChange={e => setReview(e.target.value)}
        placeholder="Write your review..."
        rows={4}
        style={{ width: "100%", marginTop: 12 }}
      />
      <button type="submit" style={{ marginTop: 8 }}>Submit Review</button>
    </form>
  );
}

export default ProductReview;

// WHAT TO NOTICE:
// 1. hovered || value — elegant: hovered takes priority, falls back to selected
// 2. onChange?.() — optional chaining lets it work without onChange prop
// 3. Array.from({ length: totalStars }) — clean way to create star array
// 4. userSelect: "none" — prevents text selection on rapid clicks
// 5. Controlled pattern — value and onChange as props, no internal state for
rating
```

Project 5 — Accordion / FAQ Component [Level: Medium]

Problem Statement

Build an FAQ accordion. Clicking a question expands its answer. Only ONE question can be open at a time. Add smooth animation. Make it reusable with data passed as props.

Concepts Covered

Lifting state (single open index in parent), conditional rendering, CSS transitions, reusable component with data prop.

```
// Accordion.jsx
```

```
import { useState } from "react";

function AccordionItem({ question, answer, isOpen, onToggle }) {
  return (
    <div style={{ borderBottom: "1px solid #e0e0e0" }}>
      <button
        onClick={onToggle}
        style={{
          width: "100%",
          padding: "16px 0",
          background: "none",
          border: "none",
          display: "flex",
          justifyContent: "space-between",
          alignItems: "center",
          cursor: "pointer",
          fontSize: 16,
          fontWeight: 500,
          textAlign: "left",
        }}
      >
        {question}
        <span style={{
          transform: isOpen ? "rotate(180deg)" : "rotate(0deg)",
          transition: "transform 0.25s ease",
          fontSize: 12,
        }}>▼</span>
      </button>

      {/* Answer panel — animated with max-height trick */}
      <div style={{
        maxHeight: isOpen ? 500 : 0,
        overflow: "hidden",
        transition: "max-height 0.3s ease",
      }}>
        <p style={{ padding: "0 0 16px", margin: 0, color: "#555", lineHeight:
1.6 }}>
          {answer}
        </p>
      </div>
    </div>
  );
}

function Accordion({ items, allowMultiple = false }) {
  // For single-open: track one index (null = none open)
  // For multi-open: track a Set of open indices
  const [openIndex, setOpenIndex] = useState(null);
  const [openSet,   setOpenSet]   = useState(new Set());

  function handleToggle(index) {
```

```
    if (allowMultiple) {
      setOpenSet(prev => {
        const next = new Set(prev);
        next.has(index) ? next.delete(index) : next.add(index);
        return next;
      });
    } else {
      setOpenIndex(prev => prev === index ? null : index);
    }
  }

  function isOpen(index) {
    return allowMultiple ? openSet.has(index) : openIndex === index;
  }

  return (
    <div style={{ maxWidth: 640, margin: "40px auto" }}>
      {items.map((item, index) => (
        <AccordionItem
          key={index}
          question={item.question}
          answer={item.answer}
          isOpen={isOpen(index)}
          onToggle={() => handleToggle(index)}
        />
      ))}
    </div>
  );
}

// Usage
const FAQ_DATA = [
  { question: "What is React?",         answer: "A JavaScript library for
building user interfaces." },
  { question: "What are Hooks?",         answer: "Functions that let you use
state and lifecycle in function components." },
  { question: "What is JSX?",            answer: "Syntax extension that looks
like HTML but compiles to React.createElement() calls." },
];

function FAQPage() {
  return <Accordion items={FAQ_DATA} allowMultiple={false} />;
}

export default FAQPage;

// WHAT TO NOTICE:
// 1. maxHeight animation trick — CSS cannot animate "height: auto", use
maxHeight
// 2. allowMultiple prop — same component handles both behaviours
// 3. openIndex === index ? null : index — toggle: clicking open item closes it
```

```
// 4. Set for multi-open — O(1) lookup vs Array.includes which is O(n)
```

Project 6 — Infinite Scroll Product Feed [Level: Medium-Hard]

Problem Statement

Build a product feed that loads more products when the user scrolls to the bottom. Show a loading spinner while fetching. Show "All products loaded" when there are no more. Handle network errors with a retry button.

Concepts Covered

useEffect, IntersectionObserver, pagination state, appending to lists, loading/error/empty states, AbortController.

```
// InfiniteProductFeed.jsx
import { useState, useEffect, useRef, useCallback } from "react";

// Simulated API — replace with real fetch in interviews
function mockFetchProducts(page) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (page > 5) return resolve({ products: [], hasMore: false });
      resolve({
        products: Array.from({ length: 8 }, (_, i) => ({
          id:    (page - 1) * 8 + i + 1,
          name:  `Product ${(page - 1) * 8 + i + 1}`,
          price: Math.floor(Math.random() * 5000) + 500,
        })),
        hasMore: page < 5,
      });
    }, 800);
  });
}

function InfiniteProductFeed() {
  const [products, setProducts] = useState([]);
  const [page,     setPage]     = useState(1);
  const [hasMore,  setHasMore]  = useState(true);
  const [loading,  setLoading]  = useState(false);
  const [error,    setError]    = useState(null);
  const sentinelRef = useRef(null);

  // Fetch page data
  const loadProducts = useCallback(async (pageNum) => {
    try {
      setLoading(true);
      setError(null);
```

```
      const data = await mockFetchProducts(pageNum);
      setProducts(prev => [...prev, ...data.products]);  // APPEND
      setHasMore(data.hasMore);
    } catch (err) {
      setError("Failed to load products. Please try again.");
    } finally {
      setLoading(false);
    }
  }, []);

  // Initial load
  useEffect(() => { loadProducts(1); }, [loadProducts]);

  // IntersectionObserver — watches the sentinel div at the bottom
  useEffect(() => {
    if (!hasMore || loading) return;
    const observer = new IntersectionObserver(([entry]) => {
      if (entry.isIntersecting) {
        setPage(p => {
          const next = p + 1;
          loadProducts(next);
          return next;
        });
      }
    }, { threshold: 0.1 });
    if (sentinelRef.current) observer.observe(sentinelRef.current);
    return () => observer.disconnect();
  }, [hasMore, loading, loadProducts]);

  function handleRetry() {
    setError(null);
    loadProducts(page);
  }

  return (
    <div style={{ maxWidth: 700, margin: "0 auto", padding: 16 }}>
      <h1>Products</h1>

      <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fill,
minmax(150px, 1fr))", gap: 16 }}>
        {products.map(p => (
          <div key={p.id} style={{ border: "1px solid #ddd", borderRadius: 8,
padding: 12 }}>
            <div style={{ background: "#f5f5f5", height: 80, borderRadius: 4,
marginBottom: 8 }} />
            <p style={{ margin: "0 0 4px", fontWeight: 500 }}>{p.name}</p>
            <p style={{ margin: 0, color: "#1a73e8" }}>₹{p.price}</p>
          </div>
        ))}
      </div>
```

```
      {/* Loading spinner */}
      {loading && <p style={{ textAlign: "center", padding: 16
}}>Loading...</p>}

      {/* Error with retry */}
      {error && (
        <div style={{ textAlign: "center", padding: 16 }}>
          <p style={{ color: "red" }}>{error}</p>
          <button onClick={handleRetry}>Try Again</button>
        </div>
      )}

      {/* End of list */}
      {!hasMore && !loading && <p style={{ textAlign: "center", color: "#888",
padding: 16 }}>All products loaded ✓</p>}

      {/* Sentinel — invisible div at bottom, triggers next page load */}
      {hasMore && <div ref={sentinelRef} style={{ height: 1 }} />}
    </div>
  );
}

export default InfiniteProductFeed;

// WHAT TO NOTICE:
// 1. APPEND with [...prev, ...data.products] — not replace
// 2. Sentinel div — 1px tall, invisible, just a trigger point
// 3. observer.disconnect() cleanup — prevents memory leaks
// 4. if (!hasMore || loading) return — do not attach observer when not needed
// 5. Error + retry — professional UX, not just "error occurred"
```

Project 7 — Multi-Step Form [Level: Hard]

Problem Statement

Build a 3-step registration form: Step 1 — Personal Info (name, email). Step 2 — Account Setup (password, confirm). Step 3 — Review & Submit. Each step validates before proceeding. Show a progress bar. On submit, show a success screen.

Concepts Covered

useReducer for complex state, multi-step logic, per-step validation, progress indicator, form reset.

```
// MultiStepForm.jsx
import { useReducer } from "react";

const STEPS = ["Personal Info", "Account Setup", "Review"];
```

```
const initialState = {
  step:    0,
  values:  { name: "", email: "", password: "", confirm: "" },
  errors:  {},
  submitted: false,
};

function validate(step, values) {
  const errors = {};
  if (step === 0) {
    if (!values.name.trim())        errors.name  = "Name is required";
    if (!/\S+@\S+/.test(values.email)) errors.email = "Valid email required";
  }
  if (step === 1) {
    if (values.password.length < 8)         errors.password = "Min 8
characters";
    if (values.confirm !== values.password)  errors.confirm  = "Passwords do not
match";
  }
  return errors;
}

function reducer(state, action) {
  switch (action.type) {
    case "CHANGE":
      return { ...state, values: { ...state.values, [action.field]: action.value
}, errors: {} };
    case "NEXT": {
      const errors = validate(state.step, state.values);
      if (Object.keys(errors).length > 0) return { ...state, errors };
      return { ...state, step: state.step + 1, errors: {} };
    }
    case "BACK":
      return { ...state, step: Math.max(0, state.step - 1), errors: {} };
    case "SUBMIT":
      return { ...state, submitted: true };
    case "RESET":
      return initialState;
    default: return state;
  }
}

function MultiStepForm() {
  const [state, dispatch] = useReducer(reducer, initialState);
  const { step, values, errors, submitted } = state;

  function handleChange(e) {
    dispatch({ type: "CHANGE", field: e.target.name, value: e.target.value });
  }

  if (submitted) {
```

```
    return (
      <div style={{ textAlign: "center", padding: 40 }}>
        <h1>🎉 Registration Complete!</h1>
        <p>Welcome, {values.name}! Check your email at {values.email}.</p>
        <button onClick={() => dispatch({ type: "RESET" })}>Register
Another</button>
      </div>
    );
  }

  const progress = ((step) / STEPS.length) * 100;

  return (
    <div style={{ maxWidth: 480, margin: "40px auto", padding: 24 }}>

      {/* STEP INDICATORS */}
      <div style={{ display: "flex", gap: 8, marginBottom: 8 }}>
        {STEPS.map((s, i) => (
          <div key={i} style={{
            flex: 1, textAlign: "center", fontSize: 13,
            color: i === step ? "#1a73e8" : i < step ? "#34a853" : "#999",
            fontWeight: i === step ? "bold" : "normal",
          }}>
            {i < step ? "✓ " : `${i + 1}. `}{s}
          </div>
        ))}
      </div>

      {/* PROGRESS BAR */}
      <div style={{ height: 4, background: "#e0e0e0", borderRadius: 2,
marginBottom: 24 }}>
        <div style={{ height: "100%", width: `${progress}%`, background:
"#1a73e8", borderRadius: 2, transition: "width 0.3s" }} />
      </div>

      {/* STEP 0 — PERSONAL INFO */}
      {step === 0 && (
        <div>
          <h2>Personal Info</h2>
          <div style={{ marginBottom: 16 }}>
            <label>Full Name<br />
              <input name="name" value={values.name} onChange={handleChange}
style={{ width: "100%", marginTop: 4 }} />
              {errors.name && <p style={{ color: "red", margin: "4px 0 0",
fontSize: 13 }}>{errors.name}</p>}
            </label>
          </div>
          <div style={{ marginBottom: 16 }}>
            <label>Email<br />
              <input name="email" type="email" value={values.email}
onChange={handleChange} style={{ width: "100%", marginTop: 4 }} />
```

```
              {errors.email && <p style={{ color: "red", margin: "4px 0 0",
fontSize: 13 }}>{errors.email}</p>}
            </label>
          </div>
        </div>
      )}

      {/* STEP 1 — ACCOUNT SETUP */}
      {step === 1 && (
        <div>
          <h2>Account Setup</h2>
          <div style={{ marginBottom: 16 }}>
            <label>Password<br />
              <input name="password" type="password" value={values.password}
onChange={handleChange} style={{ width: "100%", marginTop: 4 }} />
              {errors.password && <p style={{ color: "red", margin: "4px 0 0",
fontSize: 13 }}>{errors.password}</p>}
            </label>
          </div>
          <div style={{ marginBottom: 16 }}>
            <label>Confirm Password<br />
              <input name="confirm" type="password" value={values.confirm}
onChange={handleChange} style={{ width: "100%", marginTop: 4 }} />
              {errors.confirm && <p style={{ color: "red", margin: "4px 0 0",
fontSize: 13 }}>{errors.confirm}</p>}
            </label>
          </div>
        </div>
      )}

      {/* STEP 2 — REVIEW */}
      {step === 2 && (
        <div>
          <h2>Review Your Details</h2>
          <table style={{ width: "100%", borderCollapse: "collapse" }}>
            {[["Name", values.name], ["Email", values.email], ["Password",
"••••••••"]].map(([label, val]) => (
              <tr key={label}><td style={{ padding: "8px 0", color: "#666",
width: "40%" }}>{label}</td><td style={{ fontWeight: 500 }}>{val}</td></tr>
            ))}
          </table>
        </div>
      )}

      {/* NAVIGATION */}
      <div style={{ display: "flex", justifyContent: "space-between", marginTop:
24 }}>
        <button onClick={() => dispatch({ type: "BACK" })} disabled={step ===
0}>← Back</button>
        {step < STEPS.length - 1
          ? <button onClick={() => dispatch({ type: "NEXT" })}>Next →</button>
```

```
          : <button onClick={() => dispatch({ type: "SUBMIT" })}>Submit
✓</button>
        }
      </div>
    </div>
  );
}

export default MultiStepForm;

// WHAT TO NOTICE:
// 1. useReducer — ideal for this: multiple fields + multiple actions +
validation logic
// 2. validate() runs INSIDE "NEXT" action — validation gate before advancing
// 3. Progress bar = (step / total) * 100 — simple derived value
// 4. Step indicators show ✓ for completed steps — professional UX
// 5. All form data in ONE values object — not separate states for each field
```

Project 8 — Live Search with Debounce & API [Level: Hard]

Problem Statement

Build a GitHub user search. As the user types, search the GitHub API (debounced 500ms). Show user cards with avatar, name, bio, followers. Show loading skeleton while fetching. Handle "user not found" and network errors. Cancel in-flight requests when query changes.

Concepts Covered

Debouncing, AbortController, race condition prevention, skeleton loading, real API integration, custom hooks composition.

```
// useDebounce.js (from Chapter 12)
function useDebounce(value, delay = 500) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}
```

```
// GitHubSearch.jsx
import { useState, useEffect } from "react";

function UserCardSkeleton() {
  return (
    <div style={{ display: "flex", gap: 16, padding: 16, border: "1px solid
#eee", borderRadius: 8 }}>
```

```
      <div style={{ width: 64, height: 64, borderRadius: "50%", background:
"#e0e0e0" }} />
      <div style={{ flex: 1 }}>
        <div style={{ height: 18, background: "#e0e0e0", borderRadius: 4,
marginBottom: 8, width: "40%" }} />
        <div style={{ height: 14, background: "#e0e0e0", borderRadius: 4, width:
"80%" }} />
      </div>
    </div>
  );
}

function UserCard({ user }) {
  return (
    <div style={{ display: "flex", gap: 16, padding: 16, border: "1px solid
#eee", borderRadius: 8 }}>
      <img src={user.avatar_url} alt={user.login} style={{ width: 64, height:
64, borderRadius: "50%", objectFit: "cover" }} />
      <div>
        <h3 style={{ margin: "0 0 4px" }}>{user.name || user.login}</h3>
        <p style={{ margin: "0 0 4px", color: "#666", fontSize: 14 }}>{user.bio
|| "No bio available"}</p>
        <p style={{ margin: 0, fontSize: 13, color: "#1a73e8" }}>
          👥 {user.followers} followers · ⭐ {user.public_repos} repos
        </p>
      </div>
    </div>
  );
}

function GitHubSearch() {
  const [query,   setQuery]   = useState("");
  const [user,    setUser]    = useState(null);
  const [loading, setLoading] = useState(false);
  const [error,   setError]   = useState(null);

  const debouncedQuery = useDebounce(query, 500);

  useEffect(() => {
    if (!debouncedQuery.trim()) {
      setUser(null); setError(null); return;
    }

    const controller = new AbortController();

    async function fetchUser() {
      try {
        setLoading(true); setError(null); setUser(null);
        const res = await fetch(
          `https://api.github.com/users/${debouncedQuery}`,
          { signal: controller.signal }
```

```
        );
        if (res.status === 404) throw new Error(`User "${debouncedQuery}" not
found`);
        if (!res.ok) throw new Error("GitHub API error. Try again.");
        const data = await res.json();
        setUser(data);
      } catch (err) {
        if (err.name !== "AbortError") setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    fetchUser();
    return () => controller.abort();  // cancel on query change
  }, [debouncedQuery]);

  return (
    <div style={{ maxWidth: 520, margin: "40px auto", padding: "0 16px" }}>
      <h1>GitHub User Search</h1>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
        placeholder="Enter GitHub username..."
        style={{ width: "100%", padding: "10px 12px", fontSize: 16,
borderRadius: 6, border: "1px solid #ccc", marginBottom: 16 }}
      />

      {loading && <UserCardSkeleton />}
      {error && <p style={{ color: "red" }}>⚠️ {error}</p>}
      {user && !loading && <UserCard user={user} />}
      {!query.trim() && <p style={{ color: "#999" }}>Start typing a GitHub
username to search.</p>}
    </div>
  );
}

export default GitHubSearch;

// WHAT TO NOTICE:
// 1. useDebounce — only fires API 500ms after user stops typing
// 2. AbortController — cancel previous request when query changes
// 3. if (!debouncedQuery.trim()) return — clear state when input is empty
// 4. 404 handled separately — meaningful error message
// 5. Skeleton while loading — better UX than just "Loading..."
// 6. user && !loading — ensure old user data does not flash while new loads
```

Project 9 — Shopping Cart with Context [Level: Hard]

Problem Statement

Build a product listing page + cart. Any component can add/remove items. CartIcon in the header shows live count. Cart drawer slides in from the right. Totals computed automatically. Persist cart in localStorage.

Concepts Covered

Context API, custom Provider hook, useLocalStorage, derived state (totals), multiple components sharing one store.

```
// CartContext.jsx — the complete cart state management
import { createContext, useContext, useReducer, useEffect } from "react";

const CartContext = createContext(null);

function cartReducer(state, action) {
  switch (action.type) {
    case "ADD": {
      const exists = state.items.find(i => i.id === action.product.id);
      if (exists) {
        return { ...state, items: state.items.map(i =>
          i.id === action.product.id ? { ...i, qty: i.qty + 1 } : i
        )};
      }
      return { ...state, items: [...state.items, { ...action.product, qty:
1 }] };
    }
    case "REMOVE":
      return { ...state, items: state.items.filter(i => i.id !== action.id) };
    case "UPDATE_QTY":
      if (action.qty <= 0)
        return { ...state, items: state.items.filter(i => i.id !== action.id) };
      return { ...state, items: state.items.map(i =>
        i.id === action.id ? { ...i, qty: action.qty } : i
      )};
    case "CLEAR":
      return { ...state, items: [] };
    case "TOGGLE_DRAWER":
      return { ...state, isOpen: !state.isOpen };
    case "LOAD":
      return { ...state, items: action.items };
    default: return state;
  }
}

export function CartProvider({ children }) {
  const [state, dispatch] = useReducer(cartReducer, { items: [], isOpen:
false });
```

```
  // Load from localStorage on mount
  useEffect(() => {
    try {
      const saved = localStorage.getItem("cart");
      if (saved) dispatch({ type: "LOAD", items: JSON.parse(saved) });
    } catch {}
  }, []);

  // Save to localStorage whenever items change
  useEffect(() => {
    localStorage.setItem("cart", JSON.stringify(state.items));
  }, [state.items]);

  // Derived values — computed, not stored
  const cartCount = state.items.reduce((sum, i) => sum + i.qty, 0);
  const cartTotal = state.items.reduce((sum, i) => sum + i.price * i.qty, 0);

  const value = {
    items: state.items,
    isOpen: state.isOpen,
    cartCount,
    cartTotal,
    addToCart:    (product) => dispatch({ type: "ADD",         product }),
    removeItem:   (id)      => dispatch({ type: "REMOVE",      id }),
    updateQty:    (id, qty) => dispatch({ type: "UPDATE_QTY",  id, qty }),
    clearCart:    ()        => dispatch({ type: "CLEAR" }),
    toggleDrawer: ()        => dispatch({ type: "TOGGLE_DRAWER" }),
  };

  return <CartContext.Provider value={value}>{children}</CartContext.Provider>;
}

export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error("useCart must be used inside CartProvider");
  return ctx;
}
```

```
// Components using the cart
```

```
// Header.jsx
function Header() {
  const { cartCount, toggleDrawer } = useCart();
  return (
    <header style={{ display: "flex", justifyContent: "space-between", padding:
"12px 24px", borderBottom: "1px solid #e0e0e0" }}>
      <h1 style={{ margin: 0, fontSize: 20 }}>MyStore</h1>
      <button onClick={toggleDrawer} style={{ position: "relative", background:
"none", border: "none", cursor: "pointer", fontSize: 24 }}>
        🛒
        {cartCount > 0 && (
```

```
          <span style={{ position: "absolute", top: -6, right: -8, background:
"#e53935", color: "#fff",
            borderRadius: "50%", width: 20, height: 20, fontSize: 11, display:
"flex",
            alignItems: "center", justifyContent: "center" }}>
            {cartCount}
          </span>
        )}
      </button>
    </header>
  );
}

// ProductCard.jsx
function ProductCard({ product }) {
  const { addToCart } = useCart();
  return (
    <div style={{ border: "1px solid #e0e0e0", borderRadius: 8, padding: 16 }}>
      <h3 style={{ margin: "0 0 8px" }}>{product.name}</h3>
      <p style={{ margin: "0 0 12px", color: "#1a73e8", fontWeight: "bold" }}>₹
{product.price}</p>
      <button onClick={() => addToCart(product)} style={{ width: "100%" }}>Add
to Cart</button>
    </div>
  );
}

// CartDrawer.jsx
function CartDrawer() {
  const { items, isOpen, cartTotal, removeItem, updateQty, clearCart,
toggleDrawer } = useCart();
  return (
    <>
      {/* Overlay */}
      {isOpen && <div onClick={toggleDrawer} style={{ position: "fixed", inset:
0, background: "rgba(0,0,0,0.4)", zIndex: 99 }} />}

      {/* Drawer */}
      <div style={{ position: "fixed", top: 0, right: 0, bottom: 0, width: 360,
background: "#fff",
        boxShadow: "-4px 0 12px rgba(0,0,0,0.1)", zIndex: 100,
        transform: isOpen ? "translateX(0)" : "translateX(100%)", transition:
"transform 0.3s ease",
        display: "flex", flexDirection: "column" }}>

        <div style={{ padding: "16px 20px", borderBottom: "1px solid #e0e0e0",
display: "flex", justifyContent: "space-between" }}>
          <h2 style={{ margin: 0 }}>Your Cart ({items.length})</h2>
          <button onClick={toggleDrawer} style={{ background: "none", border:
"none", fontSize: 20, cursor: "pointer" }}>x</button>
        </div>
```

```
        <div style={{ flex: 1, overflowY: "auto", padding: "12px 20px" }}>
          {items.length === 0
            ? <p style={{ color: "#999", textAlign: "center", marginTop:
40 }}>Your cart is empty</p>
            : items.map(item => (
              <div key={item.id} style={{ display: "flex", gap: 12, padding:
"12px 0", borderBottom: "1px solid #f0f0f0" }}>
                <div style={{ flex: 1 }}>
                  <p style={{ margin: "0 0 4px", fontWeight:
500 }}>{item.name}</p>
                  <p style={{ margin: 0, color: "#1a73e8" }}>₹{item.price}</p>
                </div>
                <div style={{ display: "flex", alignItems: "center", gap: 8 }}>
                  <button onClick={() => updateQty(item.id, item.qty -
1)}>−</button>
                  <span>{item.qty}</span>
                  <button onClick={() => updateQty(item.id, item.qty +
1)}>+</button>
                  <button onClick={() => removeItem(item.id)} style={{ color:
"red", background: "none", border: "none", cursor: "pointer" }}>🗑️</button>
                </div>
              </div>
            ))
          }
        </div>

        {items.length > 0 && (
          <div style={{ padding: "16px 20px", borderTop: "1px solid #e0e0e0" }}>
            <p style={{ margin: "0 0 12px", fontWeight: "bold", fontSize:
18 }}>Total: ₹{cartTotal}</p>
            <button style={{ width: "100%", padding: 12, background: "#1a73e8",
color: "#fff", border: "none", borderRadius: 6, cursor: "pointer", fontSize:
16 }}>
              Checkout</button>
            <button onClick={clearCart} style={{ width: "100%", marginTop: 8,
padding: 10, background: "none", border: "1px solid #ccc", borderRadius: 6,
cursor: "pointer" }}>
              Clear Cart</button>
          </div>
        )}
      </div>
    </>
  );
}

// App.jsx
const PRODUCTS = [
  { id: 1, name: "Wireless Headphones", price: 2999 },
  { id: 2, name: "Mechanical Keyboard",  price: 4500 },
  { id: 3, name: "USB-C Hub",            price: 1200 },
  { id: 4, name: "Webcam HD",            price: 3200 },
];
```

```
function App() {
  return (
    <CartProvider>
      <Header />
      <main style={{ padding: 24 }}>
        <div style={{ display: "grid", gridTemplateColumns: "repeat(auto-fill,
minmax(200px, 1fr))", gap: 16 }}>
          {PRODUCTS.map(p => <ProductCard key={p.id} product={p} />)}
        </div>
      </main>
      <CartDrawer />
    </CartProvider>
  );
}

// WHAT TO NOTICE:
// 1. useReducer — complex state with many actions is cleaner with reducer
// 2. useCart() throws if used outside provider — good developer experience
// 3. cartCount and cartTotal are DERIVED — never stored in state
// 4. translateX slide animation — CSS transform, not display toggle
// 5. Overlay click closes drawer — same UX as mobile apps
// 6. updateQty(id, qty - 1) with qty <= 0 removes item — clean UX
// 7. localStorage persist — cart survives page refresh
```

Project 10 — Kanban Board [Level: Senior]

Problem Statement

Build a Kanban board with 3 columns: Todo, In Progress, Done. Cards can be moved between columns by clicking a "Move →" button. Cards can be added to any column. Cards can be deleted. Show card count per column. Persist state in localStorage.

Concepts Covered

useReducer with complex nested state, data modelling for multi-column boards, immutable state updates on nested structures, localStorage persistence, derived counts.

```
// KanbanBoard.jsx
import { useReducer, useEffect, useState } from "react";

const COLUMNS = ["todo", "in-progress", "done"];
const COLUMN_LABELS = { "todo": "📋 Todo", "in-progress": "⚙️ In Progress",
"done": "✅ Done" };

let nextId = 100;

const initialState = {
  cards: {
```

```
    "todo":        [{ id: 1, text: "Setup React project" }, { id: 2, text:
"Design database schema" }],
    "in-progress": [{ id: 3, text: "Build auth system" }],
    "done":        [{ id: 4, text: "Project kickoff" }],
  }
};

function kanbanReducer(state, action) {
  switch (action.type) {

    case "ADD_CARD": {
      const newCard = { id: nextId++, text: action.text };
      return {
        ...state,
        cards: {
          ...state.cards,
          [action.column]: [...state.cards[action.column], newCard],
        }
      };
    }

    case "DELETE_CARD": {
      return {
        ...state,
        cards: {
          ...state.cards,
          [action.column]: state.cards[action.column].filter(c => c.id !==
action.id)
        }
      };
    }

    case "MOVE_CARD": {
      const card = state.cards[action.fromCol].find(c => c.id === action.id);
      return {
        ...state,
        cards: {
          ...state.cards,
          // Remove from source column
          [action.fromCol]: state.cards[action.fromCol].filter(c => c.id !==
action.id),
          // Add to destination column
          [action.toCol]:   [...state.cards[action.toCol], card],
        }
      };
    }

    case "LOAD":
      return action.state;

    default: return state;
```

```
  }
}

function KanbanBoard() {
  const [state, dispatch] = useReducer(kanbanReducer, initialState);

  // Persist to localStorage
  useEffect(() => {
    const saved = localStorage.getItem("kanban");
    if (saved) dispatch({ type: "LOAD", state: JSON.parse(saved) });
  }, []);

  useEffect(() => {
    localStorage.setItem("kanban", JSON.stringify(state));
  }, [state]);

  return (
    <div style={{ minHeight: "100vh", background: "#f4f5f7", padding: 24 }}>
      <h1 style={{ margin: "0 0 24px" }}>Project Board</h1>
      <div style={{ display: "flex", gap: 16, alignItems: "flex-start",
overflowX: "auto" }}>
        {COLUMNS.map(col => (
          <KanbanColumn
            key={col}
            column={col}
            cards={state.cards[col]}
            allColumns={COLUMNS}
            onAdd={(text)            => dispatch({ type: "ADD_CARD",    column:
col, text })}
            onDelete={(id)           => dispatch({ type: "DELETE_CARD", column:
col, id   })}
            onMove={(id, toCol)      => dispatch({ type: "MOVE_CARD",   fromCol:
col, id, toCol })}
          />
        ))}
      </div>
    </div>
  );
}

function KanbanColumn({ column, cards, allColumns, onAdd, onDelete, onMove }) {
  const [adding, setAdding] = useState(false);
  const [newText, setNewText] = useState("");

  function handleAdd(e) {
    e.preventDefault();
    if (!newText.trim()) return;
    onAdd(newText.trim());
    setNewText("");
    setAdding(false);
  }
```

```
  // Other columns to move to (not self)
  const targetCols = allColumns.filter(c => c !== column);

  return (
    <div style={{ width: 280, flexShrink: 0, background: "#ebecf0",
borderRadius: 8, padding: 12 }}>
      {/* Column header */}
      <div style={{ display: "flex", justifyContent: "space-between",
marginBottom: 12 }}>
        <h3 style={{ margin: 0, fontSize: 15 }}>{COLUMN_LABELS[column]}</h3>
        <span style={{ background: "#ddd", borderRadius: 12, padding: "2px
10px", fontSize: 13 }}>{cards.length}</span>
      </div>

      {/* Cards */}
      {cards.map(card => (
        <div key={card.id} style={{ background: "#fff", borderRadius: 6,
padding: 12, marginBottom: 8, boxShadow: "0 1px 3px rgba(0,0,0,0.1)" }}>
          <p style={{ margin: "0 0 8px", fontSize: 14 }}>{card.text}</p>
          <div style={{ display: "flex", gap: 6, flexWrap: "wrap" }}>
            {targetCols.map(toCol => (
              <button
                key={toCol}
                onClick={() => onMove(card.id, toCol)}
                style={{ fontSize: 11, padding: "3px 8px", borderRadius: 4,
background: "#e8f0fe", border: "none", cursor: "pointer", color: "#1a73e8" }}
              >
                → {COLUMN_LABELS[toCol]}
              </button>
            ))}
            <button
              onClick={() => onDelete(card.id)}
              style={{ fontSize: 11, padding: "3px 8px", borderRadius: 4,
background: "#fce8e8", border: "none", cursor: "pointer", color: "#c62828" }}
            >
              Delete
            </button>
          </div>
        </div>
      ))}

      {/* Add card */}
      {adding ? (
        <form onSubmit={handleAdd}>
          <textarea
            value={newText}
            onChange={e => setNewText(e.target.value)}
            placeholder="Enter card title..."
            autoFocus
            rows={2}
```

```
            style={{ width: "100%", borderRadius: 4, border: "1px solid #ccc",
padding: 8, resize: "none" }}
          />
          <div style={{ display: "flex", gap: 8, marginTop: 6 }}>
            <button type="submit" style={{ flex: 1, background: "#1a73e8",
color: "#fff", border: "none", borderRadius: 4, padding: "6px 0", cursor:
"pointer" }}>Add Card</button>
            <button type="button" onClick={() => setAdding(false)}
style={{ background: "none", border: "none", cursor: "pointer", fontSize:
18 }}>x</button>
          </div>
        </form>
      ) : (
        <button
          onClick={() => setAdding(true)}
          style={{ width: "100%", padding: "8px 0", background: "none", border:
"2px dashed #bbb", borderRadius: 6, cursor: "pointer", color: "#666", fontSize:
14 }}
        >
          + Add card
        </button>
      )}
    </div>
  );
}

export default KanbanBoard;

// WHAT TO NOTICE:
// 1. Nested state update — must spread state AND cards AND the specific column
// 2. MOVE_CARD — find card, remove from source, add to destination in one
action
// 3. targetCols = allColumns.filter(c => c !== column) — dynamic move targets
// 4. adding state is LOCAL (per column) — not in the reducer — right level
// 5. autoFocus on textarea — keyboard-friendly UX
// 6. Cards count badge per column — informational, derived from cards.length
// 7. localStorage persistence — board survives refresh
```

Machine Coding Round — Winning Strategy

The 5-Minute Planning Phase (Do This FIRST — Always)

```
// Before writing ANY code, spend 5 minutes planning:

// 1. LIST the components
//    "I need: App, ProductList, ProductCard, SearchBar, FilterBar"

// 2. IDENTIFY the state
```

```
//    "Where does state live? What shape is it?"
//    products: [] — in App (multiple children need it)
//    searchQuery: "" — in App (SearchBar writes, ProductList reads)
//    selectedCategory: "all" — in App
//    loading, error — in App (or custom hook)

// 3. MAP the data flow
//    "Who passes what to whom?"
//    App → searchQuery → SearchBar (as value prop)
//    SearchBar → setSearchQuery → App (as onChange prop)
//    App → filteredProducts → ProductList

// 4. SPOT the derived state
//    "What can I COMPUTE instead of STORE?"
//    filteredProducts = useMemo(products.filter..., [products, query])
//    totalCount = filteredProducts.length

// THEN start coding — top down: App shell → child components → logic
```

Common Machine Coding Mistakes to Avoid

Mistake	What Interviewers Think	How to Fix
Storing derived state in useState	"Does not understand React rendering"	Compute from existing state, do not duplicate
Using array index as key	"Does not understand reconciliation"	Use unique id from data — always
No empty/loading/error state	"Not production-aware, junior thinking"	Handle all 3 states — always
Giant single component	"Cannot think in components"	Break up at 50+ lines — one concern per component
Mutating state directly	"Does not know React fundamentals"	Always [...prev, newItem], never prev.push()
No form e.preventDefault()	"Never built a real form"	First line of every submit handler
No input validation	"Ships broken code"	Guard with if (!input.trim()) return
console.log left in code	"Sloppy coder"	Clean up before presenting solution

The 10 Questions Interviewers Will Ask After You Build

```
1. "How would you make this performant with 10,000 items?"
   → Virtualise the list with react-window. useMemo on filter/sort.
```

```
2. "How would you persist this data?"
   → localStorage with useLocalStorage custom hook.
   → Or API calls with React Query.

3. "How would you add real API integration?"
   → Replace mock data with useFetch / React Query useQuery.
   → Add loading skeleton + error + retry.

4. "What if two users are using this simultaneously?"
   → Polling (usePolling hook every 5s) or WebSocket for real-time.

5. "How would you test this?"
   → Unit test reducers (pure functions — easy).
   → Integration test with React Testing Library.
   → Custom hooks with renderHook from @testing-library/react.

6. "How would you make this accessible?"
   → aria-label on icon buttons. role="list" on lists.
   → keyboard navigation (onKeyDown for Enter/Space).
   → useId() for form label/input connections.

7. "How would you add undo/redo?"
   → Store history array of states. useReducer with "UNDO"/"REDO" actions.

8. "How would you extract the logic?"
   → Move all state + handlers into a custom hook (useTodo, useCart, useKanban).
   → Component becomes pure rendering.

9. "How would you scale this to a large team?"
   → Feature-based folder structure. TypeScript for type safety.
   → Each feature has its own component, hook, and test file.

10. "What would you do differently with more time?"
    → Always have 2-3 honest improvements ready:
    → "I would add optimistic updates, proper error boundaries per section,
       and extract form logic into a reusable useForm hook."
```

✅ **Remember**

Plan 5 minutes before coding — components, state shape, data flow.
Components → State → Logic → Render — build in this order.
Always handle 3 states — loading, error, empty — not just the happy path.
Derived state = compute, not store — filteredProducts, totalCount, cartTotal.
useReducer > useState when you have 3+ related state fields or complex transitions.
Name things clearly — handleAddTodo not h1, products not data, isLoading not flag.
No array index as key — always use unique id.
e.preventDefault() — first line of every form onSubmit.
Clean code beats fancy code — readable, correct, and complete beats clever.

> **Be ready to extend** — interviewers WILL ask follow-ups about performance, persistence, API.

🏆 **You are ready. Go ace your machine coding round!**

End of Chapter 16 • End of React JS Interview Guide